

# Projeto de Desenvolvimento do Site LCQUI

Planejamento Funcional, Arquitetural, de Dados e de Implementação

Zadoque Carneiro

20 de setembro de 2026

## Sumário

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                                 | <b>9</b>  |
| <b>2</b> | <b>Como vai ser o Sistema</b>                                     | <b>9</b>  |
| <b>3</b> | <b>Stakeholders</b>                                               | <b>9</b>  |
| 3.1      | Chefe Geral . . . . .                                             | 9         |
| 3.2      | Gestor de Almoxarifado . . . . .                                  | 10        |
| 3.3      | Gestor de Bens Patrimoniais . . . . .                             | 10        |
| 3.4      | Professor . . . . .                                               | 11        |
| 3.5      | Aluno . . . . .                                                   | 12        |
| 3.6      | Bolsista . . . . .                                                | 12        |
| 3.7      | Matriz de Papéis e Permissões . . . . .                           | 13        |
| <b>4</b> | <b>Modelagem Entidades SQL - 3FN</b>                              | <b>14</b> |
| 4.1      | Usuario . . . . .                                                 | 14        |
| 4.2      | Chefe Geral . . . . .                                             | 14        |
| 4.3      | Gestor de Almoxarifado . . . . .                                  | 14        |
| 4.4      | Tabela Gestor de Almoxarifado × Almoxarifado (N para N) . . . . . | 14        |
| 4.5      | Gestor de bens patrimoniais . . . . .                             | 15        |
| 4.6      | Bem patrimonial . . . . .                                         | 15        |
| 4.7      | Resumo bem patrimonial . . . . .                                  | 15        |
| 4.8      | Historico bem patrimonial . . . . .                               | 17        |
| 4.9      | Alteracao bem patrimonial . . . . .                               | 17        |
| 4.10     | Requisicao Professor edicao Bem patrimonial . . . . .             | 18        |
| 4.11     | Requisicao Professor adicao Bem patrimonial . . . . .             | 19        |
| 4.12     | Lock de Requisição de Patrimônio . . . . .                        | 19        |

|                                                               |    |
|---------------------------------------------------------------|----|
| 4.13 Impressão de Etiquetas de Frasco . . . . .               | 20 |
| 4.14 Almoxarifado . . . . .                                   | 21 |
| 4.15 Substância Química . . . . .                             | 21 |
| 4.16 Resumo do Reagente / Solvente . . . . .                  | 22 |
| 4.17 Composição do Reagente . . . . .                         | 25 |
| 4.18 Especificação do Reagente / Solvente . . . . .           | 26 |
| 4.19 Estoque Mínimo por Almoxarifado . . . . .                | 26 |
| 4.20 Lote . . . . .                                           | 27 |
| 4.21 Frasco de Reagente / Solvente . . . . .                  | 28 |
| 4.22 Histórico do Frasco de Reagente / Solvente . . . . .     | 29 |
| 4.23 Empréstimo de Reagente / Solvente . . . . .              | 30 |
| 4.24 Métricas Agregadas do Reagente . . . . .                 | 32 |
| 4.25 Turma . . . . .                                          | 32 |
| 4.26 Historico de Alunos na Turma . . . . .                   | 33 |
| 4.27 Historico de Posts na Turma . . . . .                    | 33 |
| 4.28 Historico de Comentario em um Post . . . . .             | 34 |
| 4.29 Roteiro de Experimento . . . . .                         | 34 |
| 4.30 Roteiro Professor Compartilhado . . . . .                | 34 |
| 4.31 Professor . . . . .                                      | 35 |
| 4.32 Materia . . . . .                                        | 35 |
| 4.33 Tabela Professor $\times$ Materia (N para N) . . . . .   | 35 |
| 4.34 Aluno . . . . .                                          | 35 |
| 4.35 Bolsista . . . . .                                       | 35 |
| 4.36 Convite para Aluno . . . . .                             | 36 |
| 4.37 Notificação (Unificada) . . . . .                        | 36 |
| 4.38 Tabela Aluno Turma (N para N) . . . . .                  | 38 |
| 4.39 Local . . . . .                                          | 38 |
| 4.40 Post . . . . .                                           | 38 |
| 4.41 Comentario . . . . .                                     | 39 |
| 4.42 Registro de Auditoria . . . . .                          | 39 |
| 4.43 Especificação de Integridade do Modelo 3FN . . . . .     | 40 |
| 4.43.1 Cardinalidades . . . . .                               | 40 |
| 4.43.2 Nullabilidade . . . . .                                | 41 |
| 4.43.3 Chaves e constraints . . . . .                         | 41 |
| 4.43.4 Regras químicas fechadas . . . . .                     | 42 |
| 4.43.5 Máquina de estados e invariantes do frasco . . . . .   | 42 |
| 4.43.6 Fechamento dos estados automáticos . . . . .           | 43 |
| 4.44 Checklist de Fechamento do Domínio e do Modelo . . . . . | 43 |

|          |                                                                                                         |           |
|----------|---------------------------------------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Notas de Mapeamento para Firestore</b>                                                               | <b>45</b> |
| 5.1      | Reintrodução de <i>letra_inicial</i> . . . . .                                                          | 45        |
| 5.2      | Estado físico canônico do catálogo . . . . .                                                            | 46        |
| 5.3      | Espelhamento de <i>nome_equipamento</i> e Localização em <i>Bem_Patrimonial</i> . . . . .               | 46        |
| 5.4      | Composição embutida na especificação . . . . .                                                          | 46        |
| 5.5      | Tabelas “Materialized” são coleções próprias, não <i>views</i> do SGBD . . . . .                        | 47        |
| 5.6      | Unicidade de <i>numero_patrimonio_proposto</i> no Firestore . . . . .                                   | 47        |
| 5.7      | Mecanismo de Sequenciamento Atômico via Documento Singleton ( <i>Contador_Codigo_Frasco</i> ) . . . . . | 47        |
| 5.8      | Índices Compostos Obrigatórios . . . . .                                                                | 47        |
| 5.9      | Mapeamento Completo de Coleções e Sub-coleções Firestore . . . . .                                      | 48        |
| 5.9.1    | Dicionário de dados completo e convenções físicas . . . . .                                             | 53        |
| 5.10     | Estratégia de Busca Textual no Firestore . . . . .                                                      | 86        |
| 5.10.1   | Reagentes . . . . .                                                                                     | 86        |
| 5.10.2   | Bens Patrimoniais . . . . .                                                                             | 87        |
| 5.10.3   | Alunos . . . . .                                                                                        | 87        |
| 5.10.4   | Professores . . . . .                                                                                   | 87        |
| 5.11     | Espelhamento Bidirecional da Associação Aluno–Turma . . . . .                                           | 87        |
| <b>6</b> | <b>Materialized (Views)</b>                                                                             | <b>89</b> |
| 6.1      | Tabela Materialized para Lote . . . . .                                                                 | 89        |
| 6.2      | Tabela Resumo de Almoxarifado - Diário . . . . .                                                        | 89        |
| 6.3      | Tabela Resumo de Reagente/Solvente - Diário . . . . .                                                   | 92        |
| 6.4      | Tabela Resumo de Bens Patrimoniais - Diário . . . . .                                                   | 93        |
| 6.5      | Tabela de Atividade Mensal - Gestor de Almoxarifado . . . . .                                           | 93        |
| 6.6      | Tabela de Atividade Mensal - Gestor de Bens Patrimoniais . . . . .                                      | 94        |
| 6.7      | Catálogo de Métricas Disponíveis por Domínio . . . . .                                                  | 94        |
| <b>7</b> | <b>Requisitos e Regras de Negócio</b>                                                                   | <b>95</b> |
| 7.1      | Requisitos funcionais . . . . .                                                                         | 95        |
| 7.2      | Regras de Negócio Gerais . . . . .                                                                      | 96        |
| 7.2.1    | Status de um frasco de reagente . . . . .                                                               | 96        |
| 7.2.2    | Seleção do tipo em Resumo de Reagente . . . . .                                                         | 96        |
| 7.2.3    | Excluindo um Aluno de uma turma . . . . .                                                               | 97        |
| 7.2.4    | Chefe Geral . . . . .                                                                                   | 97        |
| 7.2.5    | O responsável pelo bem patrimonial não é um Usuário do sistema LCQUI . . . . .                          | 97        |
| 7.2.6    | Historico de Posts na Turma . . . . .                                                                   | 97        |
| 7.2.7    | Não compartilhar o mesmo roteiro 2 vezes com um professor . . . . .                                     | 97        |

|        |                                                                                                    |     |
|--------|----------------------------------------------------------------------------------------------------|-----|
| 7.2.8  | Densidade e concentração na especificação do reagente . . . . .                                    | 98  |
| 7.2.9  | Regra para código do frasco de Reagente . . . . .                                                  | 98  |
| 7.2.10 | Segurança na Reimpressão de Frascos Cadastrados . . . . .                                          | 98  |
| 7.2.11 | Regra em Notificação para Professor . . . . .                                                      | 98  |
| 7.2.12 | Regra para convidar um aluno que não está no sistema . . . . .                                     | 99  |
| 7.2.13 | Regra para registrar um patrimônio como dado baixa . . . . .                                       | 99  |
| 7.2.14 | Cálculo de Volume e Peso de Reagentes (Padrão de Laboratório) . . . . .                            | 100 |
| 7.2.15 | Exclusividade na retirada de reagentes . . . . .                                                   | 101 |
| 7.2.16 | Usuário Multi-role, exceto Chefe Geral . . . . .                                                   | 101 |
| 7.2.17 | Usuário Multi-role, Aluno não pode ser professor . . . . .                                         | 101 |
| 7.2.18 | Usuário Multi-role, Quais multi papéis podem existir . . . . .                                     | 101 |
| 7.2.19 | Aluno que é bolsista . . . . .                                                                     | 102 |
| 7.2.20 | Status, vencimento e descarte de frascos . . . . .                                                 | 102 |
| 7.2.21 | Limites de Geração de Relatórios . . . . .                                                         | 103 |
| 7.2.22 | Almoxarifado inativo: operações bloqueadas e encerramento or-<br>denado . . . . .                  | 103 |
| 7.3    | Matriz formal de obrigatoriedade e dependência de campos . . . . .                                 | 103 |
| 7.4    | Matriz de combinações Multi-Role . . . . .                                                         | 104 |
| 7.5    | Fluxo obrigatório para mutações críticas . . . . .                                                 | 105 |
| 7.6    | Atribuição e revogação de papéis . . . . .                                                         | 105 |
| 7.6.1  | RN-ROLE-01 — Chefe Geral não pode possuir múltiplos papéis . . . . .                               | 105 |
| 7.6.2  | RN-ROLE-02 — Chefe Geral não pode revogar o papel de outro<br>Chefe Geral . . . . .                | 105 |
| 7.6.3  | RN-ROLE-03 — Auto-revogação de Chefe Geral . . . . .                                               | 106 |
| 7.6.4  | RN-ROLE-04 — Revogação de Chefe Geral desativa a conta . . . . .                                   | 106 |
| 7.6.5  | RN-ROLE-05 — Almoxarifado deve possuir pelo menos um gestor . . . . .                              | 106 |
| 7.6.6  | RN-ROLE-06 — Gestores de Almoxarifado não possuem exclusi-<br>vidade sobre movimentações . . . . . | 106 |
| 7.6.7  | RN-ROLE-07 — Revogação de Gestor de Almoxarifado em usuá-<br>rio Multi-Role . . . . .              | 106 |
| 7.6.8  | RN-ROLE-08 — Revogação de Gestor de Bens Patrimoniais em<br>usuário Multi-Role . . . . .           | 107 |
| 7.6.9  | RN-ROLE-09 — Gestor de Bens Patrimoniais não pode ser o único . . . . .                            | 107 |
| 7.6.10 | RN-ROLE-10 — Revogação do último papel desativa a conta . . . . .                                  | 107 |
| 7.6.11 | RN-ROLE-11 — Revogação de papel não significa exclusão do<br>usuário . . . . .                     | 107 |
| 7.6.12 | RN-ROLE-12 — Usuário Multi-Role . . . . .                                                          | 107 |
| 7.6.13 | RN-ROLE-13 — Usuário com apenas um papel . . . . .                                                 | 107 |

|          |                                                                   |            |
|----------|-------------------------------------------------------------------|------------|
| 7.6.14   | RN-ROLE-14 — Recalculo dos Custom Claims . . . . .                | 108        |
| 7.6.15   | RN-ROLE-15 — Auditoria das alterações de papéis . . . . .         | 108        |
| 7.6.16   | RN-ROLE-16 — Invariante Bolsista . . . . .                        | 108        |
| <b>8</b> | <b>Descrição das telas (Dashboards)</b>                           | <b>109</b> |
| 8.1      | Header em comum em todos os papeis . . . . .                      | 109        |
| 8.2      | Header quando o usuário é Multi-Role . . . . .                    | 109        |
| 8.3      | Dashboard-Chefe Geral . . . . .                                   | 109        |
| 8.3.1    | Aba: Almoxarifados . . . . .                                      | 110        |
| 8.3.2    | Aba: Bens Patrimoniais . . . . .                                  | 110        |
| 8.3.3    | Aba: Professores . . . . .                                        | 111        |
| 8.3.4    | Aba: Alunos . . . . .                                             | 111        |
| 8.4      | Dashboard-Professor . . . . .                                     | 111        |
| 8.5      | Dashboard-Gestor de Almoxarifado . . . . .                        | 112        |
| 8.6      | Dashboard-Gestor de Bens Patrimoniais . . . . .                   | 114        |
| 8.7      | Dashboard-Aluno . . . . .                                         | 115        |
| 8.8      | Contratos detalhados das telas e modais . . . . .                 | 116        |
| 8.8.1    | UI-01: acesso, sessão, perfil e troca de papel . . . . .          | 116        |
| 8.8.2    | UI-02: usuários, gestores e atribuição de papéis . . . . .        | 117        |
| 8.8.3    | UI-03: almoxarifado, local e matéria . . . . .                    | 117        |
| 8.8.4    | UI-04: pesquisa, catálogo e detalhes . . . . .                    | 118        |
| 8.8.5    | UI-05: resumo químico, substância, especificação e lote . . . . . | 118        |
| 8.8.6    | UI-06: cadastrar, editar e pesar frasco . . . . .                 | 119        |
| 8.8.7    | UI-07: retirada, devolução, quarentena e descarte . . . . .       | 119        |
| 8.8.8    | UI-08: etiquetas virgens e segunda via . . . . .                  | 120        |
| 8.8.9    | UI-09: patrimônio e análise de requisições . . . . .              | 121        |
| 8.8.10   | UI-10: turmas, membros e convites . . . . .                       | 121        |
| 8.8.11   | UI-11: roteiros, posts e comentários . . . . .                    | 122        |
| 8.8.12   | UI-12: notificações, empréstimos próprios e relatórios . . . . .  | 123        |
| 8.8.13   | UI-13: consultas, cache e armazenamento no navegador . . . . .    | 123        |
| <b>9</b> | <b>Exemplos de fluxos</b>                                         | <b>125</b> |
| 9.1      | Fluxos complementares e critérios de completude . . . . .         | 125        |
| 9.1.1    | Concessão do papel Bolsista . . . . .                             | 125        |
| 9.1.2    | Aceitação de convite . . . . .                                    | 125        |
| 9.1.3    | Arquivamento e desarquivamento de turma . . . . .                 | 125        |
| 9.1.4    | Retirada de reagente . . . . .                                    | 125        |
| 9.1.5    | Devolução de reagente . . . . .                                   | 125        |
| 9.2      | Fluxos do usuário Gestor de Bem Patrimonial . . . . .             | 126        |

|        |                                                                                             |     |
|--------|---------------------------------------------------------------------------------------------|-----|
| 9.2.1  | Adicionando um bem patrimonial do LCQUI no sistema . . . . .                                | 126 |
| 9.2.2  | Baixando a lista de bens inservíveis para dar baixa no SEI . . . .                          | 126 |
| 9.2.3  | Baixando o relatório mensal Geral do Mês de Agosto . . . . .                                | 126 |
| 9.2.4  | Baixando relatório de bens materiais filtrado (Prédio P5, Andar 1)                          | 126 |
| 9.2.5  | Verificando requisições pendentes de professores . . . . .                                  | 127 |
| 9.2.6  | Aprovando uma requisição pendente . . . . .                                                 | 127 |
| 9.2.7  | Alterando um bem patrimonial para: Dado baixa . . . . .                                     | 127 |
| 9.3    | Fluxos do usuário Professor . . . . .                                                       | 127 |
| 9.3.1  | Criando uma turma . . . . .                                                                 | 127 |
| 9.3.2  | Adicionando Alunos em uma turma . . . . .                                                   | 128 |
| 9.3.3  | Criando um Post em uma turma . . . . .                                                      | 128 |
| 9.3.4  | Excluindo um Aluno de uma turma . . . . .                                                   | 128 |
| 9.3.5  | Respondendo um comentário de um aluno . . . . .                                             | 128 |
| 9.3.6  | Pesquisando Reagente Químico para checar disponibilidade . . .                              | 128 |
| 9.3.7  | Pesquisando um bem patrimonial pelo Nome . . . . .                                          | 128 |
| 9.3.8  | Pesquisando um bem patrimonial pelo número de patrimônio . .                                | 129 |
| 9.3.9  | Requisitando alteração de conservação (de Bom para Ruim) . .                                | 129 |
| 9.3.10 | Upload de Roteiro e compartilhamento com outros professores .                               | 129 |
| 9.4    | Fluxos do usuário Chefe Geral . . . . .                                                     | 129 |
| 9.4.1  | Criando um usuário Professor e usuário Gestor de Almoxarifado                               | 129 |
| 9.5    | Fluxos do usuário Aluno . . . . .                                                           | 129 |
| 9.5.1  | Ingressando em uma Turma . . . . .                                                          | 129 |
| 9.5.2  | Baixando um roteiro e realizando um comentário . . . . .                                    | 130 |
| 9.6    | Fluxos do Usuário Gestor de Almoxarifado . . . . .                                          | 130 |
| 9.6.1  | Adicionando um frasco fechado de um reagente existente . . . .                              | 130 |
| 9.6.2  | Adicionando um frasco já aberto de um novo reagente . . . . .                               | 130 |
| 9.6.3  | Realizando empréstimo (Retirada) de um frasco . . . . .                                     | 130 |
| 9.6.4  | Devolvendo um frasco (Cálculo de Consumo) . . . . .                                         | 131 |
| 9.7    | Cobertura operacional por papel e ação . . . . .                                            | 131 |
| 9.7.1  | COM-01 — Todos: Login, recuperar senha, perfil, sair e alternar<br>papel . . . . .          | 131 |
| 9.7.2  | CHE-01 — Chefe: Criar usuário e atribuir Bolsista ou outro papel                            | 131 |
| 9.7.3  | CHE-02 — Chefe: Revogar papel ou autorrevogar chefia . . . .                                | 132 |
| 9.7.4  | CHE-03 — Chefe: Criar/ativar almoxarifado e vincular gestores                               | 132 |
| 9.7.5  | CHE-04 — Chefe: Consultar pessoas, turmas e atividade mensal                                | 132 |
| 9.7.6  | CHE-05 — Chefe: Operar catálogo, patrimônio, matéria, etique-<br>tas e relatórios . . . . . | 132 |
| 9.7.7  | MAT-01 — Professor/Chefe: Cadastrar e editar matéria . . . .                                | 132 |

|        |                                                                                                              |     |
|--------|--------------------------------------------------------------------------------------------------------------|-----|
| 9.7.8  | PRO-01 — Professor: Criar turma e copiar código . . . . .                                                    | 132 |
| 9.7.9  | PRO-02 — Professor: Arquivar e desarquivar turma . . . . .                                                   | 132 |
| 9.7.10 | PRO-03 — Professor: Convidar aluno com ou sem turma . . . . .                                                | 133 |
| 9.7.11 | PRO-04 — Professor: Gerenciar membros e remover aluno . . . . .                                              | 133 |
| 9.7.12 | PRO-05 — Professor: Cadastrar, visualizar e baixar roteiro . . . . .                                         | 133 |
| 9.7.13 | PRO-06 — Professor: Compartilhar, acompanhar e revogar compartilhamento . . . . .                            | 133 |
| 9.7.14 | PRO-07 — Professor: Publicar/editar post e vincular roteiro próprio ou recebido . . . . .                    | 133 |
| 9.7.15 | PRO-08 — Professor: Responder comentário e navegar por notificação . . . . .                                 | 133 |
| 9.7.16 | PRO-09 — Professor: Consultar reagentes, receber empréstimo e acompanhar consumo . . . . .                   | 134 |
| 9.7.17 | PRO-10 — Professor: Pesquisar patrimônio e requisitar adição/edição . . . . .                                | 134 |
| 9.7.18 | ALU-01 — Aluno: Aceitar convite ou ingressar por código . . . . .                                            | 134 |
| 9.7.19 | ALU-02 — Aluno: Visualizar turmas, colegas, posts e baixar roteiro                                           | 134 |
| 9.7.20 | ALU-03 — Aluno: Comentar e ler notificações . . . . .                                                        | 134 |
| 9.7.21 | ALU-04 — Aluno: Consultar estoque sem retirar . . . . .                                                      | 134 |
| 9.7.22 | BOL-01 — Bolsista: Ações acadêmicas, consulta e retirada/devolução como destinatário . . . . .               | 134 |
| 9.7.23 | ALM-01 — Gestor Almoz.: Cadastrar/editar resumo, substância, especificação e lote . . . . .                  | 135 |
| 9.7.24 | ALM-02 — Gestor Almoz.: Cadastrar frasco fechado ou aberto                                                   | 135 |
| 9.7.25 | ALM-03 — Gestor Almoz.: Pesquisar, editar localização, abrir e ajustar estoque por pesagem . . . . .         | 135 |
| 9.7.26 | ALM-04 — Gestor Almoz.: Registrar retirada e autorizar uso didático de vencido . . . . .                     | 135 |
| 9.7.27 | ALM-05 — Gestor Almoz.: Receber devolução de empréstimo próprio ou de outro gestor . . . . .                 | 135 |
| 9.7.28 | ALM-06 — Gestor Almoz.: Tratar alertas, quarentena, extravio, reencontro, vazio, quebra e descarte . . . . . | 135 |
| 9.7.29 | ALM-07 — Gestor Almoz.: Imprimir etiquetas virgens ou segunda via . . . . .                                  | 136 |
| 9.7.30 | ALM-08 — Gestor Almoz.: Gerar relatório de um ou todos os almoxarifados autorizados . . . . .                | 136 |
| 9.7.31 | PAT-01 — Gestor Patr.: Criar resumo, cadastrar e editar bem                                                  | 136 |
| 9.7.32 | PAT-02 — Gestor Patr.: Pesquisar, filtrar e consultar histórico                                              | 136 |

|           |                                                                                             |            |
|-----------|---------------------------------------------------------------------------------------------|------------|
| 9.7.33    | PAT-03 — Gestor Patr.: Analisar, aprovar ou rejeitar pedido . .                             | 136        |
| 9.7.34    | PAT-04 — Gestor Patr.: Gerar relação de inservíveis e registrar<br>baixa após SEI . . . . . | 136        |
| 9.7.35    | PAT-05 — Gestor Patr.: Gerar relatório mensal, personalizado<br>ou individual . . . . .     | 137        |
| <b>10</b> | <b>Tecnologia e Relatórios (Vercel / Firebase)</b>                                          | <b>138</b> |
| 10.1      | Para relatórios . . . . .                                                                   | 138        |
| 10.1.1    | Assinatura Lógica de Relatórios (Hash Canônico) . . . . .                                   | 138        |
| 10.1.2    | Prevenção de Limites de Leitura Firestore (N+1) . . . . .                                   | 138        |
| 10.1.3    | Módulo de Geração de Etiquetas (PDF) . . . . .                                              | 138        |
| 10.2      | Princípio: nunca confiar no <i>client-side</i> . . . . .                                    | 139        |
| 10.2.1    | Funções de Apoio (Autorização) . . . . .                                                    | 139        |
| 10.2.2    | Revogação de Papel com Singleton de Controle de Concorrência<br>(M-14, N-01) . . . . .      | 142        |
| 10.2.3    | Fluxo de Reagentes: Cadastro, Retirada e Devolução . . . . .                                | 146        |
| 10.2.4    | Validação síncrona de validade no cadastro e na abertura . . . . .                          | 171        |
| 10.2.5    | Jobs Agendados: Vencimento, Atraso e Materialização . . . . .                               | 172        |
| 10.2.6    | Fluxo de Bens Patrimoniais: Requisições . . . . .                                           | 183        |
| 10.2.7    | Relatórios em PDF . . . . .                                                                 | 199        |
| 10.3      | Consolidação do planejamento complementar e contrato de execução . .                        | 206        |
| 10.3.1    | Contratos complementares pós-auditoria . . . . .                                            | 208        |
| <b>11</b> | <b>Regras de Segurança do Firestore (Security Rules)</b>                                    | <b>216</b> |
| 11.1      | Regras por Coleção . . . . .                                                                | 216        |
| 11.2      | Contrato de acesso para os fluxos detalhados . . . . .                                      | 218        |
| 11.3      | Moderação: autorização por documento e resposta filtrada . . . . .                          | 219        |
| <b>12</b> | <b>Implementações em Estudo para Versões Futuras</b>                                        | <b>220</b> |
| 12.1      | Por que GASOSO sozinho não basta . . . . .                                                  | 220        |
| 12.2      | Proposta de campos (apenas ilustrativa) . . . . .                                           | 220        |
| 12.3      | Além do enum: medição de gases não é pesagem . . . . .                                      | 221        |
| 12.4      | Horários de Turma . . . . .                                                                 | 221        |
| 12.5      | Lotação física de professores (V2) . . . . .                                                | 221        |
| <b>13</b> | <b>Camada formal incremental: prova de infraestrutura M0</b>                                | <b>223</b> |
| <b>14</b> | <b>Camada formal M1: catálogo de reagentes</b>                                              | <b>225</b> |

## 1 Introdução

O LCQUI é o Laboratório de Ciências Químicas da Universidade Estadual do Norte Fluminense Darcy Ribeiro (UENF), vinculado ao Centro de Ciência e Tecnologia (CCT). Ele funciona como uma unidade de ensino, pesquisa e suporte técnico voltada para a área de química.

## 2 Como vai ser o Sistema

Criar um sistema online que contenha as informações dos bens patrimoniais, reagentes e materiais do almoxarifado do setor de Graduação do LCQUI, bem como criar uma plataforma geral de acesso aos roteiros das aulas experimentais desenvolvidas nos laboratórios desse setor. A intenção é, dessa forma, auxiliar os professores do LCQUI na consulta dos itens existentes, registro de novos itens com maior agilidade, colaborando na identificação de itens inservíveis, em escassez ou que necessitam de reposição ou reparos, auxiliando assim na gestão da chefia do LCQUI. Em relação aos roteiros experimentais, os professores poderão utilizar a plataforma para lançamento ou troca de arquivos e divulgação junto aos alunos.

## 3 Stakeholders

### i Importante

Um mesmo usuário pode possuir múltiplos papéis, observadas as combinações permitidas na Matriz de Papéis e Permissões da seção de Regras de Negócio. O papel *Chefe Geral* é mutuamente exclusivo com os demais papéis para uma mesma conta. O papel *Bolsista* possui como pré-condição existencial e estrutural o papel de *Aluno* ( $Bolsista \implies Aluno$ ).

### 3.1 Chefe Geral

Esse é o maior cargo do sistema: Esse cargo gerencia os outros cargos.

- Poder cadastrar outros usuários e definir os papéis deles no sistema, de acordo com as regras descritas em *Regras de Negócio* e na *Descrição das telas*.
- Pode cadastrar um Almoxarifado.
- Ao cadastrar um Gestor de Almoxarifado, pode colocá-lo responsável por mais de um almoxarifado.

- Ao cadastrar um Gestor de Almoxarifado, pode vinculá-lo imediatamente para gerenciar um ou mais almoxarifados, ou apenas cadastrar o usuário (deixando a vinculação para um momento futuro, resultando em uma dashboard sem almoxarifados ativos inicialmente).
- Atribuir o papel de Bolsista a um usuário que já possua o papel Aluno.

### 3.2 Gestor de Almoxarifado

Pode haver mais de um gestor, ele(s) pode(m):

- Gerar um pdf de etiquetas de frasco
- Adicionar/alterar a quantidade de um reagente no estoque.
- Pesquisar por um reagente no estoque.
- Cadastrar uma nova especificação de reagente, informando os dados obrigatórios definidos pelo tipo de substância e pelo estado físico.
- Marcar um reagente como removido (não exclui do banco, apenas muda o status).
- Alterar o status de um reagente: marcar empréstimo, devolução (através de pesagem), descarte, vazio ou quebrado.
- Autorizar expressamente o uso de frascos vencidos para fins didáticos, demonstrações e pesquisa excepcional conforme Q04.
- Gerar relatório mensal (ou de um período específico) para um almoxarifado ou para todos.
- Editar/adicionar propriedades de um reagente.

### 3.3 Gestor de Bens Patrimoniais

Pode haver mais de um gestor de bem patrimonial, ele(s) pode(m):

- Visualizar as requisições dos professores para adicionar/editar um bem patrimonial.
- Quando a requisição do professor é para colocar um bem como inservível e o gestor aprova, o gestor deve dar baixa do bem patrimonial através do processo da UENF (abrir processo no SEI etc.). Após esse processo, ele marca no sistema que foi dada baixa.
- Adicionar um bem patrimonial e criar resumos de catálogo.

- Editar um bem patrimonial e suas propriedades.
- Gerar e baixar relatório de bens patrimoniais mensal (ou de período específico).
- Visualizar os bens patrimoniais existentes.
- Pesquisar bens patrimoniais por número de patrimônio ou nome.
- Filtrar bens patrimoniais por local (prédio, andar e sala), estado de conservação ou status.
- Imprimir Relatório mensal (ou período personalizado), ou sobre um Bem patrimonial específico podendo filtrar por prédio, andar, sala, Nome do responsável, etc.

### 3.4 Professor

Com certeza haverá mais de um, eles podem:

- Criar uma turma (especificando nome e capacidade máxima de alunos).
- Copiar o código de uma turma dele.
- Criar um post em uma turma (ao criar um post, o professor pode selecionar um roteiro que ele já tenha feito o upload, um roteiro que outro professor compartilhou com ele, ou ainda fazer upload de um roteiro do próprio computador – isso abre um modal para cadastrar o roteiro).
- Adicionar usuários apenas como alunos por e-mail (mandando o link para eles trocarem a senha).
- Pesquisar por bens patrimoniais.
- Abrir uma requisição de adição ou edição de bem patrimonial (analisada por um gestor de bens patrimoniais).
- Adicionar novos Locais (Prédio, Andar e Sala) no sistema de forma autônoma (por exemplo, ao fazer uma requisição, selecionar a opção "Novo Local" e registrá-lo).
- Solicitar/receber empréstimo de reagentes do almoxarifado; o registro operacional da retirada é efetuado pelo Gestor de Almoxarifado.
- Adicionar um roteiro de experimento (formato PDF).
- Vincular um roteiro de experimento a uma turma.

- Compartilhar um roteiro de experimento com outros professores.
- Acessar a aba de roteiros compartilhados (dividida em: “Compartilhados comigo” e “Eu compartilhei”).
- Baixar um roteiro (seja adicionado por ele mesmo ou compartilhado com ele por outro professor).
- Vincular um roteiro de experimento compartilhado com ele por outro professor a uma turma dele.
- Arquivar e desarquivar turmas sob sua responsabilidade.

### 3.5 Aluno

Esses com certeza serão maioria. Eles podem:

- Entrar em uma turma usando o código disponibilizado pelo professor (respeitado o limite de capacidade da turma).
- Visualizar suas turmas e o professor respectivo de cada turma.
- Baixar qualquer roteiro de qualquer turma da qual façam parte.
- Visualizar os colegas que estão na mesma turma.
- Comentar em um post (o comentário é visível tanto para o professor quanto para os colegas).

### 3.6 Bolsista

Seja de graduação ou pós-graduação, este papel identifica um usuário que possui o papel de Aluno e uma autorização adicional para retirada de reagentes (*Bolsista*  $\Rightarrow$  *Aluno*). Ele é mutuamente compatível com Aluno, mas possui restrições próprias para acumulação com papéis de gestor.

- Ele pode ver os reagentes e seus estoques assim como os professores.
- O gestor de reagentes pode registrar retiradas e devoluções para esses usuários.
- Bolsista e Gestor de Almoxarifado são mutuamente exclusivos (SoD). Aluno sem Bolsista pode exercer Gestor de Almoxarifado, conforme DP-C01.

### 3.7 Matriz de Papéis e Permissões

Para eliminar ambiguidades de autorização, a regra de acesso é centralizada na matriz abaixo. As permissões são do papel ativo do usuário e nunca são determinadas por um papel informado pelo cliente.

| Ação                               | Chefe | Gestor Almox. | Gestor Patr. | Professor | Aluno | Bolsista  |
|------------------------------------|-------|---------------|--------------|-----------|-------|-----------|
| Gerenciar usuários e papéis        | ✓     | –             | –            | –         | –     | –         |
| Gerenciar almoxarifados            | ✓     | –             | –            | –         | –     | –         |
| Consultar estoque de reagentes     | ✓     | ✓             | –            | ✓         | ✓     | ✓         |
| Cadastrar/editar reagente e frasco | ✓     | ✓             | –            | –         | –     | –         |
| Registrar retirada/devolução       | ✓     | ✓             | –            | retirante | –     | retirante |
| Gerenciar bens patrimoniais        | ✓     | –             | ✓            | leitura   | –     | –         |
| Abrir requisição de patrimônio     | ✓     | –             | –            | ✓         | –     | –         |
| Responder requisição de patrimônio | ✓     | –             | ✓            | –         | –     | –         |
| Gerenciar turmas próprias          | ✓     | –             | –            | ✓         | –     | –         |
| Ingressar em turma                 | –     | –             | –            | –         | ✓     | ✓         |
| Criar/comentar posts permitidos    | –     | –             | –            | ✓         | ✓     | ✓         |
| Gerenciar roteiros próprios        | ✓     | –             | –            | ✓         | –     | –         |
| Compartilhar roteiros              | ✓     | –             | –            | ✓         | –     | –         |
| Cadastrar/editar matérias          | ✓     | –             | –            | ✓         | –     | –         |
| Gerar etiquetas de frasco          | ✓     | ✓             | –            | –         | –     | –         |

#### i Regra de autorização

A Cloud Function responsável por cada mutação deve resolver os papéis a partir dos registros persistidos e, quando aplicável, validar também o escopo do recurso. Por exemplo, um Gestor de Almoxarifado só atua sobre almoxarifados aos quais está

vinculado. O frontend pode ocultar ou desabilitar ações, mas não constitui autorização.

## 4 Modelagem Entidades SQL - 3FN

### i Como ler as caixas de entidade

Cada caixa a seguir representa uma **tabela do PostgreSQL** na Terceira Forma Normal (3FN). Todos os atributos são **NOT NULL** por padrão, salvo quando explicitamente indicado **NULL**. O nome do campo aparece à esquerda e o **tipo de dado** sugerido para o PostgreSQL aparece à direita, em cinza. Quando aplicável, um selo adicional indica **PK** (chave primária), **FK** (chave estrangeira) ou uma restrição como **NULL** (aceita valor nulo) ou **UNIQUE**.

### 4.1 Usuario

#### Usuario

|                            |                               |
|----------------------------|-------------------------------|
| <i>id</i>                  | SERIAL <b>PK</b>              |
| <i>nome</i>                | VARCHAR(150) NOT NULL         |
| <i>email</i>               | VARCHAR(150) UNIQUE, NOT NULL |
| <i>profile__url__photo</i> | TEXT NULL                     |

### 4.2 Chefe Geral

#### Chefe\_Geral

|                   |                |
|-------------------|----------------|
| <i>id_usuario</i> | INTEGER PK, FK |
|-------------------|----------------|

### 4.3 Gestor de Almoxarifado

#### Gestor\_Almoxarifado

|                   |                |
|-------------------|----------------|
| <i>id_usuario</i> | INTEGER PK, FK |
|-------------------|----------------|

### 4.4 Tabela Gestor de Almoxarifado × Almoxarifado (N para N)

#### Gestor\_Almoxarifado\_x\_Almoxarifado

|                               |                |
|-------------------------------|----------------|
| <i>id_gestor_almoxarifado</i> | INTEGER PK, FK |
| <i>id_almoxarifado</i>        | INTEGER PK, FK |

**Regra de negócio**

A chave primária composta (*id\_gestor\_almojarifado*, *id\_almojarifado*) impede vincular o mesmo gestor duas vezes ao mesmo almojarifado.

**4.5 Gestor de bens patrimoniais****Gestor\_Bens\_Patrimoniais***id\_usuario*

INTEGER PK, FK

**4.6 Bem patrimonial****Bem\_Patrimonial***id*

SERIAL PK

*id\_resumo\_bem\_patrimonial*

INTEGER FK, NOT NULL

*numero\_patrimonio*

VARCHAR(30) UNIQUE, NOT NULL

*estado\_conservacao*

ENUM BOM, REGULAR, RUIM, NOT NULL

*id\_local*

INTEGER FK, NOT NULL

*photo\_url*

TEXT NOT NULL

*documento\_dado\_baixa\_pdf\_url*

TEXT NULL

*nome\_responsavel\_sei*

VARCHAR(150) NOT NULL

*status*

ENUM Ativo, Inservivel, Ja\_dado\_baixa, NOT NULL

*descricao\_complementar*

VARCHAR(200) NULL

*versao*

INTEGER DEFAULT 1, NOT NULL

**i Observação**

*id\_resumo\_bem\_patrimonial*, *numero\_patrimonio*, *estado\_conservacao*, *id\_local*, *photo\_url*, *nome\_responsavel\_sei* e *status* são estritamente **NOT NULL**. Apenas *documento\_dado\_baixa\_pdf\_url* admite NULL (preenchido exclusivamente quando o bem atinge o status *Ja\_dado\_baixa*) e *descricao\_complementar* (opcional para notas individuais daquela unidade física).

**4.7 Resumo bem patrimonial****Resumo\_Bem\_Patrimonial***id*

SERIAL PK

*nome*

VARCHAR(150) NOT NULL

*descricao*

TEXT NOT NULL

**i Regra de Identidade de Bens Patrimoniais**

*Resumo\_Bem\_Patrimonial* representa o modelo catalográfico do equipamento (ex.: “Microscópio Óptico Binocular Coleman”). Vários bens físicos compartilham o mesmo resumo. A alteração do nome do resumo é restrita ao Gestor de Bens e reflete em todos os itens do modelo. Caso um professor solicite alteração de nome para um bem individual, a operação representa reclassificação para outro resumo existente ou criação de um novo modelo, preservando a integridade dos demais equipamentos. Expressamente, resumos de catálogo não possuem vínculo com arquivos fotográficos, pertencendo a imagem unicamente à unidade física cadastrada em *Bem\_Patrimonial*.

## 4.8 Historico bem patrimonial

### Historico\_Bem\_Patrimonial

|                           |           |                                   |
|---------------------------|-----------|-----------------------------------|
| <i>id</i>                 | SERIAL    | PK                                |
| <i>id_bem_patrimonial</i> | INTEGER   | FK, NOT NULL                      |
| <i>timestamp</i>          | TIMESTAMP | NOT NULL                          |
| <i>tipo</i>               | ENUM      | cadastro, edicao, baixa, NOT NULL |
| <i>id_usuario</i>         | INTEGER   | FK, NOT NULL                      |

## 4.9 Alteracao bem patrimonial

### Alteracao\_Bem\_Patrimonial

|                       |             |              |
|-----------------------|-------------|--------------|
| <i>id</i>             | SERIAL      | PK           |
| <i>id_historico</i>   | INTEGER     | FK, NOT NULL |
| <i>campo</i>          | VARCHAR(60) | NOT NULL     |
| <i>valor_anterior</i> | TEXT        | NOT NULL     |
| <i>valor_novo</i>     | TEXT        | NOT NULL     |

## 4.10 Requisicao Professor edicao Bem patrimonial

### Requisicao\_Edicao\_Bem\_Patrimonial

|                                              |              |                                         |
|----------------------------------------------|--------------|-----------------------------------------|
| <b><i>id</i></b>                             | SERIAL       | <b>PK</b>                               |
| <b><i>id_bem_patrimonial</i></b>             | INTEGER      | FK, NOT NULL                            |
| <b><i>status</i></b>                         | ENUM         | pendente, aprovada, rejeitada, NOT NULL |
| <b><i>feita_em</i></b>                       | TIMESTAMP    | NOT NULL                                |
| <b><i>respondida_em</i></b>                  | TIMESTAMP    | NULL                                    |
| <b><i>id_usuario_solicitante</i></b>         | INTEGER      | FK, NOT NULL                            |
| <b><i>id_usuario_respondente</i></b>         | INTEGER      | FK, NULL                                |
| <b><i>novo_nome</i></b>                      | VARCHAR(150) | NULL                                    |
| <b><i>novo_status</i></b>                    | ENUM         | Ativo, Inservivel, NULL                 |
| <b><i>novo_estado_conservacao</i></b>        | ENUM         | BOM, REGULAR, RUIM, NULL                |
| <b><i>novo_id_local</i></b>                  | INTEGER      | FK, NULL                                |
| <b><i>nova_photo_url</i></b>                 | TEXT         | NULL                                    |
| <b><i>motivo</i></b>                         | TEXT         | NOT NULL                                |
| <b><i>justificativa_resposta</i></b>         | TEXT         | NULL                                    |
| <b><i>novo_id_resumo_bem_patrimonial</i></b> | INTEGER      | FK, NULL                                |
| <b><i>versao_bem_origem</i></b>              | INTEGER      | NOT NULL                                |

### Regra de negócio

RF10: *UNIQUE(id\_bem\_patrimonial) WHERE status = 'pendente'* – impede mais de uma requisição de edição pendente simultânea para o mesmo bem patrimonial. Um índice único parcial resolve isso no PostgreSQL; no Firestore, a checagem precisa ser feita na própria *Cloud Function* de criação, dentro de uma transação (ver seção *Tecnologia e Relatórios*).

## 4.11 Requisicao Professor adicao Bem patrimonial

### Requisicao\_Adicao\_Bem\_Patrimonial

|                                       |              |                                         |
|---------------------------------------|--------------|-----------------------------------------|
| <i>id</i>                             | SERIAL       | PK                                      |
| <i>numero_patrimonio_proposto</i>     | VARCHAR(30)  | NOT NULL                                |
| <i>status</i>                         | ENUM         | pendente, aprovada, rejeitada, NOT NULL |
| <i>feita_em</i>                       | TIMESTAMP    | NOT NULL                                |
| <i>id_bem_patrimonial_se_aprovado</i> | INTEGER      | FK, NULL                                |
| <i>respondida_em</i>                  | TIMESTAMP    | NULL                                    |
| <i>id_usuario_solicitante</i>         | INTEGER      | FK, NOT NULL                            |
| <i>id_usuario_respondente</i>         | INTEGER      | FK, NULL                                |
| <i>estado_conservacao_proposto</i>    | ENUM         | BOM, REGULAR, RUIM, NOT NULL            |
| <i>photo_url_proposta</i>             | TEXT         | NOT NULL                                |
| <i>nome_responsavel_proposto</i>      | VARCHAR(150) | NOT NULL                                |
| <i>id_local</i>                       | INTEGER      | FK, NOT NULL                            |
| <i>id_resumo_bem_patrimonial</i>      | INTEGER      | FK, NULL                                |
| <i>nome_resumo_proposto</i>           | VARCHAR(150) | NULL                                    |
| <i>descricao_resumo_proposta</i>      | TEXT         | NULL                                    |
| <i>motivo</i>                         | TEXT         | NOT NULL                                |
| <i>justificativa_resposta</i>         | TEXT         | NULL                                    |

### i Observação

Se o local desejado não existir no banco, o professor deve primeiro criar o *Local* no sistema, para então vincular a respectiva FK (*id\_local*) nesta requisição.

### Regra de negócio

RF10b: *UNIQUE(numero\_patrimonio\_proposto) WHERE status = 'pendente'* – impede mais de uma requisição de adição pendente simultânea para o mesmo número de patrimônio. No PostgreSQL, isso é implementado como índice único parcial; no Firestore, a verificação é reimplementada na Cloud Function de criação da requisição, dentro de uma transação.

## 4.12 Lock de Requisição de Patrimônio

### Locks\_Requisicao\_Patrimonio

|                                  |              |          |
|----------------------------------|--------------|----------|
| <i>id (docId determinístico)</i> | VARCHAR(100) | PK       |
| <i>criado_em</i>                 | TIMESTAMP    | NOT NULL |

### Lock Determinístico para Unicidade de Requisição Pendente

O Firestore não adquire trava pessimista sobre **ausência** de documento em uma query transacional. Por isso, a garantia de que não existam duas requisições pendentes para o mesmo bem ou plaqueta **não pode** ser implementada pela consulta `WHERE status = 'pendente'` dentro de uma transação – dois clientes simultâneos lerão conjunto vazio, ambos passarão na validação e ambos gravarão, violando RF10/RF10b.

A solução é o padrão *deterministic lock*: a Cloud Function tenta criar um documento cujo **docId** é **derivado deterministicamente** do recurso disputado. O Firestore garante que apenas uma gravação com o mesmo docId vence quando executadas concorrentemente dentro de uma transação.

- Para requisição de **edição**: `docId = bem_edicao_{id_bem_patrimonial}`
- Para requisição de **adição**: `docId = bem_adicao_{numero_patrimonio_proposto}`

O lock é removido na transação de aprovação/rejeição ordinária. O lock `Locks_Requisicao_Patrimonio/bem_edicao_{id}` DEVE ser removido pela transação também em caso de rejeição por divergência de versão do bem patrimonial, solucionando deadlocks permanentes. Lock e requisição são criados atomicamente; não há commit parcial dessa transação. Limpeza verifica o vínculo e jamais remove um lock apenas porque a requisição pendente tem mais de 24 horas.

## 4.13 Impressão de Etiquetas de Frasco

### Impressao\_Etiqueta\_Frasco

|                       |           |              |
|-----------------------|-----------|--------------|
| <i>id</i>             | SERIAL    | PK           |
| <i>gerado_em</i>      | TIMESTAMP | NOT NULL     |
| <i>gerado_por</i>     | INTEGER   | FK, NOT NULL |
| <i>codigo_inicial</i> | INTEGER   | NOT NULL     |
| <i>codigo_final</i>   | INTEGER   | NOT NULL     |

#### i Observação

Esta entidade audita de forma isolada as sessões de geração em lote de etiquetas virgens contínuas. *codigo\_inicial* e *codigo\_final* referem-se aos números sequenciais gerados (ex.: LCQUI-1 a LCQUI-50). Não há vínculo de chave estrangeira com a tabela *Frasco\_Reagente*, dado que a impressão em branco é puramente utilitária e não cria reservas de ID. A transação e consolidação final do código só ocorrem no banco no momento do cadastro físico.

## 4.14 Almoxarifado

### Almoxarifado

|                  |              |                        |
|------------------|--------------|------------------------|
| <i>id</i>        | SERIAL       | PK                     |
| <i>id_local</i>  | INTEGER      | FK, NOT NULL           |
| <i>nome</i>      | VARCHAR(100) | NOT NULL               |
| <i>descricao</i> | VARCHAR(500) | NOT NULL               |
| <i>ativo</i>     | BOOLEAN      | DEFAULT TRUE, NOT NULL |

#### i Observação

O campo *ativo* indica se o almoxarifado está operacional. Quando *ativo* = **false**, o almoxarifado está desativado e as regras de bloqueio da Seção 7.2.22 se aplicam. A desativação é reversível e não exclui registros históricos.

## 4.15 Substância Química

### Substancia\_Quimica

|                        |              |                        |
|------------------------|--------------|------------------------|
| <i>id</i>              | SERIAL       | PK                     |
| <i>nome</i>            | VARCHAR(150) | NOT NULL               |
| <i>cas_number</i>      | VARCHAR(15)  | UNIQUE, NULL           |
| <i>formula_quimica</i> | VARCHAR(50)  | NULL                   |
| <i>ativo</i>           | BOOLEAN      | DEFAULT TRUE, NOT NULL |

#### i Observação

A entidade *Substancia\_Quimica* representa uma substância quimicamente identificável, independentemente de fabricante, concentração, pureza, lote ou frasco.

O campo *cas\_number* identifica a substância e permanece **UNIQUE** (permitindo múltiplos NULL, já que algumas substâncias podem não ter CAS cadastrado). Uma mesma substância pode participar da composição de diversas especificações de reagentes diferentes.

## 4.16 Resumo do Reagente / Solvente

### Resumo\_Reagente

|                                 |              |                                                   |
|---------------------------------|--------------|---------------------------------------------------|
| <i>id</i>                       | SERIAL       | PK                                                |
| <i>nome</i>                     | VARCHAR(150) | NOT NULL                                          |
| <i>tipo_substancia</i>          | ENUM         | PURA, MISTURA, NOT NULL                           |
| <i>natureza_quimica</i>         | ENUM         | ORGANICO, INORGANICO, ELEMENTO, HIBRIDO, NOT NULL |
| <i>requer_pesagem_frequente</i> | BOOLEAN      | DEFAULT FALSE, NOT NULL                           |
| <i>frequencia_pesagem_dias</i>  | INTEGER      | NULL                                              |
| <i>estado_fisico</i>            | ENUM         | SOLIDO, LIQUIDO, NOT NULL                         |
| <i>eh_higroscopico</i>          | BOOLEAN      | DEFAULT FALSE, NOT NULL                           |

### i Regra Fechada de Identidade do Resumo de Reagente

O *Resumo\_Reagente* representa o item químico agrupador apresentado no catálogo do sistema, permitindo agrupar diferentes especificações comerciais ou formulações do mesmo reagente.

Dois reagentes pertencem ao **mesmo** *Resumo\_Reagente* quando compartilham a mesma identidade química nominal de catálogo (ex.: “Ácido Clorídrico”, “Etanol”, “Hidróxido de Sódio”).

- Variações de pureza (P.A., Técnico, Grau HPLC)  $\Rightarrow$  **mesmo resumo**, especificações distintas.
- Variações de concentração (HCl 37%, HCl 10%, Etanol 99.5%, Etanol 70%)  $\Rightarrow$  **mesmo resumo**, especificações distintas.
- Fabricante, código de catálogo e densidade  $\Rightarrow$  atributos de especificação, **mesmo resumo**.

Devem ser cadastrados como **resumos diferentes**:

- Espécies químicas distintas com fórmulas ou números CAS bases diferentes (ex.: Sulfato de Cobre II vs Sulfato de Ferro II);
- Soluções formuladas consagradas com finalidade analítica própria (ex.: “Reagente de Benedict”, “Solução de Fehling A”, “Solução de Lugol”)  $\Rightarrow$  cada uma constitui um *Resumo\_Reagente* próprio do tipo MISTURA.

Os campos *medida\_total\_usado* e *unidade\_de\_medida* foram removidos desta entidade: ambos são métricas agregadas derivadas de *Frasco\_Reagente* e *Emprestimo\_Reagente*, e devem existir apenas na view materializada descrita na seção *Métricas Agregadas*, nunca como coluna editável de uma tabela cadastral.

A unidade operacional é derivada de *Resumo\_Reagente.estado\_fisico*: SOLIDO usa g e LIQUIDO usa mL. Não persistir cópia redundante da unidade no modelo 3FN.

DP-A01: *estado\_fisico* e *eh\_higroscopico* pertencem ao *Resumo\_Reagente*. Estados físicos distintos nas condições de catálogo (25 graus Celsius, 1 atm) exigem resumos distintos; solução aquosa e sólido puro não são o mesmo resumo. Densidade e concentração permanecem na especificação comercial.

*natureza\_quimica* classifica o item do catálogo como um todo (não uma *Substancia\_Quimica* individual): **ELEMENTO** para substâncias simples (não se classificam como orgânicas nem inorgânicas), **ORGANICO/INORGANICO** para os demais casos, e **HIBRIDO** para fluidos complexos que não se decompõem em uma lista simples de substâncias em *Composicao\_Reagente* (ex.: Soro Fetal Bovino). Como **HIBRIDO** não é derivável a partir das substâncias componentes, o campo é curado no nível do resumo, e não em *Substancia\_Quimica*, evitando duas fontes de verdade diferentes para itens PUROS e

para MISTURA/HÍBRIDO. O rótulo exibido na interface (ex.: “Híbrido”, “Complexo” ou “Biológico”) é independente do valor salvo no banco e pode ser alterado livremente. O campo *letra\_inicial*, presente em versões anteriores deste documento, foi removido do modelo 3FN: por ser derivável do próprio *nome*, sua persistência aqui violaria a forma normal. Ele reaparece como uma denormalização proposital exclusiva da implementação em Firestore – ver seção *Notas de Mapeamento para Firestore*.

## 4.17 Composição do Reagente

### Composicao\_Reagente

|                                    |               |                                        |
|------------------------------------|---------------|----------------------------------------|
| <i>id</i>                          | SERIAL        | PK                                     |
| <i>id_especificacao_reagente</i>   | INTEGER       | FK, NOT NULL                           |
| <i>id_substancia_quimica</i>       | INTEGER       | FK, NOT NULL                           |
| <i>valor_min</i>                   | NUMERIC(10,5) | NULL                                   |
| <i>valor_max</i>                   | NUMERIC(10,5) | NULL                                   |
| <i>tipo_concentracao</i>           | ENUM          | M_M, V_V, M_V, MOL_L, MOL_KG, PPM, PPB |
| <i>notacao_original_fabricante</i> | VARCHAR(50)   | NULL                                   |

### i Observação

A entidade *Composicao\_Reagente* representa a relação entre uma especificação de reagente cadastrada e as substâncias químicas que a compõem.

A relação é do tipo N:N:

$$Especificacao\_Reagente \leftrightarrow Substancia\_Quimica$$

A composição foi movida do nível de *Resumo\_Reagente* para o nível de *Especificacao\_Reagente*, pois a concentração (37%, 10%, 1 mol/L etc.) é uma característica que varia entre especificações de um mesmo resumo, não uma propriedade do resumo em si. Vincular a composição ao resumo impediria registrar corretamente, por exemplo, HCl 37% e HCl 10% sob o mesmo *Resumo\_Reagente* com valores de composição diferentes. Uma substância pura normalmente terá apenas uma linha associada, enquanto uma mistura poderá possuir diversas substâncias.

Q03/DP-A03: concentração pontual usa *valor\_min* = *valor\_max* ou *valor\_max* NULL; faixa usa limites numéricos ordenados. CHECK (*valor\_max* IS NULL OR *valor\_min* ≤ *valor\_max*). O servidor rejeita faixa com máximo informado e mínimo ausente; preserva a notação original do fabricante separadamente, sem usá-la como autoridade numérica.

### Regra de negócio

*UNIQUE(id\_especificacao\_reagente, id\_substancia\_quimica)* – impede duas linhas de composição para a mesma substância dentro da mesma especificação.

## 4.18 Especificação do Reagente / Solvente

### Especificacao\_Reagente

|                                                        |              |                         |
|--------------------------------------------------------|--------------|-------------------------|
| <i>id</i>                                              | SERIAL       | PK                      |
| <i>id_resumo_reagente</i>                              | INTEGER      | FK, NOT NULL            |
| <i>descricao</i>                                       | VARCHAR(150) | NOT NULL                |
| <i>fabricante</i>                                      | VARCHAR(150) | NULL                    |
| <i>codigo_produto_fabricante</i>                       | VARCHAR(100) | NULL                    |
| <i>grau_pureza</i>                                     | VARCHAR(30)  | NULL                    |
| <i>densidade</i>                                       | NUMERIC(8,4) | NULL                    |
| <i>classe_inflamabilidade</i>                          |              | ENUM                    |
| NAO_INFLAMAVEL, CLASSE_1, CLASSE_2, CLASSE_3, NOT NULL |              |                         |
| <i>eh_controlado_pf</i>                                | BOOLEAN      | DEFAULT FALSE, NOT NULL |
| <i>eh_controlado_eb</i>                                | BOOLEAN      | DEFAULT FALSE, NOT NULL |
| <i>link_fds_fispq</i>                                  | VARCHAR(255) | NULL                    |

### i Observação

A especificação referencia o resumo, fonte do estado físico; a unidade é derivada desse estado. Frasco e empréstimo chegam ao resumo pela cadeia de FKs. Snapshots de consulta e de cálculo são documentados separadamente no Firestore.

*GASOSO* foi removido de *estado\_fisico* nesta versão: nenhum reagente gasoso está listado para o almoxarifado atual, e reagentes gasosos precisam de campos e método de medição próprios (pressão, não peso em balança) em vez de reaproveitar os campos já pensados para sólido/líquido. O estudo detalhado para reintrodução está na seção *Implementações em Estudo para Versões Futuras*.

## 4.19 Estoque Mínimo por Almoxarifado

### Estoque\_Minimo\_Almoxarifado

|                                  |         |                        |
|----------------------------------|---------|------------------------|
| <i>id_especificacao_reagente</i> | INTEGER | FK, PK, NOT NULL       |
| <i>id_almoxarifado</i>           | INTEGER | FK, PK, NOT NULL       |
| <i>qtd_limiar_escassez</i>       | INTEGER | NOT NULL               |
| <i>ativo</i>                     | BOOLEAN | DEFAULT TRUE, NOT NULL |
| <i>notificacao_ativa</i>         | BOOLEAN | DEFAULT TRUE, NOT NULL |

**i Observação**

A entidade *Estoque\_Minimo\_Almojarifado* estabelece o limiar de escassez (em quantidade de frascos) de uma *Especificacao\_Reagente* para um determinado *Almojarifado*. O par Especificação × Almojarifado é a identidade unívoca da configuração. No Fires-tore, esta entidade é implementada como a subcoleção *Almojarifado/{id}/Estoques\_Configurados/{idResumo\_idEspec}*. O campo *ativo* indica que o par continua confi-gurado ou intencionalmente estocado. O campo *notificacao\_ativa* indica que a emissão automática do alerta está habilitada (útil para silenciar alertas sem excluir a intenção de estocar a especificação).

**4.20 Lote****Lote**

|                                  |                             |
|----------------------------------|-----------------------------|
| <i>id</i>                        | SERIAL <b>PK</b>            |
| <i>id_especificacao_reagente</i> | INTEGER FK, NOT NULL        |
| <i>data_aquisicao</i>            | TIMESTAMP NOT NULL          |
| <i>qtd_frascos_comprados</i>     | INTEGER DEFAULT 0, NOT NULL |
| <i>nome_fornecedor</i>           | VARCHAR(80) NOT NULL        |
| <i>numero_lote</i>               | VARCHAR(100) NOT NULL       |
| <i>nota_fiscal</i>               | VARCHAR(100) NOT NULL       |
| <i>data_fabricacao</i>           | DATE NOT NULL               |
| <i>data_validade</i>             | DATE NOT NULL               |

**Regra de negócio**

*UNIQUE(id\_especificacao\_reagente, nome\_fornecedor, numero\_lote).*

**i Observação**

*qtd\_frascos\_comprados* é dado de entrada (o que consta na nota fiscal) e permanece nesta tabela. Já *qtd\_frascos\_cadastrados* – quantos frascos deste lote já foram individualmente registrados em *Frasco\_Reagente* – é uma contagem agregada (*COUNT(\*) FROM Frasco\_Reagente WHERE id\_lote = X*) e, por isso, **não** é coluna desta tabela cadastral: ela vive exclusivamente em *Lote\_Materializado* (ver seção *Materialized (Views)*), mantida por mecanismo transacional a cada frasco cadastrado ou removido.

## 4.21 Frasco de Reagente / Solvente

### Frasco\_Reagente

|                                        |               |                                                                    |
|----------------------------------------|---------------|--------------------------------------------------------------------|
| <i>id</i>                              | SERIAL        | PK                                                                 |
| <i>id_almojarifado</i>                 | INTEGER       | FK, NOT NULL                                                       |
| <i>id_lote</i>                         | INTEGER       | FK, NULL                                                           |
| <i>id_especificacao_reagente</i>       | INTEGER       | FK, NULL                                                           |
| <i>detalhe_local_armazenamento</i>     | TEXT          | NULL                                                               |
| <i>codigo_frasco</i>                   | TEXT          | UNIQUE, NOT NULL                                                   |
| <i>data_ultima_pesagem</i>             | TIMESTAMP     | NULL                                                               |
| <i>conteudo_nominal</i>                | NUMERIC(10,3) | NULL                                                               |
| <i>peso_no_cadastrado</i>              | NUMERIC(10,3) | NOT NULL                                                           |
| <i>peso_atual</i>                      | NUMERIC(10,3) | NOT NULL                                                           |
| <i>peso_frasco_vazio</i>               | NUMERIC(10,3) | NULL                                                               |
| <i>medida_usada</i>                    | NUMERIC(10,3) | DEFAULT 0, NOT NULL                                                |
| <i>data_abertura</i>                   | DATE          | NULL                                                               |
| <i>validade_fechado</i>                | DATE          | NULL                                                               |
| <i>validade_efetiva</i>                | DATE          | NULL                                                               |
| <i>validade_desconhecida</i>           | BOOLEAN       | DEFAULT FALSE, NOT NULL                                            |
| <i>validade_apos_aberto_dias</i>       | INTEGER       | NULL                                                               |
| <i>estado_fisico_frasco</i>            | ENUM          | FECHADO, ABERTO, VAZIO, QUEBRADO, DESCARTADO, EXTRAVIADO, NOT NULL |
| <i>disponibilidade</i>                 | ENUM          | DISPONIVEL, EMPRESTADO, INDISPONIVEL, NOT NULL                     |
| <i>vencido</i>                         | BOOLEAN       | DEFAULT FALSE, NOT NULL                                            |
| <i>em_quarentena</i>                   | BOOLEAN       | DEFAULT FALSE, NOT NULL                                            |
| <i>uso_vencido_autorizado</i>          | BOOLEAN       | DEFAULT FALSE, NOT NULL                                            |
| <i>detalhe_status</i>                  | TEXT          | NULL                                                               |
| <i>cadastrado_em</i>                   | TIMESTAMP     | NOT NULL                                                           |
| <i>cadastrado_por</i>                  | INTEGER       | FK, NOT NULL                                                       |
| <i>abertura_historica_desconhecida</i> | BOOLEAN       | DEFAULT FALSE, NOT NULL                                            |

### Constraint de Identidade Química do Frasco

*CHECK (id\_lote IS NOT NULL OR id\_especificacao\_reagente IS NOT NULL)* – pelo menos um dos dois deve existir. Quando *id\_lote* é informado, *id\_especificacao\_reagente* fica NULL (a especificação é derivável via *Lote.id\_especificacao\_reagente*). Quando *id\_lote* é NULL, *id\_especificacao\_reagente* é obrigatório para manter o caminho até a densidade, unidade de medida e resumo do reagente.

**i Observação**

*conteudo\_nominal* (antigo *capacidade\_nominal*) representa o conteúdo declarado no cadastro: para líquidos, é o volume nominal em mL; para sólidos, é a massa nominal em g. Se NULL, o valor original do rótulo é desconhecido ou ilegível (não usar zero para representar desconhecido).

*peso\_no\_cadastrado* é o peso total (frasco + conteúdo) registrado no momento do cadastro. Este campo é **imutável** após o cadastro – nunca é sobrescrito. *peso\_atual* é o peso total corrente, inicializado com o valor de *peso\_no\_cadastrado* e atualizado a cada devolução de empréstimo ou pesagem de rotina.

Os campos *peso\_no\_cadastrado*, *peso\_atual*, *peso\_frasco\_vazio* e *medida\_usada* são sempre leituras de balança em gramas, independentemente de o conteúdo ser sólido ou líquido, já que a operação real do laboratório consiste em pesar o frasco; o volume, quando aplicável, é sempre derivado por conversão via densidade.

O campo *unidade\_de\_medida*, antes duplicado nesta entidade, foi removido: a unidade do produto (ml ou g) é obtida via *Frasco\_Reagente* → *Lote* → *Especificacao\_Reagente* (ou diretamente via *id\_especificacao\_reagente* quando o lote é NULL), chegando ao *Resumo\_Reagente*, cujo estado determina a unidade.

O antigo campo único *status* (com valores combinados como **VENCIDO\_DISPONIVEL**, **VENCIDO\_EMPRESTADO**) foi decomposto em dimensões ortogonais – *estado\_fisico\_frasco*, *disponibilidade*, *vencido* e *em\_quarentena* – evitando o crescimento combinatorial do enum e simplificando consultas e regras de notificação.

O campo *id\_usuario\_em\_uso* foi removido: quem está com o frasco no momento é obtido consultando *Emprestimo\_Reagente* pelo empréstimo ativo (*status* **EM\_USO** ou **ATRASADO**) daquele *id\_frasco\_reagente*.

*validade\_fechado* é mantido de forma independente de *Lote.data\_validade* porque *id\_lote* é opcional: um frasco pode ser cadastrado sem lote associado, e mesmo quando o lote existe, o formulário apenas pré-preenche este campo a partir dele – o valor permanece editável e não é uma cópia somente-leitura.

O campo *uso\_vencido\_autorizado* fecha a distinção de negócio entre um frasco vencido que permanece apto ao uso didático e um frasco vencido que aguarda descarte.

**4.22 Histórico do Frasco de Reagente / Solvente****Historico\_Frasco\_Reagente***id*SERIAL **PK***id\_frasco\_reagente*

INTEGER FK, NOT NULL

*id\_almojarifado*

INTEGER FK, NOT NULL

*id\_gestor*

INTEGER FK, NOT NULL

Para ações automáticas do servidor (jobs de vencimento, atraso), no 3FN usa-se o `id_usuario = 0` (SISTEMA LCQUI); no Firestore, usa-se a string sentinela `__SISTEMA__`.

**tipo**

ENUM

CADASTRO, SAIU, ENTROU, FICOU\_VAZIO, QUEBROU, FOI\_DESCARTADO, VENCEU, INVENTARIO\_ROTINA, EVAPORACAO, AJUSTE, CONCLUSAO, ENTROU\_EM\_QUARENTENA, LIBERADO\_QUARENTENA, PENDENTE\_DE\_DESCARTE, USO\_VENCIDO\_AUTORIZADO, EXTRAVIO, REENCONTRO

**NOT NULL campo\_ajustado**

VARCHAR(30) NULL

**id\_emprestimo\_reagente**

INTEGER FK, NULL

**peso\_anterior**

NUMERIC(10,3) NULL

**peso\_novo**

NUMERIC(10,3) NULL

**medida\_ajustada**

NUMERIC(10,3) NULL

**unidade\_medida\_ajustada**

ENUM ml,g, NULL

**motivo**

TEXT NULL

**timestamp**

TIMESTAMP NOT NULL

**i Observação**

`peso_anterior` e `peso_novo` são sempre leituras de balança em gramas. Já `unidade_medida_ajustada` qualifica exclusivamente `medida_ajustada`, indicando se o ajuste concluído é reportado em massa ou volume. Podem ser NULL caso o evento histórico não seja de natureza quantitativa. O campo `motivo` é genérico e aceita NULL, porém é obrigatório para eventos que exigem justificativa (ex.: EXTRAVIO, REENCONTRO e ajustes manuais). No modelo relacional 3FN, reserva-se a linha canônica `Usuario.id = 0` com nome SISTEMA LCQUI para satisfazer a *foreign key* não nula em rotinas automáticas de servidor; no Firestore, o documento recebe a string sentinela `__SISTEMA__`.

**4.23 Empréstimo de Reagente / Solvente****Emprestimo\_Reagente****id**

SERIAL PK

**id\_frasco\_reagente**

INTEGER FK, NOT NULL

**id\_usuario\_retirou**

INTEGER FK, NOT NULL

**id\_gestor\_retirada**

INTEGER FK, NOT NULL

**status**

ENUM

EM\_USO, ATRASADO, DEVOLVIDO, DEVOLVIDO\_COM\_ATRASO, ENCERRADO\_EXTRAORDINARIO, NOT NULL

**data\_retirada**

TIMESTAMP NOT NULL

**data\_devolucao\_prevista**

DATE NOT NULL

|                                                                          |               |                         |
|--------------------------------------------------------------------------|---------------|-------------------------|
| <i>data_devolucao_efetuada</i>                                           | TIMESTAMP     | NULL                    |
| <i>id_local_usado</i>                                                    | INTEGER       | FK, NOT NULL            |
| <i>id_usuario_devolveu</i>                                               | INTEGER       | FK, NULL                |
| <i>id_gestor_devolucao</i>                                               | INTEGER       | FK, NULL                |
| <i>peso_saida</i>                                                        | NUMERIC(10,3) | NOT NULL                |
| <i>peso_retorno</i>                                                      | NUMERIC(10,3) | NULL                    |
| <i>medida_utilizada</i>                                                  | NUMERIC(10,3) | NULL                    |
| <i>unidade_medida_utilizada</i>                                          | ENUM          | ml,g, NOT NULL          |
| <i>peso_perda_evaporacao</i>                                             | NUMERIC(10,3) | DEFAULT 0, NOT NULL     |
| <i>uso_vencido_aceito</i>                                                | BOOLEAN       | DEFAULT FALSE, NOT NULL |
| <i>finalidade_uso</i>                                                    | ENUM          | NOT NULL                |
| AULA_PRATICA, DEMONSTRACAO, PESQUISA_TCC_POS, ESTUDO_DEGRADACAO_RESIDUOS |               |                         |
| <i>auto_atendimento</i>                                                  | BOOLEAN       | DEFAULT FALSE, NOT NULL |
| <i>justificativa_metodologica</i>                                        | TEXT          | NULL                    |
| <i>tcr_versao</i>                                                        | VARCHAR(100)  | NULL                    |
| <i>tcr_aceito_em</i>                                                     | TIMESTAMP     | NULL                    |
| <i>tcr_aceito_por</i>                                                    | INTEGER       | FK, NULL                |
| <i>tcr_auditoria_id</i>                                                  | INTEGER       | FK, NULL                |
| <i>tipo_encerramento_excepcional</i>                                     | ENUM          |                         |
| QUEBRA_ACIDENTAL, EXTRAVIO_SINISTRO, NULL                                |               |                         |
| <i>motivo_encerramento_excepcional</i>                                   | TEXT          | NULL                    |
| <i>id_gestor_encerramento</i>                                            | INTEGER       | FK, NULL                |
| <i>massa_perda_estimada_g</i>                                            | NUMERIC(10,3) | NULL                    |

### i Observação

**Semântica dos atores da retirada:** *id\_usuario\_retirou* identifica o **portador físico** – o Professor ou Bolsista que receberá e usará o reagente. *id\_gestor\_retirada* identifica o **operador do almoxarifado** – o Gestor de Almoxarifado (ou Chefe Geral) que registrou a operação no sistema. No autoatendimento, os dois UUIDs coincidem e a flag *auto\_atendimento* fica **true**. Esta separação é essencial para rastreabilidade de auditoria: o relatório de atividade do gestor parte de *id\_gestor\_retirada*, enquanto o painel “Meus Reagentes” do professor parte de *id\_usuario\_retirou*.

**Semântica dos atores da devolução:** *id\_usuario\_devolveu* identifica o portador físico que entrega o frasco ao balcão, enquanto *id\_gestor\_devolucao* identifica o Gestor de Almoxarifado que operou e registrou a devolução na balança.

*peso\_saida* e *peso\_retorno* são sempre leituras de balança em gramas. *unidade\_medida\_utilizada* qualifica exclusivamente *medida\_utilizada* – o consumo calculado, reportado em massa (g) para sólidos ou em volume (ml) para líquidos, via  $V = \Delta m / \rho$ . O campo *data\_devolucao\_prevista* em nível NoSQL físico armazena uma string civil no padrão YYYY-MM-DD (fuso America/Sao\_Paulo), e o vencimento legal encerra-se estritamente às 23:59:59.999 do referido dia civil.

## 4.24 Métricas Agregadas do Reagente

### i Observação

As seguintes informações devem ser tratadas preferencialmente como **views materializadas, consultas agregadas ou campos derivados mantidos por mecanismo transacional**:

- *medida\_total\_usado* e *unidade\_de\_medida* (agregados por *Resumo\_Reagente*);
- *qtd\_frascos\_fechados\_disponiveis*;
- *qtd\_frascos\_abertos\_disponiveis*;
- *qtd\_frascos\_em\_uso*;
- *qtd\_frascos\_vazios*;
- *qtd\_frascos\_descartados*;
- *qtd\_frascos\_vencidos*;
- *qtd\_frascos\_quebrados*.

## 4.25 Turma

### Turma

|                     |              |                                     |
|---------------------|--------------|-------------------------------------|
| <i>id</i>           | SERIAL       | PK                                  |
| <i>id_materia</i>   | INTEGER      | FK, NOT NULL                        |
| <i>id_professor</i> | INTEGER      | FK, NOT NULL                        |
| <i>status</i>       | ENUM         | Ativo, Arquivada, NOT NULL          |
| <i>nome_turma</i>   | VARCHAR(100) | NOT NULL                            |
| <i>ano</i>          | INTEGER      | NOT NULL                            |
| <i>semestre</i>     | SMALLINT     | CHECK (semestre IN (1,2)), NOT NULL |
| <i>capacidade</i>   | INTEGER      | NOT NULL                            |
| <i>codigo_turma</i> | VARCHAR(20)  | UNIQUE, NOT NULL                    |
| <i>data_criacao</i> | TIMESTAMP    | NOT NULL                            |

**i Observação**

Os campos *ano* e *semestre* representam o período letivo da turma (ex.: 2026/1). Ambos são exibidos na listagem de turmas arquivadas do professor – eliminando a necessidade de parsing do *nome\_turma* ou inferência a partir de *data\_criacao*. O modal de criação de turma solicita esses dois campos além dos demais.

**RN-TUR-01: Controle de Capacidade da Turma**

O ingresso de aluno por código de turma exige obrigatoriamente que a quantidade de alunos matriculados ativos seja estritamente menor que a capacidade da turma:

COUNT(Aluno\_x\_Turma WHERE id\_turma = X) < Turma.capacidade

Caso a capacidade seja atingida, o sistema rejeita o ingresso por código. Apenas o professor responsável pode ultrapassar a capacidade mediante envio de convite nominal por e-mail, registrando justificativa em auditoria.

**4.26 Historico de Alunos na Turma****Historico\_Alunos\_Turma**

|                                                   |           |                                          |
|---------------------------------------------------|-----------|------------------------------------------|
| <i>id</i>                                         | SERIAL    | PK                                       |
| <i>id_turma</i>                                   | INTEGER   | FK, NOT NULL                             |
| <i>tipo</i>                                       | ENUM      | inclusao_aluno, exclusao_aluno, NOT NULL |
| <i>id_aluno</i>                                   | INTEGER   | FK, NOT NULL                             |
| <i>modo_ingresso</i>                              | ENUM      | CODIGO, CONVITE, NULL                    |
| <i>justificativa</i>                              | TEXT      | NULL                                     |
| <i>timestamp</i>                                  | TIMESTAMP | NOT NULL                                 |
| <i>removido_por</i>                               | INTEGER   | FK, NULL                                 |
| Obrigatório quando <i>tipo</i> = 'exclusao_aluno' |           |                                          |

**4.27 Historico de Posts na Turma****Historico\_Posts\_Turma**

|                        |              |              |
|------------------------|--------------|--------------|
| <i>id</i>              | SERIAL       | PK           |
| <i>id_post</i>         | INTEGER      | FK, NOT NULL |
| <i>editado_por</i>     | INTEGER      | FK, NOT NULL |
| <i>novo_titulo</i>     | VARCHAR(150) | NULL         |
| <i>novo_id_roteiro</i> | INTEGER      | FK, NULL     |
| <i>nova_descricao</i>  | TEXT         | NULL         |
| <i>antigo_titulo</i>   | VARCHAR(150) | NULL         |

*antigo\_id\_roteiro*

INTEGER FK, NULL

*antiga\_descricao*

TEXT NULL

*timestamp*

TIMESTAMP NOT NULL

**i Observação**

O campo *editado\_por* registra o UID do usuário que realizou a edição. Normalmente é o Professor dono da turma, mas o Chefe Geral também pode editar ou remover posts em caráter excepcional de moderação institucional (Q13). O *id\_turma* não é armazenado nesta entidade 3FN porque pode ser obtido por *Historico\_Posts\_Turma.id\_post* → *Post.id\_turma*; se a consulta direta por turma for necessária no Firestore, esse vínculo poderá ser denormalizado na coleção de histórico.

**4.28 Historico de Comentario em um Post****Historico\_Comentario***id*

SERIAL PK

*id\_comentario*

INTEGER FK, NOT NULL

*novo\_texto*

TEXT NOT NULL

*texto\_antigo*

TEXT NOT NULL

*editado\_em*

TIMESTAMP NOT NULL

*editado\_por*

INTEGER FK, NOT NULL

**4.29 Roteiro de Experimento****Roteiro\_Experimento***id*

SERIAL PK

*id\_professor\_upload*

INTEGER FK, NOT NULL

*file\_url*

TEXT NOT NULL

*nome*

VARCHAR(150) NOT NULL

*descricao*

TEXT NOT NULL

**4.30 Roteiro Professor Compartilhado****Roteiro\_Professor\_Compartilhado***id\_roteiro\_experimento*

INTEGER PK, FK

*id\_professor*

INTEGER PK, FK

### 4.31 Professor

#### Professor

|                    |              |                               |
|--------------------|--------------|-------------------------------|
| <i>id_usuario</i>  | INTEGER      | PK, FK                        |
| <i>centro</i>      | ENUM         | CCT, CCTA, CBB, CCH, NOT NULL |
| <i>laboratorio</i> | VARCHAR(100) | NOT NULL                      |

### 4.32 Materia

#### Materia

|                       |              |                  |
|-----------------------|--------------|------------------|
| <i>id</i>             | SERIAL       | PK               |
| <i>nome</i>           | VARCHAR(100) | NOT NULL         |
| <i>codigo_materia</i> | VARCHAR(10)  | UNIQUE, NOT NULL |

### 4.33 Tabela Professor × Materia (N para N)

#### Professor\_x\_Materia

|                     |         |        |
|---------------------|---------|--------|
| <i>id_professor</i> | INTEGER | PK, FK |
| <i>id_materia</i>   | INTEGER | PK, FK |

#### Regra de negócio

A chave primária composta (*id\_professor*, *id\_materia*) impede vincular o mesmo professor duas vezes à mesma matéria.

### 4.34 Aluno

#### Aluno

|                         |             |                  |
|-------------------------|-------------|------------------|
| <i>id_usuario</i>       | INTEGER     | PK, FK           |
| <i>numero_matricula</i> | VARCHAR(20) | UNIQUE, NOT NULL |

### 4.35 Bolsista

#### Bolsista

|                   |         |        |
|-------------------|---------|--------|
| <i>id_usuario</i> | INTEGER | PK, FK |
|-------------------|---------|--------|

## 4.36 Convite para Aluno

### Convite\_Aluno

|                              |              |                                        |
|------------------------------|--------------|----------------------------------------|
| <i>id</i>                    | SERIAL       | PK                                     |
| <i>id_turma</i>              | INTEGER      | FK, NULL                               |
| <i>email</i>                 | VARCHAR(150) | NOT NULL                               |
| <i>convidado_em</i>          | TIMESTAMP    | NOT NULL                               |
| <i>status</i>                | ENUM         | pendente, aceitado, expirado, NOT NULL |
| <i>expira_em</i>             | TIMESTAMP    | NOT NULL                               |
| <i>convidado_por</i>         | INTEGER      | FK, NOT NULL                           |
| <i>numero_matricula</i>      | VARCHAR(20)  | NULL                                   |
| <i>token_hash</i>            | VARCHAR(100) | NOT NULL                               |
| <i>ultimo_reenvio_por</i>    | INTEGER      | FK, NULL                               |
| <i>exceder_capacidade</i>    | BOOLEAN      | DEFAULT FALSE, NOT NULL                |
| <i>justificativa_excecao</i> | TEXT         | NULL                                   |
| <i>aceitado_por</i>          | INTEGER      | FK, NULL                               |
| <i>aceitado_em</i>           | TIMESTAMP    | NULL                                   |

### Regra de negócio

A combinação *email* + *id\_turma* não pode possuir mais de um convite *pendente*. Um mesmo e-mail pode possuir convites para turmas diferentes e um aluno pode participar de várias turmas; por isso, o e-mail não é globalmente único nesta entidade.

### i Observação

*numero\_matricula* aqui é opcional e **não é UNIQUE**, pois serve apenas para pré-declarar a matrícula quando o professor a conhece no momento do convite. Ao aceitar o convite, o sistema cria as linhas *Usuario* e *Aluno* correspondentes, copiando *email* e *numero\_matricula* para a nova linha de *Aluno*, onde a restrição **UNIQUE** é rigorosamente aplicada.

## 4.37 Notificação (Unificada)

### Notificacao

|                           |         |                                                                                        |
|---------------------------|---------|----------------------------------------------------------------------------------------|
| <i>id</i>                 | SERIAL  | PK                                                                                     |
| <i>id_destinatario</i>    | INTEGER | FK, NOT NULL                                                                           |
| <i>papel_destinatario</i> | ENUM    | Aluno, Professor, Bolsista, Gestor_Bens_Patrimoniais, Gestor_Almozarifado, Chefe_Geral |

**NOT NULL *tipo***

ENUM

ADICIONADO, POST, COMENTARIO, REMOVIDO,  
 TURMA\_ARQUIVADA, TURMA\_DESARQUIVADA,  
 REQUISICAO\_BEM, DATA\_DEVOLUCAO\_REAGENTE,  
 ROTEIRO\_COMPARTILHADO,  
 REQUISICAO\_EDICAO\_BEM, REQUISICAO\_ADICAO\_BEM,  
 BEM\_INSERTIVEL,  
 ENTREGA\_ATRASADA, FRASCOS\_VAZIOS, FRASCOS\_QUEBRADOS,  
 FRASCOS\_VENCIDOS, FRASCOS\_A\_SEREM\_PESADOS,  
 FRASCOS\_EM\_QUARENTENA, ESCASSEZ\_ESTOQUE,  
 AUTO\_ATENDIMENTO\_RETIRADA

**NOT NULL *id\_quem\_fez\_acao***

INTEGER FK, NULL

***id\_turma***

INTEGER FK, NULL

***quantidade***

INTEGER NULL

***entidade\_alvo***

VARCHAR(40) NOT NULL

***id\_alvo***

VARCHAR(200) NOT NULL

***lida***

BOOLEAN DEFAULT FALSE, NOT NULL

***lida\_em***

TIMESTAMP NULL

***emitida\_em***

TIMESTAMP NOT NULL

***expira\_em***

TIMESTAMP NULL

**i Unificação das Notificações**

As quatro tabelas de notificação anteriores (*Notificacao\_para\_Aluno*, *Notificacao\_para\_Professor*, *Notificacao\_para\_Gestor\_de\_bens\_Patrimoniais*, *Notificacao\_para\_Gestor\_de\_Reagentes*) foram consolidadas em uma única entidade *Notificacao*. O campo *papel\_destinatario* indica em qual contexto de papel a notificação deve ser exibida, incluindo Aluno, Professor, Bolsista, Gestor\_Bens\_Patrimoniais, Gestor\_Almoarifado e Chefe\_Geral.

No Firestore, esta entidade é implementada como sub-coleção *Usuarios/{uid}/Notificacoes*, eliminando a necessidade de queries em múltiplas coleções para usuários multi-role. A ação de “Limpar tudo” na interface marca *lida* = true e *lida\_em* = NOW() em todas as notificações ativas do usuário, preservando o histórico de auditoria. Os campos *entidade\_alvo* e *id\_alvo* viabilizam a navegação direta (*deep linking*) para o objeto de origem ao clicar no aviso. O campo *expira\_em* é nulo para notificações operacionais persistentes (como *ESCASSEZ\_ESTOQUE*), que não possuem expiração temporal automática.

### 4.38 Tabela Aluno Turma (N para N)

#### Aluno\_x\_Turma

*id\_aluno*

INTEGER PK, FK

*id\_turma*

INTEGER PK, FK

#### Regra de negócio

A chave primária composta (*id\_aluno*, *id\_turma*) impede matricular o mesmo aluno duas vezes na mesma turma.

### 4.39 Local

#### Local

*id*

SERIAL PK

*predio*

VARCHAR(30) NOT NULL

*andar*

VARCHAR(10) NOT NULL

*sala*

VARCHAR(30) NOT NULL

*criado\_por*

INTEGER FK, NOT NULL

*criado\_em*

TIMESTAMP NOT NULL

#### Regra de negócio

A combinação de prédio, andar e sala deve ser única (*UNIQUE(predio, andar, sala)*).

### 4.40 Post

#### Post

*id*

SERIAL PK

*id\_professor*

INTEGER FK, NOT NULL

*id\_turma*

INTEGER FK, NOT NULL

*id\_roteiro\_experimento*

INTEGER FK, NULL

*titulo*

VARCHAR(150) NOT NULL

*descricao*

TEXT NOT NULL

*criado\_em*

TIMESTAMP NOT NULL

## 4.41 Comentario

### Comentario

|                         |           |                         |
|-------------------------|-----------|-------------------------|
| <i>id</i>               | SERIAL    | PK                      |
| <i>id_post</i>          | INTEGER   | FK, NOT NULL            |
| <i>id_usuario</i>       | INTEGER   | FK, NOT NULL            |
| <i>texto</i>            | TEXT      | NOT NULL                |
| <i>criado_em</i>        | TIMESTAMP | NOT NULL                |
| <i>moderado</i>         | BOOLEAN   | DEFAULT FALSE, NOT NULL |
| <i>motivo_moderacao</i> | TEXT      | NULL                    |
| <i>moderado_por</i>     | INTEGER   | FK, NULL                |

## 4.42 Registro de Auditoria

### Registro\_de\_Auditoria

|                                                                                                                                                                                                                                               |              |              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|--------------|
| <i>id</i>                                                                                                                                                                                                                                     | SERIAL       | PK           |
| <i>id_usuario</i>                                                                                                                                                                                                                             | INTEGER      | FK, NOT NULL |
| <i>acao</i>                                                                                                                                                                                                                                   | VARCHAR(200) | NOT NULL     |
| <i>tipo_entidade_sofre_acao</i>                                                                                                                                                                                                               | ENUM         |              |
| TURMA, ALUNO, PROFESSOR, ALMOXARIFADO, RESUMO_REAGENTE, ESPECIFICACAO_REAGENTE, LOTE, FRASCO_REAGENTE, EMPRESTIMO_REAGENTE, BEM_PATRIMONIAL, LOCAL, MATERIA, COMENTARIO, POST, ROTEIRO, REQUISICAO_ADICAO_BEM, REQUISICAO_EDICAO_BEM, USUARIO |              |              |
| NOT NULL <i>id_do_objeto_da_entidade</i>                                                                                                                                                                                                      | VARCHAR(200) | NOT NULL     |
| <i>acao_feita_em</i>                                                                                                                                                                                                                          | TIMESTAMP    | NOT NULL     |
| <i>metadata</i>                                                                                                                                                                                                                               | JSONB        | NOT NULL     |

### i Observação

*acao* e *id\_do\_objeto\_da\_entidade* tinham o tamanho 200 escrito como se fosse um selo de restrição em vez de parte do tipo; corrigido para VARCHAR(200). O valor REAGENTE do enum foi substituído por valores granulares. **Nota sobre Impressão:** A reimpressão avulsa (2ª via) de um frasco já cadastrado é auditada diretamente aqui (*tipo* = FRASCO\_REAGENTE, *acao* = REIMPRESSAO\_ETIQUETA) em vez de usar a entidade *Impressao\_Etiqueta\_Frasco*.

### 4.43 Especificação de Integridade do Modelo 3FN

Esta subseção fecha as propriedades lógicas que devem permanecer verdadeiras antes de qualquer denormalização específica do Firestore.

#### 4.43.1 Cardinalidades

| Relacionamento                     | Card.          | Regra                                                                                                                                                                |
|------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Usuario – cada papel específico    | 0..1 por papel | Um usuário pode ter no máximo uma linha em cada tabela de papel; várias tabelas de papel podem estar associadas ao mesmo usuário conforme as combinações permitidas. |
| Gestor Almojarifado – Almojarifado | N:N            | A mesma associação não pode ocorrer duas vezes.                                                                                                                      |
| Resumo Bem – Bem Patrimonial       | 1:N            | Cada bem pertence a um resumo.                                                                                                                                       |
| Local – Bem Patrimonial            | 1:N            | Cada bem possui um local cadastrado.                                                                                                                                 |
| Resumo Reagente – Especificação    | 1:N            | Um resumo agrupa especificações; cada especificação pertence a um resumo.                                                                                            |
| Especificação – Substância         | N:N            | A composição liga múltiplas substâncias a múltiplas especificações.                                                                                                  |
| Especificação – Lote               | 1:N            | Um lote pertence a uma única especificação.                                                                                                                          |
| Lote – Frasco                      | 1:N opcional   | Um frasco pode ter lote NULL quando o lote não for conhecido.                                                                                                        |
| Almojarifado – Frasco              | 1:N            | Cada frasco está em exatamente um almojarifado.                                                                                                                      |
| Frasco – Empréstimo                | 1:N            | Histórico completo; no máximo um empréstimo ativo por frasco.                                                                                                        |
| Turma – Aluno                      | N:N            | Alunos podem participar de várias turmas (via sub-coleção <code>Turma/{id}/Alunos</code> ).                                                                          |
| Turma – Post                       | 1:N            | Cada post pertence a uma turma e ao professor autor.                                                                                                                 |
| Post – Comentário                  | 1:N            | Cada comentário pertence a um post.                                                                                                                                  |
| Professor – Matéria                | N:N            | A associação não pode ser duplicada.                                                                                                                                 |
| Roteiro – Professor                | N:N            | O mesmo roteiro não pode ser compartilhado duas vezes com o mesmo professor.                                                                                         |

#### 4.43.2 Nullabilidade

Todo atributo é **NOT NULL** por padrão. Um campo só pode ser NULL quando isso representa uma ausência legítima do domínio. As exceções são:

- foto de perfil;
- documento de baixa antes da baixa patrimonial;
- campos de resposta de requisição antes da resposta;
- campos opcionais de especificação (fabricante, código do fabricante, FDS/FISPQ e demais dados não disponíveis);
- CAS e fórmula química quando não cadastrados;
- frequência de pesagem quando o reagente não requer pesagem frequente;
- lote e dados de validade/pesagem quando desconhecidos ou ainda não aplicáveis;
- validade\_efetiva antes da abertura quando sua definição depende do prazo após abertura;
- dados de devolução de empréstimo antes da devolução;
- id\_local da linha agregada “Geral” no resumo patrimonial diário;
- id\_turma da notificação de professor quando o tipo não é COMENTARIO;
- atributos de composição cuja ausência é necessária para representar faixa ou notação textual do fabricante.

#### 4.43.3 Chaves e constraints

- Usuario.email, Bem\_Patrimonial.numero\_patrimonio, Frasco\_Reagente.codigo\_frasco, Turma.codigo\_turma, Materia.codigo\_materia e CAS não nulo são únicos.
- As tabelas associativas usam chave primária composta por suas duas FKs.
- Composicao\_Reagente usa UNIQUE(id\_especificacao\_reagente, id\_substancia\_quimica).
- Lote usa UNIQUE(id\_especificacao\_reagente, nome\_fornecedor, numero\_lote), evitando a identificação de dois lotes equivalentes do mesmo fornecedor e produto.
- Convite\_Aluno permite o mesmo e-mail em turmas diferentes, mas impede mais de um convite PENDENTE para a mesma combinação de e-mail normalizado e turma; para convite global (*id\_turma* NULL), impede mais de um convite PENDENTE para o mesmo e-mail normalizado. No PostgreSQL, isso é representado por dois índices únicos parciais: UNIQUE(email\_normalizado, id\_turma) WHERE id\_turma IS NOT NULL AND status = 'pendente' e UNIQUE(email\_normalizado) WHERE id\_turma IS NULL AND status = 'pendente'. No Firestore, a Cloud Function deve realizar a mesma validação dentro de transação.

- Requisicao\_Edicao\_Bem\_Patrimonial permite no máximo uma requisição PENDENTE por bem.
- Todos os valores quantitativos de massa, volume, capacidade, contagens, mês, dia de frequência e quantidades agregadas devem obedecer aos limites não negativos ou positivos definidos pelas regras do domínio.
- Datas que representam intervalos devem respeitar sua ordem temporal; exemplos: fabricação  $\leq$  validade e devolução  $\geq$  retirada.

#### 4.43.4 Regras químicas fechadas

- **PURA** e **MISTURA** são tipos de substância do resumo. MISTURA exige composição na especificação; PURA não depende de múltiplas linhas de composição.
- A concentração, quando aplicável, é atributo da composição da especificação, e não do resumo.
- A densidade é atributo da especificação. É obrigatória e positiva para LIQUIDO e pode ser NULL para SOLIDO.
- LIQUIDO usa ml como unidade operacional; SOLIDO usa g. O par estado/unidade deve ser sempre coerente.
- Duas especificações com o mesmo nome podem coexistir sob o mesmo resumo quando diferirem em concentração, densidade, fabricante ou código do produto e representarem o mesmo item agrupador. Diferença de especificação não implica automaticamente novo resumo.
- Gases estão fora do escopo da V1. Não se deve misturar sua futura regra de pressão com os campos de pesagem de sólidos e líquidos.

#### 4.43.5 Máquina de estados e invariantes do frasco

| Estado/ação                              | Invariante                                                                           |
|------------------------------------------|--------------------------------------------------------------------------------------|
| FECHADO + DISPONIVEL                     | Pode ser retirado ou aberto.                                                         |
| ABERTO + DISPONIVEL                      | Pode ser retirado ou ajustado por pesagem.                                           |
| EMPRESTADO                               | Deve possuir exatamente um empréstimo ativo EM_USO ou ATRASADO.                      |
| VAZIO/QUEBRADO                           | Não pode estar EMPRESTADO e deve aparecer como pendente de descarte.                 |
| QUARENTENA                               | Exige detalhe de motivo e bloqueia nova retirada.                                    |
| VENCIDO + uso_vencido_authorized = false | Exibe PENDENTE DE DESCARTE quando não estiver em quarentena; bloqueia nova retirada. |

| Estado/ação                             | Invariante (continuação)                                                                                                                                                    |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VENCIDO + uso_vencido_authorized = true | Pode permanecer DISPONIVEL para uso excepcional após confirmação explícita no empréstimo.                                                                                   |
| DESCARTADO                              | É terminal; não retorna a DISPONIVEL.                                                                                                                                       |
| Devolução                               | Só pode ocorrer para empréstimo EM_USO ou ATRASADO e aplica tolerância híbrida Q06: ganho tolerado produz consumo zero e evento AJUSTE; ganho acima da tolerância bloqueia. |

#### 4.43.6 Fechamento dos estados automáticos

*vencido* é definido pelo servidor a partir da validade efetiva. Na abertura do frasco, *validade\_efetiva* é recalculada conforme a regra de validade pós-abertura. *ATRASADO* é promovido pelo job servidor quando a data de devolução prevista é ultrapassada. Nenhum desses estados pode ser inventado pelo frontend. O vencimento também é recalculado sincronicamente em operações críticas de cadastro e devolução, para que o resultado não dependa de o job diário já ter executado.

### 4.44 Checklist de Fechamento do Domínio e do Modelo

Os onze pontos que precisavam ser resolvidos antes da modelagem física foram consolidados da seguinte forma:

- 1. Matriz completa de permissões:** formalizada na seção de Stakeholders e referenciada como fonte normativa de autorização.
- 2. Chefe Geral e multi-role:** Chefe Geral é exclusivo por conta; eventual dupla função institucional exige contas independentes.
- 3. Obrigatoriedade de campos:** formalizada por contexto, estado, tipo de substância e operação.
- 4. Densidade:** reposicionada semanticamente como propriedade de Especificacao\_Reagente e usada nos cálculos sem duplicação no resumo.
- 5. Máquina de estados:** estados do frasco e transições de retirada, devolução, quarentena, vencimento, esvaziamento, quebra e descarte foram explicitados.
- 6. NULL/NOT NULL:** todo campo passa a ter semântica explícita, com NULL restrito a ausências legítimas do domínio.
- 7. Cardinalidades:** relacionamentos 1:N, N:N e especializações de papel foram consolidados.
- 8. Constraints:** unicidades, chaves compostas, regras temporais e limites quantitativos foram formalizados, indicando quais precisarão ser simulados por transação no Firestore.

9. **Métricas e unidades:** limiar de escassez passa a significar quantidade de frascos; massa de balança e consumo convertido permanecem conceitos distintos.
10. **Gases:** explicitamente fora do escopo da V1, com a extensão futura isolada para não contaminar o modelo atual.
11. **Vencimento e destino:** cadastro, abertura e devolução distinguem explicitamente quarentena, pendência de descarte e uso vencido autorizado.

#### i Critério de passagem para a modelagem Firestore

O modelo 3FN é considerado fechado quando cada requisito funcional consegue ser rastreado até entidade, atributo, constraint e fluxo correspondente sem depender de uma suposição não documentada. A seção seguinte pode então introduzir apenas as denormalizações necessárias às consultas do Firestore, sem alterar a semântica do modelo relacional.

#### DP-C02: moderação

Autor vê o original marcado como moderado; demais alunos e bolsistas veem aviso institucional. Professor responsável e Chefe Geral veem o original e histórico necessário à moderação/auditoria. Moderação exige motivo e moderador; proteção ocorre no servidor e nas Rules (seção 11).

#### DP-C03: segredo de convite

Gerar token com CSPRNG de 32 bytes e armazenar somente SHA-256 do token aleatório. Comparar em tempo constante no backend, validar expiração de sete dias, identidade verificada e consumo único transacional. Reenvio substitui hash e expiração no mesmo documento e registra o último operador; link anterior deixa de funcionar. No Firestore, docId determinístico é derivado no servidor por HMAC-SHA-256 com segredo do servidor sobre serialização inequívoca de contexto turma/global e e-mail normalizado. Não expor hash simples de e-mail previsível. HMAC é pseudonimização; documento e e-mail continuam restritos. Nunca registrar token em logs, auditoria ou banco.

#### Q06: massa, consumo e tolerância

Massa consumida =  $\max(0, peso_{saída} - peso_{retorno})$  em g. Medida utilizada é não negativa (g para sólido, mL via densidade para líquido). Medida usada no frasco acumula apenas gramas. Tolerância normal:  $\max(1\text{ g}, 0,005\text{ }peso_{saída})$ ; higroscópico:  $\max(2\text{ g}, 0,02\text{ }peso_{saída})$ . A fórmula de tolerância higroscópica e de variação instrumental incide estritamente sobre o **Peso Bruto de Saída** ( $peso_{saída}$ ). Ganho dentro da tolerância preserva peso físico real, consumo zero e evento AJUSTE/ganho\_massa\_higroscopia. Retorno abaixo da tara ( $peso_{retorno} < peso_{vazio}$ ): diferença até 5 g exige confirmação de esgotamento para VAZIO e ajuste auditado; diferença maior com produto restante bloqueia e exige recalibração formal de tara com justificativa. Não

converter ajuste em consumo negativo.

#### Q14 e Q04: rastreabilidade na retirada

Autoatendimento é calculado pelo servidor: só o único gestor ativo do almoxarifado pode retirar para si, com justificativa obrigatória e notificação auditável à chefia; se houver outro gestor ativo, bloquear. Para vencido ou validade desconhecida liberado com termo nas finalidades PESQUISA\_TCC\_POS ou ESTUDO\_DEGRADACAO\_RESIDUOS, exigir justificativa metodológica de 20–2000 caracteres, versão do TCR, instante servidor, UID autenticado do retirante que aceitou e referência ao Registro\_de\_Auditoria correspondente. O backend valida aceite vinculado ao frasco, finalidade e operação, nunca declaração do gestor em nome de terceiro. Sem essas condições, rejeitar. TCR registra ciência e responsabilidade, sem substituir regras institucionais de segurança química. Quarentena e pendência de descarte continuam bloqueadas.

#### Reclassificação patrimonial

Novo nome individual exige resolver/criar resumo alvo, preencher novo\_id\_resumo\_bem\_patrimonial e alterar só o vínculo do bem solicitado; não renomear resumo compartilhado. Versao\_bem\_origem captura a versão observada e impede aprovação sobre edição concorrente.

## 5 Notas de Mapeamento para Firestore

### i Por que esta seção existe

A modelagem da seção anterior é deliberadamente relacional e normalizada (3FN), pensada para ser lida e avaliada como um modelo de dados independente de tecnologia. O banco real do LCQUI, porém, é o **Firestore** (Firebase), um banco de documentos sem JOIN e sem LIKE/busca por substring. Esta seção documenta as denormalizações propositalmente necessárias para a implementação – nenhuma delas deve ser lida como parte do modelo 3FN, e nenhuma delas deve alterar a seção 4.

### 5.1 Reintrodução de *letra\_inicial*

O campo *letra\_inicial*, removido de *Resumo\_Reagente* por ser derivável de *nome* (violaria 3FN), volta a existir como campo próprio **apenas** no documento Firestore da coleção *Resumo\_Reagente*. Motivo: o Firestore não tem LIKE/busca por substring, então a busca alfabética do catálogo precisa de um campo indexável para filtro de igualdade (ou de *range query* por prefixo). Como esse filtro normalmente é combinado com outros filtros de igualdade na mesma consulta (estado físico, natureza química), um campo de igualdade simples é mais previsível dentro das limitações de índice composto do Firestore do que uma *range query* de prefixo sobre *nome* combinada com filtros adicionais.

**Consistência obrigatória**

*letra\_inicial* deve ser recalculado e reescrito pela mesma *Cloud Function* que grava *nome*, nunca aceito diretamente do cliente – ver princípio “nunca confiar no *client-side*” na seção *Tecnologia e Relatórios*.

## 5.2 Estado físico canônico do catálogo

DP-A01: *Resumo\_Reagente.estado\_fisico* é escalar SOLIDO/LIQUIDO; *eh\_higroscopico* também pertence ao resumo. Filtro usa igualdade: `.where("estado_fisico", "==", "SOLIDO")`. A unidade operacional é derivada, g para sólido e mL para líquido.

## 5.3 Espelhamento de *nome\_equipamento* e Localização em *Bem\_Patrimonial*

No modelo 3FN, o nome do tipo de bem pertence exclusivamente a *Resumo\_Bem\_Patrimonial.nome*; *Bem\_Patrimonial* mantém apenas *id\_resumo\_bem\_patrimonial* e *id\_local*. No Firestore, porém, consultas de listagem e relatórios frequentemente partem diretamente de *Bem\_Patrimonial*, sem um *JOIN* para resolver o resumo ou o prédio. Por isso, o documento Firestore de *Bem\_Patrimonial* deve conter os campos denormalizados *nome\_equipamento*, *predio*, *andar* e *sala*.

**Fonte única de verdade para o nome do patrimônio**

*nome\_equipamento* é exclusivamente um campo de leitura derivado de *Resumo\_Bem\_Patrimonial.nome*. O cliente nunca pode gravá-lo diretamente. Sempre que o nome do resumo for alterado, a mesma operação transacional ou um gatilho de propagação controlado pelo servidor deve atualizar os documentos *Bem\_Patrimonial* associados. Na aprovação da requisição de edição descrita na seção 10.2.5, *novo\_nome* representa uma proposta de alteração do nome do resumo; o servidor resolve/cria outro resumo e altera a referência somente do bem solicitado; renomear globalmente um resumo é ação distinta do gestor.

## 5.4 Composição embutida na especificação

O mapeamento físico adotado é *composicao*, array de mapas na especificação em *Resumo\_Reagente/id/Especificacoes/id*. Cada mapa referencia uma substância; o servidor impede IDs repetidos e limita o tamanho do array. Não manter simultaneamente uma coleção raiz e uma subcoleção alternativa como fontes concorrentes. A relação N:N continua normalizada na seção 4.

## 5.5 Tabelas “Materialized” são coleções próprias, não *views* do SGBD

As tabelas descritas na seção *Materialized (Views)* não são *materialized views* no sentido de PostgreSQL (não existe `REFRESH MATERIALIZED VIEW` no Firestore). Cada uma delas é implementada como sua própria coleção Firestore, escrita por *Cloud Functions* – gatilhos de escrita (`onDocumentWritten`) para contadores que reagem a um evento pontual (ex.: *Lote\_Materializado*), ou funções agendadas (`onSchedule`) para os resumos diários/mensais. Ver pseudo-código na seção *Tecnologia e Relatórios*.

## 5.6 Unicidade de `numero_patrimonio_proposto` no Firestore

O PostgreSQL implementa a regra RF10b via índice único parcial `UNIQUE(numero_patrimonio_proposto) WHERE status = 'pendente'`, impedindo duas requisições simultâneas para a mesma plaqueta enquanto o bem patrimonial ainda não existe. No Firestore não há índice parcial nem *unique constraint* nativo, então a garantia é reimplementada na Cloud Function de criação da requisição de adição.

A criação lê o documento determinístico em `Locks_Requisicao_Patrimonio`, verifica a plaqueta já cadastrada e grava lock e requisição na mesma transação. Aprovação e rejeição removem o lock na mesma transação da resposta. Uma consulta prévia pode melhorar a mensagem de erro, mas não substitui a chave de unicidade. Locks vinculados a requisições pendentes não expiram por idade.

## 5.7 Mecanismo de Sequenciamento Atômico via Documento Singleton (*Contador\_Codigo\_Frasco*)

A coleção `Contador_Codigo_Frasco` não existe no modelo 3FN porque bancos relacionais possuem primitivas de `SEQUENCE/SERIAL`. Para gerar o identificador atômico LCQUI-X, utiliza-se o documento singleton `Contador_Codigo_Frasco/singleton` com o campo `ultimo_codigo_gerado`, operado atômicamente via `runTransaction` no backend. A transação garante a unicidade; contenção e custo devem ser medidos incluindo histórico, auditoria, índices e tentativas repetidas. Não se fixa aqui uma cota universal de escritas por segundo.

## 5.8 Índices Compostos Obrigatórios

Para garantir a integridade das *queries* e ordenações cruciais, os seguintes índices compostos devem ser criados explicitamente no console do Firestore:

1. `Historico_Frasco_Reagente`: `id_frasco_reagente` (ASC) + `timestamp` (DESC).
2. `Emprestimo_Reagente`: `id_frasco_reagente` (ASC) + `data_retirada` (DESC).
3. **M-23 Bem\_Patrimonial**: `predio` (ASC) + `andar` (ASC) + `sala` (ASC) – suporta o relatório personalizado por localização (Seção 9 fluxo ALM-01).

4. **M-23** Bem\_Patrimonial: id\_resumo\_bem\_patrimonial (ASC) + status (ASC) – suporta a listagem de bens por tipo e status.
5. **M-23 / N-08** Collection-group Historico (sub-coleção de Bem\_Patrimonial): predio (ASC) + timestamp (DESC) – suporta relatório histórico por prédio/período.
6. **M-23** Collection-group Historico: id\_gestor (ASC) + timestamp (DESC) – suporta relatório de atividade por operador.
7. **M-23** Emprestimo\_Reagente: id\_almoxarifado (ASC) + data\_retirada (DESC) – suporta relatório mensal por almoxarifado.
8. Frasco\_Reagente: id\_almoxarifado (ASC) + id\_resumo\_reagente (ASC) + id\_especificacao\_reagente (ASC) + disponibilidade (ASC) + estado\_fisico\_frasco (ASC) + em\_quarentena (ASC) + vencido (ASC) – suporta job agendado de escassez.
9. Emprestimo\_Reagente: status (ASC) + data\_devolucao\_prevista (ASC) – suporta job agendado de vencimentos e atrasos.
10. Frasco\_Reagente: vencido (ASC) + validade\_efetiva (ASC) – suporta job agendado de frascos vencendo.
11. Emprestimo\_Reagente: id\_almoxarifado (ASC) + data\_devolucao\_efetuada (ASC) – suporta relatório de movimentação e consumo por almoxarifado.
12. Historico\_Frasco\_Reagente: id\_almoxarifado (ASC) + timestamp (ASC) – suporta relatório de histórico de eventos por almoxarifado.

## 5.9 Mapeamento Completo de Coleções e Sub-coleções Firestore

### i Como ler esta tabela

Cada linha descreve uma entidade 3FN da Seção 4, a decisão de estrutura no Firestore (coleção raiz, sub-coleção ou array de maps embutido) e a justificativa. Campos de consulta necessários podem ser projetados no documento Firestore conforme o dicionário; o símbolo **Denorm:** lista esses campos. A consistência usa transação para invariantes ou propagação versionada para apresentação; snapshots históricos não são atualizados com o cadastro atual.

| Entidade 3FN | Estrutura Firestore                                  | Justificativa / Denorm                            |
|--------------|------------------------------------------------------|---------------------------------------------------|
| Usuario      | Coleção raiz Usuarios<br>(docId = Firebase Auth UID) | Acesso direto por auth; estrutura natural do SDK. |

| Entidade 3FN                       | Estrutura Firestore                                                                                                                                  | Justificativa / Denorm<br>(continuação)                                                                                                                                                                |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chefe_Geral                        | Coleção raiz <b>Chefe_Geral</b> (docId = uid)                                                                                                        | Lido por <code>atualizarCustomClaims(uid)</code> para compor o claim <code>roles</code> do token JWT. Claims são projeção; conta ativa e versão de permissões devem ser verificadas conforme seção 11. |
| Gestor_Almojarifado                | Coleção raiz <b>Gestor_Almojarifado</b> (docId = uid)                                                                                                | Idem acima.                                                                                                                                                                                            |
| Gestor_Bens_Patrimoniais           | Coleção raiz <b>Gestor_Bens_Patrimoniais</b> (docId = uid)                                                                                           | Idem.                                                                                                                                                                                                  |
| Professor                          | Coleção raiz <b>Professor</b> (docId = uid)                                                                                                          | Idem.                                                                                                                                                                                                  |
| Aluno                              | Coleção raiz <b>Aluno</b> (docId = uid). Denorm: <code>letra_inicial</code>                                                                          | Idem. <code>letra_inicial</code> permite filtro de igualdade na busca.                                                                                                                                 |
| Bolsista                           | Coleção raiz <b>Bolsista</b> (docId = uid)                                                                                                           | Idem.                                                                                                                                                                                                  |
| Gestor_Almojarifado_x_Almojarifado | Coleção raiz                                                                                                                                         | Query bidirecional: “gestores de X” e “almojarifados de Y”.                                                                                                                                            |
| Almojarifado                       | Coleção raiz                                                                                                                                         | Poucas dezenas de documentos.                                                                                                                                                                          |
| Estoque_Minimo_Almojarifado        | Sub-coleção <b>Almojarifado/{id\}/Estoques_Configurados/{idResumo_idEspec\}</b>                                                                      | Configuração de limite mínimo para reposição.                                                                                                                                                          |
| Local                              | Coleção raiz                                                                                                                                         | Referenciado por múltiplas entidades; campos <code>predio</code> , <code>andar</code> , <code>sala</code> denormalizados em <b>Bem_Patrimonial</b> via trigger.                                        |
| Resumo_Bem_Patrimonial             | Coleção raiz. Denorm: <code>letra_inicial_nome</code>                                                                                                | Agrupador de bens; filtro alfabético por letra inicial.                                                                                                                                                |
| Bem_Patrimonial                    | Coleção raiz. Denorm: <code>nome Equipamento</code> , <code>predio</code> , <code>andar</code> , <code>sala</code> , <code>letra_inicial_nome</code> | Entidade central pesquisada diretamente; evita JOINS em listagens e relatórios.                                                                                                                        |

| Entidade 3FN                      | Estrutura Firestore                                                                                                                    | Justificativa / Denorm<br>(continuação)                                                                                                                                                                                                                                                               |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Historico_Bem_Patrimonial         | Sub-coleção Bem_Patrimonial/{id}/Historico.<br>Denorm (N-08):<br>predio, andar, sala – snapshot imutável do local no momento do evento | Sempre consultado a partir de um bem conhecido. O snapshot de localização é gravado no momento da criação do evento histórico e não é atualizado quando o bem muda de sala; isso garante rastreabilidade do bem antes e depois da mudança. Requerido pelo relatório personalizado por prédio/período. |
| Alteracao_Bem_Patrimonial         | Campo alteracoes (array de maps) dentro de cada doc de Historico                                                                       | Poucos campos por alteração; relação 1:N estreita, nunca consultada independentemente.                                                                                                                                                                                                                |
| Requisicao_Edicao_Bem_Patrimonial | Coleção raiz. Denorm: nome Equipamento, numero Patrimonio                                                                              | Listada independentemente nos painéis de gestores por qualquer filtro de status.                                                                                                                                                                                                                      |
| Requisicao_Adicao_Bem_Patrimonial | Coleção raiz                                                                                                                           | Mesma lógica; query por status e número de patrimônio proposto.                                                                                                                                                                                                                                       |
| Impressao_Etiqueta_Frasco         | Coleção raiz                                                                                                                           | Refere-se a impressão em lote (virgens). Reimpressões avulsas de frascos já cadastrados usam o Registro de Auditoria.                                                                                                                                                                                 |
| Contador_Codigo_Frasco            | Documento singleton em Contador_Codigo_Frasco/singleton                                                                                | Sequenciador atômico operado via runTransaction para gerar o LCQUI-X.                                                                                                                                                                                                                                 |
| Substancia_Quimica                | Coleção raiz                                                                                                                           | Catálogo global; raramente alterado.                                                                                                                                                                                                                                                                  |
| Resumo_Reagente                   | Coleção raiz. Denorm: letra_inicial                                                                                                    | Catálogo de reagentes; filtros de igualdade combinados exigem ambos os campos como propriedades do documento.                                                                                                                                                                                         |
| Especificacao_Reagente            | Sub-coleção Resumo_Reagente/{id}/Especificacoes                                                                                        | Sempre acessada a partir do resumo; nunca consultada globalmente.                                                                                                                                                                                                                                     |
| Composicao_Reagente               | Array de maps embutido no documento de Especificacao_Reagente                                                                          | N pequeno e fixo (poucas substâncias por especificação); não cresce sem limite.                                                                                                                                                                                                                       |

| Entidade 3FN                  | Estrutura Firestore                                                                                                                        | Justificativa / Denorm<br>(continuação)                                                                                                                                                   |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lote                          | Coleção raiz. Denorm:<br>id_especificacao_<br>reagente, nome_<br>reagente                                                                  | Relatórios consultam todos os lotes de um período sem partir de uma especificação conhecida.                                                                                              |
| Frasco_Reagente               | Coleção raiz. Denorm:<br>id_especificacao_<br>reagente, id_resumo_<br>reagente, nome_<br>reagente, unidade_<br>medida, id_<br>almoxarifado | Entidade central; queries cruzadas constantes por almoxarifado, status e vencimento.                                                                                                      |
| Historico_Frasco_<br>Reagente | Coleção raiz (não sub-coleção). Denorm:<br>id_almoxarifado                                                                                 | Jobs diários consultam “todos os históricos do dia” sem partir de um frasco conhecido; coleção raiz simplifica esse padrão; collection-group também permitiria históricos em subcoleções. |
| Emprestimo_Reagente           | Coleção raiz. Denorm:<br>id_almoxarifado,<br>nome_reagente, nome_<br>usuario_retirou                                                       | Relatórios por almoxarifado e por período; queries de status global (EM_USO).                                                                                                             |
| Materia                       | Coleção raiz                                                                                                                               | Catálogo global; raramente alterado.                                                                                                                                                      |
| Professor_x_Materia           | Coleção raiz                                                                                                                               | Query bidirecional: “matérias de X” e “professores de Y”.                                                                                                                                 |
| Turma                         | Coleção raiz. Denorm:<br>nome_materia                                                                                                      | Acesso direto por codigo_turma; nome da matéria necessário na listagem sem JOIN.                                                                                                          |

| Entidade 3FN             | Estrutura Firestore                                                                         | Justificativa / Denorm<br>(continuação)                                                                                                                                                                                                                                                                                                                                        |
|--------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aluno_x_Turma            | Subcoleção dupla<br>(espelhada):<br>1.<br>Turma/{id}/Alunos<br>2. Usuarios/{uid}/<br>Turmas | <b>1. Turma/{id}/Alunos:</b> Visão da disciplina; contagem de capacidade e listagem de chamada para o professor.<br><b>2. Usuarios/{uid}/Turmas:</b> Visão do aluno; viabiliza o feed em tempo real no menu lateral de “Minhas Turmas” com segurança isolada por UID e zero leituras $N + 1$ . Consistência garantida via transação nas Cloud Functions de matrícula/exclusão. |
| Historico__Alunos__Turma | Sub-coleção<br>Turma/{id}/<br>HistoricoAlunos                                               | Auditoria de inclusão/exclusão sempre por turma.                                                                                                                                                                                                                                                                                                                               |
| Post                     | Sub-coleção<br>Turma/{id}/Posts                                                             | Feed cronológico da turma; nunca consultado globalmente.                                                                                                                                                                                                                                                                                                                       |
| Comentario               | Sub-coleção<br>Turma/{id}/Posts/<br>{postId}/<br>Comentarios                                | Thread aninhada ao post.                                                                                                                                                                                                                                                                                                                                                       |
| Historico_Posts_Turma    | Sub-coleção<br>Turma/{id}/Posts/<br>{postId}/Historico                                      | Auditoria de edição sempre a partir do post.                                                                                                                                                                                                                                                                                                                                   |
| Historico_Comentario     | Sub-coleção<br>Turma/{id}/Posts/<br>{postId}/<br>Comentarios/{id}/<br>Historico             | Auditoria de edição sempre a partir do comentário.                                                                                                                                                                                                                                                                                                                             |
| Roteiro_Experimento      | Coleção raiz                                                                                | Compartilhamento cross-professor exige acesso sem partir de uma turma.                                                                                                                                                                                                                                                                                                         |
| Convite_Aluno            | Coleção raiz                                                                                | Query por e-mail normalizado e status independentemente de turma.                                                                                                                                                                                                                                                                                                              |

| Entidade 3FN                                      | Estrutura Firestore                           | Justificativa / Denorm<br>(continuação)                                                                                  |
|---------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Notificacao                                       | Sub-coleção Usuarios/<br>\{uid\}/Notificacoes | Query natural por usuário;<br>funciona automaticamente para<br>usuários multi-role sem necessidade<br>de multiple reads. |
| Registro_de_Auditoria                             | Coleção raiz                                  | Log global queryável por tipo de<br>entidade e período.                                                                  |
| Lote_Materializado                                | Coleção raiz (docId =<br>id do lote)          | Atualizada por trigger<br>onFrascoCriado/onFrascoRemovido.                                                               |
| Resumo_Almojarifado_<br>Diario                    | Coleção raiz                                  | Job agendado diário.                                                                                                     |
| Resumo_Reagente_<br>Diario                        | Coleção raiz                                  | Job agendado diário.                                                                                                     |
| Resumo_Bem_<br>Patrimonial_Diario                 | Coleção raiz                                  | Job agendado diário.                                                                                                     |
| Atividade_Gestor_<br>Almojarifado_Mensal          | Coleção raiz                                  | Job agendado diário (acumulado<br>por mês).                                                                              |
| Atividade_Gestor_<br>Bens_Patrimoniais_<br>Mensal | Coleção raiz                                  | Job agendado diário (acumulado<br>por mês).                                                                              |

#### Regra geral de denormalização

Copiar apenas atributos necessários às consultas documentadas. Cada projeção identifica sua origem e mecanismo de atualização; snapshots de eventos são imutáveis. Vínculos e invariantes críticos são transacionais, enquanto projeções de apresentação podem ser assíncronas com reconciliação.

#### 5.9.1 Dicionário de dados completo e convenções físicas

Este dicionário especifica o alvo documental. Divergências com os caminhos do código atual são listadas na auditoria; não se pressupõe migração executada. IDs relacionais SERIAL tornam-se docIds string; FKs tornam-se IDs string, sem JOIN implícito. Datas civis usam string YYYY-MM-DD; instantes usam Timestamp UTC, exibidos em America/Sao\_Paulo. Valores quantitativos usam number finito, com precisão/escala da seção 4; rejeitar NaN, infinito, sinal e casas decimais incompatíveis. Todos os limites VARCHAR continuam válidos. O = obrigatório; N = null permitido; C = condicional. Campos calculados, autoria, timestamps, IDs e estados são gravados pelo servidor. Arrays de composição e alterações são limitados pelo esquema, nunca históricos crescentes. O docId substitui o campo id SERIAL; em associações a chave é

determinística pelas FKs.

### Usuario

Caminho: `Usuarios/{uid}`.

**id** `docId` string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**nome** string; O. Nome de apresentação; não substitui a chave do documento. Máximo 150 caracteres.

**email** string; O. Endereço normalizado para identidade/convite; não divulgar em catálogo público interno. Máximo 150 caracteres. Unicidade garantida por chave determinística/registro de unicidade no servidor.

**profile\_url\_photo** string; N. Foto opcional de apresentação do usuário; ausência usa avatar padrão, com acesso validado ao objeto.

### Chefe\_Geral

Caminho: `Chefe_Geral/{id}`.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

### Gestor\_Almojarifado

Caminho: `Gestor_Almojarifado/{id}`.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

### Gestor\_Almojarifado\_x\_Almojarifado

Caminho: `Gestor_Almojarifado_x_Almojarifado/{id}`.

**id\_gestor\_almojarifado** string; O. Identificador da entidade gestor almojarifado, cuja existência e escopo são verificados no servidor.

**id\_almojarifado** string; O. Identificador da entidade almojarifado, cuja existência e escopo são verificados no servidor.

### Gestor\_Bens\_Patrimoniais

Caminho: `Gestor_Bens_Patrimoniais/{id}`.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

**Bem\_Patrimonial**

Caminho: Bem\_Patrimonial/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_resumo\_bem\_patrimonial** string; O. Identificador da entidade resumo bem patrimonial, cuja existência e escopo são verificados no servidor.

**numero\_patrimonio** string; O. Plaqueta institucional em texto, preservando zeros iniciais; identifica unicamente a unidade física. Máximo 30 caracteres.

**estado\_conservacao** string enum; O. Valor de estado conservacao, validado pelo formulário e pelo servidor conforme regras do domínio. Valores: BOM, REGULAR, RUIM.

**id\_local** string; O. Identificador da entidade local, cuja existência e escopo são verificados no servidor.

**photo\_url** string; O. Referência da foto validada; não confiar em URL arbitrária.

**documento\_dado\_baixa\_pdf\_url** string; N. Referência do comprovante PDF, nula antes da baixa e obrigatória em Ja\_dado\_baixa; preservar após vínculo.

**nome\_responsavel\_sei** string; O. Responsável institucional em texto; não é FK de usuário. Máximo 150 caracteres.

**status** string enum; O. Estado controlado pelo domínio; transições são validadas no servidor. Valores: Ativo, Inservível, Ja\_dado\_baixa.

**descricao\_complementar** string; N. Observação opcional da unidade física, sem alterar a descrição compartilhada do modelo. Máximo 200 caracteres.

**Resumo\_Bem\_Patrimonial**

Caminho: Resumo\_Bem\_Patrimonial/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**nome** string; O. Nome de apresentação; não substitui a chave do documento. Máximo 150 caracteres.

**descricao** string; O. Valor de descricao, validado pelo formulário e pelo servidor conforme regras do domínio.

### Historico\_Bem\_Patrimonial

Caminho: Bem\_Patrimonial/{id}/Historico/{eventoId}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_bem\_patrimonial** string; O. Identificador da entidade bem patrimonial, cuja existência e escopo são verificados no servidor.

**timestamp** Timestamp; O. Data/instante de timestamp, com ordem temporal validada e autoria do relógio do servidor para eventos.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

**tipo** string enum; O. Valor de tipo, validado pelo formulário e pelo servidor conforme regras do domínio. Valores: cadastro, edicao, baixa.

### Alteracao\_Bem\_Patrimonial

Caminho: Bem\_Patrimonial/{id}/Historico/{eventoId}: alteracoes[].

**campo** string; O. Valor de campo, validado pelo formulário e pelo servidor conforme regras do domínio. Máximo 60 caracteres.

**valor\_anterior** string; O. Valor de valor anterior, validado pelo formulário e pelo servidor conforme regras do domínio.

**valor\_novo** string; O. Valor de valor novo, validado pelo formulário e pelo servidor conforme regras do domínio.

### Requisicao\_Edicao\_Bem\_Patrimonial

Caminho: Requisicao\_Edicao\_Bem\_Patrimonial/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_bem\_patrimonial** string; O. Identificador da entidade bem patrimonial, cuja existência e escopo são verificados no servidor.

**status** string enum; O. Estado controlado pelo domínio; transições são validadas no servidor. Valores: pendente, aprovada, rejeitada.

**feita\_em** Timestamp; O. Data/instante de feita em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**respondida\_em** Timestamp; N. Data/instante de respondida em, com ordem temporal validade e autoria do relógio do servidor para eventos.

**id\_usuario\_solicitante** string; O. Identificador da entidade usuario solicitante, cuja existência e escopo são verificados no servidor.

**id\_usuario\_respondente** string; N. Identificador da entidade usuario respondente, cuja existência e escopo são verificados no servidor.

**novo\_nome** string; N. Valor novo nome, restrito ao campo correspondente da entidade de origem; preservar ausência legítima. Máximo 150 caracteres.

**novo\_status** string enum; N. Valor novo status, restrito ao campo correspondente da entidade de origem; preservar ausência legítima. Valores: Ativo, Inservível.

**novo\_estado\_conservacao** string enum; N. Valor novo estado conservacao, restrito ao campo correspondente da entidade de origem; preservar ausência legítima. Valores: BOM, REGULAR, RUIM.

**novo\_id\_local** string; N. Valor novo id local, restrito ao campo correspondente da entidade de origem; preservar ausência legítima.

**nova\_photo\_url** string; N. Valor nova photo url, restrito ao campo correspondente da entidade de origem; preservar ausência legítima.

**motivo** string; O. Valor de motivo, validado pelo formulário e pelo servidor conforme regras do domínio.

**justificativa\_resposta** string; N. Justificativa obrigatória ao aprovar ou rejeitar; ausente enquanto pendente.

**versao\_bem\_origem** number; O. Versão do bem observada pelo solicitante para controle otimista de concorrência.

**novo\_id\_resumo\_bem\_patrimonial** string; N. Novo modelo catalográfico em caso de reclassificação do equipamento.

### **Requisicao\_Adicao\_Bem\_Patrimonial**

Caminho: `Requisicao_Adicao_Bem_Patrimonial/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**numero\_patrimonio\_proposto** string; O. Plaqueta solicitada para o novo bem; unicidade entre pendências e bens existentes. Máximo 30 caracteres.

**status** string enum; O. Estado controlado pelo domínio; transições são validadas no servidor. Valores: pendente, aprovada, rejeitada.

**feita\_em** Timestamp; O. Data/instante de feita em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**id\_bem\_patrimonial\_se\_aprovado** string; N. Identificador da entidade bem patrimonial se aprovado, cuja existência e escopo são verificados no servidor.

**respondida\_em** Timestamp; N. Data/instante de respondida em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**id\_usuario\_solicitante** string; O. Identificador da entidade usuario solicitante, cuja existência e escopo são verificados no servidor.

**id\_usuario\_respondente** string; N. Identificador da entidade usuario respondente, cuja existência e escopo são verificados no servidor.

**estado\_conservacao\_proposto** string enum; O. Valor de estado conservacao proposto, validado pelo formulário e pelo servidor conforme regras do domínio. Valores: BOM, REGULAR, RUIM.

**photo\_url\_proposta** string; O. Foto obrigatória na submissão, validada pelo backend. Valor de photo url proposta, validado pelo formulário e pelo servidor conforme regras do domínio.

**nome\_responsavel\_proposto** string; O. Valor de nome responsavel proposto, validado pelo formulário e pelo servidor conforme regras do domínio. Máximo 150 caracteres.

**id\_local** string; O. Identificador da entidade local, cuja existência e escopo são verificados no servidor.

**id\_resumo\_bem\_patrimonial** string; N. Identificador da entidade resumo bem patrimonial, cuja existência e escopo são verificados no servidor.

**nome\_resumo\_proposto** string; N. Valor de nome resumo proposto, validado pelo formulário e pelo servidor conforme regras do domínio. Máximo 150 caracteres.

**descricao\_resumo\_proposta** string; N. Valor de descricao resumo proposta, validado pelo formulário e pelo servidor conforme regras do domínio.

**motivo** string; O. Valor de motivo, validado pelo formulário e pelo servidor conforme regras do domínio.

**justificativa\_resposta** string; N. Justificativa obrigatória ao aprovar ou rejeitar; ausente enquanto pendente.

### **Locks\_Requisicao\_Patrimonio**

Caminho: Locks\_Requisicao\_Patrimonio/{id}.

**id (docId determinístico)** string; O. Identificador da entidade id (docId determinístico), cuja existência e escopo são verificados no servidor. Máximo 100 caracteres.

**criado\_em** Timestamp; O. Data/instante de criado em, com ordem temporal validada e autoria do relógio do servidor para eventos.

### **Impressao\_Etiqueta\_Frasco**

Caminho: Impressao\_Etiqueta\_Frasco/{id}.

**id docId** string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**gerado\_em** Timestamp; O. Data/instante de gerado em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**gerado\_por** string; O. UID do operador que solicitou a geração de etiquetas, obtido da sessão autenticada.

**codigo\_inicial** number; O. Primeiro número inteiro positivo do intervalo inclusivo de etiquetas virgens.

**codigo\_final** number; O. Último número do intervalo inclusivo, maior ou igual ao inicial e limitado a 50 etiquetas.

### **Almoxarifado**

Caminho: Almoxarifado/{id}.

**id docId** string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_local** string; O. Identificador da entidade local, cuja existência e escopo são verificados no servidor.

**nome** string; O. Nome de apresentação; não substitui a chave do documento. Máximo 100 caracteres.

**descricao** string; O. Valor de descricao, validado pelo formulário e pelo servidor conforme regras do domínio. Máximo 500 caracteres.

**ativo** boolean; O. Indica se o almoxarifado está ativo (necessário para jobs automáticos).

### Estoques\_Configurados

Caminho: Almoxarifado/{almoxId}/Estoques\_Configurados/{configId}.

**configId** docId string; O. Chave do documento, gerada de forma determinística: {id\_resumo\_reagente}\_{id\_especificacao\_reagente}, garantindo unicidade do par.

**id\_especificacao\_reagente** string; O. Identificador da especificação do reagente configurada.

**id\_resumo\_reagente** string; O. Identificador do resumo do reagente, mantido no documento para consultas otimizadas e manter a identidade composta.

**id\_almoxarifado** string; O. Identificador do almoxarifado (redundante à URL, porém canônico).

**qtd\_limiar\_escassez** number; O. Limiar inteiro não negativo de quantidade de frascos para este almoxarifado.

**ativo** boolean; O. Indica se o par continua configurado/intencionalmente estocado.

**notificacao\_ativa** boolean; O. Indica se a emissão automática do alerta está habilitada (permitindo silenciar sem apagar a configuração).

### Substancia\_Quimica

Caminho: Substancia\_Quimica/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**nome** string; O. Nome de apresentação; não substitui a chave do documento. Máximo 150 caracteres.

**cas\_number** string; N. Identificador CAS opcional da substância; quando informado, validar formato, dígito e unicidade. Máximo 15 caracteres.

**formula\_quimica** string; N. Fórmula declarada da substância; apresentação textual, sem executar marcação arbitrária. Máximo 50 caracteres.

**ativo** boolean; O. Habilitação lógica; desativação preserva identidade e histórico.

**Resumo\_Reagente**

Caminho: `Resumo_Reagente/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**nome** string; O. Nome de apresentação; não substitui a chave do documento. Máximo 150 caracteres.

**tipo\_substancia** string enum; O. Valor de tipo substancia, validado pelo formulário e pelo servidor conforme regras do domínio. Valores: PURA, MISTURA.

**natureza\_quimica** string enum; O. Classificação curada do item do catálogo, independente de fabricante ou frasco. Valores: ORGANICO, INORGANICO, ELEMENTO, HIBRIDO.

**requer\_pesagem\_frequente** boolean; O. Indica necessidade de alertas periódicos de pesagem; true exige frequência positiva.

**frequencia\_pesagem\_dias** number; N. Intervalo inteiro positivo entre verificações; null quando não exige pesagem frequente.

**estado\_fisico** string enum; O. SOLIDO/LIQUIDO; fonte canônica do catálogo, filtro por igualdade.

**eh\_higroscopico** boolean; O, padrão false. Fonte canônica da higroscopicidade; não propagar mudanças para snapshots de frascos já cadastrados.

**Composicao\_Reagente**

Caminho: `Resumo_Reagente/{resumoId}/Especificacoes/{id}:composicao[]`.

**id\_substancia\_quimica** string; O. Identificador da entidade substancia quimica, cuja existência e escopo são verificados no servidor.

**valor\_min** number; N. Concentração pontual ou limite inferior não negativo.

**valor\_max** number; N. Limite superior; quando informado exige mínimo e `valor_min <= valor_max`.

**tipo\_concentracao** string enum; O. Base da concentração; enum M\_M, V\_V, M\_V, MOL\_L, MOL\_KG, PPM ou PPB. Valores: M\_M,V\_V,M\_V,MOL\_L,MOL\_KG,PPM,PPB.

**notacao\_original\_fabricante** string; N. Rótulo original, até 50 caracteres; não substitui limites numéricos.

**Especificacao\_Reagente**

Caminho: `Resumo_Reagente/{resumoId}/Especificacoes/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_resumo\_reagente** string; O. Identificador da entidade resumo reagente, cuja existência e escopo são verificados no servidor.

**descricao** string; O. Valor de descricao, validado pelo formulário e pelo servidor conforme regras do domínio. Máximo 150 caracteres.

**fabricante** string; N. Nome do fabricante da especificação comercial; não é o fornecedor do lote. Máximo 150 caracteres.

**codigo\_produto\_fabricante** string; N. Código comercial do fabricante para distinguir versões de produto sob o mesmo resumo. Máximo 100 caracteres.

**grau\_pureza** string; N. Grau declarado pelo fabricante, por exemplo P.A. ou HPLC; texto opcional, não percentual obrigatório. Máximo 30 caracteres.

**densidade** number; N. Densidade em g/ml; positiva e obrigatória para líquido, imutável após confirmação.

**unidade\_de\_medida** string enum; projeção física opcional derivada pelo servidor do estado do resumo, g/ml; não é fonte canônica nem campo editável.

**classe\_inflamabilidade** string enum; O. Classe selecionada para a especificação; não derivar apenas do nome comercial. Valores: NAO\_INFLAMAVEL, CLASSE\_1, CLASSE\_2, CLASSE\_3.

**eh\_controlado\_pf** boolean; O. Indicador cadastral de controle PF declarado pelo gestor; não é inferência automática nem parecer regulatório.

**eh\_controlado\_eb** boolean; O. Indicador cadastral de controle EB declarado pelo gestor, independente do indicador PF.

**link\_fds\_fispq** string; N. Endereço HTTPS opcional para a ficha de segurança; abrir com proteção de navegação externa. Máximo 255 caracteres.

**Lote**

Caminho: `Lote/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_resumo\_reagente** string; O. Projeção física imutável do documento pai da especificação, preenchida no cadastro para permitir agregações por resumo (desvio de 3FN focado em leitura). A base de dados requer backfill.

**id\_especificacao\_reagente** string; O. Identificador da entidade especificacao reagente, cuja existência e escopo são verificados no servidor.

**data\_aquisicao** Timestamp; O. Data da aquisição do lote; não pode estar no futuro no cadastro de entrada já realizada.

**qtd\_frascos\_comprados** number; O. Quantidade adquirida, distinta da contagem materializada de frascos cadastrados.

**nome\_fornecedor** string; O. Fornecedor da aquisição; participa da chave composta do lote. Máximo 80 caracteres.

**numero\_lote** string; O. Identificação do lote do fornecedor/fabricante, distinta do código LCQUI de cada frasco. Máximo 100 caracteres.

**nota\_fiscal** string; O. Referência textual do documento de aquisição; não contém o binário da nota. Máximo 100 caracteres.

**data\_fabricacao** string YYYY-MM-DD; O. Data civil declarada de fabricação; não posterior à validade.

**data\_validade** string YYYY-MM-DD; O. Data civil de validade do lote; preenche sugestão de validade fechada do frasco.

### Frasco\_Reagente

Caminho: Frasco\_Reagente/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_almoxarifado** string; O. Identificador da entidade almoxarifado, cuja existência e escopo são verificados no servidor.

**id\_lote** string; N. Identificador da entidade lote, cuja existência e escopo são verificados no servidor.

**id\_especificacao\_reagente** string; O. Identificador da entidade especificacao reagente, cuja existência e escopo são verificados no servidor. Projeção imutável obrigatória no Firestore.

**id\_resumo\_reagente** string; O. Identificador da entidade resumo reagente, cuja existência e escopo são verificados no servidor. Projeção imutável obrigatória no Firestore.

**detalhe\_local\_armazenamento** string; N. Valor de detalhe local armazenamento, validado pelo formulário e pelo servidor conforme regras do domínio.

**codigo\_frasco** string; O. Identificador LCQUI-N único, gerado pelo servidor. Unicidade garantida por chave determinística/registro de unicidade no servidor.

**data\_ultima\_pesagem** Timestamp; N. Data/instante de data ultima pesagem, com ordem temporal validada e autoria do relógio do servidor para eventos.

**conteudo\_nominal** number; N. Conteúdo inicial declarado em ml ou g conforme especificação. Se NULL, valor original do rótulo desconhecido ou ilegível (não usar 0 para isso).

**peso\_no\_cadastrado** number; O. Peso bruto inicial em g; imutável.

**peso\_atual** number; O. Último peso bruto confirmado em g.

**peso\_frasco\_vazio** number; N. Tara em g, calculada ou informada conforme modalidade.

**medida\_usada** number; O. Acumulado gravimétrico em g; não misturar com volume.

**data\_abertura** string YYYY-MM-DD; N. Data efetiva da abertura; não inventar data histórica desconhecida (Q05).

**validade\_fechado** string YYYY-MM-DD; N. Validade declarada antes da abertura; pode ser sugerida pelo lote, com origem preservada.

**validade\_efetiva** string YYYY-MM-DD; N. Prazo aplicável calculado com abertura e validade fechada; null quando não determinável.

**validade\_desconhecida** boolean; O. Ausência explícita de informação de validade; não significa validade ilimitada.

**validade\_apos\_aberto\_dias** number; N. Prazo inteiro positivo após abertura quando declarado; compõe o mínimo da validade efetiva.

**estado\_fisico\_frasco** string enum; O. Condição do recipiente/conteúdo; quarentena e vencimento são dimensões separadas. Valores: FECHADO,ABERTO,VAZIO,QUEBRADO,DESCARTADO,EXTRAVIADO.

**disponibilidade** string enum; O. Disponível, emprestado ou indisponível; exige coerência com empréstimo ativo. Valores: DISPONIVEL, EMPRESTADO, INDISPONIVEL.

**vencido** boolean; O. Projeção de validade comparada ao relógio do servidor; reavaliar em cada operação crítica.

**em\_quarentena** boolean; O. Bloqueio operacional independente de abertura e vencimento; true exige motivo.

**uso\_vencido\_autorizado** boolean; O. Decisão expressa de gestor para uso excepcional conforme Q04, auditada; pesquisa exige TCR e justificativa.

**detalhe\_status** string; N. Justificativa obrigatória para quarentena e decisões que a exigem.

**cadastrado\_em** Timestamp; O. Data/instante de cadastrado em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**cadastrado\_por** string; O. Valor de cadastrado por, validado pelo formulário e pelo servidor conforme regras do domínio.

**eh\_higroscopico** boolean; O. Snapshot físico denormalizado, imutável, preenchido pelo backend no cadastro a partir de Resumo\_Reagente.eh\_higroscopico (fonte canônica 3FN). Alterações posteriores do resumo não reescrevem frascos existentes. Não aceitar valor do cliente nem atualização direta. Permite aplicar Q06 na devolução sem leitura adicional do resumo. Frascos legados sem snapshot exigem reconciliação explícita antes da operação.

**abertura\_historica\_desconhecida** boolean; O, padrão false. True exige data\_abertura null e estado coerente com abertura anterior, sem inventar data.

### Historico\_Frasco\_Reagente

Caminho: Historico\_Frasco\_Reagente/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_frasco\_reagente** string; O. Identificador da entidade frasco reagente, cuja existência e escopo são verificados no servidor.

**id\_almoxarifado** string; O. Identificador da entidade almoxarifado, cuja existência e escopo são verificados no servidor.

**id\_gestor** string; O. Identificador da entidade gestor, cuja existência e escopo são verificados no servidor.

**tipo** string enum; O. Valor de tipo, validado pelo formulário e pelo servidor conforme regras do domínio.

**campo\_ajustado** string; N. Nome permitido do atributo alterado no evento; não é caminho arbitrário para atualização de documento. Máximo 30 caracteres.

**id\_emprestimo\_reagente** string; N. Identificador da entidade emprestimo reagente, cuja existência e escopo são verificados no servidor.

**peso\_anterior** number; N. Peso bruto em g imediatamente antes do evento de ajuste/pesagem, quando aplicável.

**peso\_novo** number; N. Peso bruto em g confirmado no evento, quando aplicável.

**medida\_ajustada** number; N. Diferença operacional na unidade indicada, com sinal definido pelo tipo de evento; não somar automaticamente como consumo.

**unidade\_medida\_ajustada** string enum; N. Valor de unidade medida ajustada, validado pelo formulário e pelo servidor conforme regras do domínio. Valores: ml,g. (NULL se evento não for quantitativo).

**timestamp** Timestamp; O. Data/instante de timestamp, com ordem temporal validada e autoria do relógio do servidor para eventos.

### **Emprestimo\_Reagente**

Caminho: `Emprestimo_Reagente/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_frasco\_reagente** string; O. Identificador da entidade frasco reagente, cuja existência e escopo são verificados no servidor.

**id\_usuario\_retirou** string; O. Identificador da entidade usuario retirou, cuja existência e escopo são verificados no servidor.

**status** string enum; O. Estado controlado pelo domínio; transições são validadas no servidor. Valores: EM\_USO, ATRASADO, DEVOLVIDO, DEVOLVIDO\_COM\_ATRASO, ENCERRADO\_EXTRAORDINARIO.

**data\_retirada** Timestamp; O. Data/instante de data retirada, com ordem temporal validada e autoria do relógio do servidor para eventos.

**data\_devolucao\_prevista** string civil YYYY-MM-DD (America/Sao\_Paulo); O. O vencimento legal encerra-se às 23:59:59.999 do referido dia civil.

**data\_devolucao\_efetuada** Timestamp; N. Data/instante de data devolucao efetuada, com ordem temporal validada e autoria do relógio do servidor para eventos.

**id\_local\_usado** string; O. Identificador da entidade local usado, cuja existência e escopo são verificados no servidor.

**id\_usuario\_devolveu** string; N. Identificador da entidade usuario devolveu, cuja existência e escopo são verificados no servidor.

**id\_gestor\_devolucao** string; N. Identificador do gestor de almoxarifado operador que recebeu e registrou a devolução na balança, preenchido no retorno.

**peso\_saida** number; O. Peso bruto em g medido na retirada; referência imutável para o consumo deste empréstimo.

**peso\_retorno** number; N. Peso bruto em g medido na devolução; null antes do retorno e validado contra tara/saída.

**medida\_utilizada** number; N. Consumo não negativo calculado na unidade do empréstimo; massa = max(0, saída menos retorno), conversão em mL só para líquido com densidade histórica positiva.

**unidade\_medida\_utilizada** string enum; O. Valor de unidade medida utilizada, validado pelo formulário e pelo servidor conforme regras do domínio. Valores: ml,g.

**peso\_perda\_evaporacao** number; O. Perda em g registrada separadamente de consumo; não inferir arbitrariamente.

**uso\_vencido\_aceito** boolean; O. Ciência explícita do empréstimo específico; não concede nem substitui autorização do gestor.

**finalidade\_uso** string enum; O. Finalidade validada na retirada. Valores: AULA\_PRATICA, DEMONSTRACAO, PESQUISA\_TCC\_POS, ESTUDO\_DEGRADACAO\_RESIDUOS. Uso excepcional de pesquisa exige rastreabilidade TCR de Q04.

**auto\_atendimento** boolean; O, padrão false. Calculado pelo backend conforme Q14, justificativa e notificação à chefia obrigatórias quando true.

**justificativa\_metodologica** string; C no uso excepcional Q04, 20–2000 caracteres não vazios.

**tcr\_versao** string; C em Q04, até 100. Versão institucional aceita, validada pelo backend.

**tcr\_aceito\_em** Timestamp; C em Q04. Instante servidor do aceite.

**tcr\_aceito\_por** string; C em Q04. UID autenticado do solicitante/retirante, não do gestor que atende.

**tcr\_auditoria\_id** string; C em Q04. Evento imutável em Registro\_de\_Auditoria com frasco, operação, finalidade, versão, instante e aceitante; backend verifica correspondência e consumo único do aceite.

**tipo\_encerramento\_excepcional** string enum; N. Valores: QUEBRA\_ACIDENTAL, EXTRAVIO\_SINISTRO.

**motivo\_encerramento\_excepcional** string; N. Justificativa circunstanciada obrigatória em caso de encerramento extraordinário.

**id\_gestor\_encerramento** string; N. UID do gestor que homologou o sinistro.

**massa\_perda\_estimada\_g** number; N. Estimativa gravimétrica de resíduo perdido no sinistro.

## Turma

Caminho: Turma/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_materia** string; O. Identificador da entidade materia, cuja existência e escopo são verificados no servidor.

**id\_professor** string; O. Identificador da entidade professor, cuja existência e escopo são verificados no servidor.

**status** string enum; O. Estado controlado pelo domínio; transições são validadas no servidor. Valores: Ativo, Arquivada.

**nome\_turma** string; O. Nome de apresentação da turma, sem inferir ano/semestre a partir do texto. Máximo 100 caracteres.

**ano** number; O. Ano letivo da turma ou ano da competência do agregado mensal, conforme documento.

**semestre** number; O. Semestre letivo inteiro, exclusivamente 1 ou 2.

**capacidade** number; O. Quantidade máxima ordinária de alunos; inteiro positivo, validado na transação de ingresso.

**codigo\_turma** string; O. Código único de ingresso, não concede acesso sem validação. Máximo 20 caracteres. Unicidade garantida por chave determinística/registro de unicidade no servidor.

**data\_criacao** Timestamp; O. Data/instante de data criacao, com ordem temporal validada e autoria do relógio do servidor para eventos.

**qtd\_alunos** number; O. Contador transacional atômico de alunos matriculados ativos.

**versao** number; O. Versão inteira para detecção de edições concorrentes.

**nome\_materia** string; O. Nome denormalizado da matéria para exibição direta em listagens.

### **Historico\_\_Alunos\_\_Turma**

Caminho: Turma/{id}/HistoricoAlunos/{eventoId}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_turma** string; O. Identificador da entidade turma, cuja existência e escopo são verificados no servidor.

**tipo** string enum; O. Valor de tipo, validado pelo formulário e pelo servidor conforme regras do domínio. Valores: inclusao\_aluno, exclusao\_aluno.

**id\_aluno** string; O. Identificador da entidade aluno, cuja existência e escopo são verificados no servidor.

**timestamp** Timestamp; O. Data/instante de timestamp, com ordem temporal validada e autoria do relógio do servidor para eventos.

**modo\_ingresso** string enum; N. Modo de ingresso na turma (CODIGO ou CONVITE), obrigatório se tipo == "inclusao\_aluno".

**justificativa** string; N. Texto descritivo para exceção de capacidade ou exclusão.

**removido\_por** string; N. UID do operador, obrigatório se tipo == "exclusao\_aluno".

### Historico\_Posts\_Turma

Caminho: Turma/{id}/Posts/{postId}/Historico/{eventoId}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_post** string; O. Identificador da entidade post, cuja existência e escopo são verificados no servidor.

**novo\_titulo** string; N. Valor novo titulo, restrito ao campo correspondente da entidade de origem; preservar ausência legítima. Máximo 150 caracteres.

**novo\_id\_roteiro** string; N. Valor novo id roteiro, restrito ao campo correspondente da entidade de origem; preservar ausência legítima.

**nova\_descricao** string; N. Valor nova descricao, restrito ao campo correspondente da entidade de origem; preservar ausência legítima.

**antigo\_titulo** string; N. Valor antigo titulo, restrito ao campo correspondente da entidade de origem; preservar ausência legítima. Máximo 150 caracteres.

**antigo\_id\_roteiro** string; N. Valor antigo id roteiro, restrito ao campo correspondente da entidade de origem; preservar ausência legítima.

**antiga\_descricao** string; N. Valor antiga descricao, restrito ao campo correspondente da entidade de origem; preservar ausência legítima.

**timestamp** Timestamp; O. Data/instante de timestamp, com ordem temporal validada e autoria do relógio do servidor para eventos.

### Historico\_Comentario

Caminho: Turma/{id}/Posts/{postId}/Comentarios/{id}/Historico/{eventoId}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_comentario** string; O. Identificador da entidade comentario, cuja existência e escopo são verificados no servidor.

**novo\_texto** string; O. Valor novo texto, restrito ao campo correspondente da entidade de origem; preservar ausência legítima.

**texto\_antigo** string; O. Valor de texto antigo, validado pelo formulário e pelo servidor conforme regras do domínio.

**editado\_em** Timestamp; O. Data/instante de editado em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**editado\_por** string; O. Valor de editado por, validado pelo formulário e pelo servidor conforme regras do domínio.

### Roteiro\_Experimento

Caminho: Roteiro\_Experimento/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_professor\_upload** string; O. Identificador da entidade professor upload, cuja existência e escopo são verificados no servidor.

**file\_url** string; O. Referência do PDF; download exige revalidar acesso.

**nome** string; O. Nome de apresentação; não substitui a chave do documento. Máximo 150 caracteres.

**descricao** string; O. Valor de descricao, validado pelo formulário e pelo servidor conforme regras do domínio.

**professores\_compartilhados** array de strings; O, padrão vazio. UIDs autorizados, geridos exclusivamente pelo servidor com arrayUnion/arrayRemove; sem escrita direta cliente.

A associação Roteiro\_Professor\_Compartilhado existe somente no modelo relacional 3FN; no Firestore é o array ACL professores\_compartilhados em Roteiro\_Experimento. Consultar Meus por id\_professor\_upload == uid; Compartilhados comigo por array-contains no UID; Eu compartilhei filtra os roteiros próprios carregados com ACL não vazia.

### Professor

Caminho: Professor/{id}.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

**centro** string enum; O. Centro institucional do professor: CCT, CCTA, CBB ou CCH. Valores: CCT, CCTA, CBB, CCH.

**laboratorio** string; O. Laboratório de lotação do professor; não fornece automaticamente prédio/andar/sala. Máximo 100 caracteres.

## Materia

Caminho: `Materia/{id}`.

**id** `docId` string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**nome** string; O. Nome de apresentação; não substitui a chave do documento. Máximo 100 caracteres.

**codigo\_materia** string; O. Código acadêmico único da matéria, normalizado de modo uniforme. Máximo 10 caracteres.

## Professor\_x\_Materia

Caminho: `Professor_x_Materia/{id}`.

**id\_professor** string; O. Identificador da entidade professor, cuja existência e escopo são verificados no servidor.

**id\_materia** string; O. Identificador da entidade materia, cuja existência e escopo são verificados no servidor.

## Aluno

Caminho: `Aluno/{id}`.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

**numero\_matricula** string; O. Matrícula institucional textual, com zeros iniciais; obrigatória e única em Aluno, opcional e não única em Convite. Máximo 20 caracteres.

## Bolsista

Caminho: `Bolsista/{id}`.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

## Convite\_Aluno

Caminho: `Convite_Aluno/{id}`.

**id** `docId` string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_turma** string; N. Identificador da entidade turma, cuja existência e escopo são verificados no servidor.

**email** string; O. Endereço normalizado para identidade/convite; não divulgar em catálogo público interno. Máximo 150 caracteres.

**convidado\_em** Timestamp; O. Data/instante de convidado em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**status** string enum; O. Estado controlado pelo domínio; transições são validadas no servidor. Valores: pendente, aceitado, expirado.

**expira\_em** Timestamp; O. Instante de expiração de convite/notificação; não equivale a autorização de excluir auditoria.

**convidado\_por** string; O. Valor de convidado por, validado pelo formulário e pelo servidor conforme regras do domínio.

**numero\_matricula** string; N. Matrícula institucional textual, com zeros iniciais; obrigatória e única em Aluno, opcional e não única em Convite. Máximo 20 caracteres.

**token\_hash** string; O. SHA-256 do token aleatório de 32 bytes (CSPRNG); nunca token em claro. Reenvio substitui hash e expiração atomicamente; comparação constante e aceitação única no servidor.

**ultimo\_reenvio\_por** string; N. UID servidor do operador do último reenvio. DocId derivado por HMAC-SHA-256 de contexto e e-mail normalizado com segredo servidor; não usar hash simples público de e-mail.

**exceder\_capacidade** boolean; O, padrão false. Indicador de exceção nominal de capacidade autorizada pelo docente.

**justificativa\_excecao** string; C quando `exceder_capacidade` for true.

**aceitado\_por** string; N. UID do aluno que aceitou o convite.

**aceitado\_em** Timestamp; N. Instante em que o convite foi aceito.

## Notificacao

Caminho: `Usuarios/{uid}/Notificacoes/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_destinatario** string; O. Identificador da entidade destinatario, cuja existência e escopo são verificados no servidor.

**papel\_destinatario** string enum; O. Contexto de exibição da notificação, sem conceder esse papel ao usuário. Inclui Aluno, Professor, Bolsista, Gestor\_Bens\_Patrimoniais, Gestor\_Almoхарifado e Chefe\_Geral.

**id\_quem\_fez\_acao** string; N. Identificador da entidade quem fez acao, cuja existência e escopo são verificados no servidor.

**id\_turma** string; N. Identificador da entidade turma, cuja existência e escopo são verificados no servidor.

**quantidade** number; N. Quantidade agregada de ocorrências agrupadas na notificação, quando aplicável.

**entidade\_alvo** string; O. Tipo autorizado de recurso para navegação; mapear a rotas conhecidas, não URL arbitrária. Máximo 40 caracteres.

**id\_alvo** string; O. ID do recurso da notificação; acesso é revalidado ao abrir. Máximo 200 caracteres.

**lida** boolean; O. Estado de leitura da notificação; true preserva o registro em vez de excluí-lo.

**lida\_em** Timestamp; N. Instante servidor de marcação de leitura; null enquanto não lida.

**emitida\_em** Timestamp; O. Data/instante de emitida em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**expira\_em** Timestamp; N. Instante de expiração de convite/notificação. Condicional: é NULL para notificações que constituem registros operacionais persistentes (ex: ESCASSEZ\_ESTOQUE). Expiração não equivale à exclusão automática sem regra específica.

**tipo** string enum; O. Valor de tipo, validado pelo formulário e pelo servidor conforme regras do domínio. Valores: ADICIONADO, POST, COMENTARIO, REMOVIDO, TURMA\_ARQUIVADA, TURMA\_DESARQUIVADA, REQUISICAO\_BEM, DATA\_DEVOLUCAO\_REAGENTE, ROTEIRO\_COMPARTILHADO, REQUISICAO\_EDICAO\_BEM, REQUISICAO\_ADICAO\_BEM, BEM\_INSERTIVEL, ENTREGA\_ATRASADA, FRASCOS\_VAZIOS, FRASCOS\_QUEBRADOS, FRASCOS\_VENCIDOS, FRASCOS\_A\_SEREM\_PESADOS, FRASCOS\_EM\_QUARENTENA, ESCASSEZ\_ESTOQUE, AUTO\_ATENDIMENTO\_RETIRADA.

**Aluno\_x\_Turma**

Caminho: Turma/{id}/Alunos/{uid}.

**id\_aluno** string; O. Identificador da entidade aluno, cuja existência e escopo são verificados no servidor.

**id\_turma** string; O. Identificador da entidade turma, cuja existência e escopo são verificados no servidor.

**Local**

Caminho: Local/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**predio** string; O. Identificador textual do prédio no cadastro de locais. Máximo 30 caracteres.

**andar** string; O. Identificador textual do piso; permite térreo/subsolo, sem impor número inteiro. Máximo 10 caracteres.

**sala** string; O. Identificador textual da sala, único no contexto prédio/andar. Máximo 30 caracteres.

**criado\_por** string; O. Valor de criado por, validado pelo formulário e pelo servidor conforme regras do domínio.

**criado\_em** Timestamp; O. Data/instante de criado em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**Post**

Caminho: Turma/{id}/Posts/{postId}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_professor** string; O. Identificador da entidade professor, cuja existência e escopo são verificados no servidor.

**id\_turma** string; O. Identificador da entidade turma, cuja existência e escopo são verificados no servidor.

**id\_roteiro\_experimento** string; N. Identificador da entidade roteiro experimento, cuja existência e escopo são verificados no servidor.

**titulo** string; O. Título obrigatório de publicação, exibido no feed e na navegação de notificações. Máximo 150 caracteres.

**descricao** string; O. Valor de descricao, validado pelo formulário e pelo servidor conforme regras do domínio.

**criado\_em** Timestamp; O. Data/instante de criado em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**roteiro\_anexo** map; N. Snapshot imutável da publicação: id\_roteiro (string O), nome\_arquivo (string O, até 150), tamanho\_bytes (inteiro positivo O), storage\_path (string O). Backend valida acesso ao roteiro antes de publicar. Revogação posterior da ACL não remove o anexo histórico. Aluno recebe os metadados no feed; download via obterUrlDownloadRoteiro valida vínculo atual e emite URL assinada por 15 minutos, sem leitura cliente da coleção raiz.

## Comentario

Caminho: Turma/{id}/Posts/{postId}/Comentarios/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_post** string; O. Identificador da entidade post, cuja existência e escopo são verificados no servidor.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

**texto** string; O. Conteúdo textual do comentário, escapado na renderização; rejeitar texto vazio após trim.

**criado\_em** Timestamp; O. Data/instante de criado em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**moderado** boolean; O, padrão false. Visibilidade conforme DP-C02.

**motivo\_moderacao** string; C quando moderado, até 2000 caracteres não vazios.

**moderado\_por** string; C quando moderado. UID autenticado do moderador, preenchido pelo servidor.

**Registro\_de\_Auditoria**

Caminho: `Registro_de_Auditoria/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_usuario** string; O. Identificador da entidade usuario, cuja existência e escopo são verificados no servidor.

**acao** string; O. Nome da ação auditada; distinguir solicitação, conclusão e rejeição quando relevante. Máximo 200 caracteres.

**tipo\_entidade\_sofre\_acao** string enum; O. Valor de tipo entidade sofre acao, validado pelo formulário e pelo servidor conforme regras do domínio.

**id\_do\_objeto\_da\_entidade** string; O. Identificador da entidade do objeto da entidade, cuja existência e escopo são verificados no servidor. Máximo 200 caracteres.

**acao\_feita\_em** Timestamp; O. Data/instante de acao feita em, com ordem temporal validada e autoria do relógio do servidor para eventos.

**metadata** map; O. Mapa auditável e limitado ao evento, sem senha, token ou conteúdo binário.

**Lote\_Materializado**

Caminho: `Lote_Materializado/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_lote** string; O. Identificador da entidade lote, cuja existência e escopo são verificados no servidor. Unicidade garantida por chave determinística/registro de unicidade no servidor.

**qtd\_frascos\_cadastrados** number; O. Contagem de frascos cadastrados, inteira não negativa; não editável pelo cliente quando materializada.

**ultimo\_reconciliador** Timestamp; O (se já reconciliado). Marca temporal absoluta, preenchida/atualizada pelo job de reconciliação. Eventos de trigger gerados antes deste instante são ignorados, evitando que eventos assíncronos retardados somem duplamente sobre a contagem absoluta já aferida.

**Auditoria\_Reconciliacao**

Caminho: Auditoria\_Reconciliacao/{id}.

**id** docId string; O. Chave gerada pelo servidor para o registro da cura.

**id\_lote** string; O. Lote alvo da reconciliação onde ocorreu divergência.

**qtd\_anterior** number; O. Contagem que constava no documento Lote\_Materializado antes da correção (drift).

**qtd\_corrigida** number; O. Contagem absoluta efetivamente encontrada no banco de dados e aplicada (count real).

**data** Timestamp; O. Instante exato em que a correção transacional foi aplicada e a marca d'água elevada.

**Resumo\_Almojarifado\_Diario**

Caminho: Resumo\_Almojarifado\_Diario/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**data** string YYYY-MM-DD; O. Data/instante de data, com ordem temporal validada e autoria do relógio do servidor para eventos.

**id\_almojarifado** string; O. Identificador da entidade almojarifado, cuja existência e escopo são verificados no servidor.

**qtd\_frascos\_fechados\_no\_fim\_do\_dia** number; O. Contagem de frascos fechados no fim do dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_abertos\_no\_fim\_do\_dia** number; O. Contagem de frascos abertos no fim do dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_emprestados\_no\_fim\_do\_dia** number; O. Contagem de frascos emprestados no fim do dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_cadastrados\_fechados\_durante\_o\_dia** number; O. Contagem de frascos cadastrados fechados durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_cadastrados\_abertos\_durante\_o\_dia** number; O. Contagem de frascos cadastrados abertos durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_emprestados\_durante\_o\_dia** number; O. Contagem de frascos emprestados durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_devolvidos\_durante\_o\_dia** number; O. Contagem de frascos devolvidos durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**volume\_total\_usado\_nos\_frascos\_devolvidos\_durante\_o\_dia** number; O. Métrica de volume total usado nos frascos devolvidos durante o dia, não negativa e calculada a partir dos eventos/saldos do domínio.

**volume\_total\_disponivel\_ml** number; O. Métrica de volume total disponivel ml, não negativa e calculada a partir dos eventos/saldos do domínio.

**volume\_utilizado\_no\_dia\_ml** number; O. Métrica de volume utilizado no dia ml, não negativa e calculada a partir dos eventos/saldos do domínio.

**massa\_total\_disponivel\_g** number; O. Métrica de massa total disponivel g, não negativa e calculada a partir dos eventos/saldos do domínio.

**massa\_utilizada\_no\_dia\_g** number; O. Métrica de massa utilizada no dia g, não negativa e calculada a partir dos eventos/saldos do domínio.

**created\_at** Timestamp; O. Instante servidor da primeira criação da materialização.

**updated\_at** Timestamp; O. Instante servidor da última atualização da materialização; informa atualidade do resultado.

### **Resumo\_Reagente\_Diario**

Caminho: `Resumo_Reagente_Diario/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**data** string YYYY-MM-DD; O. Data/instante de data, com ordem temporal validada e autoria do relógio do servidor para eventos.

**id\_resumo\_reagente** string; O. Identificador da entidade resumo reagente, cuja existência e escopo são verificados no servidor.

**id\_almoxarifado** string; O. Identificador da entidade almoxarifado, cuja existência e escopo são verificados no servidor.

**qtd\_frascos\_fechados\_disponiveis** number; O. Contagem de frascos fechados disponiveis, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_abertos\_disponiveis** number; O. Contagem de frascos abertos disponiveis, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_em\_uso** number; O. Contagem de frascos em uso, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_vazios** number; O. Contagem de frascos vazios, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_descartados\_dia** number; O. Contagem de frascos descartados dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_extraviados\_dia** number; O. Contagem de frascos extraviados no dia, inteira não negativa; não editável pelo cliente quando materializada.

**volume\_total\_disponivel\_ml** number; O. Métrica de volume total disponivel ml, não negativa e calculada a partir dos eventos/saldos do domínio.

**volume\_utilizado\_no\_dia\_ml** number; O. Métrica de volume utilizado no dia ml, não negativa e calculada a partir dos eventos/saldos do domínio.

**volume\_evaporado\_no\_dia\_ml** number; O. Métrica de volume evaporado no dia ml, não negativa e calculada a partir dos eventos/saldos do domínio.

**massa\_total\_disponivel\_g** number; O. Métrica de massa total disponivel g, não negativa e calculada a partir dos eventos/saldos do domínio.

**massa\_utilizada\_no\_dia\_g** number; O. Métrica de massa utilizada no dia g, não negativa e calculada a partir dos eventos/saldos do domínio.

**massa\_evaporada\_no\_dia\_g** number; O. Métrica de massa evaporada no dia g, não negativa e calculada a partir dos eventos/saldos do domínio.

**created\_at** Timestamp; O. Instante servidor da primeira criação da materialização.

**updated\_at** Timestamp; O. Instante servidor da última atualização da materialização; informa atualidade do resultado.

## Resumo\_Bem\_Patrimonial\_Diario

Caminho: `Resumo_Bem_Patrimonial_Diario/{id}`.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**data** string YYYY-MM-DD; O. Data/instante de data, com ordem temporal validada e autoria do relógio do servidor para eventos.

**id\_local** string; N. Identificador da entidade local, cuja existência e escopo são verificados no servidor.

**qtd\_bens\_ativos\_no\_fim\_do\_dia** number; O. Contagem de bens ativos no fim do dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_bens\_inservivel\_no\_fim\_do\_dia** number; O. Contagem de bens inservivel no fim do dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_bens\_dado\_baixa\_no\_fim\_do\_dia** number; O. Contagem de bens dado baixa no fim do dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_bens\_cadastrados\_durante\_o\_dia** number; O. Contagem de bens cadastrados durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_bens\_marcados\_inservivel\_durante\_o\_dia** number; O. Contagem de bens marcados inservivel durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_bens\_dados\_baixa\_durante\_o\_dia** number; O. Contagem de bens dados baixa durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_requisicoes\_abertas\_durante\_o\_dia** number; O. Contagem de requisicoes abertas durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_requisicoes\_aprovadas\_durante\_o\_dia** number; O. Contagem de requisicoes aprovadas durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_requisicoes\_rejeitadas\_durante\_o\_dia** number; O. Contagem de requisicoes rejeitadas durante o dia, inteira não negativa; não editável pelo cliente quando materializada.

**created\_at** Timestamp; O. Instante servidor da primeira criação da materialização.

**updated\_at** Timestamp; O. Instante servidor da última atualização da materialização; informa atualidade do resultado.

### **Atividade\_Gestor\_Almoхарifado\_Mensal**

Caminho: Atividade\_Gestor\_Almoхарifado\_Mensal/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_gestor** string; O. Identificador da entidade gestor, cuja existência e escopo são verificados no servidor.

**ano** number; O. Ano letivo da turma ou ano da competência do agregado mensal, conforme documento.

**mes** number; O. Mês da competência mensal, inteiro de 1 a 12.

**qtd\_reagentes\_adicionados** number; O. Contagem de reagentes adicionados, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_frascos\_adicionados** number; O. Contagem de frascos adicionados, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_movimentacoes\_registradas** number; O. Contagem de movimentacoes registradas, inteira não negativa; não editável pelo cliente quando materializada.

**created\_at** Timestamp; O. Instante servidor da primeira criação da materialização.

**updated\_at** Timestamp; O. Instante servidor da última atualização da materialização; informa atualidade do resultado.

### **Atividade\_Gestor\_Bens\_Patrimoniais\_Mensal**

Caminho: Atividade\_Gestor\_Bens\_Patrimoniais\_Mensal/{id}.

**id** docId string; O. Chave do documento, gerada pelo servidor; não duplicar como contador relacional.

**id\_gestor** string; O. Identificador da entidade gestor, cuja existência e escopo são verificados no servidor.

**ano** number; O. Ano letivo da turma ou ano da competência do agregado mensal, conforme documento.

**mes** number; O. Mês da competência mensal, inteiro de 1 a 12.

**qtd\_bens\_adicionados** number; O. Contagem de bens adicionados, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_bens\_editados** number; O. Contagem de bens editados, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_requisicoes\_aprovadas** number; O. Contagem de requisicoes aprovadas, inteira não negativa; não editável pelo cliente quando materializada.

**qtd\_requisicoes\_rejeitadas** number; O. Contagem de requisicoes rejeitadas, inteira não negativa; não editável pelo cliente quando materializada.

**created\_at** Timestamp; O. Instante servidor da primeira criação da materialização.

**updated\_at** Timestamp; O. Instante servidor da última atualização da materialização; informa atualidade do resultado.

### Complementos físicos obrigatórios e fontes de verdade.

**Usuarios** ativo (boolean, O), **versao\_permissoes** (inteiro, O), **claims\_pendentes** (boolean, O), **atualizado\_em** (Timestamp, O): controle servidor de ativação e sincronização de autorização. Não replicar papéis editáveis no navegador.

**Perfis** docId = UID em Chefe\_Geral, Professor, Aluno, Bolsista e gestores; **id\_usuario** é esse UID. **nome**, **email** e **letra\_inicial**, se usados em busca de perfil, são projeções de Usuarios, com leitura restrita e atualização pelo servidor.

**Almoxarifado** ativo (boolean, O) e **qtd\_gestores\_ativos** (inteiro não negativo, O) são necessários à ativação e revogação concorrente. O contador é derivado dos vínculos; ativo exige contador positivo.

**Turma** **qtd\_alunos** (inteiro não negativo, O) é contador transacional do vínculo ativo, não count lido fora da transação. **versao** (inteiro positivo, O) permite detectar edições concorrentes.

**Espelho Aluno–Turma** Usuarios/uid/Turmas/turmaId: **id\_turma**, **id\_professor**, **id\_materia**, **nome\_turma**, **nome\_materia** (strings O), **ano**, **semestre** (inteiros O), **status** (enum O), **ingressou\_em** (Timestamp O). Todos derivados de Turma/Matéria ou evento de ingresso; não usar espelho desatualizado como autorização. Vínculo canônico em Turma/id/Alunos/uid contém **id\_aluno** (string O), **ingressou\_em** (Timestamp O); nome/foto para colegas são projeções mínimas, sem matrícula/e-mail.

**Reagentes** Resumo possui `letra_inicial` (string O, derivada do nome). O estado físico escalar e a higroscopicidade são canônicos no resumo; filtro por igualdade. Especificação embute `composicao` (array de maps com os campos da composição; IDs de substâncias únicos). O resumo pai fornece seu ID; manter ID de resumo em Lote e Frasco para resolver o caminho completo da especificação.

**Lote e Frasco** `id_resumo_reagente`, `id_especificacao_reagente`, `nome_reagente` (strings O) são projeções servidor; em Frasco, especificação permanece resolvida mesmo com lote, como desnormalização deliberada. `unidade_medida` (ml/g, O), `localizacao_detalhada` (string até 500, N), `versao` (inteiro O), `ultima_pesagem_em` (Timestamp, N) e `modalidade_cadastro` (FECHADO/CONHECE\_TARA/ESTIMA\_VOLUME/ESTIMA\_MASSA, O) sustentam UI-06. `id_emprestimo_ativo` (string N) é ponteiro transacional, limpo no retorno. `id_almoxarifado` é fonte da localização operacional, nunca aceito sem escopo.

**Empréstimo** `id_almoxarifado`, `id_resumo_reagente`, `id_especificacao_reagente` (strings O), `densidade_aplicada` (number positiva C para líquido), `id_gestor_retirada` (string O), `id_gestor_devolucao` (string N até retorno). Snapshot químico da retirada preserva a memória de cálculo; não atualizar retroativamente. Retirante, devolvente e operador são papéis distintos.

**Patrimônio** `nome Equipamento`, `letra_inicial_nome`, `predio`, `andar`, `sala` (strings O) são projeções de Resumo/Local. `versao` (inteiro O) detecta conflitos. Histórico embute `alteracoes` (array de maps campo/valor anterior/novo, admitindo null para ausência) e snapshot do local quando necessário a relatório histórico; não propagar local atual sobre fatos passados.

**Requisições** Adição inclui `justificativa_resposta` (string N, obrigatória após resposta). Edição inclui `versao_bem_origem` (inteiro O), `novo_id_resumo_bem_patrimonial` (string N, reclassificação) e `novo_documento_baixa` (string N, C para proposta de baixa). Projeções nome/plaqueta são identificadores de leitura, não autorização. Resposta nunca inventa foto ausente.

**Convite** `exceder_capacidade` (boolean O, default false) e `justificativa_excecao` (string C quando true), `aceitado_por` (string N), `aceitado_em` (Timestamp N), `token_hash` (string O, SHA-256 do token aleatório). Nunca guardar token em claro. Status aceitado exige UID e instante; matrícula obrigatória na identidade final.

**Arquivos** Roteiros, fotos e comprovantes mantêm `storage_path` (string O quando upload), `content_type` (string O), `tamanho_bytes` (inteiro positivo O), `owner_`

`uid` (string O), `criado_em` (Timestamp O) no registro/metadados aplicável. `file_url/photo_url` são referências legadas; URL assinada transitória não é fonte de autorização nem conteúdo persistente. Arquivo e Firestore não têm transação única: usar upload provisório e reconciliação de órfãos, preservando anexos referenciados.

**Etiquetas** `Contador_Codigo_Frasco/singleton`: `ultimo_codigo_gerado` (inteiro não negativo O). Não criar simultaneamente Contadores/codigo\_frasco. `Impressao_Etiqueta_Frasco` acrescenta `start_row` (1–10), `start_col` (1–3), `id_operacao` (string O) e `status_geracao` (SOLICITADA/CONCLUIDA/FALHOU). Segunda via fica na auditoria; metadata inclui operação, frasco, motivo e resultado, sem duplicar registro como lote virgem.

**Locks** `Locks_Requisicao_Patrimonio` contém `id_requisicao`, `tipo` (ADICAO/EDICAO), `chave_recurso` (string O) e `criado_em` (Timestamp O). Lock e requisição são atômicos; limpeza só após verificar inexistência de pendência vinculada.

**Unicidade técnica** `Chaves_Unicas/chaveHash`: `tipo`, `id_recurso` (strings O), `criado_em` (Timestamp O). Chave determinística derivada do valor normalizado e domínio (ex: `Bem_Patrimonial__{numero}` para plaqueta, matrícula, CAS, matéria, local, código turma, lote ou convite pendente); nunca expor e-mail na chave. Criar/liberar na mesma transação do recurso. A implementação exige **backfill** populando esta coleção para todos os registros preexistentes antes de ativar a verificação transacional.

**Deduplicação de Eventos** `Eventos_Processados/eventId`: `processado_em` (Timestamp O). Usado em transações originadas por triggers (ex: `onDocumentCreated`) para converter a entrega "at-least-once" do Cloud Functions em processamento "exactly-once".

**Operações técnicas** `Operacoes/id`: `uid`, `tipo_operacao`, `payload_hash` (strings O, hash SHA-256 canônico do payload), `status` (PENDENTE/CONCLUIDA/FALHOU), `resultado` (map N), `criado_em`, `atualizado_em` (Timestamp O). Chave é vinculada ao ator/ação e payload; repetição com entrada diferente é rejeitada com `ALREADY_EXISTS` (idempotência semântica rigorosa). Leitura restrita ao solicitante por resposta mínima; escrita servidor.

**Controle de papéis** `Controle_Papeis/singleton`: `chefes_ativos`, `gestores_patrimoniais_ativos` (inteiros não negativos O), `versao` (inteiro O). Atualizado

na mesma transação das concessões/revogações para impedir retirada concorrente do último responsável.

**Materializações** Chaves por lote; data+almoxarifado; data+resumo+almoxarifado; data+local (sentinela GERAL); gestor+ano+mês. Todos os campos de métricas do dicionário são calculados no servidor. Acrescentar `calculado_em` (Timestamp O) e `versao_calculo` (inteiro O); reprocessar a mesma janela substitui resultado ou deduplica eventos, sem somar duas vezes. Não somar linha GERAL às linhas locais. Métrica legada de volume total usado é alias do consumo ml e não uma parcela adicional.

**Consultas e integridade do dicionário.** A entidade 3FN mantém dependências e referências canônicas; desnormalização é exceção documentada para consulta, não replicação automática de toda FK. Atualização transacional é obrigatória em vínculos, saldos e exclusões lógicas; propagação de nomes pode ser assíncrona, com versão da origem e reconciliação. Nunca aplicar trigger de nome atual a snapshot histórico. Histórico por bem admite collection-group autorizado para relatórios, portanto subcoleção não impede consulta global.

As consultas UI exigem índices versionados em `firestore.indexes.json`: empréstimo por `almoxarifado/status/data` prevista e por `retirante/data`; histórico de frasco por `frasco/timestamp` e `almoxarifado/timestamp`; requisição por `solicitante/status/feita_em`; turma por `professor/status/ano`; notificações por `papel/lida/emitida_em`; resumo diário por `escopo/data`. Acrescentar `docId` como desempate da paginação. Combinações opcionais de filtros precisam de inventário de consultas e validação em emulador/ambiente de integração, não promessa de qualquer combinação arbitrária.

## 5.10 Estratégia de Busca Textual no Firestore

### i Por que não existe LIKE no Firestore

O Firestore não oferece busca por substring (sem LIKE, sem `icontains`). A estratégia adotada é: aplicar **filtros obrigatórios de igualdade** no Firestore para reduzir drasticamente o conjunto de documentos retornados, e então aplicar **substring no frontend** sobre os poucos resultados restantes.

### 5.10.1 Reagentes

O usuário deve obrigatoriamente escolher ao menos um dos filtros: estado físico (**SOLIDO** ou **LIQUIDO**), natureza química (**ORGANICO**, **INORGANICO**, **ELEMENTO** ou **HIBRIDO**), tipo de substância (**PURA** ou **MISTURA**) e/ou letra inicial (campo `letra_inicial` denormalizado).

A query Firestore usa esses filtros de igualdade e retorna poucos documentos; o frontend aplica a busca por substring sobre o campo `nome`.

### 5.10.2 Bens Patrimoniais

Combinação obrigatória de pelo menos um: (Prédio) ou (letra inicial do nome do equipamento) ou (Status). O Firestore filtra com os campos denormalizados `predio`, `andar`, `sala`, `status` e `letra_inicial_nome`. O frontend aplica substring sobre `nome_equipamento` e `numero_patrimonio` nos resultados. Bigramas não serão implementados na V1: o filtro de (Prédio + Andar) já reduz os candidatos suficientemente.

### 5.10.3 Alunos

Busca via campo denormalizado `letra_inicial` (filtro de igualdade). O frontend aplica substring sobre `nome` nos resultados. A *range query* por prefixo (`nome >= "A"`, `nome < "B"`) é alternativa válida, mas não combina facilmente com outros filtros de igualdade em índice composto no Firestore; o campo `letra_inicial` é preferível.

### 5.10.4 Professores

Poucos no sistema (dezenas). Usar listagem autorizada e paginada com campos mínimos; filtrar por substring apenas os registros carregados. UI-13 define cache e ciclo de vida.

#### Listeners e cache com escopo

Aplicar UI-13: listeners limitados às telas visíveis, histórico paginado, memória por padrão e persistência somente em dispositivo confiável. Cache não garante atualização nem ausência de novas leituras faturadas ao reconectar. Encerrar listeners e limpar dados ao sair ou trocar identidade.

## 5.11 Espelhamento Bidirecional da Associação Aluno–Turma

- **Justificativa NoSQL:** No modelo 3FN, a tabela associativa `Aluno_x_Turma` responde bidirecionalmente às consultas via chave composta. No Firestore, como subcoleções não podem ser filtradas a partir da coleção pai (`Turma`), e regras de segurança não atuam como filtros de busca, a relação  $N : N$  é intencionalmente denormalizada e espelhada em duas estruturas complementares:
  1. **`Turma/{turmaId}/Alunos/{alunoId}` (Visão da Turma):** Utilizada pelo professor para gerenciar os matriculados e validar a regra de capacidade máxima (`qtd_alunos < capacidade`, atualizado na transação).
  2. **`Usuarios/{uid}/Turmas/{turmaId}` (Visão do Aluno):** Utilizada exclusivamente pelo frontend do Aluno para alimentar o painel lateral de “Minhas

Turmas” via listener em tempo real (`onSnapshot`), contendo os metadados essenciais (`nome_turma`, `nome_materia`, `ano`, `semestre`, `id_professor`).

- **Consistência Transacional:** A criação e a remoção desse espelho são executadas de forma estritamente atômica pelo backend nas Cloud Functions. Na matrícula (`ingressarEmTurmaPorCodigo`), o espelho é criado. Na remoção (`removerAlunoTurma`), a transação deve abranger: exclusão de `Turma/{id\}/Alunos/{uid\}`, exclusão de `Usuarios/{uid\}/Turmas/{turmaId\}`, gravação no histórico da turma, decremento do contador de vagas e gravação da auditoria.

## 6 Materialized (Views)

### 6.1 Tabela Materialized para Lote

#### Lote\_Materializado

|                                |           |                     |
|--------------------------------|-----------|---------------------|
| <i>id</i>                      | SERIAL    | PK                  |
| <i>id_lote</i>                 | INTEGER   | FK, UNIQUE          |
| <i>qtd_frascos_cadastrados</i> | INTEGER   | DEFAULT 0, NOT NULL |
| <i>ultimo_reconciliador</i>    | TIMESTAMP | NULL                |

#### i Observação

Corrigido o typo SERAIL → SERIAL. *id\_lote* recebeu UNIQUE: esta é uma tabela 1:1 com *Lote* (uma linha por lote), então o par (*id*, *id\_lote*) sem essa restrição permitiria duas linhas materializadas divergentes para o mesmo lote. Mantida **exclusivamente pela transação síncrona** da *Cloud Function* a cada frasco cadastrado, sem depender de triggers assíncronas. Adicionada a marca d'água *ultimo\_reconciliador* que é preenchida por *jobs* agendados de reconciliação para bloquear o duplo cômputo de eventos atrasados (drift silenciado, ver seção *Tecnologia e Relatórios*).

### 6.2 Tabela Resumo de Almojarifado - Diário

#### Resumo\_Almojarifado\_Diario

|                                                                |         |                     |
|----------------------------------------------------------------|---------|---------------------|
| <i>id</i>                                                      | SERIAL  | PK                  |
| <i>data</i>                                                    | DATE    | NOT NULL            |
| <i>id_almojarifado</i>                                         | INTEGER | FK, NOT NULL        |
| <i>qtd_frascos_fechados_no_fim_do_dia</i>                      | INTEGER | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_abertos_no_fim_do_dia</i>                       | INTEGER | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_emprestados_no_fim_do_dia</i>                   | INTEGER | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_cadastrados_fechados_durante_o_dia</i>          | INTEGER | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_cadastrados_abertos_durante_o_dia</i>           | INTEGER | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_emprestados_durante_o_dia</i>                   | INTEGER | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_devolvidos_durante_o_dia</i>                    | INTEGER | DEFAULT 0, NOT NULL |
| <i>volume_total_usado_nos_frascos_devolvidos_durante_o_dia</i> |         |                     |

|                                   |               |                                   |
|-----------------------------------|---------------|-----------------------------------|
|                                   | NUMERIC(12,2) | DEFAULT 0, NOT NULL – LEGADO N-07 |
| <i>volume_total_disponivel_ml</i> | NUMERIC(12,2) | DEFAULT 0, NOT NULL               |
| <i>volume_utilizado_no_dia_ml</i> | NUMERIC(12,2) | DEFAULT 0, NOT NULL               |
| <i>massa_total_disponivel_g</i>   | NUMERIC(12,2) | DEFAULT 0, NOT NULL               |
| <i>massa_utilizada_no_dia_g</i>   | NUMERIC(12,2) | DEFAULT 0, NOT NULL               |
| <i>created_at</i>                 | TIMESTAMP     | NOT NULL                          |
| <i>updated_at</i>                 | TIMESTAMP     | NOT NULL                          |

**i Separação Física Rigorosa: Massa (*g*) vs Volume (*ml*)**

Esta tabela consolida uma distinção única e consistente entre **estoque** (...*\_no\_fim\_do\_dia*, fotografia às 23h59) e **fluxo** (...*\_durante\_o\_dia*, eventos ocorridos naquele dia). Para eliminar distorções métricas em sólidos com densidade desconhecida ou nula, as colunas agregadas de saldo e consumo são formalmente segregadas nas duas dimensões físicas fundamentais: reagentes líquidos acumulam em *volume\_...\_ml* e reagentes sólidos acumulam em *massa\_...\_g*. Nunca se tenta converter gravimetria de sólidos para volume sem densidade aferida.

**N-07 – campo legado:** *volume\_total\_usado\_nos\_frascos\_devolvidos\_durante\_o\_dia* pré-existe à separação massa vs. volume e mistura unidades. Sua semântica é coberta por *volume\_utilizado\_no\_dia\_ml* e *massa\_utilizada\_no\_dia\_g*. Não preencher em novos agregadores; manter apenas para compatibilidade de relatórios históricos até a migração.

### 6.3 Tabela Resumo de Reagente/Solvente - Diário

#### Resumo\_Reagente\_Diario

|                                         |               |                     |
|-----------------------------------------|---------------|---------------------|
| <i>id</i>                               | SERIAL        | PK                  |
| <i>data</i>                             | DATE          | NOT NULL            |
| <i>id_resumo_reagente</i>               | INTEGER       | FK, NOT NULL        |
| <i>id_almojarifado</i>                  | INTEGER       | FK, NOT NULL        |
| <i>qtd_frascos_fechados_disponiveis</i> | INTEGER       | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_abertos_disponiveis</i>  | INTEGER       | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_em_uso</i>               | INTEGER       | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_vazios</i>               | INTEGER       | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_descartados_dia</i>      | INTEGER       | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_extraviados_dia</i>      | INTEGER       | DEFAULT 0, NOT NULL |
| <i>volume_total_disponivel_ml</i>       | NUMERIC(12,2) | DEFAULT 0, NOT NULL |
| <i>volume_utilizado_no_dia_ml</i>       | NUMERIC(12,2) | DEFAULT 0, NOT NULL |
| <i>volume_evaporado_no_dia_ml</i>       | NUMERIC(12,2) | DEFAULT 0, NOT NULL |
| <i>massa_total_disponivel_g</i>         | NUMERIC(12,2) | DEFAULT 0, NOT NULL |
| <i>massa_utilizada_no_dia_g</i>         | NUMERIC(12,2) | DEFAULT 0, NOT NULL |
| <i>massa_evaporada_no_dia_g</i>         | NUMERIC(12,2) | DEFAULT 0, NOT NULL |
| <i>created_at</i>                       | TIMESTAMP     | NOT NULL            |
| <i>updated_at</i>                       | TIMESTAMP     | NOT NULL            |

#### i Contratos de Materialização e Saldo

Para garantir a exatidão das métricas agregadas e evitar divergências com a Seção 4:

- **Frascos Extraviados:** Frascos com estado **EXTRAVIADO** **nunca** são contabilizados no saldo disponível, seja na quantidade de frascos ou na massa/volume total.
- **Conteúdo Nominal vs. Saldo:** O *conteudo\_nominal* (capacidade original do fabricante) **não representa** o saldo atual do frasco e não deve ser somado no lugar de saldos medidos.
- **Saldos Desconhecidos:** Se o saldo de um frasco é desconhecido (ex.: frasco aberto sem *peso\_atual* preciso registrado), ele não deve ser somado como zero (0) nos agregados totais de volume ou massa, pois isso distorceria a

realidade física. Esses frascos devem ser segregados na camada de agregação, distinguindo claramente a “quantidade conhecida” da “quantidade com saldo desconhecido”.

## 6.4 Tabela Resumo de Bens Patrimoniais - Diário

### Resumo\_Bem\_Patrimonial\_Diario

|                                                   |           |                     |
|---------------------------------------------------|-----------|---------------------|
| <i>id</i>                                         | SERIAL    | PK                  |
| <i>data</i>                                       | DATE      | NOT NULL            |
| <i>id_local</i>                                   | INTEGER   | FK, NULL            |
| <i>qtd_bens_ativos_no_fim_do_dia</i>              | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_bens_inservivel_no_fim_do_dia</i>          | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_bens_dado_baixa_no_fim_do_dia</i>          | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_bens_cadastrados_durante_o_dia</i>         | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_bens_marcados_inservivel_durante_o_dia</i> | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_bens_dados_baixa_durante_o_dia</i>         | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_requisicoes_abertas_durante_o_dia</i>      | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_requisicoes_aprovadas_durante_o_dia</i>    | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_requisicoes_rejeitadas_durante_o_dia</i>   | INTEGER   | DEFAULT 0, NOT NULL |
| <i>created_at</i>                                 | TIMESTAMP | NOT NULL            |
| <i>updated_at</i>                                 | TIMESTAMP | NOT NULL            |

## 6.5 Tabela de Atividade Mensal - Gestor de Almoxarifado

### Atividade\_Gestor\_Almoxarifado\_Mensal

|                                      |           |                     |
|--------------------------------------|-----------|---------------------|
| <i>id</i>                            | SERIAL    | PK                  |
| <i>id_gestor</i>                     | INTEGER   | FK, NOT NULL        |
| <i>ano</i>                           | INTEGER   | NOT NULL            |
| <i>mes</i>                           | INTEGER   | NOT NULL            |
| <i>qtd_reagentes_adicionados</i>     | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_frascos_adicionados</i>       | INTEGER   | DEFAULT 0, NOT NULL |
| <i>qtd_movimentacoes_registradas</i> | INTEGER   | DEFAULT 0, NOT NULL |
| <i>created_at</i>                    | TIMESTAMP | NOT NULL            |

*updated\_at*

TIMESTAMP

NOT NULL

**Regra de negócio***UNIQUE(id\_gestor, ano, mes).***6.6 Tabela de Atividade Mensal - Gestor de Bens Patrimoniais****Atividade\_Gestor\_Bens\_Patrimoniais\_Mensal***id*

SERIAL

PK

*id\_gestor*

INTEGER

FK, NOT NULL

*ano*

INTEGER

NOT NULL

*mes*

INTEGER

NOT NULL

*qtd\_bens\_adicionados*

INTEGER

DEFAULT 0, NOT NULL

*qtd\_bens\_editados*

INTEGER

DEFAULT 0, NOT NULL

*qtd\_requisicoes\_aprovadas*

INTEGER

DEFAULT 0, NOT NULL

*qtd\_requisicoes\_rejeitadas*

INTEGER

DEFAULT 0, NOT NULL

*created\_at*

TIMESTAMP

NOT NULL

*updated\_at*

TIMESTAMP

NOT NULL

**Regra de negócio***UNIQUE(id\_gestor, ano, mes).***6.7 Catálogo de Métricas Disponíveis por Domínio**

- ***Historico\_Frasco\_Reagente*** (por *tipo*): frascos cadastrados, saídas, entradas, esvaziados, quebrados, descartados, vencidos, ajustes de inventário e evaporação – por dia → alimenta *Resumo\_Almojarifado\_Diario* e *Resumo\_Reagente\_Diario*; por gestor/mês → alimenta *Atividade\_Gestor\_Almojarifado\_Mensal*; contagem "agora" → `count()` direto sobre *Frasco\_Reagente* filtrado por *vencido = true*.
- ***Emprestimo\_Reagente***: empréstimos abertos/devolvidos/atrasados por dia, volume e massa total consumida por dia → alimenta os resumos diários de reagente.
- ***Historico\_Bem\_Patrimonial* / *Alteracao\_Bem\_Patrimonial***: bens cadastrados, editados e baixados por dia → alimenta *Resumo\_Bem\_Patrimonial\_Diario*.
- ***Requisicao\_Edicao\_Bem\_Patrimonial* / *Requisicao\_Adicao\_Bem\_Patrimonial***: requisições de bens cadastrados, editados e baixados por dia → alimenta *Resumo\_Requisicao\_Bem\_Patrimonial\_Diario*.

**Patrimonial:** requisições abertas/aprovadas/rejeitadas por dia → alimenta *Resumo\_Bem\_Patrimonial\_Diario*.

- **Registro\_de\_Auditoria:** log genérico entre todas as entidades.

## 7 Requisitos e Regras de Negócio

### 7.1 Requisitos funcionais

| ID   | Requisito                                                                                                   |
|------|-------------------------------------------------------------------------------------------------------------|
| RF01 | O sistema deve permitir login por e-mail e senha.                                                           |
| RF02 | O sistema deve permitir login com Google através do provedor de autenticação configurado.                   |
| RF03 | O sistema deve permitir recuperação de senha.                                                               |
| RF04 | O sistema deve permitir que um usuário possua múltiplos papéis.                                             |
| RF05 | O sistema deve permitir alternância de papel ativo no dashboard quando houver mais de um papel autorizado.  |
| RF06 | O sistema deve permitir cadastro e manutenção de bens patrimoniais conforme permissões.                     |
| RF07 | O sistema deve manter histórico auditável de alterações patrimoniais.                                       |
| RF08 | O sistema deve permitir requisição de adição de patrimônio.                                                 |
| RF09 | O sistema deve permitir requisição de edição de patrimônio.                                                 |
| RF10 | O sistema deve impedir mais de uma requisição de edição pendente para o mesmo bem.                          |
| RF11 | O sistema deve permitir aprovação ou rejeição de requisições.                                               |
| RF12 | O sistema deve permitir registro da baixa de um bem após o procedimento institucional.                      |
| RF13 | O sistema deve permitir cadastro de almoxarifados.                                                          |
| RF14 | O sistema deve calcular pesos e volumes de frascos de reagentes utilizando a densidade.                     |
| RF15 | O sistema deve registrar empréstimo, devolução, descarte, esvaziamento e quebra de frascos.                 |
| RF16 | O sistema deve permitir consulta por nome, número de patrimônio, status, local e demais filtros aplicáveis. |
| RF17 | O sistema deve permitir criação de turmas com capacidade máxima estabelecida.                               |
| RF18 | O sistema deve permitir entrada de alunos em turmas por código ou e-mail.                                   |

| ID   | Requisito (continuação)                                                                      |
|------|----------------------------------------------------------------------------------------------|
| RF19 | O sistema deve permitir criação de posts em turmas pelo professor responsável.               |
| RF20 | O sistema deve permitir comentários em posts conforme regras de acesso.                      |
| RF21 | O sistema deve permitir upload de roteiros em PDF.                                           |
| RF22 | O sistema deve permitir compartilhamento de roteiros entre professores.                      |
| RF23 | O sistema deve permitir vínculo de roteiro a turma por meio de post.                         |
| RF24 | O sistema deve gerar relatórios mensais e por período, incluindo consumo em mL e massa em g. |
| RF25 | O sistema deve manter dados históricos e de auditoria sem apagar fatos relevantes.           |

## 7.2 Regras de Negócio Gerais

### 7.2.1 Status de um frasco de reagente

#### Regra de negócio

Quando um frasco de reagente é cadastrado como em quarentena, deve ser obrigatório escrever o detalhe do status explicando o motivo dele estar em quarentena. Um frasco no estado de quarentena pode ser descartado ou voltar ao estado disponível (FECHADO + DISPONIVEL ou ABERTO + DISPONIVEL), conforme a máquina de estados da Seção 4.

### 7.2.2 Seleção do tipo em Resumo de Reagente

#### Regra de negócio

Ao cadastrar uma *Especificacao\_Reagente*, o estado físico do resumo (SOLIDO ou LIQUIDO) determina quais campos operacionais de frasco serão apresentados. O tipo de substância (PURA ou MISTURA) determina as propriedades químicas obrigatórias, conforme a matriz de obrigatoriedade. DP-A01: estado físico e higroscopicidade pertencem ao resumo; unidade é derivada e densidade permanece na especificação.

### 7.2.3 Excluindo um Aluno de uma turma

#### Regra de negócio

Quando o professor exclui um aluno de uma turma, ele não apaga o aluno do sistema. Apenas o retira da turma. Seu histórico nos posts é mantido. Ou seja, os alunos que ainda estão na turma conseguem ver os comentários dele e também seu nome e foto de perfil nos comentários dele.

### 7.2.4 Chefe Geral

#### Regra de negócio

O sistema pode ter mais de um chefe geral, isso não altera o sistema.

### 7.2.5 O responsável pelo bem patrimonial não é um Usuário do sistema LCQUI

#### Regra de negócio

Por isso que "*nome\_responsavel\_sei*" é um simples texto.

### 7.2.6 Histórico de Posts na Turma

#### Regra de negócio

A entidade *Historico\_Posts\_Turma* registra no campo *editado\_por* o identificador do usuário que realizou a edição. Normalmente é o Professor dono da turma, mas o Chefe Geral também pode editar ou remover posts em caráter excepcional de moderação institucional (Q13); nesses casos o campo *editado\_por* identifica a ação da chefia e o campo *acao* em *Registro\_de\_Auditoria* registra a motivação. O *id\_turma* não é armazenado nesta entidade 3FN porque pode ser obtido por *Historico\_Posts\_Turma.id\_post* → *Post.id\_turma*; se a consulta direta por turma for necessária no Firestore, esse vínculo poderá ser denormalizado na coleção de histórico.

### 7.2.7 Não compartilhar o mesmo roteiro 2 vezes com um professor

#### Regra de negócio

Na tabela: *Roteiro\_Professor\_Compartilhado*  
UNIQUE(*id\_roteiro\_experimento*, *id\_professor*)

### 7.2.8 Densidade e concentração na especificação do reagente

#### Regra de negócio

A densidade é uma propriedade da *Especificacao\_Reagente*, e não do *Resumo\_Reagente*. Ela deve ser informada quando obrigatória para a especificação e, após a confirmação cadastral, não pode ser alterada diretamente, pois participa dos cálculos de volume e consumo dos frascos. A concentração pertence à composição da especificação. Duas especificações com o mesmo nome comercial podem ter densidade, concentração, fabricante ou código de produto diferentes e, portanto, são especificações distintas sob o mesmo resumo quando ainda representam o mesmo item químico agrupador; não se deve criar um novo *Resumo\_Reagente* apenas por haver diferença de concentração ou densidade.

### 7.2.9 Regra para código do frasco de Reagente

#### Regra de negócio

O código do frasco de reagente (formato LCQUI-N) é gerado e garantido exclusivamente no momento do cadastro do frasco via transação atômica. A impressão de etiquetas virgens prévias fornece identificadores sugeridos para colar fisicamente, mas não cria frascos no sistema nem garante a reserva do código.

### 7.2.10 Segurança na Reimpressão de Frascos Cadastrados

#### Regra de negócio

Para evitar a clonagem acidental de etiquetas na bancada e risco de trocas químicas, a reimpressão de segunda via é limitada a no máximo 10 frascos por sessão. O documento impresso deve conter a Ficha de Conferência e Rastreabilidade completa e 1 única etiqueta de reposição por frasco.

### 7.2.11 Regra em Notificação para Professor

#### Regra de negócio

O campo *id\_turma* é obrigatório para todas as notificações de contexto acadêmico (COMENTARIO, POST, ADICIONADO, REMOVIDO, TURMA\_ARQUIVADA, TURMA\_DESARQUIVADA) e deve ser mantido estritamente NULL para notificações operacionais e de almoxarifado/patrimônio. A notificação *DATA\_DEVOLUCAO\_REAGENTE* é preventiva: deve ser emitida pelo backend quando um empréstimo ainda está EM\_USO e sua data de devolução é hoje ou amanhã. A detecção é responsabilidade da Cloud Function; o frontend apenas apresenta e pode agrupar visualmente os registros em “atrasados”, “vence hoje” e “vence amanhã”. O frontend não decide

se uma notificação deve ser criada.

O job pode usar uma única consulta por `status = EM_USO` e `data_devolucao_prevista <= fim_de_amanha`, mas deve classificar cada empréstimo no servidor antes de criar a notificação, separando a janela de atraso da janela preventiva e aplicando idempotência. A idempotência é garantida por **docId determinístico** composto por `{id_emprestimo}-{janela}`, onde `janela` é `VENCE_HOJE` ou `VENCE_AMANHA`; a Cloud Function usa `set(..., {merge: false})` apenas se o documento ainda não existir, evitando duplicatas em execuções repetidas do job diário. Um empréstimo que muda de janela (amanhã → hoje) cria um segundo documento; o anterior não é removido, apenas expirado.

Deve aparecer na interface:

- COMENTÁRIO: x novos comentários de Fulano de Tal na turma y;
- REQUISICAO\_BEM: Nova atualização em sua requisição de Bem patrimonial;
- DATA\_DEVOLUCAO\_REAGENTE: a data de devolução de um ou mais frasco(s) está expirando [amanhã, hoje];
- ROTEIRO\_COMPARTILHADO: Você tem x novo(s) roteiro(s) compartilhado(s) com você.

### 7.2.12 Regra para convidar um aluno que não está no sistema

#### Regra de negócio

Quando o aluno é convidado sem especificar uma turma, `id_turma` é `NULL` em *Convite para Aluno*. Quando o professor vai convidar o aluno, ele pode ou não especificar uma turma, mas ao enviar um convite, tem que aparecer uma confirmação no caso de estar convidando um aluno sem uma turma para ele.

### 7.2.13 Regra para registrar um patrimônio como dado baixa

#### Regra de negócio

É preciso fazer upload do documento pdf que comprove que o bem patrimonial foi dado baixa no sistema da UENF

### 7.2.14 Cálculo de Volume e Peso de Reagentes (Padrão de Laboratório)

#### Controle de Volume via Pesagem

O sistema adotará a pesagem como método principal de controle de volume dos frascos, utilizando a fórmula  $Volume = \frac{Peso_{total} - Peso_{vazio}}{Densidade}$ .

##### 1. Cadastro de Frasco Fechado:

O usuário informa o Peso Total (frasco + reagente) medido na balança e o Volume Nominal (ex: 100mL impresso no rótulo). O sistema calcula e salva o peso do recipiente vazio:

$$Peso_{vazio} = Peso_{total} - (Volume_{nominal} \times Densidade)$$

##### 2. Cadastro de Frasco já Aberto (Opção A - Conhece o peso do recipiente):

O usuário informa o Peso do Frasco Vazio e o Peso Total atual na balança. O sistema calcula o volume atual:

$$Volume_{atual} = \frac{Peso_{total} - Peso_{vazio}}{Densidade}$$

##### 3. Cadastro de Frasco já Aberto (Opção B - Conhece o volume atual):

O usuário informa o Volume Atual visual e o Peso Total na balança. O sistema calcula o peso do frasco vazio para uso futuro:

$$Peso_{vazio} = Peso_{total} - (Volume_{atual} \times Densidade)$$

##### 4. Empréstimo e Devolução:

No empréstimo, registra-se o *peso\_saida*. Na devolução, registra-se o *peso\_retorno*. O consumo é documentado precisamente na tabela *Emprestimo\_Reagente* como:  $Volume_{utilizado} = \frac{\max(0, Peso_{saida} - Peso_{retorno})}{Densidade}$  (líquidos); sólidos usam a diferença não negativa em g. Ganho tolerado segue Q06, com consumo zero e ajuste separado.

##### Tolerância de Devolução (Regra Q06 - PDF-025):

A margem de tolerância é estritamente baseada no **peso bruto de saída** (*Peso\_saida*). Rejeita-se explicitamente o cálculo sobre massa líquida, pois a variância incide sobre a totalidade do corpo medido na balança (incluindo o frasco).

- Reagente normal:  $\max(1\text{ g}, 0,005 \times Peso_{saida})$
- Reagente higroscópico:  $\max(2\text{ g}, 0,02 \times Peso_{saida})$

A avaliação de ganho máximo permitido segue estritamente a fórmula:  $Peso_{retorno} > Peso_{saida} + Tolerancia$ . Um ganho de massa acima dessa tolerância bloqueia a devolução por anomalia (indicando possível contaminação ou erro de balança). Retornos abaixo da tara ( $Peso_{retorno} < Peso_{vazio}$ ), no entanto, bloqueiam a devolução ordinária, exigindo transição imediata para o fluxo de esgotamento (diferença até 5g) ou recalibração metrológica (diferença maior) com

notificação e confirmação do gestor.

### 7.2.15 Exclusividade na retirada de reagentes

#### Regra de negócio

Apenas os usuários cadastrados como *Professor* ou *Bolsista* têm permissão para retirar reagentes do almoxarifado (o *Emprestimo\_Reagente.id\_usuario\_retirou* deve corresponder a um *Usuario* com uma dessas duas linhas associadas). Demais Alunos (mesmo de iniciação científica ou TCC, sem o papel de Bolsista) não possuem autorização no sistema para esta ação. Esta regra foi corrigida para não contradizer a seção *Bolsista* (3.6), que descreve explicitamente o gestor de reagentes fazendo empréstimos para usuários Bolsista. O autoatendimento é exclusivo de usuários portadores da combinação **Professor + Gestor\_Almoxarifado**. Usuários no papel **Aluno + Gestor\_Almoxarifado** são terminantemente impedidos pelo backend de retirar reagentes para si mesmos, em respeito à matriz de segregação de funções (SOD).

### 7.2.16 Usuário Multi-role, exceto Chefe Geral

#### Regra de negócio

Uma conta que possui o papel *Chefe Geral* não pode possuir nenhum outro papel. Se a mesma pessoa exercer, na instituição, a função de Chefe Geral e também uma função representada por outro papel do sistema, deverão existir duas contas independentes, cada uma com identidade e permissões próprias.

### 7.2.17 Usuário Multi-role, Aluno não pode ser professor

#### Regra de negócio

O usuário que é aluno não pode ser professor.

### 7.2.18 Usuário Multi-role, Quais multi papeis podem existir

#### Regra de negócio

Pode haver:

Professor que é Gestor de Bens Patrimoniais.

Professor que é Gestor de Almoxarifado

Professor que é Gestor de Bens Patrimoniais e Gestor de Almoxarifado

Aluno que é Bolsista

Aluno que é Bolsista e Gestor de Bens Patrimoniais

Aluno que é Gestor de Almoxarifado

Aluno que é Gestor de Bens Patrimoniais

Aluno que é Gestor de Bens Patrimoniais e Gestor de Almoxarifado

O Usuário Aluno que também é Bolsista não pode ser Gestor de Almoxarifado. Se, na vida real, um aluno é bolsista e também precisa ser Gestor de Almoxarifado, devem ser criadas duas contas para ele no sistema.

No caso do Aluno que exerce algum papel de Gestor (ou ambos): o aluno pode editar bens patrimoniais e/ou reagentes, de acordo com seus papéis respectivos, porque também tem outro papel além de Aluno.

### 7.2.19 Aluno que é bolsista

#### Regra de negócio

Apenas o chefe geral consegue colocar um aluno como bolsista.

### 7.2.20 Status, vencimento e descarte de frascos

#### Regra de negócio

- Um frasco **VAZIO** ou **QUEBRADO** deve ser exibido como “Pendente de Descarte”, salvo se já estiver **DESCARTADO**.
- Um frasco **VENCIDO** não é automaticamente descartado e também não é automaticamente bloqueado para uso. Para frasco vencido, o sistema mantém a decisão explícita: `em_quarentena = true`, ou `uso_vencido_autorizado = true`, ou situação de *PENDENTE DE DESCARTE* com `uso_vencido_autorizado = false`.
- Em cadastro, quando a validade efetiva já tiver expirado no instante da operação, o gestor é obrigado a escolher um desses três destinos: **QUARENTENA**, **PENDENTE DE DESCARTE** ou **DISPONIVEL** com uso vencido explicitamente autorizado conforme Q04.
- O frasco em quarentena não pode ser retirado; o frasco pendente de descarte não pode ser retirado; o frasco vencido com uso explicitamente autorizado pode ser retirado, mas a interface deve alertar transparentemente antes da confirmação do empréstimo.
- Se um frasco vencer enquanto estiver emprestado, o empréstimo continua válido. Na devolução, o servidor deve recalcular o vencimento com a hora atual e permitir ao gestor registrar o destino pós-validade.

### 7.2.21 Limites de Geração de Relatórios

#### Regra de negócio

Para proteger o sistema contra execuções exaustivas e travamentos em ambiente *serverless*, os relatórios personalizados possuem limite de período de no máximo 31 dias corridos. Além disso, nenhuma data de início ou término pode ser superior ao dia atual, e o mês/ano de relatórios fechados não podem estar no futuro.

### 7.2.22 Almoxarifado inativo: operações bloqueadas e encerramento ordenado

#### Regra de negócio

Quando `Almoxarifado.ativo = false`, as seguintes operações são bloqueadas pelo servidor:

- cadastro de novos frascos e lotes naquele almoxarifado;
- novas retiradas (abertura de *Emprestimo\_Reagente*) vinculadas àquele almoxarifado;
- vinculação de novos gestores ao almoxarifado inativo.

As seguintes operações permanecem permitidas para encerramento ordenado:

- registro de devoluções de empréstimos ativos (`EM_USO` ou `ATRASADO`);
- descarte e atualização de status de frascos já cadastrados;
- geração de relatórios históricos.

A reativação do almoxarifado (`ativo = true`) é possível e não exige recriação da entidade. A desativação é auditada em *Registro\_de\_Auditoria* com ação `DESATIVACAO_ALMOXARIFADO`.

## 7.3 Matriz formal de obrigatoriedade e dependência de campos

| Entidade                 | Condição                 | Obrigatoriedade                                                                                   |
|--------------------------|--------------------------|---------------------------------------------------------------------------------------------------|
| Resumo Reagente          | sempre                   | nome, tipo e natureza.                                                                            |
| Estoque Mínimo<br>Almox. | sempre                   | id_especificacao_reagente, id_almoxarifado e limiar de escassez positivo (quantidade de frascos). |
| Resumo Reagente          | requer pesagem frequente | frequência em dias positiva.                                                                      |
| Especificação            | SOLIDO                   | unidade = g; densidade opcional.                                                                  |

| Entidade          | Condição           | Obrigatoriedade (continuação)                                                                                                                |
|-------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Especificação     | LIQUIDO            | unidade = ml; densidade positiva obrigatória.                                                                                                |
| Especificação     | MISTURA            | pelo menos uma composição válida.                                                                                                            |
| Frasco            | QUARENTENA         | detalhe do status obrigatório.                                                                                                               |
| Frasco            | validade conhecida | validade_fechado e validade_efetiva devem ser consistentes; se já expirada, exige decisão de destino.                                        |
| Frasco            | validade expirada  | exige destino QUARENTENA, PENDENTE_DE_DESCARTE ou DISPONIVEL com uso_vencido_ autorizado.                                                    |
| Frasco            | FECHADO            | capacidade nominal e peso total; peso vazio calculado.                                                                                       |
| Frasco            | abertura           | data_abertura e validade_efetiva calculadas no servidor; aplicar mínimo entre validade fechada e prazo após abertura quando ambos existirem. |
| Frasco            | ABERTO             | peso total e dados de estimativa conforme modalidade escolhida.                                                                              |
| Patrimônio        | sempre             | id_resumo, numero_patrimonio, estado_conservacao, id_local, photo_url, nome_responsavel_sei e status.                                        |
| Patrimônio        | Ja_dado_baixa      | documento comprobatório obrigatório.                                                                                                         |
| Requisição edição | criação            | motivo e ao menos um campo proposto.                                                                                                         |
| Requisição        | aprovada/rejeitada | respondida_em, respondente e justificativa obrigatórios.                                                                                     |
| Convite           | turma definida     | e-mail normalizado + turma.                                                                                                                  |
| Convite           | sem turma          | e-mail normalizado + confirmação explícita do convite global.                                                                                |
| Turma             | criação            | nome, capacidade positiva e professor responsável.                                                                                           |

## 7.4 Matriz de combinações Multi-Role

As únicas combinações permitidas para uma mesma conta são:

- Professor + Gestor de Bens Patrimoniais;
- Professor + Gestor de Almoxarifado;
- Professor + Gestor de Bens Patrimoniais + Gestor de Almoxarifado;
- Aluno + Bolsista;
- Aluno + Bolsista + Gestor de Bens Patrimoniais;
- Aluno + Gestor de Almoxarifado;
- Aluno + Gestor de Bens Patrimoniais;
- Aluno + Gestor de Bens Patrimoniais + Gestor de Almoxarifado.

DP-C01: Aluno sem cargo de Bolsista pode exercer Gestor de Almoxarifado; Bolsista e Gestor de Almoxarifado são mutuamente exclusivos. Não são permitidos Chefe Geral combinado com qualquer outro papel, Aluno + Professor, nem Aluno + Bolsista + Gestor de Almoxarifado. A concessão ou remoção de papéis deve ser transacional e auditada.

## 7.5 Fluxo obrigatório para mutações críticas

Toda operação que altera dados relevantes segue: autenticar, resolver papéis, validar escopo, reler estado atual no servidor, validar transição e constraints, calcular derivados, persistir atomicamente quando necessário, registrar histórico/auditoria e disparar notificações/materializações. O frontend nunca é a autoridade final.

## 7.6 Atribuição e revogação de papéis

### 7.6.1 RN-ROLE-01 — Chefe Geral não pode possuir múltiplos papéis

O papel Chefe\_Geral é mutuamente exclusivo com todos os demais papéis do sistema. Um usuário que possua Chefe\_Geral não pode possuir simultaneamente: Professor; Aluno; Bolsista; Gestor\_Almoxarifado; Gestor\_Bens\_Patrimoniais; qualquer outro papel do sistema. Consequentemente, um usuário com Chefe\_Geral nunca é considerado Multi-Role.

### 7.6.2 RN-ROLE-02 — Chefe Geral não pode revogar o papel de outro Chefe Geral

Um usuário com o papel Chefe\_Geral não pode alterar ou revogar o papel Chefe\_Geral de outro usuário que também possua esse papel. Um Chefe Geral somente pode iniciar o processo de revogação do próprio papel.

### **7.6.3 RN-ROLE-03 — Auto-revogação de Chefe Geral**

Um Chefe Geral pode revogar o próprio papel Chefe\_Geral somente quando existir pelo menos outro usuário ativo com o papel Chefe\_Geral. A operação deve ser rejeitada quando o usuário for o único Chefe Geral ativo do sistema. A finalidade dessa regra é impedir que o sistema fique sem qualquer usuário com capacidade de administração global.

### **7.6.4 RN-ROLE-04 — Revogação de Chefe Geral desativa a conta**

Como Chefe\_Geral é um papel exclusivo, sua revogação sempre resulta na ausência de outros papéis para o usuário. Nesse caso: o papel Chefe\_Geral é removido; os Custom Claims são recalculados; a conta do usuário é marcada como desativada (`ativo = false`); o usuário permanece armazenado no sistema; seus registros históricos e de auditoria são preservados; a conta não deve ser excluída fisicamente.

### **7.6.5 RN-ROLE-05 — Almoxarifado deve possuir pelo menos um gestor**

Todo almoxarifado ativo deve possuir obrigatoriamente pelo menos um Gestor\_Almoxarifado responsável. Um Gestor de Almoxarifado somente poderá ter seu papel revogado se, após a operação, cada almoxarifado que ele gerencia continuar possuindo pelo menos um outro Gestor de Almoxarifado. Portanto, a operação deve verificar individualmente todos os almoxarifados vinculados ao gestor. Se ele for o único gestor de qualquer um deles, a revogação deverá ser rejeitada.

### **7.6.6 RN-ROLE-06 — Gestores de Almoxarifado não possuem exclusividade sobre movimentações**

Não existe exigência de que o mesmo Gestor de Almoxarifado que registrou uma movimentação precise posteriormente registrar a movimentação complementar. Quando um almoxarifado possuir dois ou mais gestores: qualquer um dos gestores autorizados poderá registrar a entrada de um frasco; qualquer um dos gestores autorizados poderá registrar a saída; qualquer um dos gestores autorizados poderá registrar a devolução; qualquer um dos gestores autorizados poderá registrar outras movimentações para as quais possua permissão. O fato de um gestor ter registrado a saída de determinado frasco não impede que outro gestor autorizado registre posteriormente sua entrada ou devolução. A autoria da operação deve permanecer registrada para fins de auditoria.

### **7.6.7 RN-ROLE-07 — Revogação de Gestor de Almoxarifado em usuário Multi-Role**

Quando um usuário possuir Gestor\_Almoxarifado e pelo menos outro papel válido, a revogação de Gestor\_Almoxarifado não desativa sua conta. O sistema deve: validar as regras de revogação dos almoxarifados; remover Gestor\_Almoxarifado dos papéis do usuário; recalcul

seus Custom Claims; manter os demais papéis; manter a conta ativa; remover da interface as funcionalidades exclusivas de Gestor de Almoxarifado.

#### **7.6.8 RN-ROLE-08 — Revogação de Gestor de Bens Patrimoniais em usuário Multi-Role**

Quando um usuário possuir Gestor\_Bens\_Patrimoniais e pelo menos outro papel válido, a revogação de Gestor\_Bens\_Patrimoniais não desativa sua conta. O sistema deve: verificar a condição necessária para a revogação; remover o papel; recalculando os Custom Claims; preservar os demais papéis; manter a conta ativa; retirar da interface as funcionalidades exclusivas do Gestor de Bens Patrimoniais.

#### **7.6.9 RN-ROLE-09 — Gestor de Bens Patrimoniais não pode ser o único**

A revogação de Gestor\_Bens\_Patrimoniais somente será permitida quando existir pelo menos outro usuário ativo que continue exercendo esse papel. Caso o usuário seja o único Gestor de Bens Patrimoniais ativo, a revogação deverá ser rejeitada. A finalidade é impedir que o sistema fique sem nenhum responsável pela gestão patrimonial.

#### **7.6.10 RN-ROLE-10 — Revogação do último papel desativa a conta**

Quando um usuário possuir apenas um papel ativo e esse papel for revogado, sua conta deverá ser desativada. Isso se aplica a todos os papéis. A conta não deve ser excluída. O usuário deverá permanecer no sistema para preservação de auditoria, histórico, autoria de operações, etc.

#### **7.6.11 RN-ROLE-11 — Revogação de papel não significa exclusão do usuário**

A revogação de qualquer papel deve preservar a identidade histórica do usuário. Fluxo correto: Revogar papel -> Validar regras -> Remover papel -> Recalcular conjunto -> Atualizar Custom Claims -> Possui outro papel? (Sim = ativo, Não = desativado).

#### **7.6.12 RN-ROLE-12 — Usuário Multi-Role**

Um usuário será considerado Multi-Role quando possuir dois ou mais papéis simultaneamente, exceto Chefe\_Geral. A remoção de um papel de um usuário Multi-Role deve afetar somente aquele papel específico.

#### **7.6.13 RN-ROLE-13 — Usuário com apenas um papel**

Se o único papel de um usuário for revogado, o usuário ficará com 0 papéis e sua conta será desativada, mas o usuário permanece armazenado para fins históricos e de auditoria.

#### 7.6.14 RN-ROLE-14 — Recalculo dos Custom Claims

Os Custom Claims devem representar o conjunto efetivo de papéis do usuário após a operação. O backend deve recalculer o conjunto completo de papéis autorizados e então atualizar os Custom Claims. Se o conjunto resultante for vazio, a conta deverá ser marcada como desativada.

#### 7.6.15 RN-ROLE-15 — Auditoria das alterações de papéis

Toda atribuição e toda revogação de papel devem gerar registro de auditoria contendo: usuário afetado, usuário que executou a operação, papel adicionado/removido, data/hora, resultado, justificativa e situação da conta. Revogações rejeitadas também devem ser registradas.

#### 7.6.16 RN-ROLE-16 — Invariante Bolsista

A revogação do papel Aluno para um usuário é **TERMINANTEMENTE BLOQUEADA** pelo backend caso ele possua o papel Bolsista ativo. A revogação do papel de Bolsista deverá ocorrer de maneira prévia ou simultânea à perda do vínculo discente.

**Execução concorrente e sincronização de identidade.** A verificação do último chefe, do último gestor patrimonial e de cada almoxarifado deve compartilhar um documento de controle transacional; duas revogações concorrentes não podem ambas remover o último responsável. Revogar Aluno exige remover Bolsista na mesma operação ou rejeitar a solicitação. O modal explicita o conjunto resultante antes de confirmar. A gravação dos papéis e da auditoria ocorre no Firestore; a atualização do Firebase Auth é uma etapa externa, recuperável e idempotente. Uma falha nessa etapa mantém a autorização bloqueada até a reconciliação, sem anunciar conclusão prematura.

**Q04 e Q14: contrato de aceite e autoatendimento.** Para PESQUISA\_TCC\_POS ou ESTUDO\_DEGRADACAO\_RESIDUOS com frasco vencido ou validade desconhecida liberado com termo, obter aceite em sessão autenticada do retirante. Persistir tcr\_versao, tcr\_aceito\_em (servidor), tcr\_aceito\_por e tcr\_auditoria\_id no empréstimo, com evento correspondente em Registro\_de\_Auditoria; exigir justificativa\_metodologica de 20–2000 caracteres. Modal/checkbox são UX: backend valida versão vigente, identidade e vínculo do aceite ao frasco, finalidade e operação antes de confirmar a retirada. O TCR registra ciência e responsabilidade e não substitui regras institucionais de segurança química. Autoatendimento só quando não houver outro gestor ativo, calculado pelo servidor com justificativa e notificação à chefia. Registrar abertura\_historica\_desconhecida quando a data anterior não for conhecida; nunca inventar a data.

**Retenção V1 (DP-D02).** Retenção indefinida de históricos e auditoria é política conservadora de rastreabilidade do LCQUI/UENF. A desativação lógica (*soft delete*) é o padrão do sistema. A exclusão física de registros deve ocorrer somente quando houver fundamento

normativo/operacional aplicável e ausência de referências impeditivas, não devendo a LGPD ser utilizada como justificativa genérica ou única base para deletar dados transacionais e históricos.

**Proteção e Escopo do `localStorage`.** O uso do `localStorage` para persistência de rascunhos de wizards deve respeitar rigorosamente os seguintes controles:

- **Namespace e Segurança:** Utilizar o `UID` como namespace das chaves, embora isso não garanta isolamento contra XSS/same-origin. Tokens, PII, documentos fiscais completos e anexos jamais devem ser armazenados.
- **Ciclo de Vida:** Aplicar TTL (Time to Live) e verificá-lo tanto no *boot* quanto na leitura. Ocorrerá limpeza compulsória após o sucesso do fluxo (salvamento no backend), no logout e em qualquer troca de identidade.
- **Robustez:** Utilizar *schema versioning* e tratar cenários de múltiplas abas, limitando o escopo estritamente aos dados de catálogo (*whitelist* de formulários autorizados).

## 8 Descrição das telas (Dashboards)

### 8.1 Header em comum em todos os papeis

```
|-----|
|Olá, [Nome completo do usuário]                [foto de perfil]|
|-----|
```

Quando se clica na foto de perfil, aparece um modal que se arrasta da direita. Nele é possível deslogar e ir para "Perfil de usuário", que por sua vez abre um modal com as informações do usuário e botão para editar (foto, nome, etc.).

### 8.2 Header quando o usuário é Multi-Role

```
|-----|
|Olá, [Nome]  [Botão Papel 1] [Botão Papel 2]      [foto de perfil]|
|-----|
```

Alterar os botões de papel no header modifica instantaneamente a página toda e o painel lateral para as funcionalidades daquele papel selecionado (ex: de Aluno para Gestor de Almoxarifado).

### 8.3 Dashboard-Chefe Geral

O Chefe Geral tem uma div *full-width* logo abaixo do header principal, alterando conforme a aba selecionada no painel esquerdo:

### 8.3.1 Aba: Almoxarifados

Antes da área principal de busca, ele verá uma div *full-width* com:

- Um botão "Mais" que ao clicar terá as opções:
  - Novo Almoxarifado: Abre um modal para criar um almoxarifado.
  - Novo Gestor de Almoxarifado: Abre um modal em que ele pode escolher: transformar um aluno em gestor de almoxarifado ou criar um Usuário que é Gestor de almoxarifado.
  - Imprimir Etiquetas: Modal bifurcado em duas abas. A Aba 1 (Novos Frascos / Virgens) oferece opções de intervalo e um Grid Visual de Offset 3×10 para escolha da célula inicial na folha A4. A Aba 2 (Frascos Cadastrados / 2ª Via) permite busca por reagente/código e seleção de até 10 frascos, gerando a respectiva Ficha de Conferência.
  - **Nova Matéria:** Abre modal com campos Nome e Código da Matéria. O código deve ser único (validado na Cloud Function).
- Um botão para "Gerenciar Gestores de Almoxarifado". Esse botão abre um novo modal que:
  - Permite pesquisar e selecionar Gestores. Ao selecionar, aparece o histórico dele nos últimos 30 dias com: (Reagentes e frascos adicionados, movimentações registradas), caso ele queira de um mês específico, terá um modal para escolher o ano e o mês.
- Um botão de "Relatórios" que abre um modal com as opções para Relatórios em PDF dos almoxarifados.

Abaixo, haverá uma aba de busca onde pode-se filtrar por almoxarifado, reagente (ou ambos) e pesquisar livremente um reagente pelo nome. Ao clicar em um reagente, abre-se uma tela de detalhes: O filtro padrão da tela de detalhes é o estado atual, mas pode ser alterado para um período específico. Ele exibe: Todas as propriedades em Resumo de reagente, inclusive as materializadas (Materialized View).

### 8.3.2 Aba: Bens Patrimoniais

Antes da aba principal de busca, ele verá uma div *full-width* com:

- Um botão "Mais" com as opções:
  - Novo Gestor de Bens Patrimoniais.
- Um botão para "Gerenciar Gestores de Bens Patrimoniais", abrindo um modal que:
  - Permite pesquisar (por nome) um gestor. Ao selecionar, visualiza-se o histórico dos últimos 30 dias, ou um mês e ano específico que for escolhido (requisições aprovadas/rejeitadas, bens adicionados, edições feitas).

- Um botão de "Relatórios" para gerar os PDFs de bens patrimoniais (mensal, por prédio, geral).

Abaixo, a aba de busca permite filtrar por responsável do bem, estado de conservação, e pesquisa livre pelo número/nome do bem. Ao clicar em um patrimônio, abre-se o detalhamento: Mostra o estado atual, quantos itens existem daquele tipo cadastrados, a divisão por status (Ativo, Inservível, Baixado) e a lista completa desses itens exibindo o número de patrimônio e o respectivo responsável logado no Sistema da UENF.

Ao clicar em um número de patrimônio específico, abre um modal com as propriedades desse patrimônio, ao mudar o status dele para já dado baixa, é preciso fazer o upload do documento pdf que comprove isso.

### 8.3.3 Aba: Professores

Antes da aba de busca, uma div *full-width* com:

- Um botão para "Novo Professor".

Na área principal ele pode pesquisar Professores pelo nome, filtrar por Matéria lecionada, Centro e Laboratório (inclusive combinando múltiplos filtros). Ao clicar em um professor, abre-se um modal para gerenciar a conta daquele docente, ver seu histórico de ações e a lista de turmas que ele administra.

### 8.3.4 Aba: Alunos

Antes da aba de busca, uma div *full-width* com:

- Um botão para "Novo Aluno".

Na área principal ele pesquisa Aluno pelo nome, podendo filtrar por Matéria que cursa ou número de matrícula. Ao clicar no Aluno, abre-se um modal de gerenciamento exibindo seu histórico e as turmas em que está matriculado.

## 8.4 Dashboard-Professor

Para o papel de Professor, a dashboard foca em organização acadêmica e gestão de recursos em uso:

- **Painel Lateral:** Fica do lado esquerdo, podendo ser expandido ou mantido em modo compacto. Contém dois grupos:
  - **Grupo 1 (Visualizar):** Opções para acessar a área de "Reagentes" e "Bens Patrimoniais".
  - **Grupo 2 (Turmas):** Lista navegável de todas as turmas ativas dele. Abaixo da lista, um botão "Turmas Arquivadas" (abre um modal com as turmas passadas exibindo Nome, Ano/Semestre e botão para Desarquivar). Ao desarquivar, a

turma volta para a lista de turmas tanto do professor, quanto dos alunos que fazem parte dela.

Na aba principal, abaixo do header em comum, ele verá uma div *full-width* com:

- **Notificações:** Botão com ícone de sino, presente globalmente no header (N-11) e consolidado por conta ativa (UID), independentemente do papel selecionado no momento. Abre um modal *dropdown* contendo todos os alertas direcionados àquele usuário: requisições de bens respondidas (Professor), reagentes com prazo atrasado ou vencendo (Professor/Bolsista), e alertas de escassez (Gestores). Cada notificação exibe visualmente o escopo/papel a que se destina. Inclui opção de marcar como lida e um botão “Limpar tudo”.
- **Botão Mais (+):** Abre *dropdown* com: Nova Turma, Novo Aluno (turma opcional, com confirmação de convite global), Novo Roteiro de Experimento, Novo Pedido de Adição de Bem Patrimonial (se o usuário selecionar "Novo Local" dentro do pedido, um modal sobreposto exigirá o cadastro prévio do Prédio/Sala), e **Nova Matéria** (abre modal simples com nome e código; código deve ser único, validado pelo backend).
- **Gerenciar Roteiros:** Abre um modal com três abas ("Meus", "Compartilhados comigo" e "Eu compartilhei"). Permite abrir para visualização ou baixar os arquivos em PDF.
- **Minhas Requisições:** Modal para acompanhar os pedidos feitos de adição/edição de Bens Patrimoniais (filtrando por Pendente, Aprovada, Rejeitada).
- **Meus Reagentes:** Modal de acompanhamento onde o professor visualiza os reagentes atualmente sob sua posse (ordenados pela data de devolução) e o histórico de uso (Padrão: últimos 30 dias). Ao clicar num reagente histórico, ele visualiza exatamente o volume (mL) que ele consumiu nesse período.

**Área Central (Feed da Turma):** Ao acessar a dashboard inicialmente, uma mensagem dirá: "*Selecione uma turma ao lado para começarmos*". Ao clicar em uma turma no painel lateral, a área central exibe o Feed dessa disciplina. Existe um bloco fixo superior para criar novos posts (podendo anexar PDFs novos ou selecioná-los da sua biblioteca/compartilhados). O feed lista os posts em ordem cronológica e cada post contém a área de comentários integrada para interação direta com os alunos.

## 8.5 Dashboard-Gestor de Almoxarifado

Para o perfil de Gestor de Almoxarifado, a interface é desenhada para a velocidade de atendimento de balcão e auditoria rigorosa de insumos:

- **Painel Lateral:** Grupos de navegação para "Estoque"(Reagentes e Frascos), "Movimentações"(Empréstimos/Retornos), e "Descartes/Alertas".

Na aba principal, uma div *full-width* com botões rápidos essenciais:

- **Alertas de Gestor (Notificações):** Sino de avisos exclusivo exibindo pendências críticas: frascos declarados como vazios, vencidos ou quebrados, ou seja, aguardando mudança para o estado DESCARTADO, e reagentes que atingiram nível de escassez no estoque, frascos que precisam de pesagem de rotina, frascos que estão em quarentena.
- **Botão Mais (+):**

- Novo Reagente -> Abre modal exigindo dados de resumo e especificação separados conforme UI-05; CAS e concentração têm obrigatoriedade condicional, não universal. Campos: Nome, CAS, Concentração, grau de pureza (texto opcional), fórmula química, se é controlado pela pf ou pela eb, a classe de inflamabilidade, link da ficha de segurança, tipo (sólido ou líquido: pede densidade, pergunta se requer pesagem frequente, se sim, pede a frequência em dias). Tem um seletor para escolher entre sólido e líquido (em temperatura ambiente):

- \* Para tipo sólido -> Sempre que for adicionar um frasco -> o tipo de medida é gramas (g) (setado pelo sistema)
- \* Para tipo líquido -> Sempre que for adicionar um frasco -> o tipo de medida é mililitros (ml) (setado pelo sistema)

Tem um seletor para a Natureza do Reagente: Orgânico, Inorgânico, Elemento ou Híbrido

- Adicionar Frasco: abre um modal para escolher o reagente a ser adicionado, o reagente pode ser pesquisado, por: nome, cas e formula química (deve ter um seletor para escolher, isso serve para não precisar pesquisar em todas as categorias de uma vez). Após selecionar o reagente a ser adicionado, fecha o modal de escolher reagente e passa a preencher o formulário para adicionar o frasco de reagente, o formulário deve conter: Selecionar o almoxarifado em que esse frasco vai ficar; Escrever o detalhamento do local (como: Segunda prateleira, na direita, etc) O número do lote em que esse frasco pertence. Pergunta o status dele: Fechado, Aberto, Quarentena (em caso de quarentena, aparece um campo em texto para ser escrito explicando o motivo de estar em quarentena)

**Feedback de Sucesso:** Ao salvar, exibe: "*Frasco cadastrado sob o código LCQUI-XX (Reagente: YY, Data: DD/MM/AAAA). Localize a etiqueta física correspondente na folha de bancada, preencha os dados à caneta e aplique no frasco com fita adesiva.*" O campo estado físico no Resumo do reagente que determina os campos a serem pedidos abaixo (sólido ou líquido)

líquido ->

- \* Se é do tipo fechado, pede apenas a volume do líquido e o peso total com o frasco. (calcula o peso do frasco, mostra o peso do frasco mas não pode ser editado.)

- \* Se é do tipo Aberto, exige peso total, o conteúdo nominal do rótulo e uma modalidade: tara conhecida ou estimativa de volume, conforme UI-06.

sólido -> exige peso total e massa nominal do rótulo (fechado), ou peso total, conteúdo nominal do rótulo e tara conhecida/estimativa de massa (aberto), conforme UI-06. O saldo inicial pode ser desconhecido.

- Imprimir Etiquetas: Modal bifurcado em duas abas. A Aba 1 (Novos Frascos / Virgens) oferece opções de intervalo (a partir do último cadastrado, após última impressão ou personalizado) e um Grid Visual de Offset 3×10 para escolha da célula inicial. A Aba 2 (Frascos Cadastrados / 2ª Via) permite busca por reagente/código e seleção de até 10 frascos, gerando a respectiva Ficha de Conferência.

- **Ações de Bancada (Destacados no centro):**

- **Registrar Retirada:** Modal para escanear/buscar o frasco, selecionar o usuário retirante (Professor ou Bolsista, conforme as regras de autorização) e anotar o Peso Inicial.
- **Registrar Devolução:** Modal crucial. O gestor informa o frasco retornado e insere o **Peso de Retorno** da balança. O sistema faz a diferença, aplica a densidade (se for líquido), e mostra em negrito o *Volume Utilizado em mL*, ou então o *Peso Utilizado em gramas*, atualizando medida no frasco e seus status automaticamente antes de fechar o modal.

- **Relatórios:** Acesso à geração de PDFs mensais.

**Área Central de Busca:** Tabela geral pesquisável por nome do reagente, status do frasco ou almoxarifado físico. Ao clicar no reagente, o modal apresenta o dashboard do químico: volume total disponível, gráfico de consumo acumulado, e lista dos frascos com localização exata e status atual.

## 8.6 Dashboard-Gestor de Bens Patrimoniais

Para o Gestor de Bens Patrimoniais, o foco é em auditoria, análise comparativa e suporte processual institucional:

- **Painel Lateral:** Grupos divididos em "Patrimônio Geral", "Requisições de Professores" e "Alertas importantes" (em alertas ele vai ter os patrimônios que estão em estado inservível para poder dar baixa- isso é externo ao sistema.)

Na div *full-width* superior:

- **Notificações:** Alerta sempre que um professor abrir uma requisição de adição de bem novo ou alteração do estado de um bem existente.
- **Botão Mais (+):** Adição direta de bens (ignorando requisição) e cadastro de novas descrições/resumos de itens; cadastro de gestores é exclusivo do Chefe Geral.

- **Analisar Requisições (Destaque):** Abre uma interface de tela dividida (Split-screen). Do lado esquerdo, os dados e foto do bem como estão atualmente no banco de dados. Do lado direito, os novos dados propostos pelo professor destacado em cores (ex: Estado de Conservação passando de Bom para Ruim). No rodapé, botões massivos de "Aprovar Alteração" ou "Rejeitar" (exigindo campo de texto com justificativa).
- **Gerar Relatórios:** Modal para compilação e download de PDF. Foco em permitir gerar o relatório de "Baixa de Bens Inservíveis" (contendo responsáveis) para anexar diretamente no processo interno (SEI) da Universidade.

**Área Central de Busca:** Pesquisa altamente filtrável (Prédio, Andar, Sala, Nome, Responsável, Status). O clique no bem patrimonial exhibe, além das propriedades atuais do equipamento (foto, conservação, etc.), um histórico de auditoria vertical, listando quem modificou, a hora e quais campos foram alterados ao longo do tempo. Também é possível modificar manualmente um equipamento para "Ja\_dado\_baixa" caso o trâmite no SEI tenha finalizado.

## 8.7 Dashboard-Aluno

Para o perfil de Aluno, a interface remove completamente as complexidades de gestão, focando apenas no acesso aos materiais acadêmicos:

- **Painel Lateral:** Um visual limpo apresentando a lista de "Minhas Turmas". No rodapé desse painel, um input text estilizado e claro dizendo "Código da Turma" com um botão "Entrar", permitindo vínculo imediato a uma nova disciplina sem modais complexos. Todo aluno tem acesso de leitura ao catálogo de reagentes conforme a seção 3. Quando também for bolsista, ele mantém o grupo Visualizar/Reagentes e ganha acompanhamento dos próprios empréstimos.

Na aba principal, abaixo do header:

- **Notificações:** Sino simples indicando novos roteiros de práticas postados, ou professores que responderam às dúvidas deixadas nos comentários.

### Área Central (Visualização da Disciplina):

- Caso nenhuma turma esteja selecionada no menu lateral, a área exhibe um aviso centralizado e amigável: "*Selecione uma turma ao lado para ver as postagens*".
- Ao clicar numa turma, a tela exhibe o Feed do aluno. Os posts feitos pelo professor aparecem em forma de *cards* cronológicos.
- Se o post contém um Roteiro de Experimento anexado, um ícone de PDF chamativo aparece ao lado do nome do arquivo com um botão de ação "Baixar" (Download direto).
- Abaixo de cada post, a seção de comentários. O aluno vê um campo simples de *input* para digitar uma dúvida ou observação, que, ao ser enviada, fica visível imediatamente na *thread* daquele post para o professor e demais colegas da sala.

## 8.8 Contratos detalhados das telas e modais

Os contratos UI abaixo complementam os painéis anteriores, preservando sua organização. Cada abertura de modal, gaveta, página ou menu descrita acima usa o contrato correspondente. Campos não marcados como opcionais são obrigatórios; limites e enums do dicionário da seção 5.9 também se aplicam ao frontend. As regras seguintes especificam o comportamento esperado e não atestam implementação.

### 8.8.1 UI-01: acesso, sessão, perfil e troca de papel

Login apresenta e-mail (tipo email, até 150 caracteres, espaços externos removidos), senha obrigatória com alternância de visibilidade, Entrar, Login com Google e Recuperar senha. Não aplicar trim à senha. Erro de credencial é genérico, sem confirmar a existência da conta. Recuperação pede somente e-mail e apresenta a mesma confirmação para endereços existentes ou não. Login não concede papel: após autenticação, carregar conta ativa e permissões antes de montar a dashboard; conta desativada vai para aviso de acesso desativado, conta sem papel para acesso não autorizado. Falha de rede oferece Tentar novamente.

A foto abre gaveta com nome, e-mail somente leitura, papéis, Perfil e Sair. Perfil contém nome obrigatório (1–150 caracteres), foto opcional com prévia, Remover foto, Cancelar e Salvar. A imagem deve ter tamanho inferior a 5 MiB; o servidor valida formato e propriedade do objeto. E-mail, matrícula e papéis não são editáveis nesse formulário. Atualizar nome propaga as projeções de busca, preservando eventos históricos. Sair encerra listeners, descarta dados da sessão, libera URLs temporárias e retorna ao login.

O seletor de papel exhibe somente papéis autorizados. Ao trocar, interromper consultas do painel anterior, limpar seleções e carregar o novo escopo; um formulário modificado exige decidir entre continuar editando ou descartar. O papel preferido é apenas preferência visual. Não constitui autorização. Após alteração de permissões, renovar o token e verificar conta ativa; se o papel corrente deixou de existir, escolher outro permitido ou encerrar a sessão.

**Padrão visual, acessibilidade e estados.** Modal central: `fixed inset-0 z-50`, fundo escurecido, conteúdo `w-full max-w-2xl max-h-[90vh] overflow-y-auto`; formulários usam uma coluna no celular e duas em telas largas. Gaveta: largura total no celular e painel lateral no desktop. Manter título acessível, foco contido, retorno do foco ao acionador, rótulos ligados aos campos, erros textuais com `aria-describedby`, resumo de erros e foco no primeiro inválido. Escape e Cancelar fecham quando não há envio em andamento. Botões têm foco visível e texto além da cor. Salvar fica indisponível durante envio; sucesso só após resposta do servidor. Em conflito, preservar entradas e oferecer recarregar/revisar. Nunca repetir silenciosamente uma movimentação cuja resposta se perdeu: consultar seu identificador idempotente. Listas apresentam carregamento, vazio, erro, sem permissão e dados desatualizados como estados distintos.

### 8.8.2 UI-02: usuários, gestores e atribuição de papéis

Novo Professor, Novo Aluno e Novo Gestor compartilham formulário do Chefe Geral: escolher usuário existente por pesquisa autorizada ou convidar por e-mail; nome e e-mail são obrigatórios na criação. Professor exige centro (CCT, CCTA, CBB ou CCH) e laboratório até 100 caracteres; Aluno exige matrícula única até 20 caracteres no cadastro concluído. Convite pode antecipar matrícula opcional, exigida na aceitação se ainda ausente. Matérias são seleção opcional de IDs existentes, sem duplicatas. Gestor de almoxarifado aceita vínculos opcionais; sem vínculos, exibir estado vazio com orientação para procurar a chefia. Não converter Aluno em Professor implicitamente.

A lista de papéis mostra combinações permitidas, motivo de bloqueio e conjunto resultante. Conceder Bolsista exige Aluno e ausência de Gestor de Almoxarifado. Revogar abre confirmação com usuário, papel, vínculos afetados, justificativa obrigatória e consequência para a conta. Aplicar RN-ROLE-01 a RN-ROLE-15, incluindo autorrevogação e proteção do último responsável. Não oferecer revogar outro Chefe Geral. O backend valida tudo novamente e registra inclusive rejeições.

Detalhes de professor/aluno/gestor têm abas Dados, Papéis e vínculos, Turmas e Atividade. Nome, matrícula, laboratório e centro seguem os mesmos limites; somente campos autorizados são editáveis. Atividade inicia nos últimos 30 dias e consulta histórico paginado por autor e instante; o seletor mensal exige mês 1–12 e ano não futuro e usa os agregados mensais. Matrícula e e-mail não aparecem na lista de colegas. Filtros por matéria usam os vínculos Professor–Matéria e Aluno–Turma–Matéria. Q01: lotação física por prédio/gabinete/sala foi postergada à V2. Na V1, filtros de professor são Nome, Matéria, Centro e Laboratório.

### 8.8.3 UI-03: almoxarifado, local e matéria

Novo Almoxarifado exige nome (100), descrição (500), Local existente e pelo menos um gestor para ativação. Seleção múltipla lista apenas gestores ativos. Pode ser salvo inativo sem gestor; ativar exige vínculo válido. Gerenciar vínculos mostra todos os responsáveis e impede remover o último de um almoxarifado ativo, inclusive numa revogação global.

Novo Local exige prédio (30), andar (10, texto para aceitar térreo/subsolo) e sala (30). Remover espaços externos, rejeitar campos vazios e verificar unicidade normalizada da combinação no servidor. Ao salvar, retornar o ID e selecionar o local no formulário de origem, preservando o restante do pedido. Evitar dois diálogos simultâneos com foco concorrente: usar etapa interna ou suspender o diálogo pai. Autoria e data vêm da sessão e do servidor.

Nova/Editar Matéria pede nome (100) e código (10), ambos não vazios; normalizar código de forma consistente e verificar unicidade transacional. Editar mantém o ID e atualiza projeções; turmas não são recriadas. Chefe e Professor acessam essa ação; selecionar matéria existente não exige cadastrar novamente.

#### 8.8.4 UI-04: pesquisa, catálogo e detalhes

Reagentes exigem pelo menos um filtro entre letra, natureza, tipo de substância e estado físico. Patrimônio exige prédio, letra ou status. Usar debounce de 300 ms como parâmetro inicial de interface, cursor estável e páginas de 25 registros. Busca textual local informa “filtrando os itens carregados”; zero correspondências numa página não significa ausência no catálogo. Oferecer carregar próxima página e refinar filtros. Busca exata por LCQUI-N, CAS, matrícula ou plaqueta tem modo próprio, valida o formato e consulta índice/chave específica, sem varrer toda a base. Não abrir automaticamente detalhes enquanto o usuário ainda digita.

Detalhes de reagente mostram resumo, especificações separadas, composição, densidade/unidade, fabricante, controle PF/EB e link FDS; frascos são paginados por almoxarifado e estado. Mostrar saldo em g separado de mL, fechados/abertos/em uso e horário da materialização. Período histórico usa resumos diários; o estado atual usa documentos atuais. Selecionar frasco abre código, localização, lote opcional, validade, pesagem, disponibilidade e histórico. Emprestado não revela dados do retirante a quem só consulta catálogo. Ações de gestão exigem papel e vínculo; Professor/Bolsista consultam seus empréstimos, e Aluno tem leitura de catálogo conforme seção 3, sem retirada.

Patrimônio apresenta resumo catalográfico e itens físicos, foto, plaqueta, local, conservação e status separados. Responsável SEI é texto, não conta LCQUI. Detalhe individual usa ID e histórico paginado; resumo agrega contagens por status. Professor tem Solicitar edição, gestores e Chefe têm Editar e Baixa. Listas administrativas de pessoas usam respostas com campos mínimos e filtros de escopo; não baixar a coleção de identidades para simular busca.

#### 8.8.5 UI-05: resumo químico, substância, especificação e lote

Novo Reagente é um assistente. Primeiro pesquisar resumo existente; criar somente se a identidade química nominal for diferente. Resumo exige nome (150), estado SOLIDO/LIQUIDO, higroscopicidade, PURA/MISTURA, natureza (ORGANICO, INORGANICO, ELEMENTO, HIBRIDO) e indicador de pesagem frequente; quando marcado, frequência inteira positiva em dias. Alterar esses dados exige confirmação do alcance sobre todas as especificações.

A segunda etapa seleciona ou cria especificação: descrição (150) e classe de inflamabilidade obrigatórios; estado herdado do resumo; unidade somente leitura (g/ml). Densidade positiva obrigatória para líquido; opcional para sólido. Fabricante (150), código do produto (100), grau de pureza (30) e FDS HTTPS (255) são opcionais. Grau de pureza é texto (ex.: P.A.), não percentual obrigatório. Controle PF/EB usa dois booleanos explícitos. Densidade confirmada não é editável: registrar nova especificação e preservar cálculos históricos.

Composição permite adicionar substâncias por nome/CAS, sem repetir ID. Nova substância pede nome (150), CAS opcional (15) e fórmula opcional (50); CAS informado deve ter formato e dígito verificador válidos e ser único. Mistura exige ao menos um componente. Tipo de concentração é enum da seção 4; valor\_min/valor\_max numéricos não negativos e ordenados seguem Q03, com notacao\_original\_fabricante opcional. A validação ocorre também no servidor. Não exigir CAS único de uma mistura como se fosse substância individual.

Lote é opcional no frasco. Novo Lote exige especificação, fornecedor (80), número (100), nota fiscal (100), aquisição, fabricação, validade e quantidade inteira não negativa comprada. Fabricação não excede validade; aquisição não é futura. Verificar unicidade especificação/fornecedor/número. Lotes existentes são filtrados pela especificação selecionada. Salvamento de cada entidade retorna seu ID; eventual falha da etapa seguinte permite retomar sem criar duplicatas.

#### 8.8.6 UI-06: cadastrar, editar e pesar frasco

Adicionar Frasco inicia pelo seletor de resumo e especificação UI-05, com modos de busca nome/CAS/fórmula. Exige almoxarifado autorizado e ativo, localização detalhada (campo opcional até 500 caracteres, recomendado para localizar a unidade), estado FECHADO/ABERTO e modo de pesagem. Lote é opcional; selecionar Quarentena é uma decisão separada do estado físico e torna motivo obrigatório. Validade fechada é opcional se marcada desconhecida; prazo após abertura, quando informado, é inteiro positivo. Nunca tratar validade desconhecida como validade ilimitada. O servidor calcula validade efetiva; decisão sobre operação com validade desconhecida está em Q04.

Fechado: conteúdo nominal positivo (mL líquido/g sólido) e peso total positivo em g. Tara calculada é somente leitura: peso total menos conteúdo vezes densidade para líquido, ou menos conteúdo para sólido. Aberto: *conteudo\_nominal* continua sendo a quantidade original do rótulo/fabricante (não exigir que represente conteúdo restante), peso total, e modo CONHECE\_TARA, ESTIMA\_VOLUME (líquido) ou ESTIMA\_MASSA (sólido). Se o saldo inicial atual for desconhecido, não inventar quantidade atual. Exigir apenas a entrada correspondente; tara deve ser positiva e não maior que o peso total. Apresentar prévia identificada como estimativa; não enviar derivados como autoridade. Data de abertura anterior desconhecida não deve ser fabricada como hoje (Q05).

Se vencido, exigir destino: quarentena com motivo, pendente de descarte ou autorização excepcional explícita conforme Q04. Exibir revisão final com identidade, unidades, destino e dados de validade. Salvar cria código, frasco, histórico e auditoria de forma idempotente; apenas o servidor atribui LCQUI-N. O retorno mostra código e instrução de localizar/preencher a etiqueta física correspondente. Não gravar código sugerido por impressão anterior.

Editar propriedades permite localização, dados corrigíveis e decisões de estado autorizadas; não altera código, identidade química, peso de cadastro ou densidade. Pesagem de rotina exige novo peso em g, motivo e revisão da diferença; confirmar relê o frasco, exige disponibilidade e registra ajuste/evaporação separadamente do consumo em empréstimo. Tara incompatível bloqueia conclusão. Aplicar tolerância híbrida, esgotamento e recalibração de tara definidos em Q06; não aceitar consumo negativo silenciosamente.

#### 8.8.7 UI-07: retirada, devolução, quarentena e descarte

Retirada exige código/ID do frasco, retirante ativo Professor/Bolsista, local de uso, peso de saída positivo em g, data prevista não anterior a hoje e finalidade (AULA\_PRATICA,

DEMONSTRACAO, PESQUISA\_TCC\_POS, ESTUDO\_DEGRADACAO\_RESIDUOS). Nome e detalhes da especificação são somente leitura. Seleção de destinatário deve ser pesquisa autorizada com campos mínimos. Quarentena, vazio, quebrado, descartado ou pendente de descarte bloqueiam envio. Primeira abertura é confirmada e recalcula validade na mesma transação. Vencido autorizado exige ciência; pesquisa excepcional segue Q04 com TCR rastreável validado pelo backend, sem alterar a autorização do gestor. Confirmar apresenta resumo; servidor relê vínculo, frasco e destinatário, cria empréstimo e muda disponibilidade atômica.

Devolução seleciona empréstimo EM\_USO/ATRASADO do frasco, apresenta retirante, prazo, peso de saída e densidade fixada. Exige peso de retorno em g e identificação de quem devolve; operador é obtido da sessão. Prévia mostra diferença em g e consumo em g/mL. Peso superior ao de saída aplica tolerância híbrida Q06; ganho tolerado gera consumo zero e ajuste. Peso inferior à tara exige o fluxo de esgotamento/recalibração Q06. Ao confirmar, servidor recalcula consumo, atraso e vencimento; resultado devolve consumo final e estado atualizado. Qualquer gestor vinculado pode receber, mesmo que outro tenha registrado a saída. Frasco vencido na devolução exige destino, sem impedir o registro do retorno físico.

Alertas abrem lista filtrada de vazios, quebrados, vencidos, quarentena, pesagem ou escassez. Selecionar item abre UI-04 com ações cabíveis. Marcar vazio/quebrado exige motivo e confirmação; empréstimo ativo deve ser encerrado de modo auditado antes da transição incompatível. Quarentena e liberação exigem motivo, revalidação de validade e pesagem quando necessária. Descartar exige confirmação explícita de descarte físico concluído, motivo e registro do operador/instante; é terminal, sem excluir documento. “Removido” significa retirada do catálogo operacional por estado/arquivamento, nunca exclusão de identidade; política específica em Q07.

#### 8.8.8 UI-08: etiquetas virgens e segunda via

Imprimir Etiquetas tem abas Novos Frascos e Frascos Cadastrados. Virgens: escolha entre próximo ao último cadastro, após última geração e intervalo personalizado; mostrar origem e instante da sugestão. Início/fim são inteiros positivos, fim maior ou igual a início e total entre 1 e 50. Grade 3 por 10 permite célula inicial por teclado ou clique, linha 1–10 e coluna 1–3, com células anteriores indisponíveis para esta impressão. Prévia informa quantidade de páginas e não reserva códigos. Última geração não prova impressão física.

Segunda via: busca autorizada por código/reagente, seleção de 1 a 10 IDs distintos e contador visível; limite bloqueia seleção adicional. Exibir identidade e localização para conferência. Servidor relê cada frasco e autoriza seu almoxarifado; falha em qualquer seleção impede gerar lote parcial silencioso. Cada página A4 contém ficha cadastral, especificação, lote/validade, último peso e tara, último empréstimo e dez últimos eventos, além de uma única etiqueta no rodapé. Ausência legítima é “não informado”, nunca inventar saldo ou status. Auditoria registra uma segunda via por frasco.

Gerar mostra progresso indeterminado, erro recuperável e botão Baixar quando pronto; download não significa impressão concluída. Orientar impressão em tamanho real (100%),

sem ajustar à página, conferindo código antes de colar. Geometria e decisões de persistência constam na seção 10.

### 8.8.9 UI-09: patrimônio e análise de requisições

Novo Bem/Editar exige resumo existente (ou Novo Resumo com nome 150 e descrição), plaqueta textual até 30 caracteres preservando zeros iniciais, conservação BOM/REGULAR/RUIM, Local, foto, responsável SEI (150) e status. Descrição complementar é opcional (200). Foto: prévia, substituição, erro e upload inferior a 5 MiB; servidor verifica existência e autorização do objeto antes de salvar. Novo Bem não permite contornar baixa sem comprovante.

Pedido de Adição do Professor pede plaqueta, conservação, responsável, local, motivo e escolha exclusiva entre resumo existente e nome/descrição de novo resumo. DP-B02: foto é obrigatória já na submissão; o servidor rejeita ausência e valida o objeto; não usar imagem fictícia. Pedido de Edição apresenta valores atuais e campos propostos opcionais, com motivo obrigatório e ao menos uma diferença. Alterar nome individual significa reclassificação de resumo, não renomear todos os equipamentos. A seção 4 define conservação e status separadamente: RUIM não equivale automaticamente a Inservível.

Minhas Requisições lista somente as do solicitante, por status/data, e abre detalhe com proposta, motivo, resposta, gestor e instante. Fila do gestor começa por pendentes e abre comparação em duas colunas (uma no celular); exibir campos alterados com texto/ícone, não apenas cor. Aprovar e Rejeitar exigem justificativa. Pedidos legados sem foto não podem ser aprovados até saneamento explícito; novas submissões já exigem foto e resumo completo. Confirmar relê requisição e versão do bem; se já respondida ou alterada, não sobrescrever silenciosamente. Persistir resposta, criação/edição, histórico e liberação do lock juntos, e notificar o professor de forma idempotente.

Baixa está no detalhe individual: selecionar Ja\_dado\_baixa exige PDF comprobatório inferior a 10 MiB, prévia/nome do arquivo e confirmação com plaqueta e responsável. O servidor verifica o objeto antes da transação; falha de upload mantém status anterior. Após sucesso, item sai do filtro Ativo e permanece consultável no histórico. Não excluir comprovante associado a evento de baixa.

### 8.8.10 UI-10: turmas, membros e convites

Nova Turma exige nome (100), matéria existente, ano inteiro, semestre 1/2 e capacidade inteira positiva; responsável vem do contexto autorizado. Código único é gerado no servidor e exibido com Copiar, feedback de sucesso/falha da área de transferência e alternativa de seleção manual. Detalhe de turma tem nome/período, matéria, professor, ocupação/capacidade, feed e Membros. Membros exibem nome/foto para alunos matriculados; matrícula/e-mail ficam restritos à gestão autorizada.

Arquivar abre confirmação com nome/período e explica preservação dos dados; turma sai das listas ativas. Turmas Arquivadas mostra lista paginada, ano/semestre, detalhe e Desarquivar para professor responsável. Restaurar usa mesmo ID e vínculos, sincroniza

espelhos e notifica membros. Q08: turma arquivada é somente leitura; Rules e backend rejeitam escrita em posts e comentários.

Novo Aluno do Professor convida exclusivamente como Aluno, com um ou mais e-mails validados e deduplicados, matrícula opcional por pessoa e turma opcional. Sem turma, exigir confirmação de convite global; portanto o botão não fica bloqueado só por inexistirem turmas. Mostrar resultado individual por destinatário e permitir repetir apenas falhas, sem afirmar envio de e-mail se houve apenas criação de registro. Exceção de capacidade por convite nominal exige justificativa do professor (RN-TUR-01), persistida para validação na aceitação.

Aceitar convite exige sessão com e-mail verificado correspondente, convite pendente não expirado, nome e matrícula válida se faltarem. Validar conta/papéis existentes, não sobrescrever identidade. Aceitação cria vínculo e espelho atômicamente e consome convite uma vez. Convite expirado orienta solicitar novo; lotação sem exceção válida mantém erro recuperável. Ingresso por código no painel Aluno exige código não vazio até 20 caracteres, turma ativa, vaga e ausência de remoção impeditiva; aluno já matriculado recebe acesso à turma sem duplicar contagem. Removido anteriormente precisa de convite explícito. Remover membro abre confirmação com nome e turma, preserva comentários e histórico, remove vínculo/espelho e reduz contador na mesma transação.

#### 8.8.11 UI-11: roteiros, posts e comentários

Novo Roteiro exige nome (150), descrição e PDF inferior a 15 MiB. Upload tem seleção/arrastar arquivo, nome/tamanho, progresso, Cancelar e Tentar novamente; não habilitar Salvar antes de validar upload e metadados no servidor. Biblioteca tem Meus, Compartilhados comigo e Eu compartilhei; cada linha mostra nome, autor, descrição e ações permitidas. Visualizar abre leitor com fallback Baixar; download revalida propriedade, compartilhamento ou matrícula numa turma cujo post vincula o roteiro. URL de arquivo não é autorização permanente.

Compartilhar abre pesquisa de professores ativos, exclui proprietário e seleciona IDs distintos. Confirmar cria relações únicas; repetir não duplica acesso. Revogar compartilhamento exige confirmação e segue Q09 quanto a posts existentes. Quem recebeu pode anexar à própria turma, mas não alterar autoria ou compartilhar em nome do dono.

Novo/Editar Post exige título (150) e descrição não vazia; roteiro é opcional e único por post. Clique abre seletor com Meus/Compartilhados ou Novo Roteiro, preservando rascunho ao voltar. Publicar valida professor responsável, turma e acesso atual ao roteiro; guarda post, histórico de edição quando aplicável e notificações. Feed pagina por data/ID com mais recentes primeiro; navegação de notificação carrega o post alvo diretamente em vez de baixar todo o feed.

Comentários são carregados por post aberto, com texto obrigatório, autor/data e indicação de edição. Limite de texto segue Q10; rejeitar só espaços, renderizar texto escapado e não HTML arbitrário. Enviar mostra pendente e confirma após servidor validar matrícula/professor e post. Q11: autor pode editar, preservando histórico e etiqueta de edição; professor responsável modera com motivo obrigatório, sem apagar o original. Responder no feed é comentário no mesmo post, sem criar hierarquia de respostas ausente no modelo. Remoção do aluno não

apaga comentários anteriores.

### 8.8.12 UI-12: notificações, empréstimos próprios e relatórios

Sino abre painel com Não lidas/Todas, tipo, resumo e instante. Consulta é pelo destinatário autenticado; Q12 inclui Bolsista e agrega alertas críticos de empréstimo independentemente do papel visual selecionado. Clicar revalida acesso ao alvo e abre o contrato correspondente; recurso indisponível apresenta mensagem sem expor dados. Limpar tudo pede confirmação de marcar notificações ativas do usuário como lidas, inclusive de outros papéis; mantém histórico e usa processamento paginado. Alertas vencidos não autorizam ações por si mesmos.

Meus Reagentes possui Em uso, ordenado por prazo, e Histórico, últimos 30 dias com filtro de período. Exibir código, reagente, peso, unidade, datas, consumo e situação; selecionar histórico abre o empréstimo e a memória de cálculo. Professor/Bolsista não registram a retirada nessa tela: procuram gestor no balcão. Não somar g e mL.

Relatórios possuem Mensal, Personalizado e, para patrimônio, Inservíveis/Bem específico. Mensal exige mês e ano não futuros; personalizado exige início/fim, início não posterior a fim, nenhuma data futura e no máximo 31 dias inclusivos. Filtros opcionais de prédio/andar/sala são dependentes: trocar prédio limpa seleções incompatíveis. Outros filtros: status, conservação, responsável, reagente e almoxarifados autorizados conforme domínio. “Todos” significa todos dentro do escopo do operador. Chefe tem escopo global. Relatório de um bem exige seu ID/plaqueta.

Prévia resume filtros, período, domínio e unidades; Gerar fica bloqueado se inválido. Servidor repete limites e escopo, usa agregados para totais e históricos para movimentos, distinguindo fotografia atual de eventos do período. Sem registros, informar ausência sem simular erro. Mostrar fonte/instante da materialização e período parcial quando ainda aberto. Ao concluir, oferecer Baixar/Imprimir; preservar filtros em falha e não persistir conteúdo do PDF em localStorage.

### 8.8.13 UI-13: consultas, cache e armazenamento no navegador

Chave de cache inclui UID, papel, recurso, filtros e cursor. Cache de sessão em memória é padrão; preferência de tema, menu compacto e filtros não sensíveis podem ir para localStorage com versão de formato e validação ao ler. Não armazenar senhas, tokens manualmente, e-mails, matrícula, PDFs ou autorização no localStorage. Persistência Firestore em disco só mediante opção de dispositivo confiável; bancada compartilhada usa memória. Logout/troca de identidade encerra listeners e elimina acesso aos dados anteriores; dados já baixados exigem política de limpeza, não desaparecem pela simples troca de claims.

Listeners ficam apenas nas listas visíveis e limitadas (feed ativo, notificações e estoque em atendimento); histórico e relatórios usam leitura pontual paginada. Cancelar ao fechar modal, mudar papel ou filtro. Invalidar cache afetado após mutação e revalidar dados críticos no servidor mesmo com cache recente. Offline permite consultar dados previamente autorizados com aviso de desatualização, mas não confirmar empréstimos, baixas, papéis ou matrículas.

Evitar N+1: carregar referências únicas em lotes e materializações por escopo; nenhuma métrica de cache substitui validação de disponibilidade.

**Q04 e Q14: contrato de aceite e autoatendimento.** Para PESQUISA\_TCC\_POS ou ESTUDO\_DEGRADACAO\_RESIDUOS com frasco vencido ou validade desconhecida liberado com termo, obter aceite em sessão autenticada do retirante. Persistir tcr\_versao, tcr\_aceito\_em (servidor), tcr\_aceito\_por e tcr\_auditoria\_id no empréstimo, com evento correspondente em Registro\_de\_Auditoria; exigir justificativa\_metodologica de 20–2000 caracteres. Modal/checkbox são UX: backend valida versão vigente, identidade e vínculo do aceite ao frasco, finalidade e operação antes de confirmar a retirada. O TCR registra ciência e responsabilidade e não substitui regras institucionais de segurança química. Autoatendimento só quando não houver outro gestor ativo, calculado pelo servidor com justificativa e notificação à chefia. Registrar abertura\_historica\_desconhecida quando a data anterior não for conhecida; nunca inventar a data.

#### UI-11: visibilidade de moderação (DP-C02)

Autor vê texto original com tarja “Comentário moderado pelo docente” e motivo. Demais alunos e bolsistas veem “Comentário ocultado pela moderação da turma”. Professor responsável e Chefe Geral veem original e histórico necessário à moderação/auditoria. Bolsista segue visibilidade de Aluno. A resposta do backend já vem filtrada; Rules impedem obter diretamente o original restrito, inclusive histórico. Ocultação por CSS ou filtro no navegador não é proteção.

## 9 Exemplos de fluxos

### 9.1 Fluxos complementares e critérios de completude

Os fluxos são considerados completos quando descrevem ator, pré-condição, entrada, validação, mudança de estado, efeitos colaterais e resultado. As operações críticas abaixo complementam os cenários de tela já descritos.

#### 9.1.1 Concessão do papel Bolsista

O Chefe Geral seleciona um usuário com papel Aluno. O backend verifica a pré-condição (*Bolsista*  $\implies$  *Aluno*), rejeita a operação se o usuário não for Aluno ou gerar combinação proibida (ex.: Aluno Bolsista atuando como Gestor de Almoxarifado), cria o vínculo de papel na coleção de perfis, registra auditoria e sincroniza claims e só então anuncia a permissão disponível após renovação do token.

#### 9.1.2 Aceitação de convite

O aluno apresenta um convite válido. O backend verifica e-mail normalizado, status, expiração e capacidade da turma. A aceitação é idempotente: o vínculo Aluno–Turma é criado uma única vez e o convite deixa de ser pendente. Usuário e papel Aluno são criados apenas quando ainda não existirem.

#### 9.1.3 Arquivamento e desarquivamento de turma

O professor responsável altera o status da turma para Arquivada. Posts, comentários, histórico e vínculos com alunos não são apagados. O evento é auditado e notificações são emitidas quando aplicável. O desarquivamento restaura o status Ativo sem recriar a turma.

#### 9.1.4 Retirada de reagente

O Gestor de Almoxarifado seleciona o frasco e o Professor ou Bolsista retirante. A transação relê o frasco, valida disponibilidade e escopo do gestor, registra o peso de saída, cria o empréstimo EM\_USO e muda a disponibilidade para EMPRESTADO. Se o frasco estiver sendo aberto pela primeira vez, sua validade é recalculada antes de aprovar. Se outro processo tiver retirado o frasco antes da confirmação, a transação falha sem criar empréstimo parcial.

#### 9.1.5 Devolução de reagente

O Gestor informa o frasco e o peso de retorno. A transação verifica que o empréstimo está EM\_USO ou ATRASADO, recalcula consumo e atraso com dados do servidor (prazo encerrando às 23:59:59.999 do dia previsto), atualiza o empréstimo, grava o evento no histórico e devolve o frasco à disponibilidade adequada ou ao fluxo de descarte/quarentena se vencido ou esgotado.

### i Nota sobre o Login

Quando não houver sessão válida, a etapa inicial é COM-01: o usuário acessa a tela de Login do LCQUI, insere suas credenciais (e-mail e senha) ou escolhe o "Login com Google". O sistema exibe um *spinner* (ícone de carregamento giratório) no botão e carrega conta ativa e permissões antes de abrir a dashboard correspondente. Na mesma sessão, os fluxos seguintes não exigem repetir login.

## 9.2 Fluxos do usuário Gestor de Bem Patrimonial

### 9.2.1 Adicionando um bem patrimonial do LCQUI no sistema

Partindo da tela de login, o Gestor acessa o sistema. Caso possua múltiplos papéis, ele clica no botão "Gestor de Bens Patrimoniais" no header. Na dashboard, clica no botão "Mais (+)" e seleciona "Novo Bem Patrimonial". Um modal se sobrepõe à tela. Ele preenche o número do patrimônio, pesquisa um *Resumo\_Bem* existente, escolhe o Local (prédio, andar, sala) via menus *dropdown* e anexa a URL da foto. Ao clicar em "Salvar", um *spinner* roda no botão e, em seguida, um *toast* (notificação flutuante) verde surge na tela: "Bem cadastrado com sucesso". O modal fecha e a grid central é recarregada mostrando o novo item.

### 9.2.2 Baixando a lista de bens inservíveis para dar baixa no SEI

Partindo da tela de login, o Gestor acessa a dashboard e clica no botão "Gerar Relatórios" na parte superior. O modal de relatórios abre. Ele seleciona o tipo "Bens Inservíveis", confirma e clica em "Gerar PDF". O botão muda para o estado *loading*. Após alguns segundos, o navegador exibe a notificação de download concluído ou abre o arquivo em uma nova aba, pronto para ser anexado ao Sistema Eletrônico de Informações (SEI) da UENF.

### 9.2.3 Baixando o relatório mensal Geral do Mês de Agosto

Após o login, na tela principal, o Gestor clica em "Gerar Relatórios". No modal que se abre, seleciona a aba "Relatório Geral". Nos seletores de data, escolhe "Agosto" e "2026". Ao clicar em "Gerar", um círculo de progresso é exibido no centro do modal. Finalizado o processo backend, o modal exibe a mensagem "PDF pronto" e inicia o download automaticamente.

### 9.2.4 Baixando relatório de bens materiais filtrado (Prédio P5, Andar 1)

Partindo da tela de login, o Gestor abre o modal de relatórios e vai até a aba "Personalizado". Ele preenche as datas (15/06/2026 a 15/07/2026) e abre o filtro avançado. Marca a *checkbox* do "Prédio P5" e digita "1" no andar. Clica em "Gerar PDF". Após a barra de carregamento preencher, o sistema entrega o documento formatado apenas com os equipamentos deste andar específico.

### 9.2.5 Verificando requisições pendentes de professores

Após efetuar o login, a dashboard do Gestor já apresenta no painel central um card de "Painel de Requisições". O botão de notificações (sino) está com uma bolinha vermelha. O Gestor clica na notificação e depois na requisição desejada. A tela faz uma transição para o modo *Split-screen*. Na metade esquerda da tela, ele vê os dados estáticos (antigos) do patrimônio em texto cinza. Na metade direita, ele vê os dados novos sugeridos pelo professor (ex: alteração de sala) destacados em fundo amarelo suave, para chamar a atenção à mudança.

### 9.2.6 Aprovando uma requisição pendente

Dando continuidade ao fluxo de verificação (após login e abertura da tela de *Split-screen*), o Gestor avalia que a alteração de sala pedida pelo professor está correta. Ele clica no botão verde massivo "Aprovar Alteração" no rodapé. Um ícone de *check* (✓) animado aparece no centro da tela. A requisição desaparece da fila de pendentes e um *toast* de sucesso no canto superior direito confirma que o banco de dados principal foi atualizado.

### 9.2.7 Alterando um bem patrimonial para: Dado baixa

Partindo da tela de login, o Gestor usa a barra de busca central e digita o número da plaqueta de um equipamento que já passou por todo o processo burocrático no SEI. A tela de detalhes do item se abre. O Gestor clica no botão "Editar Status". Um menu *dropdown* aparece e ele seleciona "Ja\_dado\_baixa". Ao selecionar esse status, aparece um novo campo obrigatório para fazer upload do documento pdf que comprova seu status. Ao anexar, o sistema libera o botão para salvar a alteração. Porém, antes de salvar, o sistema exibe um modal de atenção: "*Esta ação é irreversível no histórico ativo. Deseja prosseguir?*". O Gestor clica em "Confirmar". Um *toast* azul confirma a baixa e o item recebe instantaneamente uma etiqueta cinza de inatividade, saindo da listagem de bens ativos.

## 9.3 Fluxos do usuário Professor

### 9.3.1 Criando uma turma

Após fazer login, o Professor está na sua dashboard. Ele clica no painel lateral esquerdo em "Turmas" e depois no botão flutuante "+ Nova Turma". Um modal simples é exibido solicitando Nome da Turma, Matéria, Ano, Semestre e Capacidade. Ele preenche "Química Orgânica I", seleciona a matéria correspondente, informa ano 2026, semestre 2 e capacidade 30, e clica em "Salvar". Um *toast* de sucesso é disparado, um novo card desliza aparecendo no painel lateral, e o *codigo\_turma* recém-criado pisca rapidamente na tela com um botão de "Copiar para a área de transferência".

### 9.3.2 Adicionando Alunos em uma turma

Com login feito, o Professor clica no card da turma recém-criada no painel lateral. A área central atualiza para a visão da disciplina. No header, ele clica no botão "Novo Aluno". No modal que surge, ele copia e cola os e-mails dos alunos separados por vírgula e clica em "Enviar Convites". Uma barra de carregamento é mostrada, seguida de um *toast* verde informando que os acessos foram disparados por e-mail.

### 9.3.3 Criando um Post em uma turma

Após login, o Professor acessa a disciplina clicando nela. No topo do Feed de publicações (área central), ele clica na caixa de texto "Comece um novo post...". O componente se expande. Ele preenche título e orientações da aula, clica no ícone de "Clipe de Papel"(anexo), o que abre o modal da sua biblioteca de roteiros em PDF. Ele seleciona o "Roteiro\_Prática\_3.pdf" e clica em "Publicar". A tela rola suavemente para baixo e o novo card da publicação surge com um efeito de *fade-in*, exibindo o texto e a prévia do PDF anexado.

### 9.3.4 Excluindo um Aluno de uma turma

O Professor loga no sistema e navega até a turma desejada. Ele clica na aba "Membros" logo acima do feed. Uma lista com nome e foto dos alunos aparece. Ao lado do nome do aluno a ser removido, ele clica no ícone vermelho de lixeira. Um modal de dupla confirmação exige que ele clique em "Sim, remover aluno". A linha do aluno desaparece da lista com uma rápida animação de encolhimento (*collapse*) e um *toast* avisa "Aluno removido da turma".

### 9.3.5 Respondendo um comentário de um aluno

O Professor entra no sistema e nota uma bolinha vermelha piscando no sino de Notificações. Ele clica e vê "João comentou em um Roteiro". Ao clicar na notificação, a tela rola automaticamente até a exata postagem no feed. Ele vê a dúvida do aluno, clica na caixa de texto abaixo dizendo "Escreva uma resposta..." e aperta Enter. O comentário dele surge instantaneamente logo abaixo do aluno, com seu nome identificado com a tag "[Professor]".

### 9.3.6 Pesquisando Reagente Químico para checar disponibilidade

Partindo da tela de login, o Professor clica no atalho "Reagentes" no painel esquerdo. Na grande barra de pesquisa central, ele começa a digitar "Ácido Clori...". Após selecionar ao menos um filtro obrigatório (UI-04), o sistema usa *debounce* e já filtra a tabela antes mesmo do Enter. Ele encontra o item, clica sobre a linha e um modal lateral (gaveta) desliza da direita mostrando em verde "*Disponível: 3 frascos fechados, 1 aberto*".

### 9.3.7 Pesquisando um bem patrimonial pelo Nome

Após login, o Professor vai à área "Bens Patrimoniais" e digita "Microscópio" na busca. A tela é atualizada mostrando os equipamentos carregados que correspondem ao texto, após filtro

obrigatório, com paginação (UI-04). Ele clica em um dos cards e vê a foto e a localização exata daquele Microscópio em tela cheia.

### 9.3.8 Pesquisando um bem patrimonial pelo número de patrimônio

Professor faz login, acessa a busca de "Bens Patrimoniais" e digita exatamente "987654". O sistema detecta a correspondência exata, pula a exibição da tabela genérica e direciona o professor direto para a página exclusiva de detalhes daquele patrimônio específico.

### 9.3.9 Requisitando alteração de conservação (de Bom para Ruim)

No passo anterior, ao estar na tela de detalhes do microscópio, o Professor, após logar, percebe que a lente está trincada. Ele clica no botão amarelo "Solicitar Edição". No formulário sobreposto, ele altera o campo *dropdown* "Estado de Conservação" de "Bom" para "RUIM" (conservação); Inservível é outro campo, de status, preenche o campo de texto justificando ("Lente principal rachada") e envia. O botão vira um *spinner* rapidamente e um *toast* avisa: "Requisição enviada ao Gestor de Patrimônio".

### 9.3.10 Upload de Roteiro e compartilhamento com outros professores

Professor acessa a dashboard após login, clica em "Gerenciar Roteiros" (no header central). O modal abre na aba "Meus". Ele clica no ícone de Nuvem para upload, arrasta um PDF do seu computador para a área pontilhada e aguarda a barra de progresso encher (100%). Em seguida, no próprio arquivo recém-subido, ele clica em "Compartilhar". Uma lista de professores do sistema aparece, ele marca 3 caixinhas de seleção e clica no botão "Autorizar Acesso". Um alerta flutuante informa que o roteiro agora está visível nas bibliotecas dos colegas.

## 9.4 Fluxos do usuário Chefe Geral

### 9.4.1 Criando um usuário Professor e usuário Gestor de Almoxarifado

O Chefe Geral acessa a plataforma logando com suas credenciais. Na sua dashboard, acessa a aba "Professores" e clica no botão "+ Novo Professor". No modal, insere nome, e-mail institucional, laboratório e seleciona o Centro (ex: CCT). Ao salvar, um *toast* avisa do envio do convite. Da mesma forma, indo para a aba "Almoxarifados", ele clica em "Mais (+)", depois "Novo Gestor de Almoxarifado", preenche o e-mail, e um novo card de gestor é populado silenciosamente na lista do sistema, notificando o sucesso da operação.

## 9.5 Fluxos do usuário Aluno

### 9.5.1 Ingressando em uma Turma

O Aluno acessa o site do LCQUI pela primeira vez, cria a senha (ou entra via Google) e faz o login. A dashboard dele é muito mais limpa e a tela central diz: "Você ainda não tem turmas".

Ele foca sua atenção no canto inferior esquerdo onde há o input "Código da Turma". Ele digita o *código* de até 20 caracteres que o professor escreveu no quadro e clica em "Entrar". O botão gira um carregamento, o sistema verifica a capacidade máxima da turma no *backend*, faz o link de tabelas, e instantaneamente um Card colorido com o nome da disciplina desliza para o centro da tela com o feedback: "*Matrícula realizada com sucesso!*".

### 9.5.2 Baixando um roteiro e realizando um comentário

Logado, o aluno clica na disciplina recém-adicionada. O Feed da turma é carregado na tela. Ele rola a página e acha a postagem do professor com um arquivo. Ele clica no botão vermelho e muito visível de "Baixar PDF". O *download* inicia via navegador. Logo abaixo do mesmo *post*, na seção de interações, ele clica na caixa de texto vazia, digita "Professor, levaremos óculos de proteção?" e pressiona Enviar. O comentário surge na tela logo após uma animação suave de transição (*fade-in*).

## 9.6 Fluxos do Usuário Gestor de Almoxarifado

### 9.6.1 Adicionando um frasco fechado de um reagente existente

Partindo do login, o Gestor de Almoxarifado acessa seu painel, clica no grupo "Estoque", digita o nome de um ácido na busca e clica em "Adicionar Frasco". Ele coloca o frasco lacrado em cima da balança USB (ou lê manualmente), digita no campo "Peso Total" (ex: 610g) e no "Volume Nominal" (ex: 500mL). Ao clicar em "Salvar", o sistema calcula a tara (peso do frasco vazio), registra no banco e o modal se fecha com uma notificação verde "Frasco Fechado registrado no almoxarifado".

### 9.6.2 Adicionando um frasco já aberto de um novo reagente

Após o login, o Gestor nota que o reagente ainda não existe no sistema. Na dashboard central, ele clica em "Novo Reagente". Preenche as propriedades químicas obrigatórias, com destaque em vermelho para o campo "Densidade" quando líquido. Ao confirmar, o modal avança automaticamente para o passo "Cadastrar Frascos Vinculados". Como o frasco físico está pela metade, ele muda a chave seletora para "Informar Volume Atual (Visual)". Ele digita o peso medido agora, e estima 120mL no input visual. Salva a operação, a tela é atualizada e um aviso diz: "Estoque inicial criado e peso do frasco base arquivado".

### 9.6.3 Realizando empréstimo (Retirada) de um frasco

Com o login concluído, um Professor ou Bolsista chega no balcão. O Gestor de Almoxarifado clica rapidamente no enorme botão verde "Registrar Retirada" no centro da dashboard. Um modal abre solicitando a leitura do código do frasco ou pesquisa pelo nome. Localizado o item, o Gestor seleciona o usuário autorizado, informa o local de uso e pesa o frasco.

Antes da confirmação, a interface bloqueia frascos em quarentena ou pendentes de descarte. Se o frasco estiver vencido e seu uso vencido tiver sido autorizado, deve aparecer um alerta

explícito: “Este frasco venceu em DD/MM/AAAA. O uso vencido exige autorização e ciência conforme a finalidade; pesquisa requer TCR rastreável. Deseja confirmar o empréstimo mesmo assim?”. A confirmação é enviada ao backend, que repete a validação; o frontend não pode ignorar essa regra. Se a operação também registrar a primeira abertura do frasco, a validade efetiva é recalculada no servidor dentro da mesma transação.

#### 9.6.4 Devolvendo um frasco (Cálculo de Consumo)

O professor devolve o vidro à bancada. O Gestor faz login (se necessário) e clica no grande botão “Registrar Devolução”. Busca o item (que está na lista de pendentes). O sistema solicita “Peso de Retorno”, calcula o consumo e verifica a validade efetiva com o relógio do servidor (com prazo normalizado para as 23:59:59.999 do dia previsto). Se o peso informado for menor que a tara (N-10), a devolução ordinária é bloqueada e encaminhada ao fluxo Q06 de esgotamento ou recalibração, preservando a leitura real da balança. Diferença de até 5 g exige confirmação de esgotamento para VAZIO e ajuste auditado; diferença maior com produto restante exige recalibração formal de tara com justificativa. Não substituir o peso medido pela tara nem concluir uma devolução fictícia para contornar a validação. Se o frasco estiver vencido, ou tiver vencido enquanto permaneceu com o professor, a devolução permite registrar a decisão do gestor entre quarentena, pendência de descarte ou continuidade de uso excepcional conforme Q04; registrar descarte físico concluído é outra operação.

O sistema renderiza na tela em negrito e com fundo destacado: “**Volume utilizado neste empréstimo: 4.5 mL**” ou “**Peso utilizado nesse empréstimo: 22 g**”. Ao clicar em “Finalizar”, um sinal sonoro/visual confirma que a transação de estoque, consumo, histórico e auditoria foram concluídas.

### 9.7 Cobertura operacional por papel e ação

Cada fluxo parte de sessão ativa validada por COM-01 e usa os campos/estados do contrato UI referenciado na seção 8. Os nomes de coleções e os efeitos descritos seguem o dicionário da seção 5.9. Não repetir login entre ações da mesma sessão. A coluna de resultado inclui persistência e falhas relevantes.

#### 9.7.1 COM-01 — Todos: Login, recuperar senha, perfil, sair e alternar papel

**Contrato:** UI-01. Login ou foto do header; preencher credenciais/perfil ou escolher papel autorizado. Auth e Usuarios determinam sessão; renovar permissões antes da dashboard. Cancelamento preserva sessão anterior; revogação limpa dados e redireciona.

#### 9.7.2 CHE-01 — Chefe: Criar usuário e atribuir Bolsista ou outro papel

**Contrato:** UI-02. Aba Pessoas > Novo > selecionar existente/convidar > preencher dados do papel > revisar > confirmar. Validar combinações e identidade, gravar perfil e auditoria,

sincronizar Auth; lista atualiza após confirmação, falha de claims aparece como pendência.

### 9.7.3 CHE-02 — Chefe: Revogar papel ou autorrevogar chefia

**Contrato:** UI-02. Pessoa > Papéis > Revogar > justificar > revisar conjunto restante. Transação verifica último chefe/gestor e vínculos; rejeição mantém papéis e audita tentativa. Último papel removido desativa conta e encerra acesso.

### 9.7.4 CHE-03 — Chefe: Criar/ativar almoxarifado e vincular gestores

**Contrato:** UI-03. Almoxarifados > Mais > Novo > nome/descrição/local/gestores > Salvar. Criar local antes se necessário. Persistir almoxarifado e vínculos; ativo exige gestor. Gerenciar gestores permite trocar responsáveis sem deixar almoxarifado ativo órfão.

### 9.7.5 CHE-04 — Chefe: Consultar pessoas, turmas e atividade mensal

**Contrato:** UI-02. Selecionar aba, filtro e pessoa > Atividade/Turmas > período. Ler perfis mínimos e agregados por autor/mês; abrir eventos paginados. Sem atividade exibe zero registros, sem confundir com erro de autorização.

### 9.7.6 CHE-05 — Chefe: Operar catálogo, patrimônio, matéria, etiquetas e relatórios

**Contrato:** UI-03 a UI-09 e UI-12. Usar as mesmas entradas dos fluxos ALM/PAT/MAT abaixo com escopo global. Poder administrativo não dispensa campos, comprovante de baixa, destino de vencidos ou auditoria. Gestão de turmas/roteiros de terceiros requer Q13; não simular propriedade.

### 9.7.7 MAT-01 — Professor/Chefe: Cadastrar e editar matéria

**Contrato:** UI-03. Mais > Nova Matéria ou detalhe > Editar > nome/código > confirmar. Reservar código único e persistir Matéria; atualizar projeções e manter vínculos por ID. Código em uso retorna erro no campo.

### 9.7.8 PRO-01 — Professor: Criar turma e copiar código

**Contrato:** UI-10. Turmas > Nova > nome/matéria/ano/semestre/capacidade > Salvar. Criar Turma e código único; selecionar turma e Copiar. Falha da área de transferência não recria turma.

### 9.7.9 PRO-02 — Professor: Arquivar e desarquivar turma

**Contrato:** UI-10. Turma própria > Arquivar > confirmar; ou Turmas Arquivadas > selecionar > Desarquivar. Alterar status mantendo posts/membros; propagar espelhos e notificações. Q08: turmas arquivadas ficam somente em leitura.

**9.7.10 PRO-03 — Professor: Convidar aluno com ou sem turma**

**Contrato:** UI-10. Mais/Novo Aluno > e-mails e matrícula opcional > turma opcional > confirmar ausência de turma ou justificar exceção de capacidade > Enviar. Persistir convites sem conceder papel diferente de Aluno; resultado por destinatário. Aceitação é ALU-01.

**9.7.11 PRO-04 — Professor: Gerenciar membros e remover aluno**

**Contrato:** UI-10. Turma própria > Membros > selecionar > Remover > confirmar. O servidor remove o vínculo principal (Turma/{id}/Alunos) e também propaga a exclusão no espelho inverso (Usuarios/{uid}/Turmas) na mesma transação (N-20), atualiza contador e grava histórico/auditoria; comentários permanecem. Nova entrada por código bloqueada, retorno exige convite.

**9.7.12 PRO-05 — Professor: Cadastrar, visualizar e baixar roteiro**

**Contrato:** UI-11. Mais > Novo Roteiro ou Gerenciar Roteiros > upload > nome/descrição/PDF > registrar. Servidor valida objeto e autoria. Biblioteca > Visualizar/Baixar revalida acesso e entrega arquivo; upload falho não cria roteiro utilizável.

**9.7.13 PRO-06 — Professor: Compartilhar, acompanhar e revogar compartilhamento**

**Contrato:** UI-11. Biblioteca > Meus > Compartilhar > professores > confirmar. Criar vínculo único e notificação; Eu compartilhei mostra destinatários. Revogar exige confirmação e tratamento de posts existentes conforme Q09; recebido não autoriza redistribuição pelo destinatário.

**9.7.14 PRO-07 — Professor: Publicar/editar post e vincular roteiro próprio ou recebido**

**Contrato:** UI-11. Turma própria > compositor/Editar > título/descrição > clipe > escolher roteiro ou cadastrar novo > Publicar. Revalidar acesso ao PDF e turma; persistir Post/histórico, notificar membros e atualizar feed. Falha mantém rascunho na sessão.

**9.7.15 PRO-08 — Professor: Responder comentário e navegar por notificação**

**Contrato:** UI-11 e UI-12. Sino > aviso > post alvo > escrever comentário > Enviar. Revalidar propriedade da turma, persistir Comentário e notificar interessados; mostrar resposta no mesmo post, sem pressupor threads hierárquicas.

#### **9.7.16 PRO-09 — Professor: Consultar reagentes, receber empréstimo e acompanhar consumo**

**Contrato:** UI-04, UI-07 e UI-12. Visualizar > Reagentes > filtros > detalhe > procurar gestor para retirada ALM-04. Meus Reagentes mostra empréstimos próprios e consumo histórico; somente gestor confirma movimentação. Consulta não cria reserva de estoque.

#### **9.7.17 PRO-10 — Professor: Pesquisar patrimônio e requisitar adição/edição**

**Contrato:** UI-04 e UI-09. Bens > filtros/número > detalhe > Solicitar Edição, ou Mais > Pedido de Adição. Preencher proposta/motivo, criar Local se ausente, enviar. Lock e requisição são atômicos; Minhas Requisições acompanha resposta PAT-03.

#### **9.7.18 ALU-01 — Aluno: Aceitar convite ou ingressar por código**

**Contrato:** UI-10. Convite > autenticar e verificar e-mail > completar nome/matricula > Aceitar; ou painel > código > Entrar. Servidor verifica validade, combinações, turma e capacidade/exceção; cria vínculo/espelho/histórico uma vez. Convite inválido ou lotação não cria matrícula parcial.

#### **9.7.19 ALU-02 — Aluno: Visualizar turmas, colegas, posts e baixar roteiro**

**Contrato:** UI-10 e UI-11. Minhas Turmas > turma > Feed/Membros > post > PDF. Ler somente turma com vínculo atual, colegas com dados mínimos e roteiro vinculado. Remoção invalida consultas e acesso ao arquivo; não confiar em espelho/cache antigo.

#### **9.7.20 ALU-03 — Aluno: Comentar e ler notificações**

**Contrato:** UI-11 e UI-12. Post de turma matriculada > texto > Enviar; ou Sino > aviso > post. Validar vínculo e texto, persistir comentário e apresentar confirmação; Limpar tudo marca avisos lidos sem apagar fatos.

#### **9.7.21 ALU-04 — Aluno: Consultar estoque sem retirar**

**Contrato:** UI-04. Visualizar > Reagentes > filtro obrigatório > detalhes de catálogo. Leitura conforme matriz da seção 3, sem identidade de retirantes nem botões de movimentação; concessão de Bolsista não pode ser feita pelo próprio aluno.

#### **9.7.22 BOL-01 — Bolsista: Ações acadêmicas, consulta e retirada/devolução como destinatário**

**Contrato:** UI-04, UI-07, UI-10 a UI-12. Executar ALU-01 a ALU-04 pelo papel Aluno. Consultar reagente, comparecer ao balcão e informar finalidade/local/prazo ao gestor; ALM-04/05 registram saída/retorno. Acompanhar empréstimos próprios; nunca executar mutação de gestor nem acumular Gestor de Almoxarifado.

### 9.7.23 ALM-01 — Gestor Almox.: Cadastrar/editar resumo, substância, especificação e lote

**Contrato:** UI-05. Estoque > Novo Reagente > reutilizar/criar resumo > composição/especificação > lote opcional. Validar identidade e unidades, persistir cada etapa com IDs retornados; densidade confirmada exige nova especificação. Não duplicar resumo por concentração.

### 9.7.24 ALM-02 — Gestor Almox.: Cadastrar frasco fechado ou aberto

**Contrato:** UI-06. Estoque > Adicionar Frasco > especificação > almoxarifado/local/lote > modalidade e pesos > validade/destino > revisar > salvar. Transação gera LCQUI-N e grava frasco/histórico; código retornado guia etiqueta física. Repetição da requisição não gera segundo frasco.

### 9.7.25 ALM-03 — Gestor Almox.: Pesquisar, editar localização, abrir e ajustar estoque por pesagem

**Contrato:** UI-04 e UI-06. Estoque > filtros > frasco > ação. Abrir recalcula validade; editar localização mantém identidade; pesar exige leitura e motivo, produz histórico de ajuste/evaporação. Conflito com empréstimo obriga recarregar; não sobrescrever peso inicial.

### 9.7.26 ALM-04 — Gestor Almox.: Registrar retirada e autorizar uso didático de vencido

**Contrato:** UI-07. Registrar Retirada > código > Professor/Bolsista > local/peso/prazo/finalidade > revisão. Vencido só segue com autorização registrada e ciência explícita; pesquisa excepcional exige justificativa e TCR rastreável Q04. Transação valida disponibilidade e escopo, cria empréstimo e histórico; concorrente perdedor não retira.

### 9.7.27 ALM-05 — Gestor Almox.: Receber devolução de empréstimo próprio ou de outro gestor

**Contrato:** UI-07. Registrar Devolução > frasco/pendência > peso/quem devolveu > destino se vencido > confirmar. Servidor calcula consumo e atraso, encerra empréstimo e atualiza frasco/histórico. Peso incompatível exige revisão; qualquer gestor vinculado pode receber.

### 9.7.28 ALM-06 — Gestor Almox.: Tratar alertas, quarentena, extravio, reencontro, vazio, quebra e descarte

**Contrato:** UI-07. Alertas > categoria > frasco > ação/motivo > confirmar. Validar transição e empréstimo ativo; gravar evento, manter identidade e invalidar saldo/alerta. Descarte é terminal; política de remoção lógica em Q07. Em caso de extravio, o frasco assume estado EXTRAVIADO e um eventual empréstimo ativo sofre encerramento extraordinário. Em caso

de reencontro, o frasco retorna como ABERTO ou FECHADO (conforme constatado), porém obrigatoriamente com *em\_quarentena = true*; o empréstimo encerrado não é reaberto.

#### 9.7.29 ALM-07 — Gestor Almoz.: Imprimir etiquetas virgens ou segunda via

**Contrato:** UI-08. Mais > Imprimir > aba > intervalo/célula ou 1–10 frascos > Gerar > Baixar. Conferir código, imprimir em 100

#### 9.7.30 ALM-08 — Gestor Almoz.: Gerar relatório de um ou todos os almoxarifados autorizados

**Contrato:** UI-12. Relatórios > mensal/período > escopo/filtros > revisar > Gerar/Baixar. Servidor valida 31 dias e permissões, separa g/ml e usa fontes históricas adequadas; erro não anuncia PDF pronto.

#### 9.7.31 PAT-01 — Gestor Patr.: Criar resumo, cadastrar e editar bem

**Contrato:** UI-09. Patrimônio > Mais > Novo Resumo/Bem, ou detalhe > Editar > preencher/revisar > Salvar. Validar plaqueta única, local, foto e responsável; alteração de nome global exige ação no resumo, reclassificação individual altera referência. Histórico preserva antes/depois.

#### 9.7.32 PAT-02 — Gestor Patr.: Pesquisar, filtrar e consultar histórico

**Contrato:** UI-04. Patrimônio > prédio/status/letra e filtros opcionais > resumo > plaqueta > detalhe/histórico. Consultar itens e eventos por cursor; mostrar conservação e status distintos, sem substituir histórico pela localização atual.

#### 9.7.33 PAT-03 — Gestor Patr.: Analisar, aprovar ou rejeitar pedido

**Contrato:** UI-09. Fila/Sino > requisição pendente > comparação > completar dados obrigatórios > justificativa > Aprovar/Rejeitar. Transação confere versão, persiste decisão, altera/cria bem se aprovado e libera lock; notifica solicitante. Segundo respondente recebe conflito, sem dupla aprovação.

#### 9.7.34 PAT-04 — Gestor Patr.: Gerar relação de inservíveis e registrar baixa após SEI

**Contrato:** UI-09 e UI-12. Alertas/Relatórios > Inservíveis > filtros > PDF para procedimento externo. Após conclusão no SEI: bem > Baixa > upload comprovante > confirmar. Somente confirmação persistida altera status; histórico e arquivo permanecem disponíveis.

### 9.7.35 PAT-05 — Gestor Patr.: Gerar relatório mensal, personalizado ou individual

**Contrato:** UI-12. Relatórios ou detalhe do bem > tipo/período/filtros > Gerar > Baixar/Imprimir. Validar intervalo e datas; documento identifica filtros e instante. Mês em aberto é parcial; erro ou dados ausentes têm retorno distinto.

**Q04 e Q14: contrato de aceite e autoatendimento.** Para PESQUISA\_TCC\_POS ou ESTUDO\_DEGRADACAO\_RESIDUOS com frasco vencido ou validade desconhecida liberado com termo, obter aceite em sessão autenticada do retirante. Persistir tcr\_versao, tcr\_aceito\_em (servidor), tcr\_aceito\_por e tcr\_auditoria\_id no empréstimo, com evento correspondente em Registro\_de\_Auditoria; exigir justificativa\_metodologica de 20–2000 caracteres. Modal/checkbox são UX: backend valida versão vigente, identidade e vínculo do aceite ao frasco, finalidade e operação antes de confirmar a retirada. O TCR registra ciência e responsabilidade e não substitui regras institucionais de segurança química. Autoatendimento só quando não houver outro gestor ativo, calculado pelo servidor com justificativa e notificação à chefia. Registrar abertura\_historica\_desconhecida quando a data anterior não for conhecida; nunca inventar a data.

## 10 Tecnologia e Relatórios (Vercel / Firebase)

### 10.1 Para relatórios

Após avaliação das restrições do ambiente *serverless* (Vercel e Firebase Functions), a biblioteca escolhida para a geração de relatórios no backend é o **pdfkit** em conjunto com o plugin **pdfkit-table**.

**Justificativa da escolha:**

- **Baixo consumo de memória:** O **pdfkit** desenha o PDF nativamente, respeitando os limites de memória da Vercel/Firebase.
- **Tabelas dinâmicas:** O **pdfkit-table** resolve a paginação automática de tabelas longas.
- **Fluxo de Armazenamento:** A Cloud Function gera o PDF em um *Buffer* de memória, retorna o PDF em base64 conforme o contrato atual; o cliente cria um Blob temporário para download.

#### 10.1.1 Assinatura Lógica de Relatórios (Hash Canônico)

Todos os relatórios gerados pelo backend injetam um rodapé contendo um Hash Canônico. Este hash compila os parâmetros da requisição (ID, datas e registros consultados) concatenados com o Timestamp (`Date.now()`), servindo como identificador de rastreabilidade, sem equivaler a assinatura digital.

#### 10.1.2 Prevenção de Limites de Leitura Firestore (N+1)

Resolver referências únicas em lotes para reduzir N+1. O tamanho é definido pela API e pelo volume medido; `in` e `getAll` têm contratos distintos, sem limite universal de 200 itens.

#### 10.1.3 Módulo de Geração de Etiquetas (PDF)

Para a identificação física dos frascos, o backend também utiliza o **pdfkit** para desenhar uma matriz (Grid Visual de Offset 3×10) em tamanho A4, com etiquetas de 65,0 mm × 26,5 mm. O código de barras no formato Code 128 é gerado ativamente em memória via biblioteca **bwip-js** e embutido no buffer do PDF, garantindo alta definição na impressão e dispensa de fontes especiais no client. Para frascos já cadastrados (2ª via), o backend gera folhas individuais A4 contendo a Ficha de Conferência e Rastreabilidade com histórico recente de 10 movimentações e 1 única etiqueta de reposição centralizada no rodapé.

##### i Leitura dos exemplos legados

Os listings a seguir preservam exemplos de desenvolvimento, não uma implementação pronta para copiar. O contrato atualizado está nas seções 5.9, 8 e 11: caminhos de especificações são aninhados, datas civis são normalizadas no fuso institucional, relatórios usam base64, consultas e escritas são ordenadas em transação e claims exigem

controle de revogação. Q06 utiliza tolerância híbrida, consumo não negativo e ajuste separado quando há ganho de massa tolerado. Não adotar consumo negativo, data de abertura inventada, placeholder de foto ou simples cópia de novo nome sobre o bem. Esses exemplos devem ser adaptados ao contrato antes de implementação.

## 10.2 Princípio: nunca confiar no *client-side*

Todas as *Cloud Functions* desta seção seguem a mesma regra: **qualquer dado que o servidor consiga derivar ou verificar por conta própria é recalculado ou reconfirmado no servidor, nunca aceito como o cliente enviou**. Isso vale, em particular, para o papel do usuário (claims assinadas pelo Auth, nunca um papel declarado no payload; conta ativa verificada seletivamente conforme DP-D01), para valores calculados como peso/volume/vencimento/atraso (sempre recomputados a partir dos dados brutos e da hora do servidor), para códigos gerados pelo sistema (Regra 6.2.9), e para o estado de disponibilidade de um recurso concorrente (sempre relido dentro de uma transação antes de decidir).

### 10.2.1 Funções de Apoio (Autorização)

Resolução de Papéis via Custom Claims (zero leituras Firestore)

```
import { onCall, HttpsError } from "firebase-functions/v2/https";
import * as admin from "firebase-admin";

// C1: Papéis lidos diretamente do JWT (Firebase Custom Claims).
// Zero leituras extras no Firestore por chamada de API.
// PREREQUISITO: toda mutação de papel deve chamar atualizarCustomClaims(uid).

function resolverPapeisDoToken(auth: {token: Record<string, unknown>} | undefined): string[] {
  if (!auth) return [];
  const roles = auth.token["roles"];
  if (!Array.isArray(roles)) return [];
  return roles as string[];
}

// Chamada após TODA concessão ou remoção de papel:
// await atualizarCustomClaims(uid);
// Isso atualiza claims para o próximo token; não invalida automaticamente o atual; o cliente deve renovar com user.getIdToken(true).

export async function atualizarCustomClaims(uid: string): Promise<void> {
  const colecoes = ["Chefe_Geral", "Gestor_Almojarifado", "Gestor_Bens_Patrimoniais", "Professor", "Aluno", "Bolsista"];
}
```

```

const leituras = await Promise.all(
  colecoes.map((c) => admin.firestore().collection(c).doc(uid).get())
);
const roles = colecoes.filter((_, i) => leituras[i].exists);
await admin.auth().setCustomUserClaims(uid, { roles });
}

// PDF-020: JANELA DE CUSTOM CLAIMS E REVOGACAO
// Operacoes de baixo risco (consultas) confiam apenas no JWT para minimizar
  latencia.
// Mutações de ALTO IMPACTO DEVEM passar requerAtivo=true para mitigar a
  janela de 1 hora
// de cache do JWT. Exemplos normativos: registrarRetirada,
  registrarDevolucao,
// responderRequisicao*, concederPapel, revogarPapel.
async function validarPermissao(
  request: {auth?: {uid: string; token: Record<string, unknown>}},
  papeisPermitidos: string[],
  requerAtivo = false
): Promise<string[]> {
  if (!request.auth?.uid) throw new HttpsError("unauthenticated", "Usuário n
    ão autenticado.");
  if (requerAtivo) {
    const snap = await admin.firestore().collection("Usuarios").doc(request.
      auth.uid).get();
    if (!snap.exists || snap.data()?.ativo !== true)
      throw new HttpsError("permission-denied", "Conta desativada ou não
        habilitada.");
  }
  const papeis = resolverPapeisDoToken(request.auth);
  if (!papeisPermitidos.some((p) => papeis.includes(p)))
    throw new HttpsError("permission-denied", "Usuário sem papel autorizado
      para esta ação.");
  return papeis;
}

async function validarGestorDoAlmoxarifado(uid: string, token: Record<string,
  unknown>, idAlmoxarifado: string): Promise<void> {
  const papeis = resolverPapeisDoToken({ token });
  if (papeis.includes("Chefe_Geral")) return; // Chefe tem escopo global
  if (!papeis.includes("Gestor_Almoxarifado")) {

```

```
    throw new HttpsError("permission-denied", "Usuário não é Gestor de
    Almoxarifado.");
  }
  // 1 leitura: o vínculo de escopo (não o papel - esse já veio do token)
  const vinculo = await admin.firestore().collection("
  Gestor_Almoxarifado_x_Almoxarifado")
    .where("id_gestor_almoxarifado", "==", uid)
    .where("id_almoxarifado", "==", idAlmoxarifado)
    .limit(1).get();
  if (vinculo.empty) {
    throw new HttpsError("permission-denied", "Gestor não está designado
    para este almoxarifado.");
  }
}

// Fechamento de TOCTOU: valida conta, papel e vínculo estritamente dentro
// da transação
async function validarGestorDoAlmoxarifadoTx(tx: FirebaseFirestore.
  Transaction, uid: string, idAlmoxarifado: string): Promise<void> {
  const usuarioSnap = await tx.get(admin.firestore().collection("Usuarios").
    doc(uid));
  if (!usuarioSnap.exists || usuarioSnap.data()?.ativo !== true) {
    throw new HttpsError("permission-denied", "Conta desativada ou não
    habilitada.");
  }

  const chefeSnap = await tx.get(admin.firestore().collection("Chefe_Geral").
    doc(uid));
  if (chefeSnap.exists) return;

  const gestorSnap = await tx.get(admin.firestore().collection("
  Gestor_Almoxarifado").doc(uid));
  if (!gestorSnap.exists) {
    throw new HttpsError("permission-denied", "Usuário não é Gestor de
    Almoxarifado.");
  }

  const vinculoSnap = await tx.get(
    admin.firestore().collection("Gestor_Almoxarifado_x_Almoxarifado")
      .where("id_gestor_almoxarifado", "==", uid)
      .where("id_almoxarifado", "==", idAlmoxarifado)
```

```

        .limit(1)
    );
    if (vinculoSnap.empty) {
        throw new HttpsError("permission-denied", "Gestor não está designado
        para este almoxarifado.");
    }
}
}

// Geração atômica sequencial no Firestore via Documento Singleton (Seção
// 5.7)
// Executado estritamente dentro da transação de cadastro do frasco
async function obterProximoCodigoFrascoTx(
    tx: FirebaseFirestore.Transaction
): Promise<{ proximoNumero: number; novoCodigo: string }> {
    const contadorRef = admin.firestore().collection("Contador_Codigo_Frasco").
        doc("singleton");
    const contadorSnap = await tx.get(contadorRef);

    const ultimoCodigo = contadorSnap.data()?.ultimo_codigo_gerado || 0;
    const proximoNumero = ultimoCodigo + 1;

    tx.set(contadorRef, { ultimo_codigo_gerado: proximoNumero }, { merge: true
        });

    return { proximoNumero, novoCodigo: `LCQUI-${proximoNumero}` };
}

```

#### DP-D01: verificação seletiva e risco residual

Usar `requerAtivo=true` em retirada, devolução, baixa patrimonial, aprovação/rejeição de requisições e concessão/revogação de papéis. As funções `concederPapel` e `revogarPapel` seguem a mesma assinatura. Leituras e mutações sem essa exigência podem continuar confiando em token válido até renovação (aproximadamente uma hora). Verificar ativo não atualiza automaticamente papéis de uma conta ainda ativa. Essa limitação é risco residual aceito da DP-D01, não revogação universal instantânea.

### 10.2.2 Revogação de Papel com Singleton de Controle de Concorrência (M-14, N-01)

revogarPapel com singleton Controle\_Papeis/singleton (RN-ROLE-02 a RN-ROLE-15)

```
// M-14 / N-01: Singleton que arbitra revogações concorrentes.
```

```
// Duas revogações simultâneas leem e escrevem o singleton na mesma transação
o,
// garantindo que somente uma remoção do último responsável seja possível.
const singletonRef = admin.firestore().collection("Controle_Papeis").doc("
  singleton");

export const revogarPapel = onCall(async (request) => {
  const { uidAlvo, papel } = request.data as { uidAlvo: string; papel:
    string };
  await validarPermissao(request, ["Chefe_Geral"], true);

  return admin.firestore().runTransaction(async (tx) => {
    // 1. Leitura do singleton - bloqueia outras transacoes concorrentes que
    //    também escrevam no singleton (transacao serializada pelo Firestore
    //    )
    const singletonSnap = await tx.get(singletonRef);
    const singletonData = singletonSnap.data() || { chefes_ativos: 0,
      gestores_patrimoniais_ativos: 0 };

    // 2. Verificar invariantes de RN-ROLE antes de qualquer escrita
    let vinculos: FirebaseFirestore.QuerySnapshot<Firestore.DocumentData> | null = null;
    if (papel === "Chefe_Geral") {
      // RN-ROLE-02: Chefe Geral não pode revogar outro Chefe Geral
      if (uidAlvo !== request.auth!.uid)
        throw new HttpsError("permission-denied", "Chefe Geral não pode
          revogar outro Chefe Geral.");
      // RN-ROLE-03: auto-revogação só com outro Chefe ativo
      if (singletonData.chefes_ativos <= 1)
        throw new HttpsError("failed-precondition", "Único Chefe Geral: auto-
          revogação bloqueada (RN-ROLE-03).");
    }
    if (papel === "Gestor_Almojarifado") {
      // RN-ROLE-05: Transação bloqueia Almojarifado/{id} e valida
      qtd_gestores_ativos (C003)
      vinculos = await tx.get(
        admin.firestore().collection("Gestor_Almojarifado_x_Almojarifado")
          .where("id_gestor_almojarifado", "==", uidAlvo)
      );
      for (const v of vinculos.docs) {
        const idAlmox = v.data().id_almojarifado;
```

```

    const almoxRef = admin.firestore().collection("Almoxarifado").doc(
idAlmox);
    const almoxSnap = await tx.get(almoxRef);
    if (almoxSnap.exists) {
        const almox = almoxSnap.data()!;
        if (almox.ativo && (almox.qtd_gestores_ativos || 0) <= 1) {
            throw new HttpsError("failed-precondition",
                'Almoxarifado ${idAlmox} ficaria sem gestor ativo (RN-ROLE-05).
');
        }
    }
}

if (papel === "Gestor_Bens_Patrimoniais") {
    // RN-ROLE-09: deve existir ao menos outro gestor patrimonial
    if (singletonData.gestores_patrimoniais_ativos <= 1)
        throw new HttpsError("failed-precondition",
            "Único Gestor de Bens Patrimoniais: revogação bloqueada (RN-ROLE
-09).");
}

// PDF-002: RN-ROLE-BOLSISTA: Bolsista implica Aluno (Bolsista -> Aluno).

// Revogar Aluno com Bolsista ativo viola essa dependência estrutural.
// Política segura: o operador deve revogar Bolsista antes de revogar
Aluno.
if (papel === "Aluno") {
    const bolsistaSnap = await tx.get(
        admin.firestore().collection("Bolsista").doc(uidAlvo)
    );
    if (bolsistaSnap.exists)
        throw new HttpsError("failed-precondition",
            "Este usuario e Bolsista (Bolsista -> Aluno). Revogue o papel
Bolsista antes de revogar Aluno (RN-ROLE-BOLSISTA).");
}

// 3. Verificar se a conta deve ser desativada (RN-ROLE-10/13) -
LEITURAS
const colecoesRestantes = ["Chefe_Geral", "Gestor_Almoxarifado",
    "Gestor_Bens_Patrimoniais", "Professor", "Aluno", "Bolsista"].filter(c
=> c !== papel);
const leituras = await Promise.all(

```

```
    colecoesRestantes.map(c => tx.get(admin.firestore().collection(c).doc(
uidAlvo)))
  );
  const semPapeis = leituras.every(s => !s.exists);

  // 4. Remover o papel e dependências - ESCRITAS
  const papelRef = admin.firestore().collection(papel).doc(uidAlvo);
  tx.delete(papelRef);

  if (papel === "Gestor_Almoxxarifado" && vinculos) {
    // Limpeza em cascata dos vínculos e atualização transacional do
    contador (C003)
    vinculos.docs.forEach((doc) => {
      tx.delete(doc.ref);
      const idAlmox = doc.data().id_almoxxarifado;
      const almoxRef = admin.firestore().collection("Almoxxarifado").doc(
idAlmox);
      tx.update(almoxRef, {
        qtd_gestores_ativos: admin.firestore.FieldValue.increment(-1)
      });
    });
  }

  if (semPapeis) {
    tx.update(admin.firestore().collection("Usuarios").doc(uidAlvo), {
ativo: false });
  }

  // 5. Atualizar singleton (marca a operação, invalida leituras
  concorrentes e decrementa contadores canônicos)
  const atualizacaoSingleton: Record<string, any> = {
    ultima_operacao: { tipo: "REVOGACAO", papel, uid_alvo: uidAlvo,
      uid_operador: request.auth!.uid,
      ts: admin.firestore.FieldValue.serverTimestamp() },
  };

  if (papel === "Chefe_Geral") {
    atualizacaoSingleton.chefes_ativos = admin.firestore.FieldValue.
increment(-1);
  } else if (papel === "Gestor_Bens_Patrimoniais") {
    atualizacaoSingleton.gestores_patrimoniais_ativos = admin.firestore.
FieldValue.increment(-1);
  }
}
```

```

    }
    tx.set(singletonRef, atualizacaoSingleton, { merge: true });

    return { desativado: semPapeis };
  });
  // Fora da transação: atualizar Custom Claims e auditoria
  // (recuperável e idempotente; falha não desfaz a revogação persistida)
});

```

### 10.2.3 Fluxo de Reagentes: Cadastro, Retirada e Devolução

#### Cadastro de Frasco Fechado (Regra 6.2.13, item 1)

```

interface CadastroFrascoFechado {
  idResumoReagente: string;
  idOperacao: string;
  idEspecificacaoReagente: string;
  idAlmoxarifado: string;
  idLote?: string;
  volumeNominal?: number;
  pesoTotal: number;
  validadeFechado?: string;
  validadeDesconhecida?: boolean;
  decisaoSeJaVencido?: "QUARENTENA" | "PENDENTE_DE_DESCARTE" | "DISPONIVEL";
  detalheStatus?: string;
}

export const cadastrarFrascoFechado = onCall(async (request) => {
  const dados = request.data as CadastroFrascoFechado;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxarifado"]);

  const crypto = require("crypto");
  const { DateTime } = require("luxon");
  const { idOperacao, ...payloadRelevante } = dados;
  const payloadString = JSON.stringify(payloadRelevante, Object.keys(
    payloadRelevante).sort());
  const payloadHash = crypto.createHash("sha256").update(payloadString).
    digest("hex");

  const agora = new Date();
  const validadeFechadoStr = dados.validadeDesconhecida ? null : (dados.
    validadeFechado ?? null);

```

```
const validadeData = validadeFechadoStr ? DateTime.fromISO(
  validadeFechadoStr, { zone: "America/Sao_Paulo" }).endOf("day").toDate() : null;
const venceuNoCadastro = Boolean(validadeData && validadeData <= agora);

if (venceuNoCadastro && !dados.decisaoSeJaVencido) {
  throw new HttpsError("failed-precondition", "O frasco está vencido no cadastro e exige uma decisão do gestor.");
}
if (dados.decisaoSeJaVencido === "QUARENTENA" && !dados.detalheStatus) {
  throw new HttpsError("invalid-argument", "A entrada em quarentena exige detalhe_status.");
}

// Execução atômica no Firestore: Sequenciamento + Cadastro do Frasco
return admin.firestore().runTransaction(async (tx) => {
  await validarGestorDoAlmoxarifadoTx(tx, request.auth!.uid, dados.idAlmoxarifado);

  const operacaoRef = admin.firestore().collection("Operacoes").doc(dados.idOperacao);
  const operacaoSnap = await tx.get(operacaoRef);
  if (operacaoSnap.exists) {
    const op = operacaoSnap.data()!;
    if (op.uid === request.auth!.uid && op.tipo_operacao === "CADASTRAR_FECHADO" && op.payload_hash === payloadHash) {
      return op.resultado;
    }
    throw new HttpsError("already-exists", "id_operacao reutilizada com ator ou payload divergente.");
  }

  const resumoRef = admin.firestore().collection("Resumo_Reagente").doc(dados.idResumoReagente);
  const especRef = resumoRef.collection("Especificacoes").doc(dados.idEspecificacaoReagente);

  const [resumoSnap, especSnap] = await Promise.all([tx.get(resumoRef), tx.get(especRef)]);
```

```
if (!especSnap.exists) throw new HttpsError("not-found", "Especificação de reagente não encontrada.");
const resumo = resumoSnap.data()!;
const estadoFisico = resumo.estado_fisico;
const densidade = especSnap.data()!.densidade;

if (dados.idLote) {
  const loteSnap = await tx.get(admin.firestore().collection("Lote").doc(dados.idLote));
  if (!loteSnap.exists) throw new HttpsError("not-found", "Lote não encontrado.");
  if (loteSnap.data()!.id_especificacao_reagente !== dados.idEspecificacaoReagente)
    throw new HttpsError("failed-precondition", "Lote não pertence à especificação informada.");

  const loteMatSnap = await tx.get(admin.firestore().collection("Lote_Materializado").doc(dados.idLote));
  const countAtual = loteMatSnap.exists ? (loteMatSnap.data()!.count || 0) : 0;
  const qtdComprados = loteSnap.data()!.qtd_frascos_comprados as number;
  if (countAtual >= qtdComprados)
    throw new HttpsError("failed-precondition", 'Limite de frascos do lote atingido (${qtdComprados} comprados).');
}

let pesoVazio: number | null = null;
if (dados.volumeNominal !== null) {
  if (estadoFisico === "LIQUIDO") {
    if (!densidade || densidade <= 0) throw new HttpsError("failed-precondition", "Líquido exige densidade positiva.");
    pesoVazio = dados.pesoTotal - (dados.volumeNominal * densidade);
  } else {
    pesoVazio = dados.pesoTotal - dados.volumeNominal;
  }
  if (pesoVazio <= 0) {
    throw new HttpsError("invalid-argument", "Peso total incompatível com o volume nominal para esta densidade.");
  }
}
```

```

// 1. Gera o código sequencial estrito LCQUI-N
const { novoCodigo } = await obterProximoCodigoFrascoTx(tx);

// 2. Persiste o frasco com o código gerado
const frascoRef = admin.firestore().collection("Frasco_Reagente").doc();
tx.set(frascoRef, {
  id_almoxarifado: dados.idAlmoxarifado,
  id_lote: dados.idLote ?? null,
  id_especificacao_reagente: dados.idEspecificacaoReagente, // projeção
  fisica
  id_resumo_reagente: dados.idResumoReagente,
  eh_higroscopico: resumo.eh_higroscopico, // snapshot servidor imutável
  codigo_frasco: novoCodigo,
  conteudo_nominal: dados.volumeNominal ?? null,
  peso_no_cadastrado: dados.pesoTotal,
  peso_atual: dados.pesoTotal,
  peso_frasco_vazio: pesoVazio,
  medida_usada: 0,
  estado_fisico_frasco: "FECHADO",
  disponibilidade: "DISPONIVEL",
  validade_fechado: validadeFechadoStr,
  validade_efetiva: validadeFechadoStr,
  validade_desconhecida: dados.validadeDesconhecida ?? false,
  vencido: venceuNoCadastro,
  em_quarentena: venceuNoCadastro && dados.decisaoSeJaVencido === "
  QUARENTENA",
  uso_vencido_autorizado: venceuNoCadastro && dados.decisaoSeJaVencido
  === "DISPONIVEL",
  detalhe_status: venceuNoCadastro
    ? dados.detalheStatus ?? (dados.decisaoSeJaVencido === "
  PENDENTE_DE_DESCARTE" ? "Frasco vencido aguardando processo institucional
  de descarte." : null)
    : null,
  cadastrado_em: admin.firestore.FieldValue.serverTimestamp(),
  cadastrado_por: request.auth!.uid,
});

if (dados.idLote) {
  tx.set(admin.firestore().collection("Lote_Materializado").doc(dados.
  idLote), {
    qtd_frascos_cadastrados: admin.firestore.FieldValue.increment(1)
  });
}

```

```

    }, { merge: true });
  }

  // M-15: evento CADASTRO no histórico do frasco, gravado na mesma transação
  const historicoRef = admin.firestore().collection("Historico_Frasco_Reagente").doc();
  tx.set(historicoRef, {
    id_frasco_reagente: frascoRef.id,
    id_almoxarifado: dados.idAlmoxarifado,
    id_gestor: request.auth!.uid,
    tipo: "CADASTRO",
    timestamp: admin.firestore.FieldValue.serverTimestamp(),
  });

  const resultado = { idFrasco: frascoRef.id, codigoFrasco: novoCodigo,
    pesoVazioCalculado: pesoVazio, venceuNoCadastro };

  tx.set(operacaoRef, {
    tipo_operacao: "CADASTRAR_FECHADO",
    uid: request.auth!.uid,
    payload_hash: payloadHash,
    resultado,
    status: "CONCLUIDA",
    criado_em: admin.firestore.FieldValue.serverTimestamp(),
    atualizado_em: admin.firestore.FieldValue.serverTimestamp()
  });

  return resultado;
});
});

```

#### cadaststrarFrascoAberto com bifurcação SÓLIDO/LÍQUIDO (C5)

```

// C5: modalidade para SÓLIDO usa ESTIMA_MASSA (não ESTIMA_VOLUME)
interface CadastroFrascoAberto {
  idOperacao: string;
  idResumoReagente: string;
  idEspecificacaoReagente: string;
  idAlmoxarifado: string;
  idLote?: string;
  modalidade: "CONHECE_TARA" | "ESTIMA_VOLUME" | "ESTIMA_MASSA";
}

```

```
pesoTotalBalanca: number;
volumeNominal?: number;
pesoFrascoVazioInformado?: number;
volumeAtualEstimado?: number;
massaAtualEstimada?: number;
dataAbertura?: string;
aberturaHistoricaDesconhecida: boolean;
validadeAberto?: string;
validadeDesconhecida?: boolean;
decisaoSeJaVencido?: "QUARENTENA" | "PENDENTE_DE_DESCARTE" | "DISPONIVEL";
detalheStatus?: string;
}

export const cadastrarFrascoAberto = onCall(async (request) => {
  const dados = request.data as CadastroFrascoAberto;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxxarifado"]);

  const crypto = require("crypto");
  const { DateTime } = require("luxon");
  const { idOperacao, ...payloadRelevante } = dados;
  const payloadString = JSON.stringify(payloadRelevante, Object.keys(
    payloadRelevante).sort());
  const payloadHash = crypto.createHash("sha256").update(payloadString).
    digest("hex");

  const agora = new Date();
  const validadeAbertoStr = dados.validadeDesconhecida ? null : (dados.
    validadeAberto ?? null);
  const validadeData = validadeAbertoStr ? DateTime.fromISO(
    validadeAbertoStr, { zone: "America/Sao_Paulo" }).endOf("day").toJSDate()
    : null;
  const jaVencido = Boolean(validadeData && validadeData <= agora);

  if (jaVencido && !dados.decisaoSeJaVencido)
    throw new HttpsError("failed-precondition", "O frasco está vencido no
      cadastro e exige uma decisão do gestor.");
  if (dados.decisaoSeJaVencido === "QUARENTENA" && !dados.detalheStatus)
    throw new HttpsError("invalid-argument", "A entrada em quarentena exige
      detalhe_status.");

  return admin.firestore().runTransaction(async (tx) => {
```

```
await validarGestorDoAlmoxarifadoTx(tx, request.auth!.uid, dados.idAlmoxarifado);

const operacaoRef = admin.firestore().collection("Operacoes").doc(dados.idOperacao);
const operacaoSnap = await tx.get(operacaoRef);
if (operacaoSnap.exists) {
  const op = operacaoSnap.data()!;
  if (op.uid === request.auth!.uid && op.tipo_operacao === "CADASTRAR_ABERTO" && op.payload_hash === payloadHash) {
    return op.resultado;
  }
  throw new HttpsError("already-exists", "id_operacao reutilizada com ator ou payload divergente.");
}

const resumoRef = admin.firestore().collection("Resumo_Reagente").doc(dados.idResumoReagente);
const especRef = resumoRef.collection("Especificacoes").doc(dados.idEspecificacaoReagente);

const [resumoSnap, especSnap] = await Promise.all([tx.get(resumoRef), tx.get(especRef)]);
if (!especSnap.exists) throw new HttpsError("not-found", "Especificação não encontrada.");

const especData = especSnap.data()!;
const resumo = resumoSnap.data()!;
const estadoFisico: "SOLIDO" | "LIQUIDO" = resumo.estado_fisico;
const densidade: number | null = especData.densidade ?? null;

if (dados.idLote) {
  const loteSnap = await tx.get(admin.firestore().collection("Lote").doc(dados.idLote));
  if (!loteSnap.exists) throw new HttpsError("not-found", "Lote não encontrado.");
  if (loteSnap.data()!.id_especificacao_reagente !== dados.idEspecificacaoReagente)
    throw new HttpsError("failed-precondition", "Lote não pertence à especificação informada.");
}
```

```
    const loteMatSnap = await tx.get(admin.firestore().collection("
Lote_Materializado").doc(dados.idLote));
    const countAtual = loteMatSnap.exists ? (loteMatSnap.data()!.count ||
0) : 0;
    const qtdComprados = loteSnap.data()!.qtd_frascos_comprados as number;
    if (countAtual >= qtdComprados)
        throw new HttpsError("failed-precondition", 'Limite de frascos do
lote atingido (${qtdComprados} comprados).');
    }

    // C5: bifurcação explícita por estado físico
    let pesoVazio: number | null = null;
    const conteudoNominal = dados.volumeNominal ?? null;

    if (estadoFisico === "SOLIDO") {
        if (dados.modalidade === "CONHECE_TARA") {
            if (dados.pesoFrascoVazioInformado == null) throw new HttpsError("
invalid-argument", "CONHECE_TARA exige pesoFrascoVazioInformado.");
            pesoVazio = dados.pesoFrascoVazioInformado;
        } else {
            if (dados.massaAtualEstimada == null) throw new HttpsError("invalid-
argument", "ESTIMA_MASSA exige massaAtualEstimada.");
            pesoVazio = dados.pesoTotalBalanca - dados.massaAtualEstimada;
        }
        if (pesoVazio <= 0) throw new HttpsError("invalid-argument", "Massa
atual incompatível ou maior que o peso medido.");
    } else {
        if (!densidade || densidade <= 0) throw new HttpsError("failed-
precondition", "Líquido exige densidade positiva.");
        if (dados.modalidade === "CONHECE_TARA") {
            if (dados.pesoFrascoVazioInformado == null) throw new HttpsError("
invalid-argument", "CONHECE_TARA exige pesoFrascoVazioInformado.");
            pesoVazio = dados.pesoFrascoVazioInformado;
        } else {
            if (dados.volumeAtualEstimado == null) throw new HttpsError("invalid-
argument", "ESTIMA_VOLUME exige volumeAtualEstimado.");
            pesoVazio = dados.pesoTotalBalanca - (dados.volumeAtualEstimado *
densidade);
        }
        if (pesoVazio <= 0) throw new HttpsError("invalid-argument", "Dados
inconsistentes com a densidade.");
    }
```

```
}

const { novoCodigo } = await obterProximoCodigoFrascoTx(tx);
const frascoRef = admin.firestore().collection("Frasco_Reagente").doc();

tx.set(frascoRef, {
  id_almoxarifado: dados.idAlmoxarifado,
  id_lote: dados.idLote ?? null,
  id_especificacao_reagente: dados.idEspecificacaoReagente,
  id_resumo_reagente: dados.idResumoReagente,
  eh_higroscopico: resumo.eh_higroscopico,
  codigo_frasco: novoCodigo,
  conteudo_nominal: conteudoNominal,
  peso_no_cadastrado: dados.pesoTotalBalanca,
  peso_atual: dados.pesoTotalBalanca,
  peso_frasco_vazio: pesoVazio,
  medida_usada: 0,
  data_abertura: dados.aberturaHistoricaDesconhecida ? null : dados.
dataAbertura,
  abertura_historica_desconhecida: dados.aberturaHistoricaDesconhecida,
  estado_fisico_frasco: "ABERTO",
  disponibilidade: "DISPONIVEL",
  validade_efetiva: validadeAbertoStr,
  validade_desconhecida: dados.validadeDesconhecida ?? false,
  vencido: jaVencido,
  em_quarentena: jaVencido && dados.decisaoSeJaVencido === "QUARENTENA",
  uso_vencido_autorizado: jaVencido && dados.decisaoSeJaVencido === "
DISPONIVEL",
  detalhe_status: jaVencido
    ? (dados.detalheStatus ?? (dados.decisaoSeJaVencido === "
PENDENTE_DE_DESCARTE"
      ? "Frasco vencido aguardando processo institucional de descarte.
" : null))
    : null,
  cadastrado_em: admin.firestore.FieldValue.serverTimestamp(),
  cadastrado_por: request.auth!.uid,
});

if (dados.idLote) {
  tx.set(admin.firestore().collection("Lote_Materializado").doc(dados.
idLote), {
```

```
    qtd\_frascos\_cadastrados: admin.firestore.FieldValue.increment(1)
  }, { merge: true });
}

const historicoRef = admin.firestore().collection("
Historico_Frasco_Reagente").doc();
tx.set(historicoRef, {
  id_frasco_reagente: frascoRef.id,
  id_almoxarifado: dados.idAlmoxarifado,
  id_gestor: request.auth!.uid,
  tipo: "CADASTRO",
  timestamp: admin.firestore.FieldValue.serverTimestamp(),
});

const resultado = { idFrasco: frascoRef.id, codigoFrasco: novoCodigo,
conteudoNominal, pesoVazio, venceuNoCadastro: jaVencido };

tx.set(operacaoRef, {
  tipo_operacao: "CADASTRAR_ABERTO",
  uid: request.auth!.uid,
  payload_hash: payloadHash,
  resultado,
  status: "CONCLUIDA",
  criado_em: admin.firestore.FieldValue.serverTimestamp(),
  atualizado_em: admin.firestore.FieldValue.serverTimestamp()
});

return resultado;
});
});
```

#### Abertura e cálculo de validade efetiva no servidor

```
function calcularValidadeEfetivaNaAbertura(frasco: any, dataAbertura: Date):
Date | null {
  if (frasco.validade_desconhecida) return null;

  const validadeFechado = frasco.validade_fechado
  ? (typeof frasco.validade_fechado.toDate === "function"
    ? frasco.validade_fechado.toDate()
    : DateTime.fromISO(frasco.validade\_fechado, { zone: "America/
Sao_Paulo" }).endOf("day").toJSDate())
```

```
    : null;
const diasDepoisDeAberto = frasco.validade_apos_aberto_dias;
const validadeDepoisDaAbertura = diasDepoisDeAberto != null
  ? DateTime.fromJSDate(new Date(dataAbertura.getTime() +
    diasDepoisDeAberto * 24 * 60 * 60 * 1000), { zone: "America/Sao_Paulo" })
    .endOf("day").toJSDate()
    : null;

if (validadeFechado && validadeDepoisDaAbertura) {
  return validadeFechado <= validadeDepoisDaAbertura ? validadeFechado :
    validadeDepoisDaAbertura;
}
return validadeFechado ?? validadeDepoisDaAbertura;
}

interface AberturaFrasco {
  idFrasco: string;
  destinoSeVencerNaAbertura?: "QUARENTENA" | "PENDENTE_DE_DESCARTE" | "
    DISPONIVEL";
  detalheStatus?: string;
}

export const registrarAberturaFrasco = onCall(async (request) => {
  const dados = request.data as AberturaFrasco;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxxarifado"]);

  const frascoRef = admin.firestore().collection("Frasco_Reagente").doc(
    dados.idFrasco);
  return admin.firestore().runTransaction(async (tx) => {
    const snap = await tx.get(frascoRef);
    if (!snap.exists) throw new HttpsError("not-found", "Frasco não
      encontrado.");
    const frasco = snap.data()!;
    await validarGestorDoAlmoxxarifadoTx(tx, request.auth!.uid, frasco.
      id_almoxxarifado);

    if (frasco.estado_fisico_frasco !== "FECHADO") {
      throw new HttpsError("failed-precondition", "Somente um frasco FECHADO
        pode ser aberto por esta operação.");
    }
  })
})
```

```

const agora = new Date();
const validadeEfetiva = calcularValidadeEfetivaNaAbertura(frasco, agora);

const venceu = Boolean(validadeEfetiva && validadeEfetiva <= agora);

if (venceu && !dados.destinoSeVencerNaAbertura) {
  throw new HttpsError("failed-precondition", "A abertura tornou o
  frasco vencido. O gestor deve escolher o destino do frasco.");
}
if (venceu && dados.destinoSeVencerNaAbertura === "QUARENTENA" && !dados.
detalheStatus) {
  throw new HttpsError("invalid-argument", "A entrada em quarentena
  exige detalhe_status.");
}

tx.update(frascoRef, {
  estado_fisico_frasco: "ABERTO",
  data_abertura: DateTime.fromJSDate(agora, { zone: "America/Sao_Paulo"
}).toISODate(),
  validade_efetiva: validadeEfetiva ? DateTime.fromJSDate(
validadeEfetiva, { zone: "America/Sao_Paulo" }).toISODate() : null,
  vencido: venceu,
  em_quarentena: venceu && dados.destinoSeVencerNaAbertura === "
QUARENTENA",
  uso_vencido_autorizado: venceu && dados.destinoSeVencerNaAbertura ===
"DISPONIVEL",
  detalhe_status: venceu
    ? (dados.detalheStatus ?? (dados.destinoSeVencerNaAbertura === "
PENDENTE_DE_DESCARTE"
    ? "Frasco vencido aguardando processo institucional de descarte."
    : null))
    : null,
});

return { validadeEfetiva, venceu, destino: dados.
destinoSeVencerNaAbertura ?? null };
});
});

```

#### Registrar Retirada (Empréstimo)

```
interface RetiradaFrasco {
```

```
idFrasco: string;
idUsuarioRetirou: string;
idLocalUsado: string;
pesoSaida: number;
dataDevolucaoPrevista: string;
confirmarUsoVencido?: boolean;
abrirNoEmprestimo?: boolean;
finalidadeUso: "AULA_PRATICA" | "DEMONSTRACAO" | "PESQUISA_TCC_POS" | "
  ESTUDO_DEGRADACAO_RESIDUOS";
justificativaMetodologica?: string;
idAceiteTcr?: string; // evento prévio autenticado pelo retirante
justificativaAutoAtendimento?: string;
}

export const registrarRetirada = onCall(async (request) => {
  const dados = request.data as RetiradaFrasco;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Almojarifado"],
    true);

  // M-05: verificar conta ativa do tomador (Professor ou Bolsista)
  // Custom Claims não estão disponíveis para terceiros; leitura Firestore
  // necessária (1 ou 2 leituras).
  const [profSnap, bolsSnap, tomadorSnap] = await Promise.all([
    admin.firestore().collection("Professor").doc(dados.idUsuarioRetirou).
      get(),
    admin.firestore().collection("Bolsista").doc(dados.idUsuarioRetirou).get
      (),
    admin.firestore().collection("Usuarios").doc(dados.idUsuarioRetirou).get
      (),
  ]);
  if (!profSnap.exists && !bolsSnap.exists) {
    throw new HttpsError("failed-precondition", "Usuário selecionado não é
      Professor nem Bolsista.");
  }
  if (!tomadorSnap.exists || tomadorSnap.data()?.ativo !== true) {
    throw new HttpsError("failed-precondition", "Conta do tomador está
      desativada ou não encontrada.");
  }

  const frascoRef = admin.firestore().collection("Frasco_Reagente").doc(
    dados.idFrasco);
```

```
return admin.firestore().runTransaction(async (tx) => {
  const frascoSnap = await tx.get(frascoRef);
  if (!frascoSnap.exists) throw new HttpsError("not-found", "Frasco não encontrado.");
  const frasco = frascoSnap.data()!;

  await validarGestorDoAlmoxarifadoTx(tx, request.auth!.uid, frasco.id_almoxarifado);
  if (frasco.disponibilidade !== "DISPONIVEL") {
    throw new HttpsError("failed-precondition", "Frasco não está disponível para retirada.");
  }
  if (frasco.estado_fisico_frasco !== "FECHADO" && frasco.estado_fisico_frasco !== "ABERTO") {
    throw new HttpsError("failed-precondition", "Estado físico incompatível com retirada.");
  }
  if (frasco.em_quarentena) {
    throw new HttpsError("failed-precondition", "Frasco em quarentena não pode ser retirado.");
  }

  const agora = new Date();
  const { DateTime } = require("luxon");
  let validadeEfetiva = frasco.validade_efetiva
    ? (typeof frasco.validade_efetiva.toDate === "function"
      ? frasco.validade_efetiva.toDate()
      : DateTime.fromISO(frasco.validade_efetiva, { zone: "America/Sao_Paulo" }).endOf("day").toJSDate())
    : null;
  let frascoVencido = Boolean(validadeEfetiva && validadeEfetiva <= agora);

  if (dados.abrirNoEmprestimo && frasco.estado_fisico_frasco === "FECHADO") {
    validadeEfetiva = calcularValidadeEfetivaNaAbertura(frasco, agora);
    frascoVencido = Boolean(validadeEfetiva && validadeEfetiva <= agora);
  }

  const excepcional = frascoVencido || frasco.validade_desconhecida;
```

```
if (excepcional && (!frasco.uso_vencido_autorizado || !dados.
confirmarUsoVencido))
    throw new HttpsError("failed-precondition", "Uso excepcional exige
autorização e ciência.");
const finalidade = dados.finalidadeUso; // nunca substituir finalidade
declarada
const pesquisa = ["PESQUISA_TCC_POS", "ESTUDO_DEGRADACAO_RESIDUOS"].
includes(finalidade);
// Helpers contratuais: todas as leituras devem ocorrer antes da
primeira escrita.
// Aceite foi criado por endpoint autenticado do retirante, não pelo
gestor.
const aceite = excepcional && pesquisa
    ? await validarAceiteTcrTx(tx, dados.idAceiteTcr, {
        retirante: dados.idUsuarioRetirou, frasco: dados.idFrasco,
        finalidade, justificativa: dados.justificativaMetodologica,
    }) : null;
const autoAtendimento = await validarAutoAtendimentoTx(tx, request.auth!.
uid,
    dados.idUsuarioRetirou, frasco.id_almoxarifado, dados.
justificativaAutoAtendimento);
const quimica = await resolverSnapshotQuimicoTx(tx, frasco);

const emprestimoRef = admin.firestore().collection("Emprestimo_Reagente"
).doc();
tx.set(emprestimoRef, {
    id_frasco_reagente: dados.idFrasco,
    // M-20: id_usuario_retirou = o portador físico (Professor ou Bolsista
que vai usar o frasco)
    // id_gestor_retirada = o operador do almoxarifado que registrou a
operação (uid do requisitante)
    id_usuario_retirou: dados.idUsuarioRetirou,
    id_gestor_retirada: request.auth!.uid,
    id_almoxarifado: frasco.id_almoxarifado,
    status: "EM_USO",
    data_retirada: admin.firestore.FieldValue.serverTimestamp(),
    data_devolucao_prevista: DateTime.fromISO(dados.
dataDevolucaoPrevista, { zone: "America/Sao_Paulo" }).endOf("day").
toJSDate(),
    peso_perda_evaporacao: 0,
    id_local_usado: dados.idLocalUsado,
```

```
    peso_saida: dados.pesoSaida,
    uso_vencido_aceito: Boolean(frascoVencido),
    finalidade_uso: finalidade,
    auto_atendimento: autoAtendimento,
    justificativa_metodologica: dados.justificativaMetodologica ?? null,
    tcr_versao: aceite?.versao ?? null,
    tcr_aceito_em: aceite?.aceito_em ?? null,
    tcr_aceito_por: aceite?.uid ?? null,
    tcr_auditoria_id: aceite?.auditoria_id ?? null,
    unidade_medida_utilizada: quimica.unidade,
    densidade_aplicada: quimica.densidade,
  });

  tx.update(frascoRef, {
    disponibilidade: "EMPRESTADO",
    ...(dados.abrirNoEmprestimo && frasco.estado_fisico_frasco === "
    FECHADO"
    ? { estado_fisico_frasco: "ABERTO", data_abertura: DateTime.
    fromJSDate(agora, { zone: "America/Sao_Paulo" }).toISODate(),
      validade_efetiva: validadeEfetiva ? DateTime.fromJSDate(
    validadeEfetiva, { zone: "America/Sao_Paulo" }).toISODate() : null,
      vencido: frascoVencido } : {}),
  });

  if (aceite) vincularAceiteEAuditoriaTx(tx, aceite, emprestimoRef.id);
  if (autoAtendimento) registrarNotificacaoChefiaTx(tx, emprestimoRef.id);

  const historicoRef = admin.firestore().collection("
  Historico_Frasco_Reagente").doc();
  tx.set(historicoRef, {
    id_frasco_reagente: dados.idFrasco,
    id_almoxarifado: frasco.id_almoxarifado,
    id_gestor: request.auth!.uid,
    tipo: "SAIU",
    id_emprestimo_reagente: emprestimoRef.id,
    timestamp: admin.firestore.FieldValue.serverTimestamp(),
  });

  return { idEmprestimo: emprestimoRef.id };
});
});
```

## Registrar Devolução com atualização física e histórico completo

```
interface DevolucaoFrasco {
  idEmprestimo: string;
  pesoRetorno: number;
  idUsuarioDevolveu: string; // Portador físico que entrega o frasco ao
    almoxarifado
  destinoPosDevolucao?: "QUARENTENA" | "PENDENTE_DE_DESCARTE" | "DISPONIVEL";
}

export const registrarDevolucao = onCall(async (request) => {
  const dados = request.data as DevolucaoFrasco;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxarifado"],
    true);

  const emprestimoRef = admin.firestore().collection("Emprestimo_Reagente").
    doc(dados.idEmprestimo);

  return admin.firestore().runTransaction(async (tx) => {
    const emprestimoSnap = await tx.get(emprestimoRef);
    if (!emprestimoSnap.exists) throw new HttpsError("not-found", "Empré
      stimo não encontrado.");
    const emprestimo = emprestimoSnap.data()!;
    if (!["EM_USO", "ATRASADO"].includes(emprestimo.status)) {
      throw new HttpsError("failed-precondition", "Empréstimo já foi
        devolvido.");
    }

    const frascoRef = admin.firestore().collection("Frasco_Reagente").doc(
      emprestimo.id_frasco_reagente);
    const frascoSnap = await tx.get(frascoRef);
    if (!frascoSnap.exists) throw new HttpsError("not-found", "Frasco não
      encontrado.");
    const frasco = frascoSnap.data()!;
    await validarGestorDoAlmoxarifadoTx(tx, request.auth!.uid, frasco.
      id_almoxarifado);

    const liquido = emprestimo.unidade_medida_utilizada === "ml";
    const densidade = liquido ? emprestimo.densidade_aplicada : null;
    if (liquido && (!Number.isFinite(densidade) || densidade <= 0))
      throw new HttpsError("failed-precondition", "Densidade histórica invá
```

```
lida.");
const pesoConsumido = Math.max(0, emprestimo.peso_saida - dados.
pesoRetorno);

if (typeof frasco.eh_higroscopico !== "boolean")
  throw new HttpsError("failed-precondition", "Snapshot histórico
ausente: reconciliar frasco.");

// PDF-025: FORMULA CANONICA Q06
// A margem de tolerância é estritamente calculada sobre o PESO BRUTO (
peso_saida).
// Rejeitada explicitamente a sugestão de calcular sobre massa líquida,
// pois a variância ocorre sobre a medição total na balança.
// Fórmula:
// Normal: max(1g, 0.5% peso_saida)
// Higroscópico: max(2g, 2.0% peso_saida)

const deltaMax = frasco.eh_higroscopico
  ? Math.max(2.0, 0.02 * emprestimo.peso_saida)
  : Math.max(1.0, 0.005 * emprestimo.peso_saida);
const limiteMaxRetorno = emprestimo.peso_saida + deltaMax;
if (!Number.isFinite(dados.pesoRetorno) || dados.pesoRetorno <= 0) {
  throw new HttpsError("invalid-argument", "Peso de retorno inválido.");
}
// PDF-025 / N-10: Verificação de tara na devolução (Q06)
if (frasco.peso_frasco_vazio != null && dados.pesoRetorno < frasco.
peso_frasco_vazio) {
  throw new HttpsError("failed-precondition", "Peso de retorno menor que
a tara cadastrada. Operação ordinária bloqueada: desviar para recalibraç
ão de tara ou esgotamento (Q06).");
}
// PDF-025: Ganho acima da tolerância bloqueia por anomalia.
if (dados.pesoRetorno > limiteMaxRetorno) {
  throw new HttpsError("invalid-argument", "Ganho de massa excede tolerâ
ncia Q06 (possível erro ou contaminação). Bloqueado por anomalia.");
}
const avisoHigroscopico = dados.pesoRetorno > emprestimo.peso_saida;
const volumeUtilizado = densidade ? pesoConsumido / densidade :
pesoConsumido;

const agora = new Date();
```

```
// DP-D02: Normalização via Luxon (America/Sao_Paulo). Prazo encerra-se às 23:59:59.999 do dia previsto.
const { DateTime } = require("luxon");
const prazoLimite = DateTime.fromJSDate(emprestimo.data_devolucao_prevista.toDate(), { zone: "America/Sao_Paulo" })
  .endOf("day").toJSDate();
const atrasado = agora > prazoLimite;

const validadeEfetiva = frasco.validade_efetiva
  ? (typeof frasco.validade_efetiva.toDate === "function"
    ? frasco.validade_efetiva.toDate()
    : DateTime.fromISO(frasco.validade_efetiva, { zone: "America/Sao_Paulo" }).endOf("day").toJSDate())
  : null;
const venceuAgora = Boolean(validadeEfetiva && validadeEfetiva <= agora);

const frascoVencido = Boolean(frasco.vencido || venceuAgora);

const atualizacaoFrasco: Record<string, unknown> = {
  disponibilidade: "DISPONIVEL",
  data_ultima_pesagem: admin.firestore.FieldValue.serverTimestamp(),
  peso_atual: dados.pesoRetorno,
  medida_usada: admin.firestore.FieldValue.increment(pesoConsumido), //
  sempre g
  vencido: frascoVencido,
};

if (frascoVencido && dados.destinoPosDevolucao === "QUARENTENA") {
  atualizacaoFrasco.em_quarentena = true;
  atualizacaoFrasco.uso_vencido_autorizado = false;
  atualizacaoFrasco.detalhe_status = "Frasco vencido retido em quarentena após devolução.";
} else if (frascoVencido && dados.destinoPosDevolucao === "DISPONIVEL") {
  atualizacaoFrasco.em_quarentena = false;
  atualizacaoFrasco.uso_vencido_autorizado = true;
} else if (frascoVencido && dados.destinoPosDevolucao === "PENDENTE_DE_DESCARTE") {
  atualizacaoFrasco.em_quarentena = false;
  atualizacaoFrasco.uso_vencido_autorizado = false;
  atualizacaoFrasco.detalhe_status = "Frasco vencido aguardando processo
```

```
    institucional de descarte.";
  }

  tx.update(emprestimoRef, {
    status: atrasado ? "DEVOLVIDO_COM_ATRASO" : "DEVOLVIDO",
    data_devolucao_efetuada: admin.firestore.FieldValue.serverTimestamp(),
    id_usuario_devolveu: dados.idUsuarioDevolveu,
    id_gestor_devolucao: request.auth!.uid,
    peso_retorno: dados.pesoRetorno,
    medida_utilizada: volumeUtilizado,
    unidade_medida_utilizada: densidade ? "ml" : "g",
    ...(avisoHigroscopico && { aviso: "higroscopico_suspeito" }),
  });
  tx.update(frascoRef, atualizacaoFrasco);

  if (avisoHigroscopico) {
    tx.set(admin.firestore().collection("Historico_Frasco_Reagente").doc(), {
      id_frasco_reagente: frascoRef.id,
      id_almoxarifado: frasco.id_almoxarifado,
      id_gestor: request.auth!.uid,
      id_emprestimo_reagente: emprestimoRef.id,
      tipo: "AJUSTE", campo_ajustado: "ganho_massa_higroscopia",
      peso_anterior: emprestimo.peso_saida, peso_novo: dados.pesoRetorno,
      medida_ajustada: dados.pesoRetorno - emprestimo.peso_saida,
      unidade_medida_ajustada: "g",
      timestamp: admin.firestore.FieldValue.serverTimestamp(),
    });
  }

  // Registro obrigatório de devolução no histórico
  const histRef = admin.firestore().collection("Historico_Frasco_Reagente")
    .doc();
  tx.set(histRef, {
    id_frasco_reagente: emprestimo.id_frasco_reagente,
    id_almoxarifado: frasco.id_almoxarifado,
    id_gestor: request.auth!.uid,
    tipo: "ENTROU",
    id_emprestimo_reagente: emprestimoRef.id,
    peso_anterior: emprestimo.peso_saida,
    peso_novo: dados.pesoRetorno,
```

```
        medida_ajustada: volumeUtilizado,
        unidade_medida_ajustada: densidade ? "ml" : "g",
        timestamp: admin.firestore.FieldValue.serverTimestamp(),
    });

    return { pesoConsumido, volumeUtilizado, atrasado, frascoVencido,
        avisoHigroscopico: avisoHigroscopico || false };
    });
});

interface StatusExtraordinario {
    idFrasco: string;
    acao: "EXTRAVIAR" | "REENCONTRAR";
    motivo: string;
    estadoConstatadoAoReencontrar?: "ABERTO" | "FECHADO"; // obrigatório se
    REENCONTRAR
    id_operacao: string;
}

export const registrarExtravioOuReencontro = onCall(async (request) => {
    const dados = request.data as StatusExtraordinario;
    await validarPermissao(request, ["Chefe_Geral", "Gestor_Almojarifado"],
        true);
    if (!dados.id_operacao) throw new HttpsError("invalid-argument", "
        id_operacao é obrigatório.");

    const frascoRef = admin.firestore().collection("Frasco_Reagente").doc(
        dados.idFrasco);
    const operacaoRef = admin.firestore().collection("Operacoes").doc(dados.
        id_operacao);

    return admin.firestore().runTransaction(async (tx) => {
        const crypto = require("crypto");
        const payloadRelevante = dados.acao === "EXTRAVIAR"
            ? { acao: dados.acao, idFrasco: dados.idFrasco, motivo: dados.motivo }
            : { acao: dados.acao, idFrasco: dados.idFrasco, motivo: dados.motivo,
                estadoConstatadoAoReencontrar: dados.estadoConstatadoAoReencontrar };
        const payloadStr = JSON.stringify(payloadRelevante, Object.keys(
            payloadRelevante).sort());
        const payloadHash = crypto.createHash("sha256").update(payloadStr).
            digest("hex");
```

```
// 1. ler registro de idempotência
const opSnap = await tx.get(operacaoRef);
if (opSnap.exists) {
  const op = opSnap.data();
  if (op.uid === request.auth!.uid && op.payload_hash === payloadHash) {
    return op.resultado;
  }
  throw new HttpsError("already-exists", "id_operacao duplicada:
payload_hash ou ator divergente.");
}

// 2. ler frasco
const frascoSnap = await tx.get(frascoRef);
if (!frascoSnap.exists) throw new HttpsError("not-found", "Frasco não
encontrado.");
const frasco = frascoSnap.data();

// 3. validar versão/estado e vínculo
await validarGestorDoAlmoxarifadoTx(tx, request.auth!.uid, frasco.
id_almoxarifado);

// 4. ler eventual empréstimo EM_USO|ATRASADO (se for EXTRAVIAR)
if (dados.acao === "EXTRAVIAR") {
  if (frasco.estado_fisico_frasco === "EXTRAVIADO")
    throw new HttpsError("failed-precondition", "O frasco já consta como
extraviado.");

  const queryEmprestimo = admin.firestore().collection("
Emprestimo_Reagente")
    .where("id_frasco_reagente", "==", dados.idFrasco)
    .where("status", "in", ["EM_USO", "ATRASADO"])
    .limit(1);
  const emprestimoAberto = await tx.get(queryEmprestimo);

  tx.update(frascoRef, {
    estado\_fisico\_frasco: "EXTRAVIADO",
    disponibilidade: "INDISPONIVEL",
    detalhe\_status: dados.motivo,
  });
}
```

```
if (!emprestimoAberto.empty) {
  const docEmprestimo = emprestimoAberto.docs[0];
  if (docEmprestimo.data().peso_retorno !== null) {
    throw new HttpsError("failed-precondition", "Empréstimo já possui  
medição de peso_retorno. Reconciliação manual necessária.");
  }
  const pesoSaida = docEmprestimo.data().peso_saida || 0;
  const pesoVazio = frasco.peso_frasco_vazio !== null ? frasco.  
peso_frasco_vazio : null;
  const perdaEstimada = pesoVazio !== null ? Math.max(0, pesoSaida -  
pesoVazio) : pesoSaida;

  tx.update(docEmprestimo.ref, {
    status: "ENCERRADO_EXTRAORDINARIO",
    tipo_encerramento_excepcional: "EXTRAVIO_SINISTRO",
    motivo_encerramento_excepcional: dados.motivo,
    id_gestor_encerramento: request.auth!.uid,
    massa_perda_estimada_g: perdaEstimada,
    peso_retorno: null,
    medida_utilizada: 0,
    data_devolucao_efetuada: admin.firestore.FieldValue.  
serverTimestamp(),
  });
}

tx.set(admin.firestore().collection("Historico_Frasco_Reagente").doc(),
{
  id_frasco_reagente: dados.idFrasco,
  id_almoxarifado: frasco.id_almoxarifado,
  id_gestor: request.auth!.uid,
  tipo: "EXTRAVIO",
  motivo: dados.motivo,
  timestamp: admin.firestore.FieldValue.serverTimestamp(),
});
} else { // REENCONTRAR
  if (frasco.estado_fisico_frasco !== "EXTRAVIADO")
    throw new HttpsError("failed-precondition", "Frasco não está  
extraviado.");
  if (!dados.estadoConstatadoAoReencontrar)
    throw new HttpsError("invalid-argument", "Estado constatado é  
obrigatório.");
```

```
    tx.update(frascoRef, {
      estado_fisico_frasco: dados.estadoConstatadoAoReencontrar,
      disponibilidade: "DISPONIVEL",
      em_quarentena: true,
      detalhe_status: "Reencontrado - aguardando análise: " + dados.motivo,
    });

    tx.set(admin.firestore().collection("Historico_Frasco_Reagente").doc(),
    {
      id_frasco_reagente: dados.idFrasco,
      id_almoxarifado: frasco.id_almoxarifado,
      id_gestor: request.auth!.uid,
      tipo: "REENCONTRO",
      motivo: dados.motivo,
      timestamp: admin.firestore.FieldValue.serverTimestamp(),
    });
  }

  const resultado = { status: "OK" };
  // Grava operação para garantir idempotência
  tx.set(operacaoRef, {
    tipo_operacao: dados.acao,
    uid: request.auth!.uid,
    payload_hash: payloadHash,
    resultado: resultado,
    status: "CONCLUIDA",
    criado_em: admin.firestore.FieldValue.serverTimestamp(),
    atualizado_em: admin.firestore.FieldValue.serverTimestamp()
  });

  return resultado;
});

interface PayloadRecalibrarTara {
  id_frasco: string;
  id_operacao: string;
  peso_vazio_real: number;
  motivo: string;
```

```
}

export const recalibrarTaraFrascoEsgotado = onCall(async (request) => {
  const dados = request.data as PayloadRecalibrarTara;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxxarifado"],
    true);
  if (!dados.id_operacao) throw new HttpsError("invalid-argument", "
    id_operacao é obrigatório.");

  const frascoRef = admin.firestore().collection("Frasco_Reagente").doc(
    dados.id_frasco);
  const operacaoRef = admin.firestore().collection("Operacoes").doc(dados.
    id_operacao);

  return admin.firestore().runTransaction(async (tx) => {
    const crypto = require("crypto");
    const payloadRelevante = { acao: "RECALIBRAR_TARA", id_frasco: dados.
      id_frasco, peso_vazio_real: dados.peso_vazio_real, motivo: dados.motivo
    };
    const payloadStr = JSON.stringify(payloadRelevante, Object.keys(
      payloadRelevante).sort());
    const payloadHash = crypto.createHash("sha256").update(payloadStr).
      digest("hex");

    const opSnap = await tx.get(operacaoRef);
    if (opSnap.exists) {
      const op = opSnap.data()!;
      if (op.uid === request.auth!.uid && op.payload_hash === payloadHash) {
        return op.resultado;
      }
      throw new HttpsError("already-exists", "id_operacao duplicada:
        payload_hash ou ator divergente.");
    }

    const frascoSnap = await tx.get(frascoRef);
    if (!frascoSnap.exists) throw new HttpsError("not-found", "Frasco não
      encontrado.");
    const frasco = frascoSnap.data()!;
    await validarGestorDoAlmoxxarifadoTx(tx, request.auth!.uid, frasco.
      id_almoxxarifado);
  });
});
```

```
tx.update(frascoRef, { peso_frasco_vazio: dados.peso_vazio_real });

tx.set(admin.firestore().collection("Historico_Frasco_Reagente").doc(),
{
  id_frasco_reagente: dados.id_frasco,
  id_almoxarifado: frasco.id_almoxarifado,
  id_gestor: request.auth!.uid,
  tipo: "AJUSTE",
  campo_ajustado: "peso_frasco_vazio",
  peso_anterior: frasco.peso_frasco_vazio || null,
  peso_novo: dados.peso_vazio_real,
  motivo: dados.motivo,
  timestamp: admin.firestore.FieldValue.serverTimestamp(),
});

const resultado = { status: "OK" };
tx.set(operacaoRef, {
  tipo_operacao: "RECALIBRAR_TARA",
  uid: request.auth!.uid,
  payload_hash: payloadHash,
  resultado,
  status: "CONCLUIDA",
  criado_em: admin.firestore.FieldValue.serverTimestamp(),
  atualizado_em: admin.firestore.FieldValue.serverTimestamp()
});

return resultado;
});
});
```

#### 10.2.4 Validação síncrona de validade no cadastro e na abertura

A validade não depende apenas do job diário. No cadastro do frasco, a Cloud Function calcula *validade\_efetiva* e compara esse valor com o relógio do servidor. Se a validade já expirou, a criação só pode prosseguir com uma decisão explícita do gestor: **QUARENTENA**, **PENDENTE\_DE\_DESCARTE** ou **DISPONIVEL** com *uso\_vencido\_autorizado*. A primeira opção exige *detalhe\_status*. A terceira representa uma exceção operacional deliberada: reagentes vencidos podem continuar sendo utilizados em práticas menos exigentes ou demonstrações quando o gestor registra explicitamente essa autorização.

Na abertura do frasco, a função *calcularValidadeEfetivaNaAbertura* aplica:

$$validade\_efetiva = \min(validade\_fechado, data\_abertura + validade\_apos\_aberto\_dias),$$

quando os dois limites existem.

### 10.2.5 Jobs Agendados: Vencimento, Atraso e Materialização

Vencimento e Atraso com busca preventiva de devoluções

```
import { onSchedule } from "firebase-functions/v2/scheduler";
import { onDocumentCreated, onDocumentDeleted, onDocumentUpdated } from "
  firebase-functions/v2/firestore";

export const verificarVencimentosEAtrasos = onSchedule({
  schedule: "every day 03:00",
  timeZone: "America/Sao_Paulo"
}, async () => {
  const agora = new Date();
  const inicioHoje = inicioDoDiaInstitucional(agora);
  const fimDeAmanha = fimDoDiaInstitucional(adicionarDias(inicioHoje, 1));

  const emprestimosNaJanela = await admin.firestore().collection("
    Empréstimo_Reagente")
    .where("status", "==", "EM_USO")
    .where("data_devolucao_prevista", "<=", fimDeAmanha)
    .get();

  const batchAtrasados = admin.firestore().batch();
  const notificacoesPreventivas: Array<{ idEmpréstimo: string; idUsuario:
    string; quando: "HOJE" | "AMANHÃ" }> = [];

  emprestimosNaJanela.docs.forEach((doc) => {
    const dados = doc.data();
    const prevista = dados.data_devolucao_prevista.toDate();

    if (prevista < inicioHoje) {
      batchAtrasados.update(doc.ref, { status: "ATRASADO" });
      return;
    }

    if (mesmoDia(prevista, inicioHoje)) {
```

```
    notificacoesPreventivas.push({ idEmprestimo: doc.id, idUsuario: dados.
id_usuario_retirou, quando: "HOJE" });
  } else {
    notificacoesPreventivas.push({ idEmprestimo: doc.id, idUsuario: dados.
id_usuario_retirou, quando: "AMANHÃ" });
  }
});

await batchAtrasados.commit();

for (const item of notificacoesPreventivas) {
  await notificarProfessorDevolucaoUmaVez(item.idUsuario, item.
idEmprestimo, item.quando);
}

// Segmentação por almoxarifado para notificações de atraso (PDF-005)
const atrasadosPorAlmox: Record<string, number> = {};
emprestimosNaJanela.docs.forEach((doc) => {
  const dados = doc.data();
  const prevista = dados.data_devolucao_prevista.toDate();
  if (prevista < inicioHoje) {
    atrasadosPorAlmox[dados.id_almoxarifado] = (atrasadosPorAlmox[dados.
id_almoxarifado] || 0) + 1;
  }
});

for (const [idAlmox, count] of Object.entries(atrasadosPorAlmox)) {
  const gestoresSnap = await admin.firestore().collection("
Gestor_Almoxarifado_x_Almoxarifado")
    .where("id_almoxarifado", "==", idAlmox).get();
  for (const gDoc of gestoresSnap.docs) {
    const gestorId = gDoc.data().id_gestor_almoxarifado;
    const docIdAtraso = `atraso_${idAlmox}_${hojeISO}`;
    try {
      await admin.firestore().collection("Usuarios").doc(gestorId).
collection("Notificacoes").doc(docIdAtraso).set({
        id_destinatario: gestorId,
        papel_destinatario: "Gestor_Almoxarifado",
        tipo: "ENTREGA_ATRASADA",
        entidade_alvo: "Almoxarifado",
        id_alvo: idAlmox,
```

```
        quantidade: count,
        lida: false,
        emitida_em: admin.firestore.FieldValue.serverTimestamp(),
        expira_em: null,
      }, { merge: false });
    } catch (err: any) {
      if (err.code !== 6 && err.code !== "ALREADY_EXISTS") throw err; //
      Ignora apenas duplicidade
    }
  }
}

const frascosVencendo = await admin.firestore().collection("
  Frasco_Reagente")
  .where("vencido", "==", false)
  .where("validade_efetiva", "<=", agora)
  .get();

const vencidosPorAlmox: Record<string, number> = {};
const batchVencidos = admin.firestore().batch();
frascosVencendo.docs.forEach((doc) => {
  batchVencidos.update(doc.ref, { vencido: true });
  const idAlmox = doc.data().id_almoxarifado;
  vencidosPorAlmox[idAlmox] = (vencidosPorAlmox[idAlmox] || 0) + 1;
  const histRef = admin.firestore().collection("Historico_Frasco_Reagente"
  ).doc();
  batchVencidos.set(histRef, {
    id_frasco_reagente: doc.id,
    id_almoxarifado: idAlmox,
    id_gestor: "__SISTEMA__",
    tipo: "VENCEU",
    timestamp: admin.firestore.FieldValue.serverTimestamp(),
  });
});

await batchVencidos.commit();

for (const [idAlmox, count] of Object.entries(vencidosPorAlmox)) {
  const gestoresSnap = await admin.firestore().collection("
  Gestor_Almoxarifado_x_Almoxarifado")
    .where("id_almoxarifado", "==", idAlmox).get();
  for (const gDoc of gestoresSnap.docs) {
```

```
const gestorId = gDoc.data().id_gestor_almoxarifado;
const docIdVencidos = `vencidos_${idAlmox}_${hojeISO}`;
try {
  await admin.firestore().collection("Usuarios").doc(gestorId).
collection("Notificacoes").doc(docIdVencidos).set({
  id_destinatario: gestorId,
  papel_destinatario: "Gestor_Almoxarifado",
  tipo: "FRASCOS_VENCIDOS",
  entidade_alvo: "Almoxarifado",
  id_alvo: idAlmox,
  quantidade: count,
  lida: false,
  emitida_em: admin.firestore.FieldValue.serverTimestamp(),
  expira_em: null,
}, { merge: false });
} catch (err: any) {
  if (err.code !== 6 && err.code !== "ALREADY_EXISTS") throw err;
}
}
});

// PDF-015: INVARIANTE DE TIMEZONE
// Toda data civil em regras operacionais (retirada, vencimento, escassez,
// relatorios)
// DEVE ser derivada do timezone IANA "America/Sao_Paulo", independente do
// TZ do processo.
// NAO usar Date.setHours(0,0,0,0) sem especificar timezone (depende de TZ
// do servidor).
// NAO usar offset fixo -03:00 (nao respeita horario de verao historico).
// Usar biblioteca que implemente IANA corretamente (ex: luxon, date-fns-tz,
// dayjs+tz).
//
// Exemplo normativo com luxon:
// import { DateTime } from "luxon";
// const TZ = "America/Sao_Paulo";
// function inicioDoDiaInstitucional(ref: Date): Date {
//   return DateTime.fromJSDate(ref, { zone: TZ }).startOf("day").toJSDate
//   ();
// }
// function fimDoDiaInstitucional(ref: Date): Date {
```

```
//      return DateTime.fromJSDate(ref, { zone: TZ }).endOf("day").toJSDate();

//  }
//  function adicionarDias(data: Date, dias: number): Date {
//      return DateTime.fromJSDate(data, { zone: TZ }).plus({ days: dias }).
//          toJSDate();
//  }
//  function mesmoDia(a: Date, b: Date): boolean {
//      const da = DateTime.fromJSDate(a, { zone: TZ });
//      const db = DateTime.fromJSDate(b, { zone: TZ });
//      return da.hasSame(db, "day");
//  }
//
// O pseudocódigo abaixo é INVALIDO (setHours ignora timezone):
//      result.setHours(0, 0, 0, 0); // PROIBIDO em contexto de data civil

function adicionarDias(data: Date, dias: number): Date {
    // NAO USAR: usa o timezone do processo, que pode ser UTC em producao
    // Usar luxon: DateTime.fromJSDate(data, { zone: "America/Sao_Paulo" }).
    //     plus({ days: dias }).toJSDate()
    return DateTime.fromJSDate(data, { zone: "America/Sao_Paulo" }).plus({
        days: dias
    }).toJSDate();
}

function mesmoDia(a: Date, b: Date): boolean {
    const dtA = DateTime.fromJSDate(a, { zone: "America/Sao_Paulo" });
    const dtB = DateTime.fromJSDate(b, { zone: "America/Sao_Paulo" });
    return dtA.hasSame(dtB, "day");
}

function inicioDoDiaInstitucional(data: Date): Date {
    return DateTime.fromJSDate(data, { zone: "America/Sao_Paulo" }).startOf("
        day").toJSDate();
}

function fimDoDiaInstitucional(data: Date): Date {
    return DateTime.fromJSDate(data, { zone: "America/Sao_Paulo" }).endOf("day
        ").toJSDate();
}
```

```

export const onEventoRemocaoFrasco = onDocumentCreated("
  Eventos_Dominio_Contagem/{eventoId}", async (event) => {
const data = event.data?.data();
if (data?.tipo === "FRASCO_REMOVIDO" && data?.id_lote && data?.ocorrido_em
) {
  // PDF-016: A exclusão de um frasco não depende do 'event.time' do
  Eventarc.
  // A API/sistema que remove o frasco grava na mesma transação Firestore
  este
  // evento de domínio, com ocorrido_em = FieldValue.serverTimestamp().
  // Assim, garantimos a mesma ordem temporal exata para criação e exclusã
  o.
  await atualizarContadorLoteIdempotente(event.id, data.id_lote, -1, data.
  ocorrido_em);
}
});

// PDF-016: Coordenacao Triggers vs Reconciliacao
// Problema: Um evento atrasado (ex: trigger retida no Pub/Sub por horas)
  processado
// apos uma reconciliacao absoluta causaria somatoria dupla (o count() do
  job
// ja incluiria o frasco, e o incremento da trigger repetiria a soma).
//
// Alternativas:
// 1. Versionamento Estrito (Optimistic Concurrency): Requereria
  linearizacao e rejeitaria processamento assincrono.
// 2. Invalidacao Antecipada de Locks: O job teria que prever eventId
  pendentes, o que e impossivel.
// 3. Marca d'Agua (Recomendada): O lote guarda 'ultimo_reconciliador'. A
  trigger ignora incrementos de eventos cuja emissao ('event.time') seja
  anterior a marca.
// Impacto da Recomendacao: Adicao de 'ultimo_reconciliador' na entidade, '
  tx.get' extra na trigger para validar a marca antes do incremento.

async function atualizarContadorLoteIdempotente(eventId: string, idLote:
  string, delta: number, eventTime: admin.firestore.Timestamp) {
await admin.firestore().runTransaction(async (tx) => {
  const lockRef = admin.firestore().collection("Eventos_Processados").doc(
  eventId);
  const lock = await tx.get(lockRef);

```

```
if (lock.exists) return; // Deduplicado: evento ja processado (at-least-
once resolvida)

const loteRef = admin.firestore().collection("Lote_Materializado").doc(
idLote);
const loteSnap = await tx.get(loteRef);

let aplicarIncremento = true;
if (loteSnap.exists) {
  const lote = loteSnap.data(!);
  if (lote.ultimo_reconciliador) {
    const evTs = eventTime;
    const mcTs = lote.ultimo_reconciliador; // admin.firestore.Timestamp

    // Formalizacao do Contrato Temporal de Reconciliacao (PDF-016):
    // 1. T_evento (evTs):
    //    - Criacao: Snapshot 'createTime'. Representa o timestamp exato
    de commit no Spanner.
    //    - Exclusao: Via evento de dominio persistido com 'ocorrido_em'
(serverTimestamp).
    //    Representa o timestamp exato de commit da transacao que
    efetivou a exclusao.
    // 2. T_corte (mcTs): 'lote.ultimo_reconciliador'. Timestamp exato
    do commit da reconciliacao (serverTimestamp).
    // 3. Comparabilidade: 100% ancorada em transacoes Firestore (linha
    do tempo Spanner).
    //    Nao existe mais nenhuma dependencia na latencia de geracao do
    Eventarc (event.time).
    // 4. Se T_evento <= T_corte: A mutacao ocorreu antes ou no instante
    do fechamento da fotografia
    //    da reconciliacao. Por garantia ACID, o count() ja tratou o
    frasco. O incremento é DESCARTADO.
    // 5. Se T_evento > T_corte: A mutacao aconteceu estritamente apos o
    corte. O incremento DEVE ser aplicado.

    const isEventoVelho = evTs.seconds < mcTs.seconds ||
      (evTs.seconds === mcTs.seconds && evTs.nanoseconds <= mcTs.
nanoseconds);

    if (isEventoVelho) {
      aplicarIncremento = false; // Evento retardado engolido pela
      contagem absoluta da reconciliacao
    }
  }
}
```

```
    }
  }
}

if (aplicarIncremento) {
  tx.set(loteRef, {
    id_lote: idLote,
    qtd_frascos_cadastrados: admin.firestore.FieldValue.increment(delta)
  }, { merge: true });
}

tx.set(lockRef, { processado_em: admin.firestore.FieldValue.
serverTimestamp() });
});
}

// Alem da deduplicacao nas triggers (acima), o sistema exige um job de
// reconciliacao
// para curar eventuais divergencias (drift) causadas por operacoes fora das
// functions
// ou falhas de lock.
export const reconciliarContadoresLote = onSchedule({
  schedule: "every sunday 04:00",
  timeZone: "America/Sao_Paulo"
}, async () => {
  // Busca lotes ativos que nao foram reconciliados recentemente
  const lotesSnap = await admin.firestore().collection("Lote_Materializado").
    get();

  for (const lote of lotesSnap.docs) {
    const idLote = lote.id;
    await admin.firestore().runTransaction(async (tx) => {
      // 1. Contagem absoluta transacional
      const frascosQuery = admin.firestore().collection("Frasco_Reagente").
        where("id_lote", "==", idLote);
      const aggregate = await tx.get(frascosQuery.count());
      const qtdReal = aggregate.data().count;

      const loteSnap = await tx.get(lote.ref);
      if (!loteSnap.exists) return;
    });
  }
}
```

```

    // 2. So escrever se houver divergencia, ou apenas atualizar a marca d
    'agua
    const dados = loteSnap.data()!;
    if (dados.qtd_frascos_cadastrados !== qtdReal) {
        // Grava auditoria
        tx.set(admin.firestore().collection("Auditoria_Reconciliacao").doc(),
        {
            id_lote: idLote,
            qtd_anterior: dados.qtd_frascos_cadastrados,
            qtd_corrigida: qtdReal,
            data: admin.firestore.FieldValue.serverTimestamp()
        });
    }

    // 3. Atualizar OBRIGATORIAMENTE a marca d'agua absoluta
    tx.set(lote.ref, {
        qtd_frascos_cadastrados: qtdReal,
        ultimo_reconciliador: admin.firestore.FieldValue.serverTimestamp()
    }, { merge: true });
    });
}
});

```

**Pré-requisito: Backfill Legado e Reconciliação (R3.1)** O job de escassez baseia a projeção de estoque nas chaves `id_resumo_reagente` e `id_especificacao_reagente`. Devido a cadastros legados que não possuíam `id_especificacao_reagente`, a ativação deste job requer a execução prévia de uma rotina **one-shot administrativa** (backfill):

- Para legados com especificação determinística (ex: via lote conhecido): popular `id_especificacao_reagente`.
- Para legados ambíguos: isolar como pendência de reconciliação manual. **O sistema nunca deve inventar um `id_especificacao_reagente` para satisfazer o schema.**
- A ativação da *cron* de escassez só deve ocorrer após a conclusão desta reconciliação, sob pena de gerar falsos alertas de escassez pela omissão de frascos aptos na query de contagem.

#### Escassez de Estoque Baseada em Configuração por Almoxarifado

```

import { DateTime } from "luxon";

export const verificarEscassezDeEstoque = onSchedule({

```

```
    schedule: "every day 04:00",
    timeZone: "America/Sao_Paulo"
  }, async () => {
    const db = admin.firestore();
    const hojeISO = DateTime.now().setZone("America/Sao_Paulo").toISODate();

    const almoxarifados = await db.collection("Almoxarifado").where("ativo", "
      ==", true).get();
    const bw = db.bulkWriter();
    const promises: Promise<any>[] = [];

    for (const almoxDoc of almoxarifados.docs) {
      const idAlmox = almoxDoc.id;

      // 1. Busca configurações ativas de escassez para o almoxarifado
      const estoquesConfig = await db.collection(`Almoxarifado/${idAlmox}/
      Estoques_Configurados`)
        .where("ativo", "==", true)
        .where("notificacao_ativa", "==", true)
        .get();

      if (estoquesConfig.empty) continue;

      // Busca os gestores vinculados ao almoxarifado
      const gestoresSnap = await db.collection("
      Gestor_Almoxarifado_x_Almoxarifado")
        .where("id_almoxarifado", "==", idAlmox)
        .get();

      const gestoresIds = gestoresSnap.docs.map(d => d.data().
      id_gestor_almoxarifado);

      for (const configDoc of estoquesConfig.docs) {
        const config = configDoc.data();
        const qtdLimiar = config.qtd_limiar_escassez;

        // 2. Conta saldo físico apto para retirada
        const frascosAptosQuery = db.collection("Frasco_Reagente")
          .where("id_almoxarifado", "==", idAlmox)
          .where("id_resumo_reagente", "==", config.id_resumo_reagente)
          .where("id_especificacao_reagente", "==", config.
```

```
id_especificacao_reagente)
    .where("disponibilidade", "==", "DISPONIVEL")
    .where("estado_fisico_frasco", "in", ["FECHADO", "ABERTO"])
    .where("em_quarentena", "==", false)
    .where("vencido", "==", false);

const aptosSnap = await frascosAptosQuery.count().get();
const qtdAptos = aptosSnap.data().count;

// 3. Emite notificação idempotente se o saldo for ESTRITAMENTE MENOR
que o limiar
if (qtdAptos < qtdLimiar) {
    for (const gestorId of gestoresIds) {
        const notifId = `escassez_${idAlmox}_${configDoc.id}_${hojeISO}`;
        const notifRef = db.collection("Usuarios").doc(gestorId).
collection("Notificacoes").doc(notifId);

        // Idempotência via create() do BulkWriter. Se já existir para
hoje, ignora sem falhar o job.
        const prom = bw.create(notifRef, {
            id_destinatario: gestorId,
            papel_destinatario: "Gestor_Almojarifado",
            tipo: "ESCASSEZ_ESTOQUE",
            entidade_alvo: "Almojarifado",
            id_alvo: idAlmox,
            lida: false,
            emitida_em: admin.firestore.FieldValue.serverTimestamp(),
            expira_em: null
        }).catch((err) => {
            if (err.code !== 6) { // 6 = ALREADY_EXISTS
                console.error(`Erro ao criar notificação ${notifId} para ${
gestorId}:`, err);
            }
        });
        promises.push(prom);
    }
}

// Aguarda todas as operações agendadas serem despachadas e resolvidas
```

```

    await Promise.all(promises);
    await bw.close();
  });

```

### 10.2.6 Fluxo de Bens Patrimoniais: Requisições

Criação e Resposta de Requisição de Edição (C2: lock determinístico)

```

export const criarRequisicaoEdicaoBem = onCall(async (request) => {
  await validarPermissao(request, ["Professor"]);
  const dados = request.data as { idBemPatrimonial: string; novoNome?:
    string; novoStatus?: string; novoEstadoConservacao?: string; novoIdLocal
    ?: string; novaPhotoUrl?: string; motivo: string };
  if (dados.novoStatus === "Ja\_dado\_baixa") {
    throw new HttpsError("invalid-argument", "Professor não pode solicitar
      baixa diretamente (RF12).");
  }

  // C2: Lock determinístico - elimina phantom reads em transações Firestore.

  // Se dois professores enviarem simultaneamente, apenas um cria o lock; o
  // outro
  // recebe ALREADY\_EXISTS da transação e falha antes de criar a requisição.

  const lockId = `bem\_edicao\_${dados.idBemPatrimonial}`;
  const lockRef = admin.firestore().collection("Locks\_Requisicao\_
    _Patrimonio").doc(lockId);
  const reqRef = admin.firestore().collection("Requisicao\_Edicao\_Bem\_
    _Patrimonial").doc();

  return admin.firestore().runTransaction(async (tx) => {
    const lockSnap = await tx.get(lockRef);
    if (lockSnap.exists) {
      throw new HttpsError("failed-precondition", "Já existe uma requisição
        de edição pendente para este bem.");
    }

    const bemSnap = await tx.get(admin.firestore().collection("Bem\_
      _Patrimonial").doc(dados.idBemPatrimonial));
    if (!bemSnap.exists) throw new HttpsError("not-found", "Bem não
      encontrado.");

    // Cria lock e requisição atomicamente

```

```

    tx.set(lockRef, {
      id\_requisicao: reqRef.id,
      tipo: "EDICAO",
      chave\_recurso: dados.idBemPatrimonial,
      criado\_em: admin.firestore.FieldValue.serverTimestamp()
    });
    tx.set(reqRef, {
      id\_bem\_patrimonial: dados.idBemPatrimonial,
      novo\_nome: dados.novoNome ?? null,
      versao\_bem\_origem: bemSnap.data()!.versao,
      novo\_id\_resumo\_bem\_patrimonial: null,
      novo\_status: dados.novoStatus ?? null,
      novo\_estado\_conservacao: dados.novoEstadoConservacao ?? null,
      novo\_id\_local: dados.novoIdLocal ?? null,
      nova\_photo\_url: dados.novaPhotoUrl ?? null,
      motivo: dados.motivo,
      status: "pendente",
      feita\_em: admin.firestore.FieldValue.serverTimestamp(),
      id\_usuario\_solicitante: request.auth!.uid,
    });
    return { idRequisicao: reqRef.id };
  });
});

// N-02 (PDF-001): Algoritmo normativo de resposta a requisição de edição.
// INVARIANTE DE LOCK: o lock bem\_edicao\__{idBem} DEVE ser liberado em
//   todos os desfechos:
//   - aprovação com sucesso: liberado na linha tx.delete abaixo;
//   - rejeição: liberado na mesma transação, sem verificação de versão;
//   - conflito de versão na aprovação: a transação deve rejeitar a requisiç
//     ão E liberar o lock,
//     não apenas abortar, para evitar lock permanente irrecoverável.
// FLUXO NORMATIVO:
//   1. Leitura: requisição -> lock (implícito) -> bem;
//   2. Validação de estado da requisição (pendente?);
//   3. SE REJEIÇÃO: não verificar versão; finalizar requisição como
//     rejeitada; liberar lock;
//   4. SE APROVAÇÃO: verificar versão;
//     a. Conflito de versão -> rejeitar requisição como conflito; liberar
//       lock (não abortar sem liberar);

```

```
//      b. Sem conflito -> aplicar alterações; incrementar versão; histórico;
//      liberar lock.
// UNICIDADE DE PLAQUETA: criarRequisicaoAdicaoBem usa Lock C2 por numero\
//      _patrimonio\_proposto;
//      a aprovação cria o bem apenas depois de verificar Chaves\_Unicas/Bem\
//      _Patrimonial\__{numero}
//      dentro da mesma transação, garantindo que duas aprovações concorrentes
//      não criem duplicatas.
export const responderRequisicaoEdicaoBem = onCall(async (request) => {
  await validarPermissao(request, ["Chefe\_Geral", "Gestor\_Bens\_
    _Patrimoniais"], true);
  const { idRequisicao, aprovar, justificativa, idResumoAlvo } = request.
    data;
  const lockColl = admin.firestore().collection("Locks\_Requisicao\_
    _Patrimonio");
  return admin.firestore().runTransaction(async (tx) => {
    // PASSO 1: Todas as leituras antes de qualquer escrita (obrigatório em
    // transações Firestore)
    const chefeSnap = await tx.get(admin.firestore().collection("Chefe_Geral
    ").doc(request.auth!.uid));
    if (!chefeSnap.exists) {
      const gestorSnap = await tx.get(admin.firestore().collection("
    Gestor_Bens_Patrimoniais").doc(request.auth!.uid));
      if (!gestorSnap.exists) throw new HttpsError("permission-denied", "Usu
      ário sem papel de Gestor de Bens ativo.");
    }
    const reqRef = admin.firestore().collection("Requisicao\_Edicao\_Bem\_
    _Patrimonial").doc(idRequisicao);
    const req = (await tx.get(reqRef)).data();
    if (!req || req.status !== "pendente") throw new HttpsError("failed-
    precondition", "Requisição indisponível.");
    const bemRef = admin.firestore().collection("Bem\_Patrimonial").doc(req.
    id\_bem\_patrimonial);
    const lockRef = lockColl.doc(`bem\_edicao\__${bemRef.id}`);
    const bem = (await tx.get(bemRef)).data();

    // PASSO 2: Rejeição não depende de verificação de versão (PDF-001: lock
    // liberado em todos os desfechos)
    if (!aprovar) {
      tx.update(reqRef, {
        status: "rejeitada",
```

```
    respondida\_em: admin.firestore.FieldValue.serverTimestamp(),
    id\_usuario\_respondente: request.auth!.uid,
    justificativa\_resposta: justificativa,
  });
  tx.delete(lockRef); // INVARIANTE: lock liberado mesmo na rejeição
  return { status: "rejeitada" };
}

// PASSO 3: Aprovação -- verificar versão (conflito)
// N-02: se o bem foi alterado após a abertura da requisição, a versão
// diverge.
if (!bem || bem.versao !== req.versao\_bem\_origem) {
  tx.update(reqRef, {
    status: "rejeitada",
    respondida\_em: admin.firestore.FieldValue.serverTimestamp(),
    id\_usuario\_respondente: request.auth!.uid,
    justificativa\_resposta: "Conflito de versão: o bem foi alterado após
s a abertura desta requisição. Revise a proposta com os dados atuais e
abra nova requisição.",
  });
  tx.delete(lockRef); // INVARIANTE: lock liberado no conflito de versão
  return { status: "rejeitada", motivo: "CONFLITO\_VERSAO" };
}

// PASSO 4: Leituras secundárias do caminho de aprovação (preserva
// leituras antes das escritas deste fluxo)
let localSnap: FirebaseFirestore.DocumentSnapshot<FirebaseFirestore.
DocumentData> | null = null;
if (req.novo\_id\_local) {
  localSnap = await tx.get(admin.firestore().collection("Local").doc(req.
novo\_id\_local));
  if (!localSnap.exists) {
    // Rejeição sistêmica: local não existe, o lock precisa ser liberado.

    tx.update(reqRef, {
      status: "rejeitada",
      respondida\_em: admin.firestore.FieldValue.serverTimestamp(),
      id\_usuario\_respondente: request.auth!.uid,
      justificativa\_resposta: "Local referenciado na requisição não
encontrado ou excluído.",
    });
  }
}
```

```

        tx.delete(lockRef);
        return { status: "rejeitada", motivo: "LOCAL\_NAO\_ENCONTRADO" };
    }
}

// Resolve alvo existente ou prepara criação; helper só lê nesta etapa.
const alvo = req.novo\_nome
  ? await prepararResumoAlvoTx(tx, idResumoAlvo, req.novo\_nome) : null;
const alteracoes = calcularAlteracoesPermitidas(req, bem);
if (alvo) {
    if (alvo.novo) tx.create(alvo.ref, alvo.dados);
    alteracoes.id\_resumo\_bem\_patrimonial = alvo.ref.id;
    alteracoes.nome\_equipamento = alvo.dados.nome;
}

// M-02: quando novo\_id\_local está presente, propagar predio/andar/
sala denormalizados
if (req.novo\_id\_local && localSnap) {
    const local = localSnap.data()!;
    // Atualizar as projeções denormalizadas conforme Seção 5 (
    Espelhamento de Local)
    alteracoes.id\_local = req.novo\_id\_local;
    alteracoes.predio = local.predio;
    alteracoes.andar = local.andar;
    alteracoes.sala = local.sala;
}

// PASSO 4: Escritas -- todas após as leituras
tx.update(bemRef, { ...alteracoes, versao: bem.versao + 1 });
registrarHistoricoBemTx(tx, bemRef, bem, alteracoes, request.auth!.uid);
tx.update(reqRef, {
    status: "aprovada",
    novo\_id\_resumo\_bem\_patrimonial: alvo?.ref.id ?? null,
    respondida\_em: admin.firestore.FieldValue.serverTimestamp(),
    id\_usuario\_respondente: request.auth!.uid, justificativa\_resposta:
justificativa,
});
tx.delete(lockRef); // INVARIANTE: lock liberado na aprovação
return { status: "aprovada" };
});
});

```

criarRequisicaoAdicaoBem com lock determinístico (C2)

```
export const criarRequisicaoAdicaoBem = onCall(async (request) => {
```

```
await validarPermissao(request, ["Professor"]);

const dados = request.data as {
  numeroPatrimonioProposto: string;
  estadoConservacaoProposto: string;
  idLocal: string;
  nomeResponsavelProposto: string;
  photoUrlProposta: string; // objeto verificado pelo backend
  idResumoBemPatrimonial?: string;
  nomeResumoProposto?: string;
  descricaoResumoProposta?: string;
  motivo: string;
};

await validarFotoProposta(dados.photoUrlProposta, request.auth!.uid);

// C2: Lock determinístico por número de patrimônio proposto (normalizado)
const numeroNormalizado = dados.numeroPatrimonioProposto.trim().
  toUpperCase();
const lockId = `bem\_adicao\_${numeroNormalizado}`;
const lockRef = admin.firestore().collection("Locks\_Requisicao\_
  \_Patrimonio").doc(lockId);
const reqRef = admin.firestore().collection("Requisicao\_Adicao\_Bem\_
  \_Patrimonial").doc();

return admin.firestore().runTransaction(async (tx) => {
  const lockSnap = await tx.get(lockRef);
  if (lockSnap.exists) {
    throw new HttpsError("failed-precondition", "Já existe requisição
    pendente para este número de patrimônio.");
  }

  // Validar existência prévia do patrimônio na coleção Chaves\_Unicas
  const chaveUnicaRef = admin.firestore().collection("Chaves\_Unicas").doc
    (`Bem\_Patrimonial\_\\_${numeroNormalizado}`);
  const chaveUnicaSnap = await tx.get(chaveUnicaRef);
  if (chaveUnicaSnap.exists) {
    throw new HttpsError("already-exists", "O número de patrimônio
    proposto já está em uso por outro bem.");
  }
  tx.set(lockRef, {
```

```

        id\_requisicao: reqRef.id,
        tipo: "ADICAO",
        chave\_recurso: numeroNormalizado,
        criado\_em: admin.firestore.FieldValue.serverTimestamp()
    });
    tx.set(reqRef, {
        numero\_patrimonio\_proposto: dados.numeroPatrimonioProposto,
        estado\_conservacao\_proposto: dados.estadoConservacaoProposto,
        photo\_url\_proposta: dados.photoUrlProposta,
        nome\_responsavel\_proposto: dados.nomeResponsavelProposto,
        id\_local: dados.idLocal,
        id\_resumo\_bem\_patrimonial: dados.idResumoBemPatrimonial ?? null,
        nome\_resumo\_proposto: dados.nomeResumoProposto ?? null,
        descricao\_resumo\_proposta: dados.descricaoResumoProposta ?? null,
        motivo: dados.motivo,
        status: "pendente",
        feita\_em: admin.firestore.FieldValue.serverTimestamp(),
        id\_usuario\_solicitante: request.auth!.uid,
        respondida\_em: null,
        id\_usuario\_respondente: null,
    });
    return { idRequisicao: reqRef.id };
});
});

export const responderRequisicaoAdicaoBem = onCall(async (request) => {
    await validarPermissao(request, ["Chefe\_Geral", "Gestor\_Bens\_
        _Patrimoniais"], true);
    const { idRequisicao, aprovar, justificativa } = request.data as {
        idRequisicao: string; aprovar: boolean; justificativa: string };
    const reqRef = admin.firestore().collection("Requisicao\_Adicao\_Bem\_
        _Patrimonial").doc(idRequisicao);

    return admin.firestore().runTransaction(async (tx) => {
        // PASSO A: Leituras primárias
        const chefeSnap = await tx.get(admin.firestore().collection("Chefe_Geral
        ").doc(request.auth!.uid));
        if (!chefeSnap.exists) {
            const gestorSnap = await tx.get(admin.firestore().collection("
        Gestor_Bens_Patrimoniais").doc(request.auth!.uid));
            if (!gestorSnap.exists) throw new HttpsError("permission-denied", "Usu

```

```
ário sem papel de Gestor de Bens ativo.");
}
const reqSnap = await tx.get(reqRef);
if (!reqSnap.exists) throw new HttpsError("not-found", "Requisição não encontrada.");
const req = reqSnap.data()!;
if (req.status !== "pendente") throw new HttpsError("failed-precondition", "Requisição já respondida.");

const numeroNormalizado = req.numero\_patrimonio\_proposto.trim().toUpperCase();
const lockRef = admin.firestore().collection("Locks\_Requisicao\_Patrimonio").doc('bem\_adicao\_${numeroNormalizado}');
const lockSnap = await tx.get(lockRef);

let chaveUnicaRef: FirebaseFirestore.DocumentReference | null = null;
let resumoNome = req.nome\_resumo\_proposto ?? "";
let localSnapData: any = null;
let letraInicial = "";

// PASSO B: Decisão e validações adicionais (leituras antes das escritas mantidas)
let aprovarInterno = aprovar;
let justificativaFinal = justificativa;

if (aprovarInterno && !req.photo\_url\_proposta) {
  aprovarInterno = false;
  justificativaFinal = "Rejeição sistêmica: Foto obrigatória ausente no pedido legado.";
}

if (aprovarInterno) {
  chaveUnicaRef = admin.firestore().collection("Chaves\_Unicas").doc('Bem\_Patrimonial\_\\_${numeroNormalizado}');
  const chaveUnicaSnap = await tx.get(chaveUnicaRef);
  if (chaveUnicaSnap.exists) {
    throw new HttpsError("already-exists", "O número de patrimônio aprovado já está em uso por outro bem.");
  }

  if (req.id\_resumo\_bem\_patrimonial) {
```

```
    const resumoSnap = await tx.get(admin.firestore().collection("Resumo\n_Bem_Patrimonial").doc(req.id\_resumo\_bem\_patrimonial));
    if (!resumoSnap.exists) {
        aprovarInterno = false;
        justificativaFinal = "Rejeição sistêmica: Resumo referenciado não existe.";
    } else {
        resumoNome = resumoSnap.data()!.nome;
    }
}

if (aprovarInterno) { // Só lê local se ainda for aprovar
    const localSnap = await tx.get(admin.firestore().collection("Local").doc(req.id\_local));
    if (!localSnap.exists) {
        aprovarInterno = false;
        justificativaFinal = "Rejeição sistêmica: Local referenciado não existe.";
    } else {
        localSnapData = localSnap.data()!;
        letraInicial = resumoNome ? resumoNome.charAt(0).toUpperCase() : "
";
    }
}

if (aprovarInterno && !req.id\_resumo\_bem\_patrimonial) {
    if (!req.nome\_resumo\_proposto?.trim()) {
        aprovarInterno = false;
        justificativaFinal = "Rejeição sistêmica: A aprovação exige vincular ou criar um resumo patrimonial válido.";
    } else if (!req.descricao\_resumo\_proposta?.trim()) {
        aprovarInterno = false;
        justificativaFinal = "Rejeição sistêmica: A criação de um novo resumo exige descrição obrigatória conforme o dicionário.";
    }
}

// PASSO C: Escritas (remoção do lock e resposta)
if (lockSnap.exists) {
    const lockData = lockSnap.data()!;
```

```
    if (lockData.id\_requisicao === idRequisicao) {
      tx.delete(lockRef);
    } else {
      throw new HttpsError("failed-precondition", "Conflito de integridade:
Lock pertence a outra requisição.");
    }
  }
}

let idBemCriado: string | null = null;

if (aprovarInterno) {
  let idResumo = req.id\_resumo\_bem\_patrimonial;

  // Se não havia resumo existente, cria atômica e transacionalmente o
  Resumo\_Bem\_Patrimonial
  if (!idResumo && req.nome\_resumo\_proposto) {
    const novoResumoRef = admin.firestore().collection("Resumo\_Bem\_
_Patrimonial").doc();
    const nomeNorm = req.nome\_resumo\_proposto.trim();
    tx.set(novoResumoRef, {
      nome: nomeNorm,
      descricao: req.descricao\_resumo\_proposta!.trim(),
      versao: 1
    });
    idResumo = novoResumoRef.id;
    resumoNome = nomeNorm;
    letraInicial = nomeNorm.charAt(0).toUpperCase();
  }

  const bemRef = admin.firestore().collection("Bem\_Patrimonial").doc();
  idBemCriado = bemRef.id;

  tx.set(bemRef, {
    id\_resumo\_bem\_patrimonial: idResumo,
    nome\_equipamento: resumoNome,
    numero\_patrimonio: req.numero\_patrimonio\_proposto,
    estado\_conservacao: req.estado\_conservacao\_proposto,
    id\_local: req.id\_local,
    photo\_url: req.photo\_url\_proposta, // ausência rejeitada antes
das escritas
    documento\_dado\_baixa\_pdf\_url: null,
  });
}
```

```
nome\_responsavel\_sei: req.nome\_responsavel\_proposto,
status: "Ativo",
versao: 1,
predio: localSnapData.predio,
andar: localSnapData.andar,
sala: localSnapData.sala,
letra\_inicial\_nome: letraInicial,
cadastrado\_em: admin.firestore.FieldValue.serverTimestamp(),
});

// PDF-001: Gravação da chave única associada ao novo bem
tx.set(chaveUnicaRef!, {
  tipo: "Bem\_Patrimonial",
  id\_recurso: idBemCriado,
  criado\_em: admin.firestore.FieldValue.serverTimestamp()
});

tx.update(reqRef, {
  status: "aprovada",
  respondida\_em: admin.firestore.FieldValue.serverTimestamp(),
  id\_usuario\_respondente: request.auth!.uid,
  id\_bem\_patrimonial\_se\_aprovado: idBemCriado,
  justificativa\_resposta: justificativaFinal,
});
} else {
  tx.update(reqRef, {
    status: "rejeitada",
    respondida\_em: admin.firestore.FieldValue.serverTimestamp(),
    id\_usuario\_respondente: request.auth!.uid,
    justificativa\_resposta: justificativaFinal,
  });
}

return { status: aprovarInterno ? "aprovada" : "rejeitada", idBemCriado
};
});
});

// PDF-007/PDF-017: Mecanismo único oficial de fan-out com chunks de 400.
// Decisão normativa (MODIFICACOES sec.5.5): batches particionados em blocos
de no máximo 400
```

```
// operações por commit, garantindo que atualizações em acervos grandes (ex:
    800 bens)
// não ultrapassem o limite do Firestore. A callable concorrente
    propagarNomeResumo
// (baseada em BulkWriter) foi descontinuada; este trigger é o único
    mecanismo oficial.
async function commitEmChunks(
    operacoes: Array<{ ref: FirebaseFirestore.DocumentReference; dados: Record
        <string, unknown> }>,
    tamanhoDunk = 400
): Promise<void> {
    for (let i = 0; i < operacoes.length; i += tamanhoDunk) {
        const chunk = operacoes.slice(i, i + tamanhoDunk);
        const batch = admin.firestore().batch();
        chunk.forEach(({ ref, dados }) => batch.update(ref, dados));
        await batch.commit(); // idempotente: re-executar o mesmo chunk produz o
            mesmo estado
    }
}

export const onResumoBemPatrimonialNomeAtualizado = onDocumentUpdated(
    "Resumo\_Bem\_Patrimonial/{resumoId}",
    async (event) => {
        const antes = event.data?.before.data();
        const depois = event.data?.after.data();
        if (!antes || !depois || antes.nome === depois.nome) return;

        const bens = await admin.firestore().collection("Bem\_Patrimonial")
            .where("id\_resumo\_bem\_patrimonial", "==", event.params.resumoId)
            .get();

        // PDF-007: único mecanismo de propagação de nome (chunks de 400 -- sem
        BulkWriter concorrente)
        const ops = bens.docs.map((bem) => ({ ref: bem.ref, dados: { nome\_
            equipamento: depois.nome } }));
        await commitEmChunks(ops);
    }
);

export const onLocalAtualizado = onDocumentUpdated(
    "Local/{localId}",
```

```
async (event) => {
  const antes = event.data?.before.data();
  const depois = event.data?.after.data();
  if (!antes || !depois) return;
  if (antes.predio === depois.predio && antes.andar === depois.andar &&
    antes.sala === depois.sala) return;

  const bens = await admin.firestore().collection("Bem_Patrimonial")
    .where("id_local", "==", event.params.localId)
    .get();

  // PDF-017: chunks de 400 garantem que locais com muitos bens (ex: 600+)
  // não excedam o limite
  const ops = bens.docs.map((bem) => ({
    ref: bem.ref,
    dados: { predio: depois.predio, andar: depois.andar, sala: depois.sala
  },
  }));
  await commitEmChunks(ops);
}
);

export const ingressarEmTurmaPorCodigo = onCall(async (request) => {
  await validarPermissao(request, ["Aluno"]);
  const { codigoTurma } = request.data as { codigoTurma: string };

  const turmaSnap = await admin.firestore().collection("Turma")
    .where("codigo_turma", "==", codigoTurma)
    .where("status", "==", "Ativo")
    .limit(1).get();
  if (turmaSnap.empty) throw new HttpsError("not-found", "Turma não
    encontrada ou arquivada.");
  const turmaDoc = turmaSnap.docs[0];
  const turma = turmaDoc.data();

  return admin.firestore().runTransaction(async (tx) => {
    // Checar capacidade lendo o documento da turma na transação (PDF-008)
    const turmaLiveSnap = await tx.get(turmaDoc.ref);
    if (!turmaLiveSnap.exists) throw new HttpsError("not-found", "Turma não
    encontrada.");
    const turmaLive = turmaLiveSnap.data()!;
```

```
if (turmaLive.status !== "Ativo") throw new HttpsError("failed-
precondition", "Turma inativa.");

const qtdAlunosAtual = turmaLive.qtd_alunos || 0;
if (qtdAlunosAtual >= turmaLive.capacidade) {
  throw new HttpsError("failed-precondition", "Turma lotada.");
}

// Checar se já está matriculado lendo apenas o documento individual (
evita leitura O(N))
const alunoSnap = await tx.get(turmaDoc.ref.collection("Alunos").doc(
request.auth!.uid));
if (alunoSnap.exists) {
  throw new HttpsError("already-exists", "Você já está nesta turma.");
}

// P14: Checar se foi removido anteriormente
const historico = await tx.get(
  turmaDoc.ref.collection("HistoricoAlunos")
    .where("id\_aluno", "==", request.auth!.uid)
    .where("tipo", "==", "exclusao\_aluno")
    .limit(1)
);
if (!historico.empty) {
  throw new HttpsError("failed-precondition",
    "Você foi removido desta turma pelo professor. O re-ingresso requer
convite explícito.");
}

// PDF-003 / MODIFICACOES sec.5.2: espelhamento atômico obrigatório.
// Matrícula requer dois documentos consistentes na mesma transação:
// 1. Turma/{turmaId}/Alunos/{uid} -- o aluno vê a turma no feed;
// 2. Usuarios/{uid}/Turmas/{turmaId} -- a turma aparece na home do
aluno.
// Ambos devem ser criados ou nenhum (atomicidade). O contador qtd\
\_alunos
// é atualizado com FieldValue.increment(1) na mesma transação.
tx.set(turmaDoc.ref.collection("Alunos").doc(request.auth!.uid), {
  id\_aluno: request.auth!.uid,
  ingressou\_em: admin.firestore.FieldValue.serverTimestamp(),
});
```

```
// Espelho reverso: aluno encontra a turma em Usuarios/{uid}/Turmas
tx.set(
  admin.firestore().collection("Usuarios").doc(request.auth!.uid)
    .collection("Turmas").doc(turmaDoc.id),
  {
    id\_turma: turmaDoc.id,
    id\_professor: turma.id\_professor,
    id\_materia: turma.id\_materia,
    nome\_turma: turma.nome\_turma,
    nome\_materia: turma.nome\_materia,
    ano: turma.ano,
    semestre: turma.semestre,
    status: turma.status,
    ingressou\_em: admin.firestore.FieldValue.serverTimestamp(),
  }
);
// Contador transacional (sem leitura prévia) -- idempotente para o atômico
tx.update(turmaDoc.ref, { qtd\_alunos: admin.firestore.FieldValue.increment(1) });
tx.set(turmaDoc.ref.collection("HistoricoAlunos").doc(), {
  id\_aluno: request.auth!.uid,
  tipo: "inclusao\_aluno",
  modo\_ingresso: "CODIGO",
  timestamp: admin.firestore.FieldValue.serverTimestamp(),
});

return { idTurma: turmaDoc.id, nomeTurma: turma.nome\_turma };
});
});

export const removerAlunoTurma = onCall(async (request) => {
  await validarPermissao(request, ["Professor", "Chefe_Geral"], true);
  const { idTurma, idAluno } = request.data as { idTurma: string; idAluno: string };

  const turmaRef = admin.firestore().collection("Turma").doc(idTurma);

  return admin.firestore().runTransaction(async (tx) => {
    const turmaSnap = await tx.get(turmaRef);
    if (!turmaSnap.exists) throw new HttpsError("not-found", "Turma não
```

```
encontrada.");
const turma = turmaSnap.data()!;

// Apenas professor da turma ou Chefe Geral
const isChefe = (await tx.get(admin.firestore().collection("Chefe_Geral"
).doc(request.auth!.uid))).exists;
if (!isChefe && turma.id_professor !== request.auth!.uid) {
  throw new HttpsError("permission-denied", "Apenas o professor responsá
vel pode remover alunos da turma.");
}

const alunoDocRef = turmaRef.collection("Alunos").doc(idAluno);
const alunoSnap = await tx.get(alunoDocRef);
if (!alunoSnap.exists) throw new HttpsError("not-found", "Aluno não está
matriculado nesta turma.");

// Exclusão espelhada atômica (PDF-003 / Secção 5.11 / PRO-04)
tx.delete(alunoDocRef);
tx.delete(admin.firestore().collection("Usuarios").doc(idAluno).
collection("Turmas").doc(idTurma));

// Decremento do contador de alunos e gravação no histórico da turma
tx.update(turmaRef, { qtd_alunos: admin.firestore.FieldValue.increment
(-1) });
tx.set(turmaRef.collection("HistoricoAlunos").doc(), {
  id_aluno: idAluno,
  tipo: "exclusao_aluno",
  removido_por: request.auth!.uid,
  timestamp: admin.firestore.FieldValue.serverTimestamp(),
});

// Registro na auditoria geral
tx.set(admin.firestore().collection("Registro_de_Auditoria").doc(), {
  id_usuario: request.auth!.uid,
  acao: "REMOCAO_ALUNO_TURMA",
  tipo_entidade_sofre_acao: "TURMA",
  id_do_objeto_da_entidade: idTurma,
  acao_feita_em: admin.firestore.FieldValue.serverTimestamp(),
  metadata: { id_aluno_removido: idAluno },
});
```

```

    return { status: "ALUNO_REMOVIDO" };
  });
});

```

### 10.2.7 Relatórios em PDF

Pseudo-códigos das funções de relatório, revisados: os nomes de campo passam a corresponder exatamente ao modelo (*data\_devolucao\_efetuada*, não *data\_devolucao*; filtro por *id\_almoxarifado* denormalizado em *Emprestimo\_Reagente*, não por um inexistente *id\_local\_saida*), e *gerarRelatorioAlmoxarifado* passa a checar o escopo do gestor, não apenas o papel.

#### Configuração Base e Relatório de Almoxarifado

```

import PDFDocument from "pdfkit-table";

// PDF-026: Diferenciação Nominal vs Físico e Extravio
// 1. O relatório obedece à seguinte semântica de dados (PDF-026):
//   medida_utilizada -> consumo histórico
//   peso_atual -> última medição bruta
//   peso_frasco_vazio -> tara quando conhecida
//   conteudo_nominal -> declaração original do fabricante
//   saldo_atual -> derivado apenas quando houver dados físicos suficientes.

// 2. Frascos com estado EXTRAVIADO não devem ser contados como aptos para
//   retirada. O predicado exige estado_fisico_frasco IN {FECHADO, ABERTO}.
//   Eles devem aparecer segregados em seções de perdas/extravios.

interface FiltrosAlmoxarifado {
  idAlmoxarifado: string;
  mes: number;
  ano: number;
}

export const gerarRelatorioAlmoxarifado = onCall(async (request) => {
  const { idAlmoxarifado, mes, ano } = request.data as FiltrosAlmoxarifado;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxarifado"]);
  await validarGestorDoAlmoxarifado(request.auth!.uid, request.auth!.token,
    idAlmoxarifado);

  const { DateTime } = require("luxon");
  const dataInicio = DateTime.fromObject({ year: ano, month: mes, day: 1 },
    { zone: "America/Sao_Paulo" }).startOf("day").toJSDate();

```

```
const dataFim = DateTime.fromObject({ year: ano, month: mes }, { zone: "America/Sao_Paulo" }).endOf("month").toJSDate();

const devolucoesSnapshot = await admin.firestore().collection("Emprestimo_Reagente")
  .where("id_almoxarifado", "==", idAlmoxarifado)
  .where("data_devolucao_efetuada", ">=", dataInicio)
  .where("data_devolucao_efetuada", "<=", dataFim)
  .get();

const consumoPorUnidade = devolucoesSnapshot.docs.reduce((acc, doc) => {
  const d = doc.data();
  if (d.unidade_medida_utilizada === "ml") acc.ml += d.medida_utilizada || 0;
  else if (d.unidade_medida_utilizada === "g") acc.g += d.medida_utilizada || 0;
  return acc;
}, { g: 0, ml: 0 });

const historicoSnapshot = await admin.firestore().collection("Historico_Frasco_Reagente")
  .where("id_almoxarifado", "==", idAlmoxarifado)
  .where("timestamp", ">=", dataInicio)
  .where("timestamp", "<=", dataFim)
  .orderBy("timestamp", "asc").get();

const doc = new PDFDocument({ margin: 30, size: "A4" });
const buffer = await buildPdfBuffer(doc, historicoSnapshot.docs, consumoPorUnidade);

return { pdfBase64: buffer.toString("base64") };
});
```

### Relatório Mensal de um Prédio (Filtros Compostos)

```
interface FiltrosPredio {
  predio: string;
  mes: number;
  ano: number;
  andar?: string;
  sala?: string;
  estadoConservacao?: string;
```

```
    status?: string;
  }

export const gerarRelatorioBensPredio = onCall(async (request) => {
  const filtros = request.data as FiltrosPredio;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Bens_Patrimoniais"]);

  const dataRef = new Date();
  const isHistorico = filtros.mes && filtros.ano && (filtros.mes !== dataRef.getMonth() + 1 || filtros.ano !== dataRef.getFullYear());

  let query: FirebaseFirestore.Query = admin.firestore().collection("Bem_Patrimonial");
  if (isHistorico) {
    query = admin.firestore().collectionGroup("Historico");
    query = query.where("predio", "==", filtros.predio);
    if (filtros.andar) query = query.where("andar", "==", filtros.andar);
    if (filtros.sala) query = query.where("sala", "==", filtros.sala);
    // C004: Filtros de status e estado_conservacao omitidos aqui na query NoSQL,
    // serão resolvidos em memoria no buildPredioPdfBuffer iterando sobre o array_alteracoes.
  } else {
    query = query.where("predio", "==", filtros.predio);
    if (filtros.andar) query = query.where("andar", "==", filtros.andar);
    if (filtros.sala) query = query.where("sala", "==", filtros.sala);
    if (filtros.estadoConservacao) query = query.where("estado_conservacao", "==", filtros.estadoConservacao);
    if (filtros.status) query = query.where("status", "==", filtros.status);
  }

  const bensSnapshot = await query.get();

  const doc = new PDFDocument({ size: "A4" });
  const buffer = await buildPredioPdfBuffer(doc, bensSnapshot.docs, filtros);

  return { pdfBase64: buffer.toString("base64") };
});
```

## Relatório Geral e Período Personalizado

```
interface FiltrosGeralEPersonalizado {
  dataInicio: string;
  dataFim: string;
  entidade: "Bens_Patrimoniais" | "Reagentes";
  prediosSelecionados?: string[];
  almoxarifadosSelecionados?: string[];
}

export const gerarRelatorioPersonalizado = onCall(async (request) => {
  const { dataInicio, dataFim, entidade, prediosSelecionados,
    almoxarifadosSelecionados } = request.data as FiltrosGeralEPersonalizado;
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Bens_Patrimoniais",
    "Gestor_Almoxarifado"]);

  const { DateTime } = require("luxon");
  const dataIniObj = DateTime.fromISO(dataInicio, { zone: "America/Sao_Paulo"
    }).startOf("day").toJSDate();
  const dataFimObj = DateTime.fromISO(dataFim, { zone: "America/Sao_Paulo"
    }).endOf("day").toJSDate();
  const agora = new Date();

  const diffDias = Math.ceil((dataFimObj.getTime() - dataIniObj.getTime()) /
    (1000 * 3600 * 24));
  if (diffDias <= 0 || diffDias > 31) {
    throw new HttpsError("invalid-argument", "O período não pode ser
      negativo nem superior a 31 dias corridos.");
  }
  if (dataFimObj > agora) {
    throw new HttpsError("invalid-argument", "A data final não pode ser no
      futuro.");
  }

  let snapshot;

  if (entidade === "Bens_Patrimoniais") {
    let query = admin.firestore().collectionGroup("Historico") // somente
      caminhos Bem_Patrimonial/{id}/Historico
      .where("timestamp", ">=", dataIniObj)
      .where("timestamp", "<=", dataFimObj);
    if (prediosSelecionados && prediosSelecionados.length > 0) {
```

```
    query = query.where("predio", "in", prediosSelecionados);
  }
  snapshot = await query.get();
} else {
  let queryReq = admin.firestore().collection("Emprestimo_Reagente")
    .where("data_devolucao_efetuada", ">=", dataIniObj)
    .where("data_devolucao_efetuada", "<=", dataFimObj);

  // C005: Validar escopo de almoxarifados para Gestores de Almoxarifado
  if (request.auth!.token.Chefe_Geral !== true && request.auth!.token.
Gestor_Almoxarifado === true) {
    if (!almoxarifadosSelecionados || almoxarifadosSelecionados.length ===
0) {
      throw new HttpsError("invalid-argument", "Gestores devem especificar
os almoxarifados autorizados.");
    }
    for (const idAlmox of almoxarifadosSelecionados) {
      const snap = await admin.firestore().collection("
Gestor_Almoxarifado_x_Almoxarifado")
        .where("id_gestor_almoxarifado", "==", request.auth!.uid)
        .where("id_almoxarifado", "==", idAlmox).limit(1).get();
      if (snap.empty) throw new HttpsError("permission-denied", "Sem
autorização para o almoxarifado " + idAlmox);
    }
    queryReq = queryReq.where("id_almoxarifado", "in",
almoxarifadosSelecionados);
  } else if (almoxarifadosSelecionados && almoxarifadosSelecionados.length
> 0) {
    queryReq = queryReq.where("id_almoxarifado", "in",
almoxarifadosSelecionados);
  }

  snapshot = await queryReq.get();
}

const doc = new PDFDocument({ layout: "landscape", size: "A4" });
const buffer = await buildPersonalizadoPdfBuffer(doc, snapshot.docs,
entidade);

return { pdfBase64: buffer.toString("base64") };
});
```

## Módulo de Etiquetas em PDF: Virgens e Reimpressão

```
//  
=====
```

*// GERACAO DE ETIQUETAS VIRGENS (NOVOS FRASCOS) - ABA 1*

```
//  
=====
```

```
interface DadosEtiquetasVirgens {  
  codigoInicial: number;  
  codigoFinal: number;  
  startRow?: number; // 1 a 10 (Grid A4)  
  startCol?: number; // 1 a 3  
  idOperacao?: string;  
}  
  
export const gerarPdfEtiquetasVirgens = onCall(async (request) => {  
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Almoхарifado"]);  
  const dados = request.data as DadosEtiquetasVirgens;  
  
  const total = dados.codigoFinal - dados.codigoInicial + 1;  
  if (total <= 0 || total > 50) {  
    throw new HttpsError("invalid-argument", "0 lote deve conter entre 1 e  
    50 etiquetas por impressão.");  
  }  
  
  // Audita a sessao de geracao de etiquetas virgens (Secao 4.13)  
  await admin.firestore().collection("Impressao_Etiqueta_Frasco").add({  
    gerado_em: admin.firestore.FieldValue.serverTimestamp(),  
    gerado_por: request.auth!.uid,  
    codigo_inicial: dados.codigoInicial,  
    codigo_final: dados.codigoFinal,  
    start_row: dados.startRow || 1,  
    start_col: dados.startCol || 1,  
    status_geracao: "CONCLUIDA",  
    id_operacao: dados.idOperacao || `op_etiquetas_${Date.now()}`,  
  });  
  
  const codigos = Array.from({ length: total }, (_, i) => `LCQUI-${dados.  
    codigoInicial + i}`);  
  const buffer = await gerarPdfEtiquetasNovosFrascos(codigos, dados.startRow
```

```
    || 1, dados.startCol || 1);

    const fileName = `etiquetas/virgens_${dados.codigoInicial}_a_${dados.
        codigoFinal}_${Date.now()}.pdf`;
    const fileRef = admin.storage().bucket().file(fileName);
    await fileRef.save(buffer, { contentType: "application/pdf" });
    const [url] = await fileRef.getSignedUrl({ action: "read", expires: Date.
        now() + 3600000 });

    return { url };
});

//
=====

// REIMPRESSAO E FICHA DE CONFERENCIA (FRASCOS JA CADASTRADOS) - ABA 2
//
=====

interface DadosReimpressao {
    frascoIds: string[]; // Máximo 10 frascos
}

export const gerarPdfReimpressaoFrascos = onCall(async (request) => {
    await validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxxarifado"]);
    const { frascoIds } = request.data as DadosReimpressao;

    // Regra 7.2.10: Teto rígido de segurança química
    if (!frascoIds || frascoIds.length === 0 || frascoIds.length > 10) {
        throw new HttpsError("invalid-argument", "Selecione entre 1 e 10 frascos
            por sessão de reimpressão.");
    }

    // 1. Busca dados completos, histórico recente e último empréstimo dos
        frascos
    const dadosFrascos = await buscarDadosCompletoParaConferencia(frascoIds);

    // 2. Registra auditoria individual de 2ª via para cada frasco (Secao
        4.41)
    const batch = admin.firestore().batch();
    for (const frascoId of frascoIds) {
```

```
const auditRef = admin.firestore().collection("Registro_de_Auditoria").doc();
batch.set(auditRef, {
  id_usuario: request.auth!.uid,
  acao: "REIMPRESSAO_ETIQUETA",
  tipo_entidade_sofre_acao: "FRASCO_REAGENTE",
  id_do_objeto_da_entidade: frascoId,
  acao_feita_em: admin.firestore.FieldValue.serverTimestamp(),
  metadata: { motivo: "segunda_via_conferencia" },
});
}
await batch.commit();

// 3. Gera PDF com Ficha de Conferencia + 1 Etiqueta Centralizada por frasco
const buffer = await gerarPdfFracosCadastrados(dadosFracos);

const fileName = `etiquetas/reimpressao_${Date.now()}.pdf`;
const fileRef = admin.storage().bucket().file(fileName);
await fileRef.save(buffer, { contentType: "application/pdf" });
const [url] = await fileRef.getSignedUrl({ action: "read", expires: Date.now() + 3600000 });

return { url };
});
```

### 10.3 Consolidação do planejamento complementar e contrato de execução

O inventário complementar inclui Planejamento\_Etiquetas\_LCQUI.md, REGRAS\_MULTI\_ROLE.md, COMPILACAO\_NIX\_LCQUI.md, README.md e os quatro documentos de acompanhamento. Os README de scaffold do frontend não adicionam requisitos de negócio; instruções AGENTS/CLAUDE orientam desenvolvimento, não a especificação do produto. DUVIDAS\_PENDENTES\_LCQUI.md é o registro atual de decisões; Q01–Q14 e DP-A01–DP-D02 já foram resolvidas, conforme MODIFICACOES\_CONSOLIDADAS\_LCQUI.md.

O planejamento de etiquetas foi incorporado às seções 4 (auditoria), 5.9 (contador e campos), 7 (regras), 8 (UI-08), 9 (ALM-07), 10 (geração) e 11 (acesso). O documento de Multi-Role foi incorporado integralmente pelos IDs RN-ROLE-01 a RN-ROLE-15 na seção 7. Os Markdown de origem permanecem como histórico; esta versão explicita as decisões conflitantes em vez de criar fontes concorrentes.

**Etiquetas e geometria.** Papel A4 210 por 297 mm; três colunas e dez linhas; etiqueta 65 por 26,5 mm; margens laterais 7,5 mm e superior/inferior 16 mm; espaçamentos horizontal/vertical zero. Linhas cinza contínuas orientam corte. Offset é aplicado apenas na primeira página; ao completar a grade, reiniciar na célula 1/1 da seguinte. Etiqueta virgem contém código, espaço manuscrito para data/reagente e Code 128. Segunda via contém nome e data cadastrados e uma etiqueta centralizada por ficha. Gerar barras em memória com bwip-js e PDF com pdffit; preservar proporção e área livre ao redor do código, testar leitura física e códigos longos. Casos mínimos: uma etiqueta, coluna 3, offset 4/2, última célula 10/3, 30/36 itens, nome longo, frasco sem histórico e rejeição de 11 IDs na segunda via. Os testes relatados no planejamento são evidência histórica de protótipo, não homologação da aplicação atual.

**Contador e auditoria.** Adota-se o caminho já usado pelo sistema, Contador\_Codigo\_Frasco/singleton e ultimo\_codigo\_gerado. Contadores/codigo\_frasco no planejamento é proposta antiga, não outro contador ativo. Cadastro incrementa contador e cria frasco atomicamente; impressão não reserva código. Lotes virgens usam Impressao\_Etiqueta\_Frasco; segunda via usa Registro\_de\_Auditoria. A sugestão contraditória de armazenar segunda via também em Impressao não é adotada. Custos reais incluem todas as escritas/leitura de domínio, índices, histórico e tentativas; não reutilizar estimativa de “1 leitura e 2 escritas” como custo total.

**Relatórios, transporte e integridade.** O backend atual retorna PDF em base64; a especificação adota esse transporte para relatórios limitados, com conversão em Blob no cliente e liberação da URL temporária. Roteiros e comprovantes permanecem no Storage. Relatórios maiores exigem decisão de infraestrutura antes de ampliar limites. Um hash simples de parâmetros/data não é assinatura digital nem prova de autoria; tratá-lo como identificador de rastreabilidade. Relatórios históricos usam eventos/snapshots da época, não somente o cadastro atual.

**Garantias de execução.** Exemplos de código desta seção são ilustrativos e não substituem contratos UI/dicionário. Todas as leituras transacionais precedem escritas; callbacks podem repetir e não devem enviar e-mail ou gerar PDF como efeito externo. A transação grava evento/operação e um processador idempotente realiza efeitos externos após commit. Alterações no Auth e upload no Storage exigem reconciliação, pois não participam da transação Firestore. Consultas in e getAll não têm o mesmo contrato: tamanho de lote deve ser escolhido por API e medido; 200 não é limite universal do Firestore.

**Ambiente e documentação operacional.** README contém instruções de aplicação e emuladores; COMPILACAO\_NIX\_LCQUI.md é guia de build documental. O flake.nix atual fornece Node/Python/Git/Docker, mas não TeX Live; não presumir que nix develop disponibiliza pdflatex. Usar TeX Live instalado ou ambiente Nix dedicado e compilar main.tex até estabilizar referências. Os quatro documentos de acompanhamento registram evidências do código, não novos requisitos independentes.

**Referências técnicas verificadas.** [Firebase: transações e lotes](#); [Firebase: propagação de Custom Claims](#); [Firebase: persistência offline](#). Consulta em 11/09/2026. Estes fundamentos apoiam a execução transacional, renovação de tokens e opção por cache em dispositivo confiável.

**Contratos dos auxiliares Q04/Q14.** `validarAceiteTcrTx` lê o evento de aceite produzido em sessão autenticada do retirante, confere versão institucional vigente, UID, frasco, finalidade, operação ainda não consumida e justificativa de 20–2000 caracteres. Não aceita timestamp ou UID arbitrário no payload do gestor. `vincularAceiteEAuditoriaTx` consome o aceite e registra o empréstimo correspondente na mesma transação. `validarAutoAtendimentoTx` conta gestores ativos, bloqueia quando existe outro e exige justificativa; `registrarNotificacaoChefiaTx` grava notificação/outbox idempotente. `resolverSnapshotQuimicoTx` resolve unidade pelo resumo e densidade pela especificação, sem gravações antes de concluir leituras. Auxiliares ilustram contratos, não atestam implementação em functions/. O TCR não substitui segurança química institucional.

### 10.3.1 Contratos complementares pós-auditoria

M-12: `parseDataCivil` – datas civis normalizadas em America/Sao\_Paulo

```
// M-12: todas as datas civis YYYY-MM-DD devem ser interpretadas no fuso
// America/Sao_Paulo, não em UTC. 'new Date("2026-09-15")' retorna meia-
//   noite UTC
// (=21:00 do dia anterior em Sao_Paulo), o que causaria comparações erradas.

// Prazo de devolução encerra-se às 23:59:59.999 do dia previsto em
//   Sao_Paulo.
//
// Nota: horário de verão histórico (Brasil aboliu em 2019) não é mais risco
//   futuro,
// mas o helper usa Intl para robustez caso o fuso mude.
import { DateTime } from "luxon";

// Interpreta como data local em America/Sao_Paulo às 00:00:00
function parseDataCivil(dataStr: string): Date {
  return DateTime.fromISO(dataStr, { zone: "America/Sao_Paulo" }).startOf("
    day").toJSDate();
}

function prazoLimiteDevolucao(dataStr: string): Date {
  return DateTime.fromISO(dataStr, { zone: "America/Sao_Paulo" }).endOf("day
    ").toJSDate();
}
```

```
}

```

M-11: geração de `codigo_turma` – base32 Crockford 6 chars com retry transacional

```
// M-11: codigo_turma UNIQUE, <= 20 chars, gerado pelo servidor.
// Usa Crockford Base32 (alfabeto sem I, L, O, U) para legibilidade.
// Retry transacional via docId determinístico em coleção Chaves_Unicas.
const CROCKFORD = "0123456789ABCDEFGHJKLMNPQRSTUVWXYZ";

function gerarCodigoTurmaCandidate(): string {
  const bytes = require("crypto").randomBytes(4); // 32 bits => 6 chars
  Crockford
  let n = bytes.readUInt32BE(0);
  let codigo = "";
  for (let i = 0; i < 6; i++) {
    codigo = CROCKFORD[n & 0x1f] + codigo;
    n >>= 5;
  }
  return codigo;
}

async function gerarCodigoTurmaTx(
  tx: FirebaseFirestore.Transaction
): Promise<string> {
  for (let tentativa = 0; tentativa < 5; tentativa++) {
    const candidato = gerarCodigoTurmaCandidate();
    const lockRef = admin.firestore()
      .collection("Chaves_Unicas").doc(`Turma_${candidato}`);
    const lockSnap = await tx.get(lockRef);
    if (!lockSnap.exists) {
      tx.create(lockRef, { reservado_em: admin.firestore.FieldValue.
serverTimestamp() });
      return candidato;
    }
  }
  throw new HttpsError("internal", "Falha ao gerar codigo_turma único após 5
tentativas.");
}
```

N-12: aceitarConviteAluno com criação condicional de conta Firebase Auth

```
// N-12: ao aceitar um convite, o aluno pode já ter conta Firebase Auth (
login Google

```

```
// prévio) ou não. A Cloud Function deve reutilizar o UID existente pelo e-
// mail,
// evitando contas duplicadas. Criação é condicional; o papel Aluno é sempre
// verificado/criado na mesma transação Firestore.
export const aceitarConviteAluno = onCall(async (request) => {
  const { tokenConvite, matriculaInformada } = request.data as {
    tokenConvite: string; matriculaInformada?: string };
  if (!request.auth?.uid) throw new HttpsError("unauthenticated", "Usuario
    nao autenticado.");

  // 1. Calcular hash SHA-256 para busca do convite pelo token_hash (o HMAC
  // e usado apenas como ID deterministico do documento)
  const { createHash } = require("crypto");
  const hashToken = createHash("sha256").update(tokenConvite).digest("hex");

  // 2. Verificar e-mail do usuario autenticado no provedor
  const authUser = await admin.auth().getUser(request.auth.uid);
  const emailNorm = authUser.email?.toLowerCase();

  // 3. Transacao: revalidar convite e criar Aluno e vinculos se nao
  // existirem
  return admin.firestore().runTransaction(async (tx) => {
    // PASSO A: Todas as leituras
    const conviteQuery = await tx.get(admin.firestore().collection("
    Convite_Aluno")
      .where("token_hash", "==", hashToken).limit(1));
    if (conviteQuery.empty) throw new HttpsError("not-found", "Convite
    invalido.");

    const conviteDoc = conviteQuery.docs[0];
    const convite = conviteDoc.data();

    if (convite.status !== "pendente")
      throw new HttpsError("failed-precondition", "Convite ja aceito ou
    cancelado.");
    if (new Date() > convite.expira_em.toDate())
      throw new HttpsError("failed-precondition", "Convite expirado.");
    if (emailNorm !== convite.email)
      throw new HttpsError("permission-denied", "E-mail da conta nao
    corresponde ao convite.");
```

```
const uid = request.auth!.uid;
const alunoRef = admin.firestore().collection("Aluno").doc(uid);
const usuarioRef = admin.firestore().collection("Usuarios").doc(uid);

const alunoSnap = await tx.get(alunoRef);
const usuarioSnap = await tx.get(usuarioRef);

let turmaSnap = null;
let turma = null;
let jaMatriculado = false;
let alunosSnap = null;

if (convite.id_turma) {
  const turmaRef = admin.firestore().collection("Turma").doc(convite.id_turma);
  turmaSnap = await tx.get(turmaRef);
  if (!turmaSnap.exists) throw new HttpsError("not-found", "Turma nao encontrada.");
  turma = turmaSnap.data()!;
  if (turma.status !== "Ativo") throw new HttpsError("failed-precondition", "Turma inativa.");

  const alunoNaTurmaSnap = await tx.get(turmaRef.collection("Alunos").doc(uid));
  jaMatriculado = alunoNaTurmaSnap.exists;

  // Validacao de capacidade transacional (PDF-008: evita leitura O(N))
  if (!jaMatriculado && !convite.exceder_capacidade) {
    const qtdAlunosAtual = turma.qtd_alunos || 0;
    if (qtdAlunosAtual >= turma.capacidade) {
      throw new HttpsError("resource-exhausted", "Turma lotada.");
    }
  }
}

// PDF-003: Validar matricula e unicidade se for criar um novo Aluno
let lockRefMatricula: FirebaseFirestore.DocumentReference | null = null;
let matriculaFinal: string | null = null;
if (!alunoSnap.exists) {
  const matriculaBruta = convite.numero_matricula ?? matriculaInformada;
  if (typeof matriculaBruta !== "string") {
```

```
        throw new HttpsError("invalid-argument", "Matrícula é obrigatória e  
deve ser texto.");  
    }  
  
    matriculaFinal = matriculaBruta.trim(); // Canonicalização documental  
  
    if (!matriculaFinal) {  
        throw new HttpsError("invalid-argument", "Matrícula não pode ser  
vazia ou conter apenas espaços.");  
    }  
  
    if (matriculaFinal.length > 20) {  
        throw new HttpsError("invalid-argument", "Matrícula inválida: excede  
o limite de 20 caracteres.");  
    }  
  
    lockRefMatricula = admin.firestore().collection("Chaves_Unicas").doc(  
Aluno__${matriculaFinal});  
    const lockSnap = await tx.get(lockRefMatricula);  
    if (lockSnap.exists) throw new HttpsError("already-exists", "A matrí  
cula informada já está em uso por outro aluno.");  
    }  
  
    // PASSO B: Escritas (idempotentes para repeticao concorrente)  
    if (!usuarioSnap.exists) {  
        tx.set(usuarioRef, {  
            nome: authUser.displayName ?? emailNorm,  
            email: emailNorm,  
            ativo: true,  
            profile_url_photo: authUser.photoURL ?? null,  
        });  
    }  
    if (!alunoSnap.exists && matriculaFinal && lockRefMatricula) {  
        tx.set(alunoRef, {  
            id_usuario: uid,  
            numero_matricula: matriculaFinal,  
        });  
        tx.set(lockRefMatricula, {  
            tipo: "Aluno",  
            id_recurso: uid,  
            criado_em: admin.firestore.FieldValue.serverTimestamp()  
        });  
    }
```

```
}

// Expirar convite e reconciliar com Secao 5 (aceitado_por, aceitado_em)
const agora = admin.firestore.FieldValue.serverTimestamp();
tx.update(conviteDoc.ref, {
  status: "aceitado",
  aceitado_por: uid,
  aceitado_em: agora
});

// Vincular a turma (se aplicavel e se nao estiver matriculado ainda)
if (convite.id_turma && turmaSnap && !jaMatriculado) {
  const turmaRef = turmaSnap.ref;
  const vincRef = turmaRef.collection("Alunos").doc(uid);
  const vincRef2 = usuarioRef.collection("Turmas").doc(convite.id_turma);

  // 1o Espelho: colecao Alunos na Turma
  tx.set(vincRef, { id_aluno: uid, ingressou_em: agora });
  // 2o Espelho: colecao Turmas no Usuario
  tx.set(vincRef2, {
    id_turma: convite.id_turma,
    id_professor: turma.id_professor,
    id_materia: turma.id_materia,
    nome_turma: turma.nome_turma,
    nome_materia: turma.nome_materia,
    ano: turma.ano,
    semestre: turma.semestre,
    status: turma.status,
    ingressou_em: agora
  });

  // Registro no historico: preserva regras de reingresso sem duplicacao
  tx.set(turmaRef.collection("HistoricoAlunos").doc(), {
    id_aluno: uid,
    tipo: "inclusao_aluno",
    modo_ingresso: "CONVITE",
    justificativa: convite.justificativa_excecao || null,
    timestamp: agora
  });

  tx.update(turmaRef, { qtd_alunos: admin.firestore.FieldValue.increment
(1) });
```

```
    }

    return { vinculado: !!convite.id_turma, ja_matriculado: jaMatriculado };
  });
  // Apos commit: atualizarCustomClaims(request.auth!.uid) -- recuperavel e
  idempotente
});
```

#### M-01: registrarBaixaBemPatrimonial (RF12)

```
// M-01: Regra RF12 -- registro de baixa patrimonial após processo
institucional.
// O gestor faz upload do PDF comprobatório antes de chamar esta função.
// O backend valida a existência do documento no Storage antes de confirmar
a baixa.
export const registrarBaixaBemPatrimonial = onCall(async (request) => {
  const { idBem, documentoUrl } = request.data as { idBem: string;
    documentoUrl: string };
  await validarPermissao(request, ["Chefe_Geral", "Gestor_Bens_Patrimoniais"], true);
  await validarDocumentoStorage(documentoUrl, request.auth!.uid); //
    verifica existência no bucket
  return admin.firestore().runTransaction(async (tx) => {
    const bemRef = admin.firestore().collection("Bem_Patrimonial").doc(idBem);
    const snap = await tx.get(bemRef);
    if (!snap.exists) throw new HttpsError("not-found", "Bem não encontrado.");
    const bem = snap.data()!;
    if (bem.status !== "Inservível")
      throw new HttpsError("failed-precondition", "Bem deve estar Inservível
antes da baixa.");
    tx.update(bemRef, {
      status: "Ja_dado_baixa",
      documento_dado_baixa_pdf_url: documentoUrl,
      versao: bem.versao + 1,
    });
    registrarHistoricoBemTx(tx, bemRef, bem, { status: "Ja_dado_baixa" },
request.auth!.uid);
    return { status: "Ja_dado_baixa" };
  });
});
```

**M-07: Bounds e validação na geração de etiquetas**

O grid de etiquetas é sempre 3 colunas  $\times$  10 linhas (30 posições por página A4). A seleção de offset inicial aplica-se apenas na primeira página, validando: coluna em  $[1, 3]$  e linha em  $[1, 10]$ . Lotes de até 50 etiquetas virgens são suportados por operação; as páginas subsequentes preenchem as posições continuamente desde a coordenada (1,1). Para segunda via, o limite é de 10 frascos por sessão, validado no backend antes de iniciar a geração. IDs fora do intervalo de etiquetas virgens conhecidas são rejeitados com mensagem explícita.

## 11 Regras de Segurança do Firestore (Security Rules)

### i Por que as Security Rules são obrigatórias

Toda leitura via `onSnapshot` no frontend passa **diretamente** pelo Firestore SDK, **sem** passar pelas Cloud Functions. Portanto, as Cloud Functions são a camada de autorização para escrita, mas as *Security Rules* são a camada de autorização para **leitura direta**. Sem rules corretas, qualquer usuário autenticado poderia ler qualquer coleção.

### Princípio: Deny-All por Padrão

A regra raiz do arquivo `firestore.rules` deve ser:

```
allow read, write: if false;
```

Todas as exceções são declaradas explicitamente abaixo.

### 11.1 Regras por Coleção

| Coleção                                                                                | Regra                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Usuarios/{uid}</code>                                                            | Read: <code>request.auth.uid == uid</code> (usuário lê só o próprio documento). Write: negado ao cliente (só Cloud Function).                                                                                                                    |
| <code>Usuarios/{uid}/Turmas/{turmaId}</code>                                           | Read: <code>request.auth.uid == uid</code> (o aluno lê em tempo real apenas as turmas em que está matriculado). Write: negado ao cliente (gerenciada exclusivamente pelas Cloud Functions).                                                      |
| <code>Chefe_Geral</code> , <code>Gestor_Almojarifado</code> , etc. (coleções de papel) | Read: <code>request.auth.uid == docId</code> (cada usuário lê só o próprio documento de papel). Write: negado ao cliente.                                                                                                                        |
| <code>Turma/{turmaId}</code>                                                           | Read: usuário é professor da turma ( <code>resource.data.id_professor == request.auth.uid</code> ) OU está na sub-coleção <code>Alunos</code> OU <code>'Chefe_Geral'</code> in <code>request.auth.token.roles</code> . Write: negado ao cliente. |
| <code>Turma/{turmaId}/Alunos</code>                                                    | Read: <code>request.auth.uid == resource.data.id_aluno</code> OU membro ativo da mesma turma, apenas em projeção mínima sem matrícula/e-mail.                                                                                                    |
| <code>Turma/{turmaId}/Posts</code>                                                     | Read: membro da turma (professor ou aluno matriculado) OU <code>'Chefe_Geral'</code> in <code>request.auth.token.roles</code> . Write: negado.                                                                                                   |

| Coleção                                                       | Regra (continuação)                                                                                                                                        |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Turma/{turmaId\}/Posts/<br>\{postId\}/Comentarios             | allow read, write: if false; (Negado ao cliente.<br>Leitura filtrada provida via Cloud Function<br>listarComentariosPost).                                 |
| Usuarios/{uid\}/<br>Notificacoes                              | Read: request.auth.uid == uid. Write: negado ao<br>cliente.                                                                                                |
| Resumo_Reagente/<br>\{resumoId\}                              | Read: qualquer usuário com papel ativo no sistema<br>(token de acesso válido). Write: negado ao cliente.                                                   |
| Resumo_Reagente/<br>\{resumoId\}/<br>Especificacoes/{specId\} | Read: qualquer usuário com papel ativo no sistema<br>(catálogo público interno). Write: negado ao cliente.                                                 |
| Frasco_Reagente                                               | Read: papel roles no token contém Chefe_Geral OU<br>Gestor_Almoхарifado OU Professor OU Bolsista OU<br>Aluno. Write: negado ao cliente.                    |
| Emprestimo_Reagente                                           | Read: Chefe_Geral OU Gestor_Almoхарifado OU o<br>próprio usuário que retirou (resource.data.id_<br>usuario_retirou == request.auth.uid). Write:<br>negado. |
| Bem_Patrimonial                                               | Read: Chefe Geral, Gestor de Bens Patrimoniais ou<br>Professor. Write: negado ao cliente.                                                                  |
| Requisicao_Edicao_Bem_<br>Patrimonial                         | Read: o solicitante (resource.data.id_usuario_<br>solicitante == uid) OU gestores de bens (papel no<br>token). Write: negado.                              |
| Requisicao_Adicao_Bem_<br>Patrimonial                         | Idem acima.                                                                                                                                                |
| Locks_Requisicao_<br>Patrimonio                               | Read/Write: negado ao cliente (exclusivo de Cloud<br>Function).                                                                                            |
| Registro_de_Auditoria                                         | Read: Chefe_Geral no token. Write: negado ao cliente.                                                                                                      |
| Roteiro_Experimento                                           | Read: resource.data.id_professor_upload == uid<br>OU uid pertence ao array professores_<br>compartilhados. Write: negado.                                  |
| Contador_Codigo_Frasco                                        | Read: request.auth != null (qualquer logado lê).<br>Write: negado ao cliente (só Cloud Function atualiza o<br>sequenciador).                               |
| Almoхарifado                                                  | Read: qualquer usuário com papel ativo no sistema<br>(token de acesso válido). Write: negado ao cliente.                                                   |
| Local                                                         | Read: qualquer usuário com papel ativo no sistema.<br>Write: negado ao cliente.                                                                            |

| Coleção                                                                                                            | Regra (continuação)                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Materia                                                                                                            | Read: qualquer usuário com papel ativo no sistema.<br>Write: negado ao cliente.                                                                                        |
| Substancia_Quimica                                                                                                 | Read: qualquer usuário com papel ativo no sistema (catálogo público interno). Write: negado ao cliente.                                                                |
| Lote                                                                                                               | Read: papel <code>roles</code> no token contém <code>Chefe_Geral</code> OU <code>Gestor_Almoхарifado</code> (leitura restrita). Write: negado ao cliente.              |
| Coleções de Relatórios e Materializações ( <code>Resumo_*_Diario</code> , <code>Atividade_Gestor_*_Mensal</code> ) | Read: <code>Chefe_Geral</code> , <code>Gestor_Almoхарifado</code> ou <code>Gestor_Bens_Patrimoniais</code> conforme o respectivo domínio.<br>Write: negado ao cliente. |

#### Leitura de Custom Claims nas Security Rules

O papel do usuário fica disponível nas Security Rules como `request.auth.token.roles`. Exemplo:

```
function temPapel(papel) {
  return papel in request.auth.token.roles;
}
```

Isso garante consistência com o mesmo Custom Claim usado pelas Cloud Functions (`resolverPapeisDoToken`).

## 11.2 Contrato de acesso para os fluxos detalhados

#### Auditoria de transições críticas e estado físico

As transições para estado físico `EXTRAVIADO`, entrada ou saída de `em_quarentena`, desativação ou exclusão lógica nunca devem ser expostas para escrita direta do cliente. O backend deve validar as condições de contorno, registrar motivo e autor da ação, e garantir efeitos colaterais (como encerramento de empréstimos), persistindo toda a operação no `Historico_Frasco_Reagente` de forma atômica.

Além do papel, validar o escopo atual. DP-D01 determina consulta de conta ativa em mutações de alto impacto conforme a seção 10; leituras e demais mutações podem confiar no JWT até renovação. Atualizar Custom Claims não invalida imediatamente o token emitido. Versão de permissões e reconciliação não constituem garantia universal já implementada. Preferência de papel no cliente só altera apresentação.

Leituras de membros, posts, comentários e PDFs de turma exigem professor responsável ou vínculo ativo; colegas recebem projeção mínima, sem e-mail/matricula. Requisições são visíveis ao solicitante, gestor patrimonial e Chefe. Empréstimos completos são visíveis ao retirante, gestor vinculado ao almoxarifado e Chefe; demais leitores do catálogo recebem somente disponibilidade. Histórico operacional segue o mesmo escopo, e auditoria global

fica com a chefia. Diretórios usados em seletores devem ter endpoints/projeções mínimas autorizadas, não leitura irrestrita de Usuarios.

Mutações de domínio e locks são exclusivas do servidor, inclusive marcar notificações lidas; não permitir delete de notificações para implementar Limpar tudo. Regras devem cobrir explicitamente `Resumo_Reagente/id/Especificacoes`, `Turma` e suas subcoleções e `Bem_Patrimonial/id/Historico`. Consultas `collection-group` exigem regras/índices próprios. Contador, unicidade e controles de papéis são técnicos; somente resposta mínima autorizada é exposta ao operador.

Storage valida dono, caminho, tipo e tamanho: foto inferior a 5 MiB, baixa inferior a 10 MiB e roteiro inferior a 15 MiB. Backend verifica arquivo existente e assinatura de conteúdo antes de referenciar; metadados não bastam para provar formato. Leitura de roteiro exige propriedade, compartilhamento ou post de turma acessível. Comprovante de baixa segue escopo patrimonial e não pode ser removido após vinculação. Não conceder leitura de todo Storage apenas porque o usuário está autenticado.

Estas são exigências da especificação; `firestore.rules` e `storage.rules` atuais ainda divergem, conforme `AUDITORIA_ATUALIZADA.md`. A presença de `deny-all` genérico não anula permissões amplas de matches específicos.

### 11.3 Moderação: autorização por documento e resposta filtrada

Firestore Rules autorizam ou negam documentos completos; não fazem projeção nem substituição do campo texto. Não implementar “texto vazio via Rules”. Para o modelo físico descrito, negar leitura direta de `Comentarios` e histórico a quem não pode conhecer o original; a listagem de comentários é fornecida por endpoint autenticado que valida vínculo com a turma e devolve somente aviso institucional para comentários moderados de colegas. Autor recebe original marcado; Professor responsável e Chefe Geral recebem conteúdo necessário à auditoria. Não retornar original em campos auxiliares, cache, notificações ou mensagens de erro. Consulta ampla cliente que possa incluir documentos proibidos deve ser rejeitada e substituída pelo endpoint, nunca filtrada apenas no frontend. Backend valida também autoria da edição, motivo e papel de moderação, status ativo da turma e registra histórico/auditoria.

Coleções `Controle_Papeis`, `Operacoes` e `Chaves_Unicas` são exclusivas de Admin SDK; leitura e escrita diretas de clientes são negadas. O `deny-all` atual já as protege, sem `allow` concorrente; matches explícitos constituem hardening documental. Esta seção descreve o alvo; a implementação atual de moderação permanece pendente.

## 12 Implementações em Estudo para Versões Futuras

### i Escopo desta seção

Nada aqui faz parte do modelo 3FN da Seção 4 ou do mapeamento físico Firestore da Seção 5, nem deve ser implementado nesta primeira versão. É o registro de uma proposta para quando reagentes gasosos entrarem no almoxarifado do LCQUI – hoje nenhum reagente gasoso está cadastrado, por isso **GASOSO** foi removido de *Resumo\_Reagente.estado\_fisico* (seção 4.17) nesta versão, em vez de deixado junto de campos pensados só para sólido/líquido.

### 12.1 Por que **GASOSO** sozinho não basta

Simplesmente devolver **GASOSO** a *estado\_fisico* resolveria a classificação básica, mas um cilindro de gás carrega informação que **não** é estado físico e que não deve virar uma explosão de valores dentro do mesmo enum (**GASOSO\_COMPRIMIDO**, **GASOSO\_LIQUEFEITO**, **MISTURA\_GASOSA\_CALIBRACAO**, ...). São quatro dimensões independentes, que devem continuar independentes no banco:

| Dimensão                       | Exemplos de valor                                  |
|--------------------------------|----------------------------------------------------|
| Estado físico                  | sólido, líquido, gasoso                            |
| Forma de armazenamento         | gás comprimido, gás liquefeito, gás criogênico     |
| Tipo de substância (já existe) | pura, mistura                                      |
| Finalidade/aplicação           | reagente comum, gás de calibração, gás de processo |

Um cilindro de CO<sub>2</sub> puro, por exemplo, é *estado\_fisico* = gasoso, *forma\_armazenamento* = liquefeito (o CO<sub>2</sub> é mantido líquido sob pressão dentro do cilindro, embora seja gasoso em condição ambiente), *tipo\_substancia* = pura. Já uma mistura de calibração (ex.: 500 ppm de CO<sub>2</sub> balanceado em N<sub>2</sub>) é *estado\_fisico* = gasoso, *forma\_armazenamento* = comprimido, *tipo\_substancia* = mistura, *finalidade* = calibração. Tratar “mistura gasosa de calibração” como uma categoria física própria misturaria uma propriedade química (o que a substância é) com uma propriedade comercial/de uso (para que ela serve) – o mesmo tipo de acoplamento indevido que a decomposição do *status* de *Frasco\_Reagente* (seção 4.19) já evitou para outro campo.

### 12.2 Proposta de campos (apenas ilustrativa)

#### Especificacao\_Reagente (campos adicionais propostos)

*forma\_armazenamento*

ENUM NULL

CONVENCIONAL, GAS\_COMPRIMIDO, GAS\_LIQUEFEITO, GAS\_CRIOGENICO

***finalidade***

ENUM NULL

*REAGENTE, CALIBRACAO, PROCESSO, OUTRO*

Ambos ficam em *Especificacao\_Reagente*: são propriedades do produto comercial específico, não da substância química abstrata nem do item de catálogo. NULL para itens sólidos/líquidos, onde a dimensão não se aplica.

### 12.3 Além do enum: medição de gases não é pesagem

A Regra de Cálculo de Volume e Peso (seção 6.2.13) assume que o volume é sempre derivado de uma leitura de balança convertida por densidade. Para cilindros de gás, a prática de laboratório mais comum é ler a **pressão** no manômetro do próprio cilindro (ainda que pesar o cilindro também seja uma alternativa viável em alguns casos). Isso implica que *Frasco\_Reagente* precisaria, no mínimo, de:

- *pressao\_nominal\_bar* – pressão de enchimento de fábrica.
- *pressao\_atual\_bar* – última leitura do manômetro (substituindo/complementando *peso\_no\_cadastrado/medida\_usada* para este tipo de frasco).
- *capacidade\_litros\_cntp* – volume de gás equivalente em CNTP (Condições Normais de Temperatura e Pressão) quando o cilindro está cheio.
- *tipo\_valvula* – necessário para compatibilidade de equipamento na retirada.

O cálculo de consumo também muda: em vez de  $V = \Delta m / \rho$ , a estimativa correta usa a proporcionalidade entre pressão e quantidade de gás restante (lei dos gases ideais,  $PV = nRT$ , ou uma correlação real para gases não-ideais em alta pressão), não uma conversão por densidade. Por isso este documento não tenta encaixar gases nos campos de peso já existentes: a reintrodução de **GASOSO** deve vir acompanhada desses campos e dessa fórmula alternativa, não apenas do valor no enum.

### 12.4 Horários de Turma

A entidade *Horario\_Turma* (dia da semana, hora de início/fim e local por turma) foi removida da V1 por não possuir fluxo de uso definido nas telas nem nas regras de negócio. Em versões futuras, ela pode ser reintroduzida para permitir que o sistema exiba grades horárias, detecte conflitos de sala e envie lembretes automáticos antes do início das aulas.

### 12.5 Lotação física de professores (V2)

Q01: vinculação a prédio, gabinete e sala não integra a V1 por insuficiência de dados estruturados. Integração futura de dados institucionais de lotação e alocação de salas permitirá

filtragem espacial; depende de disponibilização e validação dos dados pela UENF. Na V1, busca de professor limita-se a Nome, Matéria, Centro e Laboratório.

## 13 Camada formal incremental: prova de infraestrutura M0

Esta seção acrescenta uma fatia formal ao baseline congelado da Fase 3B. A interface em `frontend/`, o backend em `functions/` e o Firestore permanecem a aplicação operacional. CUE descreve a estrutura; Alloy verifica propriedades abstratas; Rust apenas transforma os resultados em documentação.

O modelo inicial cobre três dimensões do frasco e o filtro físico de retirada. Esse filtro é uma condição necessária, mas não suficiente: validade recalculada, autorização, aceite de uso excepcional, finalidade, idempotência e concorrência Firestore continuam sujeitos aos contratos humanos existentes e à implementação. Não há migração integral da entidade nem nova regra de negócio nesta seção.

### Frasco\_\_Reagente — fatia M0

M0 parcial: filtro físico; não equivale à autorização completa de retirada.

**estado\_fisico\_frasco** Tipo: enum. Obrigatório: sim. Aceita nulo: não. Valores: FECHADO, ABERTO, VAZIO, QUEBRADO, DESCARTADO, EXTRAVIADO.

**disponibilidade** Tipo: enum. Obrigatório: sim. Aceita nulo: não. Valores: DISPONIVEL, EMPRESTADO.

**em\_quarentena** Tipo: bool. Obrigatório: sim. Aceita nulo: não.

### Verificação abstrata de retirada

**INV-FRASCO-001** BloqueioFisico: UNSAT.

Escopo: `check BloqueioFisico for 4 but exactly 2 Estado.`

**INV-EMPRESTIMO-001** Unicidade: UNSAT.

Escopo: `check Unicidade for 4 but exactly 2 Estado.`

**WIT-RETIRADA-001** Testemunha: SAT.

Escopo: `run Testemunha for 4 but exactly 2 Estado.`

**WIT-DISPONIBILIDADE-001** DisponivelNaoApto: SAT.

Escopo: `run DisponivelNaoApto for 4 but exactly 2 Estado.`

UNSAT significa ausência de contraexemplo no escopo declarado; SAT indica testemunha encontrada. Não certifica a implementação Firebase.

O modelo assume coerência entre disponibilidade e empréstimos ativos no estado inicial. A retirada abstrata adiciona um empréstimo e atualiza a disponibilidade; a coerência do estado final é verificada, não assumida. As buscas de testemunha exigem uma retirada possível e um frasco extraviado disponível, mas fisicamente inapto. A existência dessas instâncias evita

atribuir sucesso apenas a um modelo sem comportamento possível. Estados não envolvidos na transição não representam estados operacionais validados.

Os hashes das entradas constam de `generated/MANIFEST.json`. Os resultados são limitados ao escopo declarado e não substituem testes no Firebase Emulator.

## 14 Camada formal M1: catálogo de reagentes

A primeira ampliação cobre os registros normalizados de resumo e especificação das Seções 4 e 5. Os schemas CUE descrevem registros existentes, com todos os campos presentes: um campo que admite nulo recebe `null` explicitamente. Essa representação não é um payload de criação, não insere valores padrão e não é o corpo de um documento Firestore. Os IDs relacionais são inteiros; no Firestore, o `docId` e as referências são strings, conforme o mapeamento existente.

A condição de pesagem exige frequência inteira positiva quando habilitada e nulo quando desabilitada. O schema do par resumo/especificação exige que a FK aponte para o resumo apresentado; para líquido, exige densidade positiva. Isso não comprova existência de registros no banco nem imutabilidade após confirmação. Essas verificações continuam sendo responsabilidade das transações e das futuras especificações comportamentais.

### Resumo\_Reagente — modelo normalizado M1

Registro normalizado: todos os campos presentes, incluindo `null` explícito quando permitido. Não é payload de criação nem corpo Firestore.

**id** Tipo: int. Obrigatório: sim. Aceita nulo: não. Mínimo: 1. Identidade relacional SERIAL; Firestore usa `docId` string, fora do corpo.

**nome** Tipo: string. Obrigatório: sim. Aceita nulo: não. Máximo: 150 caracteres.

**tipo\_substancia** Tipo: enum. Obrigatório: sim. Aceita nulo: não. Valores: PURA, MISTURA.

**natureza\_quimica** Tipo: enum. Obrigatório: sim. Aceita nulo: não. Valores: ORGANICO, INORGANICO, ELEMENTO, HIBRIDO.

**requer\_pesagem\_frequente** Tipo: bool. Obrigatório: sim. Aceita nulo: não.

**frequencia\_pesagem\_dias** Tipo: int. Obrigatório: sim. Aceita nulo: sim. Mínimo: 1. Positiva quando requer pesagem frequente; `null` caso contrário (Seção 5).

**estado\_fisico** Tipo: enum. Obrigatório: sim. Aceita nulo: não. Valores: SOLIDO, LIQUIDO.

**eh\_higroscopico** Tipo: bool. Obrigatório: sim. Aceita nulo: não.

### Mapeamento: Resumo\_Reagente

Caminho: `Resumo_Reagente/{id}`.

- `id SERIAL` torna-se `docId` string fora do corpo; FKs no Firestore são strings.
- `letra_inicial` é denormalização exclusiva do Firestore, recalculada pelo servidor junto de `nome`.
- `estado_fisico` e `eh_higroscopico` pertencem ao resumo; unidade é derivada e não editável.

## Especificacao\_Reagente — modelo normalizado M1

A densidade depende do estado do resumo no par validado. Composição, imutabilidade e existência global de FKs não estão modeladas nesta fatia.

**id** Tipo: int. Obrigatório: sim. Aceita nulo: não. Mínimo: 1. Identidade relacional SERIAL; Firestore usa docId string.

**id\_resumo\_reagente** Tipo: int. Obrigatório: sim. Aceita nulo: não. Mínimo: 1. FK para Resumo\_Reagente; existência validada pelo servidor. Firestore usa string.

**descricao** Tipo: string. Obrigatório: sim. Aceita nulo: não. Máximo: 150 caracteres.

**fabricante** Tipo: string. Obrigatório: sim. Aceita nulo: sim. Máximo: 150 caracteres.

**codigo\_produto\_fabricante** Tipo: string. Obrigatório: sim. Aceita nulo: sim. Máximo: 100 caracteres.

**grau\_pureza** Tipo: string. Obrigatório: sim. Aceita nulo: sim. Máximo: 30 caracteres.

**densidade** Tipo: number. Obrigatório: sim. Aceita nulo: sim. Maior que 0. Menor que 10000. Múltiplo de 0.0001. NUMERIC(8,4), g/mL; positiva quando informada; obrigatória para líquido no par com resumo. Imutabilidade depende de transição, fora de M1.

**classe\_inflamabilidade** Tipo: enum. Obrigatório: sim. Aceita nulo: não. Valores: NAO\_INFLAMAVEL, CLASSE\_1, CLASSE\_2, CLASSE\_3.

**eh\_controlado\_pf** Tipo: bool. Obrigatório: sim. Aceita nulo: não.

**eh\_controlado\_eb** Tipo: bool. Obrigatório: sim. Aceita nulo: não.

**link\_fds\_fispq** Tipo: string. Obrigatório: sim. Aceita nulo: sim. Máximo: 255 caracteres. Padrão: `^https://[^\/?#[:space:]]+[^[:space:]]*$`. HTTPS com host não vazio; acessibilidade e autorização do recurso não verificadas por CUE.

## Mapeamento: Especificacao\_Reagente

Caminho: `Resumo_Reagente/{resumoId}/Especificacoes/{id}`.

- `id` torna-se `docId` string; `id_resumo_reagente` string deve corresponder ao pai, validado pelo servidor.
- `unidade_de_medida` é projeção opcional derivada do estado do resumo: `g` ou `ml`; não é fonte canônica.
- `composicao` é array de mapas embutido; sua migração formal ainda não está incluída no M1.
- `Precisão/escala` e `limites` VARCHAR permanecem válidos; `densidade` é positiva e obrigatória para líquido.

## Limites da migração M1

Composição, estoque mínimo, identidade química global, normalização de nomes, imutabilidade e autorização não foram migrados integralmente. As notas de mapeamento acima registram decisões existentes, sem constituir um schema Firestore completo. Nenhum campo foi movido entre resumo e especificação; nenhuma implementação de interface ou backend foi gerada ou substituída.

Os testes CUE incluem exemplos aceitos e rejeitados, limites de texto Unicode, escala decimal, ausência versus nulo, enums e condições entre registros. O IR v2 transporta várias entidades; o gerador ainda lê o IR v1 do M0 e mantém seus fragmentos. As assertions Alloy apresentadas na seção anterior continuam restritas à retirada abstrata do M0, não ao catálogo M1.